

Chatopera 云服务

用户使用文档

<https://bot.chatopera.com>



Chatopera

重新定义聊天机器人

Copyright © (2018-2026) 北京华夏春松科技有限公司版权所有

Table of Contents

[服务概述](#) [服务概述](#)

[入门教程](#) [入门教程](#)

[<1/5> 创建机器人](#) [<1/5> 创建机器人](#)

[<2/5> 生成 RAG 知识库](#) [<2/5> 生成 RAG 知识库](#)

[<3/5> 使用 FAQ 修复回答](#) [<3/5> 使用 FAQ 修复回答](#)

[<4/5> 批量导入 FAQ 知识库](#) [<4/5> 批量导入 FAQ 知识库](#)

[<5/5> 优化欢迎语](#) [<5/5> 优化欢迎语](#)

[使用指南](#) [使用指南](#)

[账号](#) [账号](#)

[注册使用 Chatopera 云服务](#) [注册使用 Chatopera 云服务](#)

[开发者工具](#) [开发者工具](#)

[VSCode 插件](#) [VSCode 插件](#)

[快速开始（中文）](#) [快速开始（中文）](#)

[Get Started\(En\)](#) [Get Started\(En\)](#)

[RAG 知识库](#) [RAG 知识库](#)

[知识库问答自动生成](#) [知识库问答自动生成](#)

[知识库修改提示词模板](#) [知识库修改提示词模板](#)

[FAQ 知识库](#) [FAQ 知识库](#)

[知识库问答对维护](#) [知识库问答对维护](#)

[知识库问答及调优测试](#) [知识库问答及调优测试](#)

[知识库问答对导入和导出](#) [知识库问答对导入和导出](#)

[自定义词典增强知识库](#) [自定义词典增强知识库](#)

[知识库热门问题查看和导出](#) [知识库热门问题查看和导出](#)

[设置欢迎语热门问题](#) [设置欢迎语热门问题](#)

[使用函数获得动态答案](#) [使用函数获得动态答案](#)

[话术助手快速检索话术](#) [话术助手快速检索话术](#)

[集成微信实现转人工客服](#) [集成微信实现转人工客服](#)

[意图识别](#) [意图识别](#)

[创建和训练意图识别模型](#) [创建和训练意图识别模型](#)

[发布上线意图识别生产版本](#) [发布上线意图识别生产版本](#)

[大语言模型](#) [大语言模型](#)

[使用大语言模型（LLM）](#) [使用大语言模型（LLM）](#)

[多轮对话](#) [多轮对话](#)

使用通配符匹配器	使用通配符匹配器
使用模糊匹配器	使用模糊匹配器
使用意图匹配器	使用意图匹配器
设置回复	设置回复
使用上下轮钩子	使用上下轮钩子
使用函数	使用函数
回复富文本内容	回复富文本内容
配置环境变量	配置环境变量
管理对话状态	管理对话状态
集成自定义外部对话系统	集成自定义外部对话系统
设置定时任务	设置定时任务
对话调优	对话调优
对话测试	对话测试
对话历史分析	对话历史分析
CLI 连接多轮对话	CLI 连接多轮对话
渠道	渠道
H5 聊天控件	H5 聊天控件
机器人聊天码	机器人聊天码
飞书	飞书
春松客服	春松客服
系统集成	系统集成
SDK	SDK
CLI 安装和配置	CLI 安装和配置
CLI 导入和导出对话语料	CLI 导入和导出对话语料
词典	词典
系统词典	系统词典
词汇表词典	词汇表词典
正则表达式词典	正则表达式词典
示例程序	示例程序
讲笑话机器人	讲笑话机器人
GuessNumberBot	GuessNumberBot
预定机票机器人	预定机票机器人
招聘面试机器人	招聘面试机器人
冰箱报修服务机器人	冰箱报修服务机器人
参考手册	参考手册
术语	术语
通配符匹配器	通配符匹配器

模糊匹配器	模糊匹配器
意图匹配器	意图匹配器
内置函数库	内置函数库
basics	basics
message	message
user	user
maestro	maestro
3rd-party	3rd-party
函数返回值	函数返回值
命令行界面 (CLI)	命令行界面 (CLI)
SDK	SDK
Chatopera 类	Chatopera 类
构造函数	构造函数
接口规范	接口规范
机器人管理	机器人管理
Chatbot 类	Chatbot 类
构造函数	构造函数
接口规范	接口规范
机器人基本管理	机器人基本管理
对话检索	对话检索
词典管理	词典管理
知识库管理	知识库管理
用户和对话历史	用户和对话历史
常见问题	常见问题
计费及保障	计费及保障
计费及发票	计费及发票
私有部署	私有部署
服务水平协议	服务水平协议
服务条款	服务条款
背景知识	背景知识
深度学习	深度学习
词典	词典
知识库	知识库
意图识别	意图识别
对话脚本	对话脚本
模糊匹配器	模糊匹配器
意图匹配器	意图匹配器

[多轮对话的检索](#) [多轮对话的检索](#)

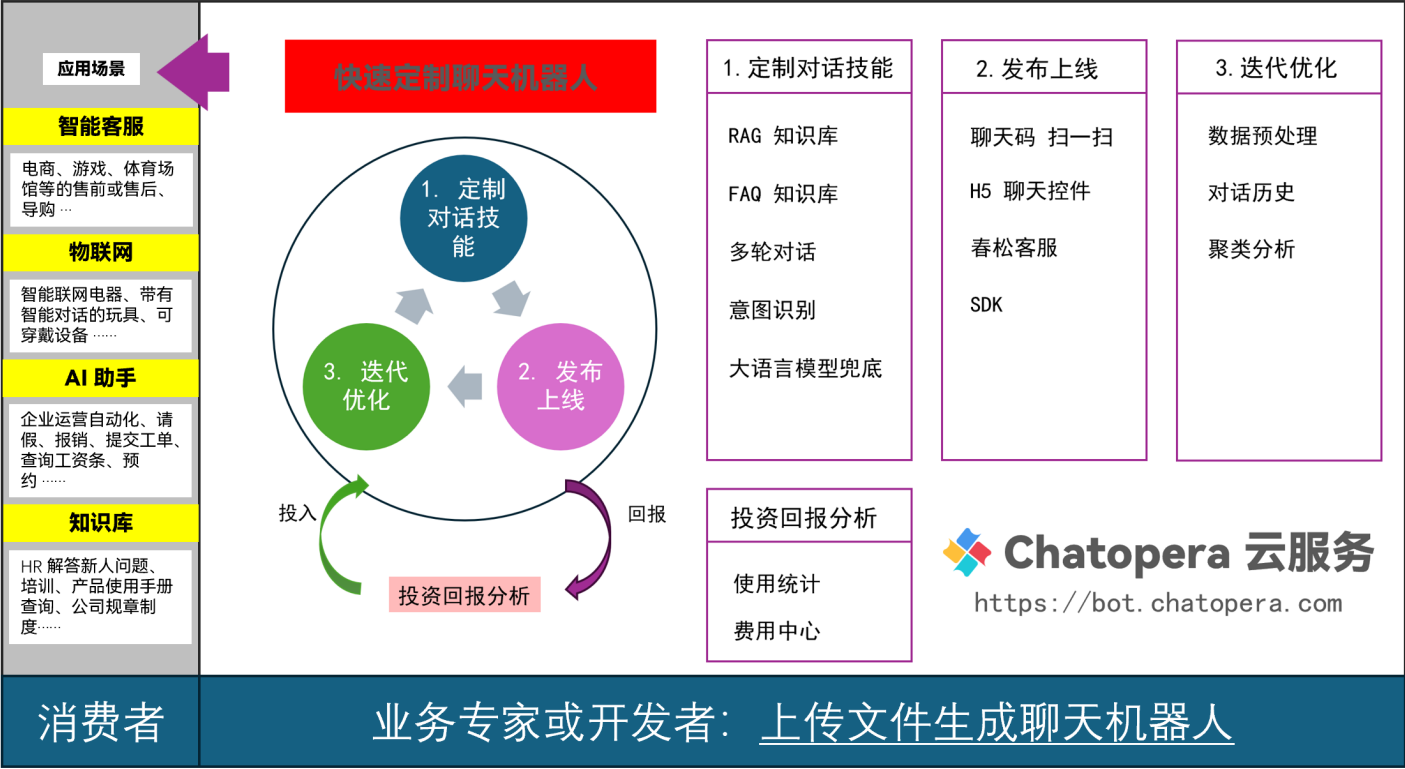
[视频教程](#) [视频教程](#)

[附录](#) [附录](#)

[词性标注](#) [词性标注](#)

应用场景

- 智能客服：电商、游戏、体育场馆等的售前或售后、导购 ...
- 物联网：智能联网电器、带有智能对话的玩具、自然语言交互控制 ...
- AI 助手：企业运营自动化、请假、报销、提交工单、查询工资条、预约 ...
- 企业知识库：HR 解答新人问题、培训、产品使用手册查询、公司规章制度 ...



Chatopera 融合了不同形式的实现聊天机器人的方法，支持定制多轮对话，从而满足企业各种需求，管理对话机器人对话内容。此外，Chatopera 机器人平台还有聊天历史管理、聚类分析和大语言模型等模块，这些服务都是紧紧围绕智能对话机器人的上线展开的。

如何使用

用聊天机器人做市场营销、客户服务和自动化流程？如何实现多轮对话？

Chatopera 机器人平台帮助企业 and 开发者使用低代码方式定制智能对话机器人。

Chatopera 云服务是Chatopera 机器人平台的公有云实例，Chatopera 云服务尤其对中小型企业非常友好，按需消费，一站式上线智能对话机器人。

Chatopera 云服务使用按量计费，提供多语言 SDK 集成；同时 Chatopera 可为企业私有部署 Chatopera 机器人平台。

Chatopera 机器人平台目前支持语言：

- 中文简体 (zh_CN)
- 中文繁体 (zh_TW)
- 英语 (en_US)
- 日语 (ja)
- 泰语 (th)

下一步

- [新手任务：使用入门教程一步步实现智能对话机器人](#)

入门教程帮助您了解和熟悉 Chatopera 云服务, 开始快速的定制聊天机器人。

体验示例机器人

Chatopera 云服务的主要应用场景是智能客服, 因此, 我们将引导您实现一个智能门锁的机器人客服示例, 在课程的最后, 您将得到类似下面的机器人服务及 H5 页面~



您也可以现在去体验试试呢! --> [Demo 机器人地址](#)

面向读者

Chatopera 云服务是一个低代码的开发、发布聊天机器人的平台, 因此, 本文档面向的读者, 需要具备以下的技能:

- 有使用过命令行终端、CLI 工具的经验, 比如 Windows PowerShell, Linux Bash, macOS 终端工具
- 有编程经验, 尤其是使用过 JavaScript 或 Node.js 的经验
- 使用过集成开发环境, 比如 VSCode, IntelliJ IDEA, Sublime Text 等

获得帮助

当您在新手任务中, 遇到任何问题, 欢迎[联系 Chatopera 工程师](#)获得帮助支持~

前提准备

CDE 和 CLI 是 Chatopera 发布的开发者工具, 它们依赖 Node.js 环境和 VSCode, 使用下面的连接获得详细的安装过程。

- 安装 [Chatopera CDE, CLI 工具](#)

分成 5 个小任务

<1/5> 创建机器人

<2/5> 回答 80% 的客服问题，使用 RAG 模块

<3/5> 快速修复不好的回答，使用 FAQ 模块

<4/5> 批量导入 FAQ 知识库，使用 CDE 和 CLI

<5/5> 优化打招呼 and 热门问题列表

<1/5> 创建机器人

<< 上一步：入门教程首页 | 阅读本节内容大约需要 15 mins

当您在新手任务中，遇到任何问题，欢迎[联系 Chatopera 工程师](#)获得帮助支持~

登录

登录 Chatopera 云服务管理控制台：<https://bot.chatopera.com/dashboard>。

创建机器人

在工具条菜单中，点击【创建机器人】。填入表单：

表单项	值
机器人名称	小 Q 机器人

其它信息默认，点击【确认】。



创建成功后，会自动跳转到小 Q 机器人的导航面板。



导航面板

- 处理数据
对话历史
聚类分析
CLI 导入导出
- 定制对话技能
RAG 知识库
FAQ 知识库
意图识别
多轮对话
测试对话
词典管理
设置
- 发布渠道
H5 聊天控件
春松客服
SDK
飞书
- 投资回报:
使用统计
费用中心

使用统计

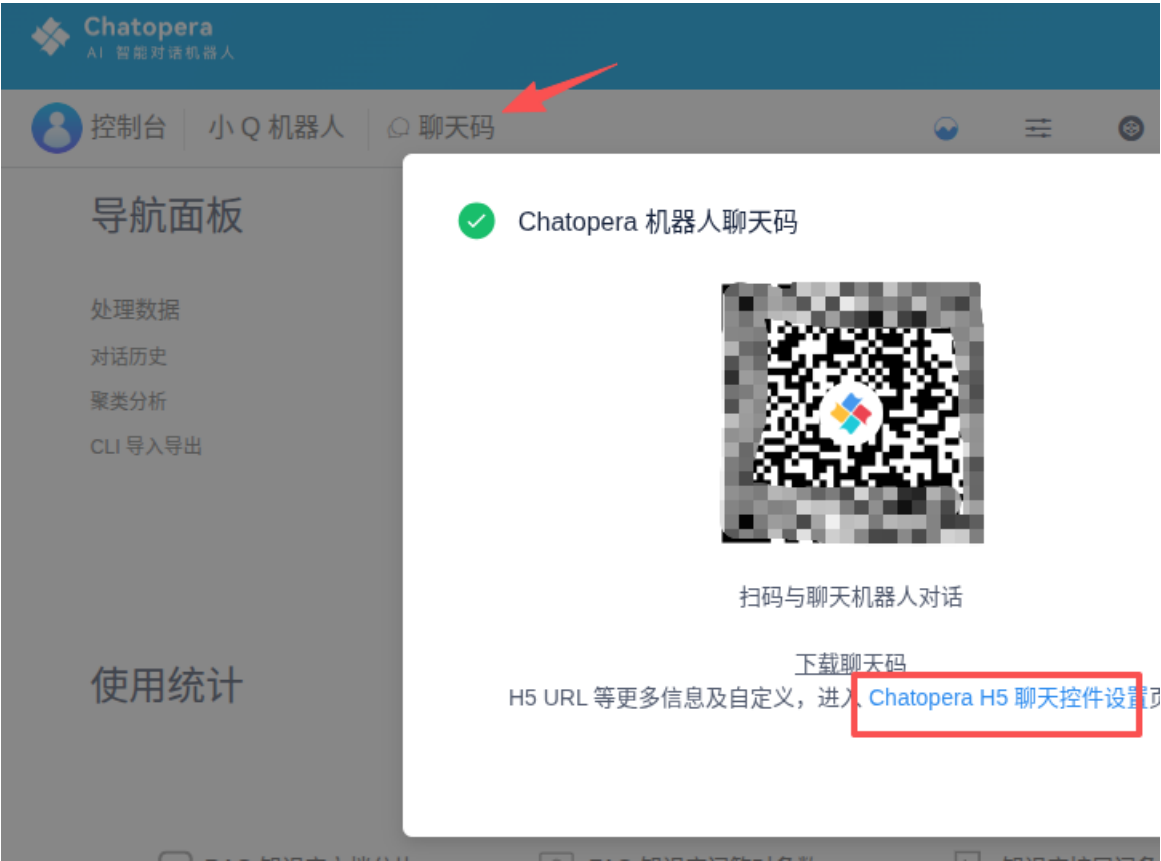
机器人对话内容概况



这就是一个立即可用的聊天机器人，在之后的任务中，我们将了解如何丰富小 Q 机器人的对话能力。但是现在您已经可以发布这个机器人给您的访客了^publish-visitor！

发布聊天机器人

步骤如下，在菜单中，点击【聊天码】，弹出类似下图的界面。



该二维码，就是您可以公开发送给访客的，用来和聊天机器人对话的二维码，Chatopera 称之为聊天码。

如果您需要 URL 的方式接入，可以点击上图中的【Chatopera H5 聊天控件设置】获得。

Chatopera
AI 智能对话机器人

控制台 | 小 Q 机器人 | 聊天码

H5 聊天控件

回到 系统集成

基本设置 | 邀请设置 | 客服信息

接入代码

请将以下代码添加到你的网站 HTML源代码中，放在<head></head>标签之间.

`<script defer="true" src="https://h5.chatopera.com/im/0cJJZw.html"></script>`

文字链代码

请将以下代码添加到你的网站链接代码上，自由定义链接的内容形式，或将以下 URL 生成

`https://h5.chatopera.com/im/text/0cJJZw.html`

访客对话入口样式

1、访客入口样式

SDK 集成

您也可以将机器人集成到其它软件和服务中，参考机器人集成页面。

Chatopera
AI 智能对话机器人

控制台 | 小 Q 机器人 | 聊天码

Java

使用说明
示例程序
接口文档

php

使用说明
示例程序
接口文档

golang

使用说明
示例程序
接口文档

nodejs

使用说明
示例程序
接口文档

Python

使用说明
示例程序
接口文档

系统集成

恭喜您完成本节任务！接下来，我们继续改进小 Q 机器人：回答 80% 的客服问题，使用 RAG 模块。



<< 上一步: [入门教程首页](#) | >> 下一步: <2/5> [回答 80% 的客服问题, 使用 RAG 模块](#)

<2/5> 回答 80% 的客服问题，使用 RAG 模块

<< 上一步：<1/5> 创建机器人 | 阅读本节内容大约需要 15 mins

当您在新手任务中，遇到任何问题，欢迎[联系 Chatopera 工程师](#)获得帮助支持~

回复访客的咨询问题

我们设定 小 Q 机器人是一款智能门锁的 AI 客服，为了模拟对话，我们从海尔官网下载了[HFD-T15 智能门锁](#)的使用说明书^[haier-explain](#)。

下载说明书：[haier-HFD-T15-manual-V1.pdf](#)。

那么，我们就可以利用这份说明书，升级聊天机器人了。并且借助这个方法，80% 的访客咨询都可以得到解答。

上传说明书到 RAG 知识库

首先，我们打开这个说明书 PDF 文件，做一点浏览，感受一下文件的内容：有文字、图片、链接等。可以说，这是一份"普通的文件"，没有特殊的格式。

下一步，我们从浏览器中，打开小 Q 机器人的【RAG 模块】控制台：点击【上传文件】。



然后，选择我们刚刚下载的 [haier-HFD-T15-manual-V1.pdf](#)，然后，可以看到文件上传成功。此处，支持的文件格式很多，比如常用的 Word, PPT, 图片，Markdown，HTML 等。

Chatopera

AI 智能对话机器人

控制台

小 Q 机器人

聊天码

RAG 知识库

上传文件

提示词模板

测试对话

文件名	文件大小	最近更新日期	操作
haier-HFD-T15-manual-V1.pdf	4 MB	2026-08-25 14:49	

接着，点击右侧的【生成 RAG 知识库】。

Chatopera

AI 智能对话机器人

控制台

小 Q 机器人

聊天码

用海量数据进行训练，使用数十亿个参数为回答问题、翻译语言和完成句子等任务生成原始输出。

支持文件后缀：jpg, png, pdf, ppt, pptx, doc, docx, html, mhtml, md, txt 和 rst；单个文件不能大于 20 MB。

上传文件

提示词模板

测试对话

文件名	文件大小	最近更新日期	操作
haier-HFD-T15-manual-V1.pdf	4 MB	2026-08-25 14:49	

待分析

文件变更后，进入待分析状态。

分析中 ...

分析后台执行，可关闭；请耐心等待~

分析完成

分析完成有短信提醒；请及时查看。

生成 RAG 知识库

分析的过程，大约 3 分钟，在状态栏，会获得实时的进度提示。

分析任务

待分析

文件变更后，进入待分析状态。

分析中 ...

分析后台执行，可关闭浏览器页面；分析任务可能持续数小时或数天，请耐心等待~

分析完成

分析完成有短信提醒；及时下载结果，下次分析时本次结果被覆盖。

生成 RAG 知识库

分析日志

2026/08/25 14:59

上传分析结果文件

2026/08/25 14:58

正在分析文件：haier-HFD-T15-manual-V1.pdf ...

分析完成后的状态，如下图。

分析任务

待分析

文件变更后，进入待分析状态。

分析中 ...

分析后台执行，可关闭浏览器页面；分析任务可能持续数小时或数天，请耐心等待~

分析完成

分析完成有短信提醒；及时下载结果，下次分析时本次结果被覆盖。

生成 RAG 知识库

测试对话

待分析完成，机器人能回答访客的咨询问题了吗？我们可以借助测试对话模块，试试！

Chatopera

AI 智能对话机器人

控制台

小 Q 机器人

聊天码

RAG 知识库

• 上传文件实现聊天机器人：执行分析，自动生成 RAG 知识

• 检索增强生成（RAG）是指对大语言模型输出进行优化，用海量数据进行训练，使用数十亿个参数为回答问题、翻

• 支持文件后缀：jpg, png, pdf, ppt, pptx, doc, docx,

上传文件

提示词模板

测试对话

文件名	文件大小
haier-HFD-T15-manual-V1.pdf	4 MB

点击【测试对话】，这样就进入了如下界面。

控制台

小 Q 机器人

聊天码

测试对话

多轮对话

RAG 知识库

FAQ 知识库

意图识别

对话

商或售后服务部门。建议出示保修证明和购机发票。不属于免费维修范围的情况包括人为损坏或超出保修期。

ME

使用时的注意事项

机器人

首次使用门锁后请尽快更改管理密码并妥善保管机械钥匙和IC卡。使用时应保持手指干净，避免门锁长时间暴晒、淋雨或接触腐蚀性化学物质。更换电池时需

调试信息

检索接口返回值，SDK 调用得到的 JSON

```
{
  "think": "",
  "content": "首次使用门锁后请尽快机械钥匙和IC卡。使用时应保持手指干燥、淋雨或接触腐蚀性化学物质。更换干电池，且新旧电池不能混装。门锁操作，用户不应私自拆装。",
  "is_logic_fallback": false,
  "slices": [
    {
      "slice_id": "6a8d3d338260",
      "distance": 0.65923976898,
      "certainty": 0.6703801155,
      "text": " ♦️在门外输入门锁管理
```

这个界面，是给访客使用前，我们调试用途的工具。发送了几个问题，感觉很不错～

智能门锁 AI 客服 - 小 Q 机器人，上线了～

接下来，我们看看上线后的优化。

<< 上一步：<1/5> 创建机器人 | >> 下一步：<3/5> 快速修复不好的回答，使用 FAQ 模块

17/290

https://bot.chatopera.com

<< 上一步：<2/5> 回答 80% 的客服问题，使用 RAG 模块 | 阅读本节内容大约需要 20 mins

当您在新手任务中，遇到任何问题，欢迎[联系 Chatopera 工程师](#)获得帮助支持~

查看访客会话

上线只是开始：机器人的对话能力的优化，是个持续迭代的过程。从上线时刻，我们就要关注机器人和访客的对话情况。

进入小 Q 机器人的【对话历史】模块。

Chatopera
AI 智能对话机器人

控制台 | 小 Q 机器人 | 聊天码

对话历史

查看对话历史，支持按照对话用户浏览对话过程、标注知识库。

【下载兜底回复对话】支持选择时间范围，导出兜底回复对话：包括大语言模型或兜底回复回复的对话。

下载

测试对话

聚类分析

时间	来访者ID	消息	机器人回复	模块	操作
2026-08-27 19:26	ac36c2e5b7a9aa5f6f3e947a875a7027	木门可以安装吗	安装前请检查您的门是实木门还是钢门或中空木门，实木门采用无天地勾版本锁体。	FAQ 知识库	更新 FAQ 问答对 查看完整对话
2026-08-27 19:25	ac36c2e5b7a9aa5f6f3e947a875a7027	预约上门安装电话	各区咨询电话 - 北京地区：158 0000xxx 上海地区：1580000xx x 全国其它地区：400-xxxxxx 支持7天无理由退货，但需商品	FAQ 知识库	更新 FAQ 问答对 查看完整对话

对话历史

在这里，您可以浏览到每个访客和机器人对话的过程，并且能看到机器人回答的技能模块：RAG 知识库，FAQ 知识库，多轮对话等。

过滤不好的回答

我们需要重点关注的是那些机器人没有能回答好的问题，尤其是模块属于【兜底回复】的问答。

比如，我们发现机器人对于问题【预约师傅安装的电话】的回答是兜底回复。

控制台 | 小 Q 机器人 | 聊天码

2026-08-27 19:07	70e49df0a543241140c00e48af03dbac	支持7天无理由退货吗	支持7天无理由退货，需商品完好、不影响二次销售。定制、生鲜等特殊商品除外，具体以商品页面说明为准。	大语言模型 (LLM)	创建 FAQ 问答对 查看完整对话
2026-08-27 19:07	70e49df0a543241140c00e48af03dbac	联系师傅上门安装的电话	各区咨询电话 - 北京地区：158 0000xxx 上海地区：1580000xx x 全国其它地区：400-xxxxxx	FAQ 知识库	更新 FAQ 问答对 查看完整对话
2026-08-27 19:07	70e49df0a543241140c00e48af03dbac	预约师傅安装的电话	我不明白您的意思。	兜底回复	创建 FAQ 问答对 查看完整对话
2026-08-27 19:06	70e49df0a543241140c00e48af03dbac	支持7天无理由退货吗	支持7天无理由退货，但需商品完好、不影响二次销售。部分特殊商品（如定制、生鲜等）除外，具体以商品页面说明为准。	大语言模型 (LLM)	创建 FAQ 问答对 查看完整对话
2026-08-27 19:06	70e49df0a543241140c00e48af03dbac	联系师傅上门安装的电话	各区咨询电话 - 北京地区：158 0000xxx 上海地区：1580000xx x 全国其它地区：400-xxxxxx	FAQ 知识库	更新 FAQ 问答对 查看完整对话

添加问答对解决

对于没有回答好的问题，我们可以通过 FAQ 知识库解决，具体做法如下。

首先，进入 FAQ 知识库模块。



点击【新建问答对】

表单项	值
问题	联系师傅上门安装的电话
答1	各区咨询电话 - 北京地区：1580000xxx， 上海地区：1580000xxx

点击【确定】。



然后，如果访客再次的问这个问题，就能得到准确的解答了～



通过 FAQ 知识库的形式，我们可以修复很多不好的回答。

上线后的优化非常重要，优化的过程，是一个长期的，比如三个月。也可能因为一些问题有时效性，我们的服务有更新，导致问答对需要长期的维护。

我们的 FAQ 知识库，也会越来越多，如何批量的导入和维护呢？下一节，您将学习高级用法：[批量导入 FAQ 知识库，使用 CDE 和 CLI](#)。

<< 上一步: [<2/5> 回答 80% 的客服问题，使用 RAG 模块](#) | >> 下一步: [<4/5> 批量导入 FAQ 知识库，使用 CDE 和 CLI](#)

<4/5> 批量导入 FAQ 知识库，使用 CDE 和 CLI

<< 上一步：<3/5> 设置热门问题 | 阅读本节内容大约需要 30 mins

当您在新手任务中，遇到任何问题，欢迎[联系 Chatopera 工程师](#)获得帮助支持~

有很多问答对

RAG 知识库和 FAQ 知识库，是小 Q 机器人的两个主要的知识来源，因为 FAQ 知识库有快速、准确的特点，会形成越来越多的问答对。我们在机器人之间迁移，或者对机器人进行备份，都需要执行批量操作。

加入我们通过程序，能通过访问数据库、爬取网站等形式，构建很多的 FAQ 问答对，是对优化小 Q 的对话能力最好的方式。

那么，如何快速批量的导入 FAQ 问答对呢？这需要我们了解两个部分：

- 1. faq 知识库问答对的数据结构 - Chatopera 云服务支持 faq.json 文件；
- 2. 如何自动化的导入 - 通过在命令行终端执行 Chatopera CLI 指令。

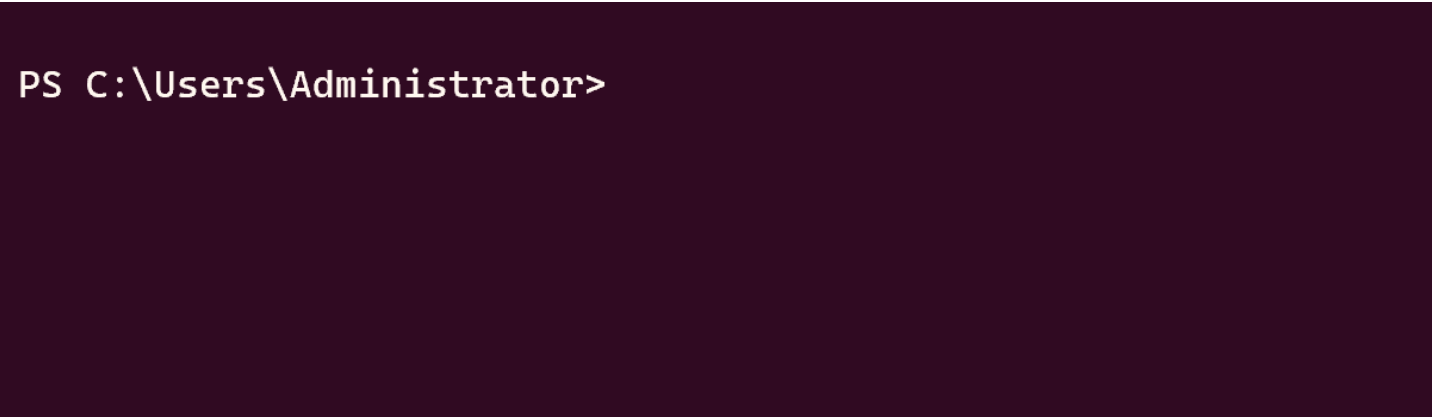
下面，我们通过实际操作来介绍。

创建机器人项目根目录

启动一个命令行终端：

操作系统	工具
Linux	Bash Shell
macOS	Terminal 应用
Windows PowerShell	PowerShell（本文档示例不支持 CMD Prompt）

比如，PowerShell 的界面如下：



打开终端后，执行下面的命令：

```
mkdir -p ~/chatopera/bot_q
```

这时，我们就创建了一个新的文件夹：`用户的HOME目录/chatopera/bot_q`，后面简称为【机器人项目根目录】。

继续使用命令行，进入机器人项目根目录。

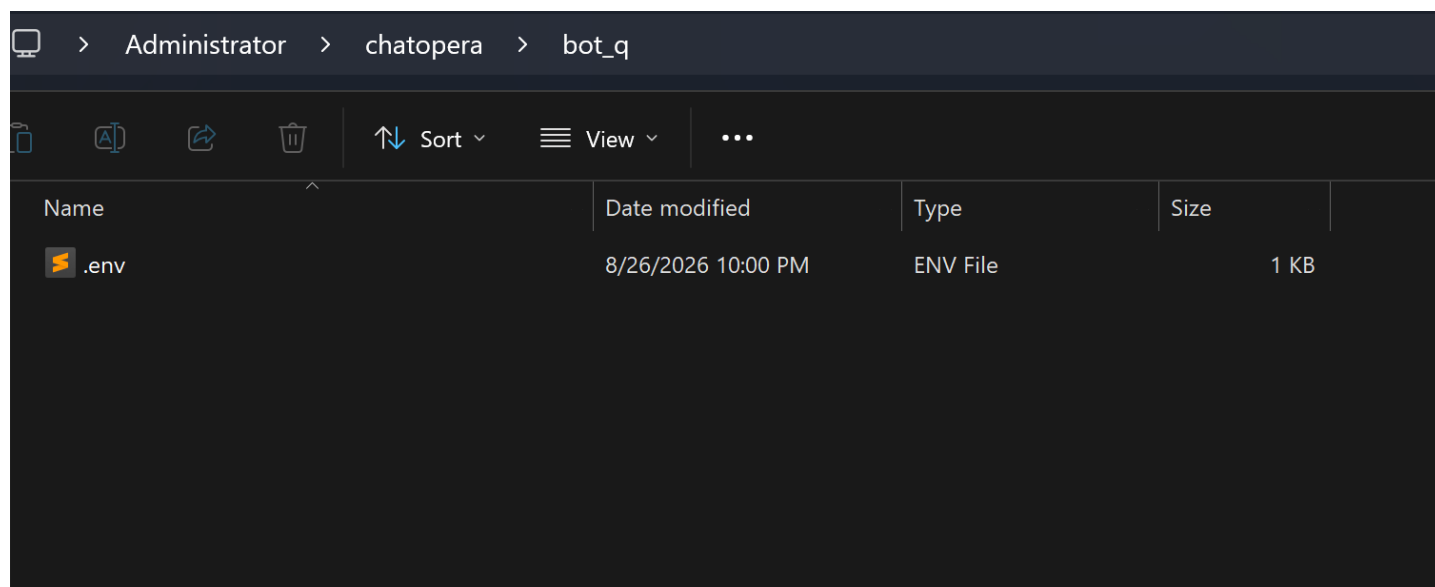
```
cd ~/chatopera/bot_q
```

接下来，使用 Chatopera CLI 创建机器人配置文件 `.env`，过程如下：

- 在命令行终端，进入机器人项目根目录
- 运行命令 `bot env`

```
PS C:\Users\Administrator\chatopera\bot_q> bot env
>> load env file C:\Users\Administrator\.env into process.env ...
.env file[C:\Users\Administrator\chatopera\bot_q\.env] is generated.
wto-guides/integration/cli-install-config.html
PS C:\Users\Administrator\chatopera\bot_q>
```

成功运行后，可以得到这样的文件和目录结构：



编辑 .env 文件

使用文本编辑工具打开 `.env` 文件，比如记事本、VSCode 或 Notepad 等。

.env

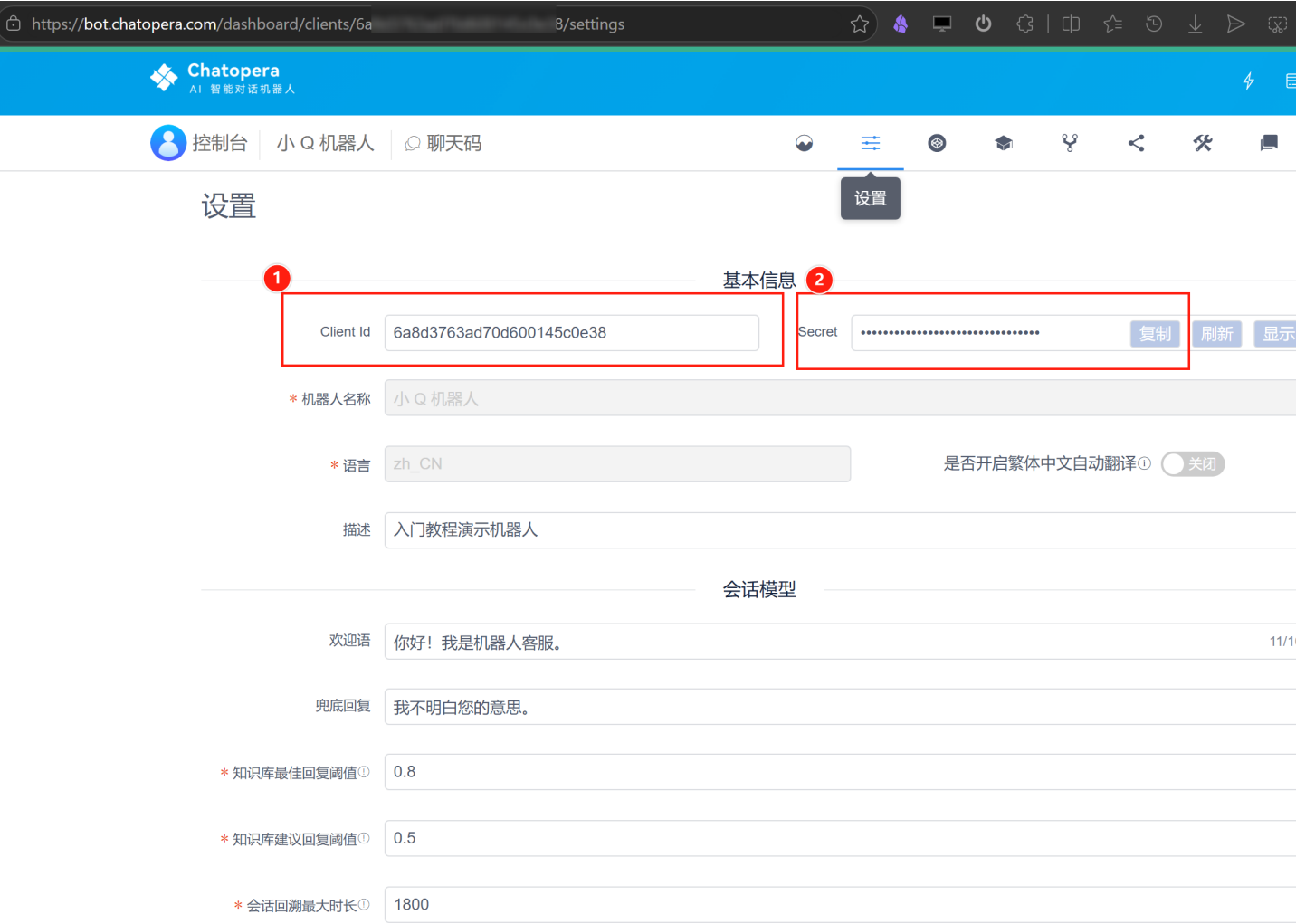
×

+

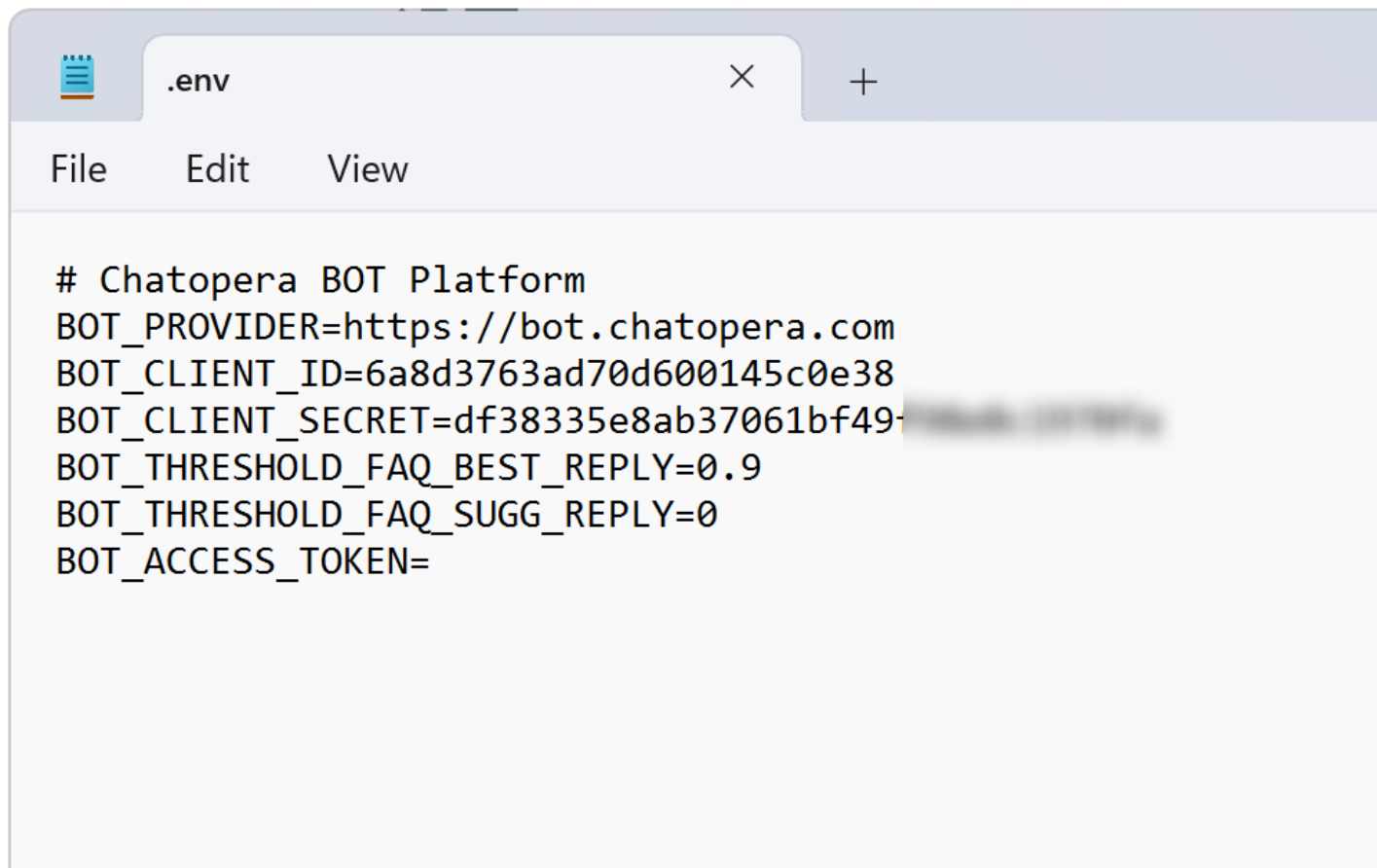
FileEditView

```
# Chatopera BOT Platform
BOT_PROVIDER=https://bot.chatopera.com
BOT_CLIENT_ID=
BOT_CLIENT_SECRET=
BOT_THRESHOLD_FAQ_BEST_REPLY=0.9
BOT_THRESHOLD_FAQ_SUGG_REPLY=0
BOT_ACCESS_TOKEN=
```

这时，我们要修改 BOT_CLIENT_ID 和 BOT_CLIENT_SECRET。进入机器人管理控制台，在设置中，找到 clientId 和 secret.



将 clientId 和 secret 粘贴到 .env 文件中，类似下图：

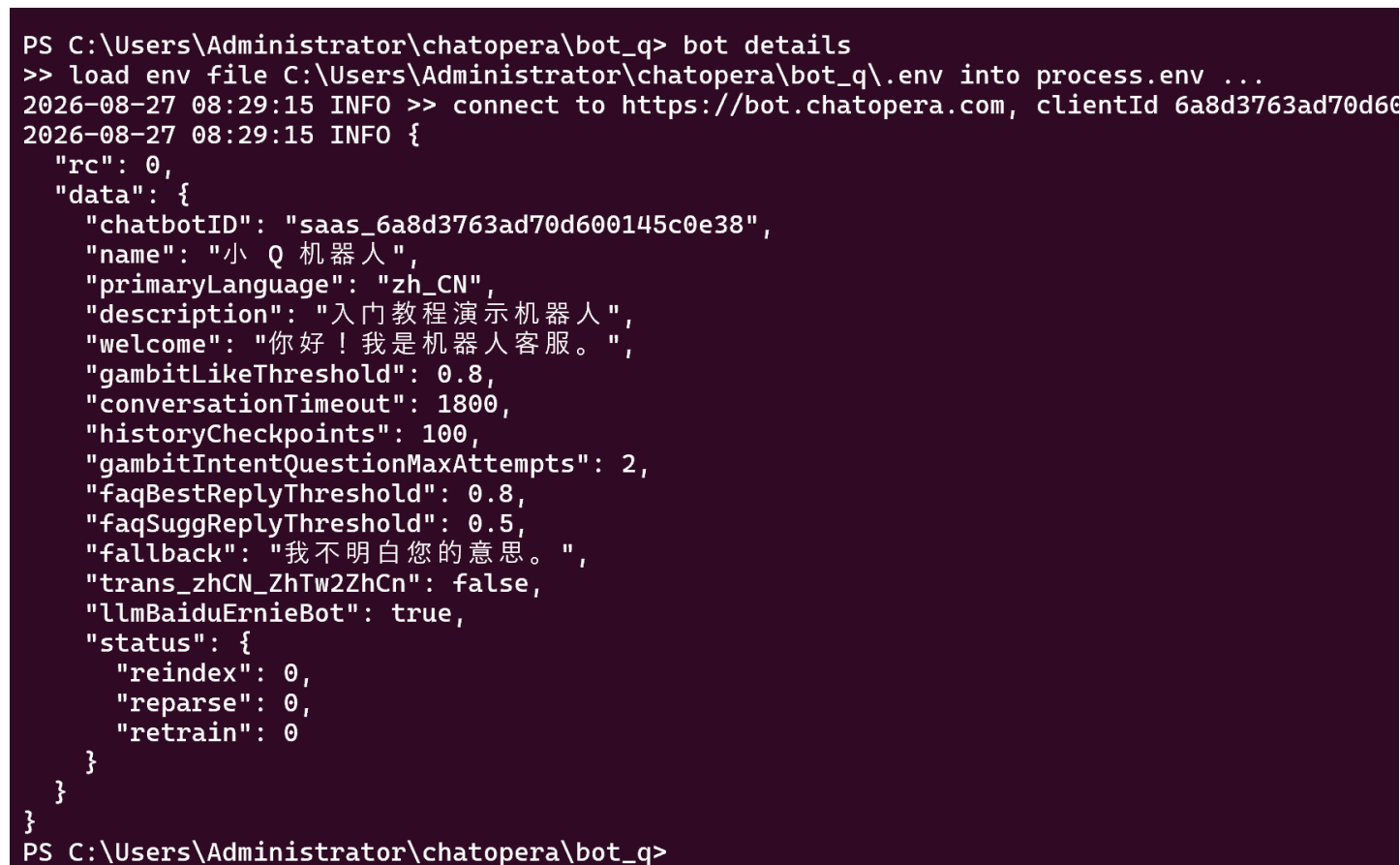


```

# Chatopera BOT Platform
BOT_PROVIDER=https://bot.chatopera.com
BOT_CLIENT_ID=6a8d3763ad70d600145c0e38
BOT_CLIENT_SECRET=df38335e8ab37061bf49-
BOT_THRESHOLD_FAQ_BEST_REPLY=0.9
BOT_THRESHOLD_FAQ_SUGG_REPLY=0
BOT_ACCESS_TOKEN=

```

保存，并关闭 .env 文件，在命令行终端，测试配置成功，执行 `bot details`，此命令要在机器人项目根目录下运行。



```

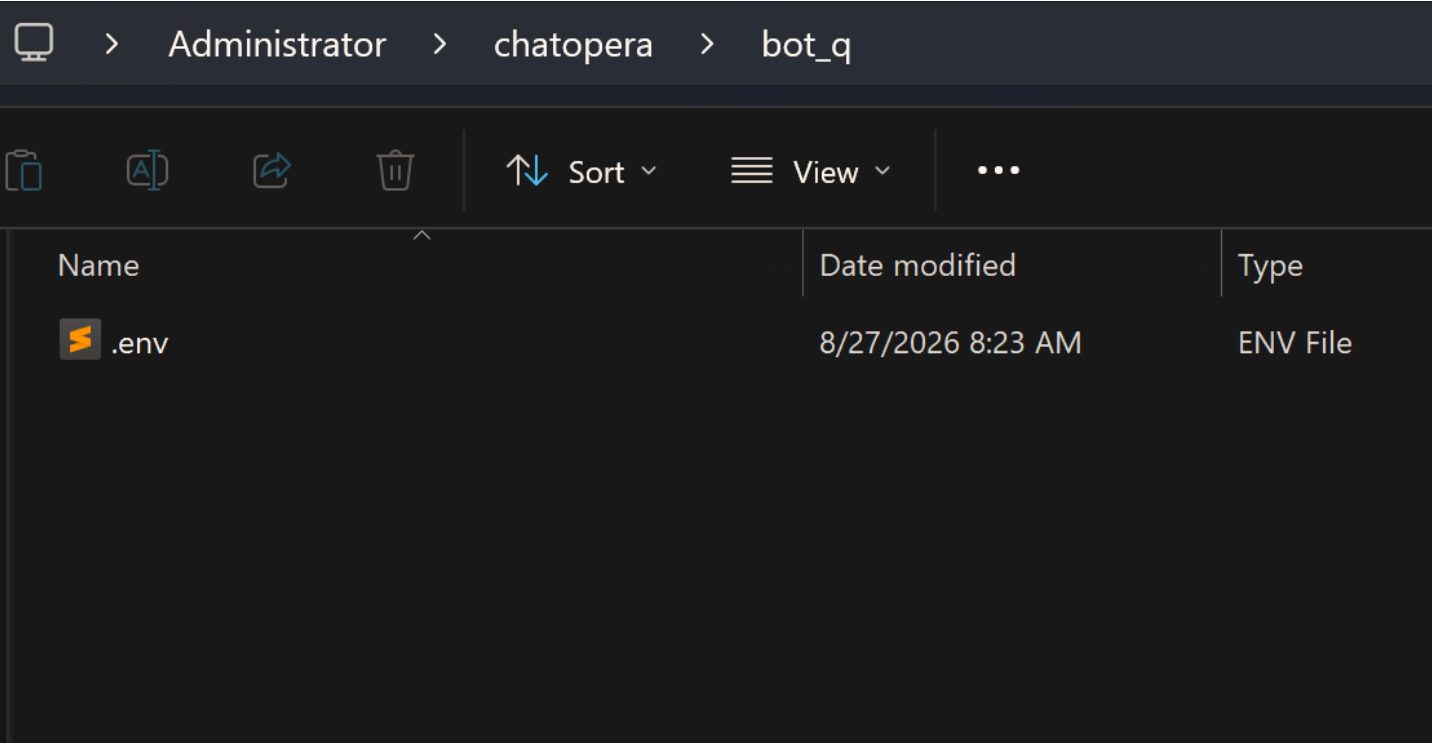
PS C:\Users\Administrator\chatopera\bot_q> bot details
>> load env file C:\Users\Administrator\chatopera\bot_q\.env into process.env ...
2026-08-27 08:29:15 INFO >> connect to https://bot.chatopera.com, clientId 6a8d3763ad70d600145c0e38
2026-08-27 08:29:15 INFO {
  "rc": 0,
  "data": {
    "chatbotID": "saas_6a8d3763ad70d600145c0e38",
    "name": "小 Q 机器人",
    "primaryLanguage": "zh_CN",
    "description": "入门教程演示机器人",
    "welcome": "你好！我是机器人客服。",
    "gambitLikeThreshold": 0.8,
    "conversationTimeout": 1800,
    "historyCheckpoints": 100,
    "gambitIntentQuestionMaxAttempts": 2,
    "faqBestReplyThreshold": 0.8,
    "faqSuggReplyThreshold": 0.5,
    "fallback": "我不明白您的意思。",
    "trans_zhCN_ZhTw2ZhCn": false,
    "llmBaiduErnieBot": true,
    "status": {
      "reindex": 0,
      "reparse": 0,
      "retrain": 0
    }
  }
}
PS C:\Users\Administrator\chatopera\bot_q>

```

这就意味着，我们的 Chatopera CLI 工具和 Chatopera 云服务中的机器人连接上了。

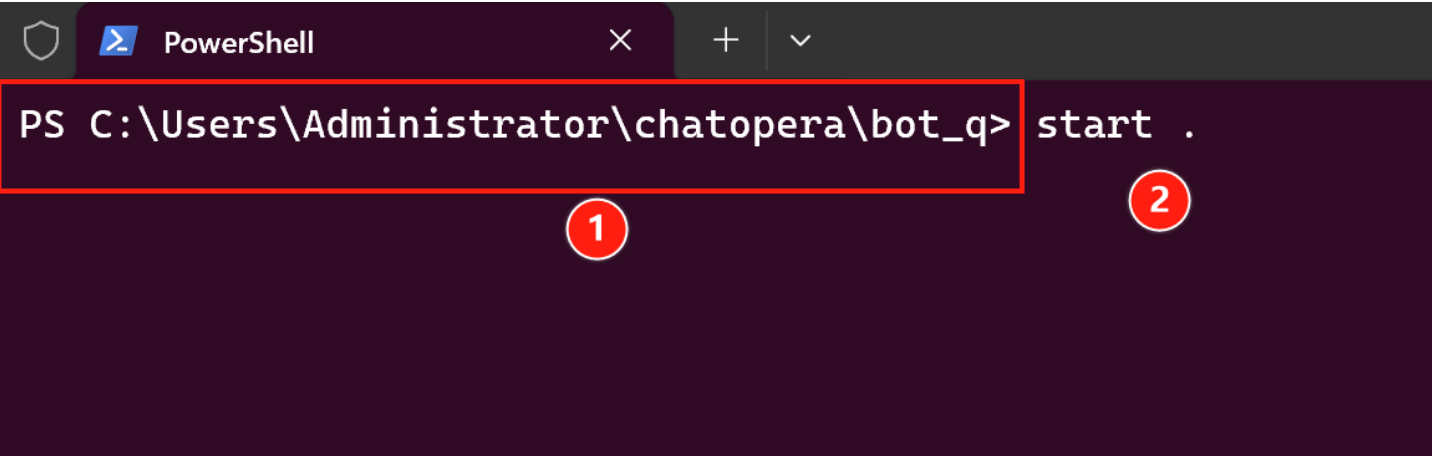
存储 FAQ JSON 文件

首先，在文件管理器中，打开【机器人项目根目录】。

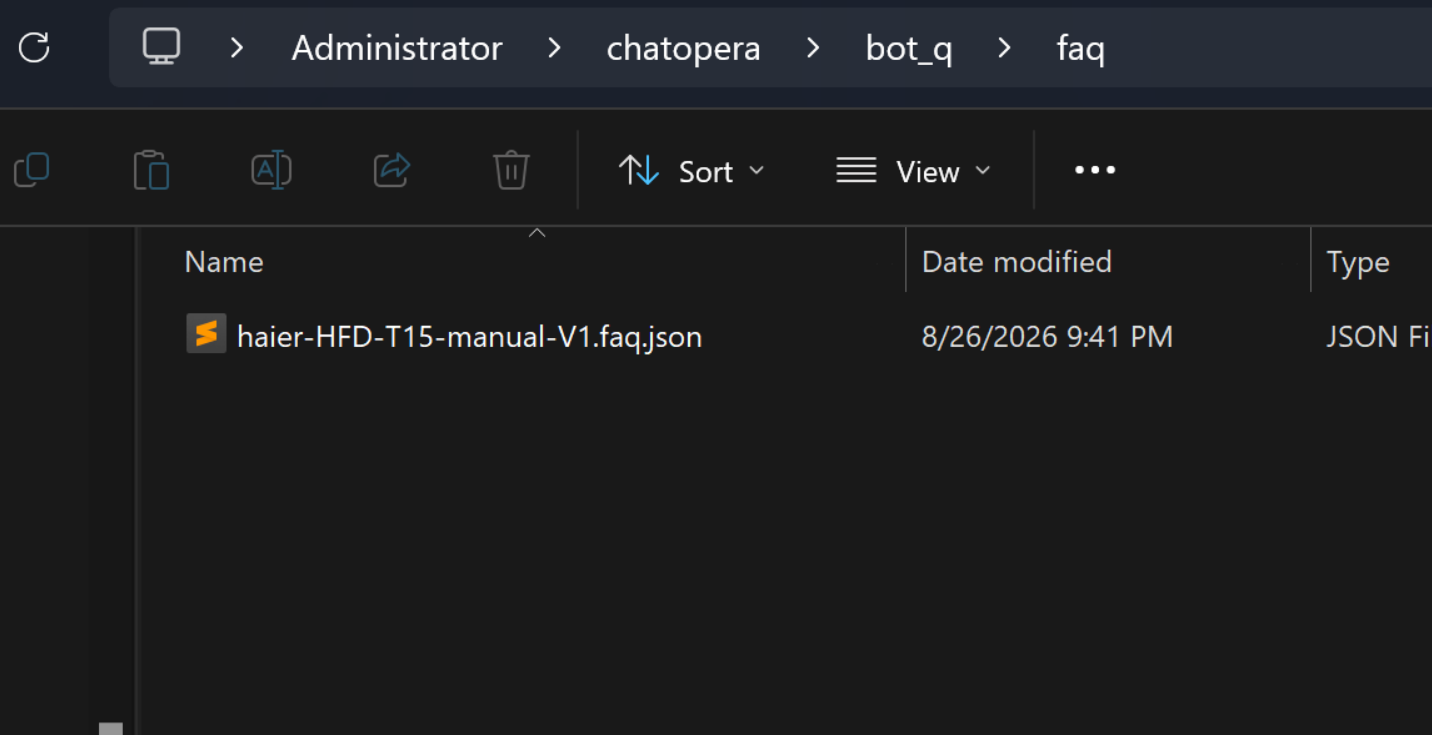


这一步，也可以通过命令完成：

操作系统	命令
Linux/macOS	<code>open .</code>
Windows PowerShell	<code>start .</code>



然后，在这个目录下，创建文件夹：faq。接着下载保存文件[haier-HFD-T15-manual-V1.faqjson](#) [^20260827082742]，然后将 haier-HFD-T15-manual-V1.faqjson 保存到 faq 文件夹。



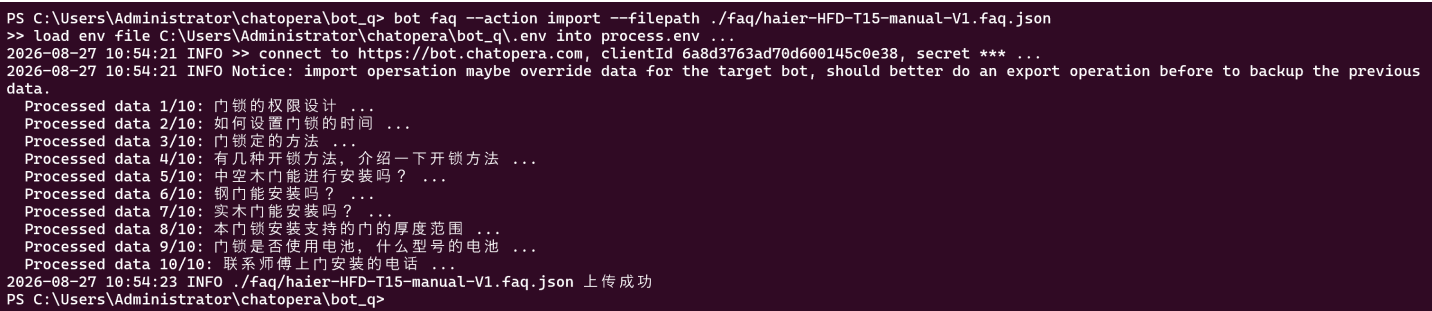
[^20260827082742]: haier-HFD-T15-manual-V1.faq.json 文件下载地址，<https://docs.chatopera.com/assets/files/haier-HFD-T15-manual-V1.faq.json>

执行导入命令

在命令行中，运行下面的命令：

```
cd ~/chatopera/bot_q
bot faq --action import --filepath ./faq/haier-HFD-T15-manual-V1.faq.json
```

- 1. 进入【机器人项目根目录】
- 2. 执行导入



FAQ 批量导入完成。

回顾一下 JSON 数据结构

FAQ JSON 文件的格式，参考 haier-HFD-T15-manual-V1.faq.json 这个文件。

```
haier-HFD-T15-manual-V1.faq.json x
1  [
2    {
3      "docId": "AaA9TzUYmhT6m4dyRvZ0",
4      "categories": [],
5      "enabled": true,
6      "post": "门锁的权限设计",
7      "replies": [
8        {
9          "rtype": "plain",
10         "content": "本产品拥有分级权限管理功能，系统将用户类型分成：管理员、普通用户
11       }
12     ],
13     "similarQuestions": []
14   },
15   {
16     "docId": "AaA9TlsgmhT6m4dyRvZz",
17     "categories": [],
18     "enabled": true,
```

1 条问答对

- FAQ JSON 文件的最外层是 `[]`
- 每条问答对是一个 JSON Object `{}`
- docId, 选填，为每个问答对分配的唯一标识，如果目前机器人有了这个 docId，那么将会进行更新，而不是新建问答对
- categories, 选填，为问答对指定分类，支持层级，比如 `["动物", "哺乳动物"]`
- post, 必填，问题
- replies, 必填，回答
- similarQuestions, 选填，问题的更多说法，比如 `["高级会员的权益", "VIP 的权益"]`

祝贺您入门了 CLI 的使用

Chatopera CLI 还有很多的用途，通过[命令行界面（CLI）](#)可以进一步的学习！

现在，我们就进入入门教程的最后一个任务：<5/5> [优化打招呼和热门问题列表](#)。

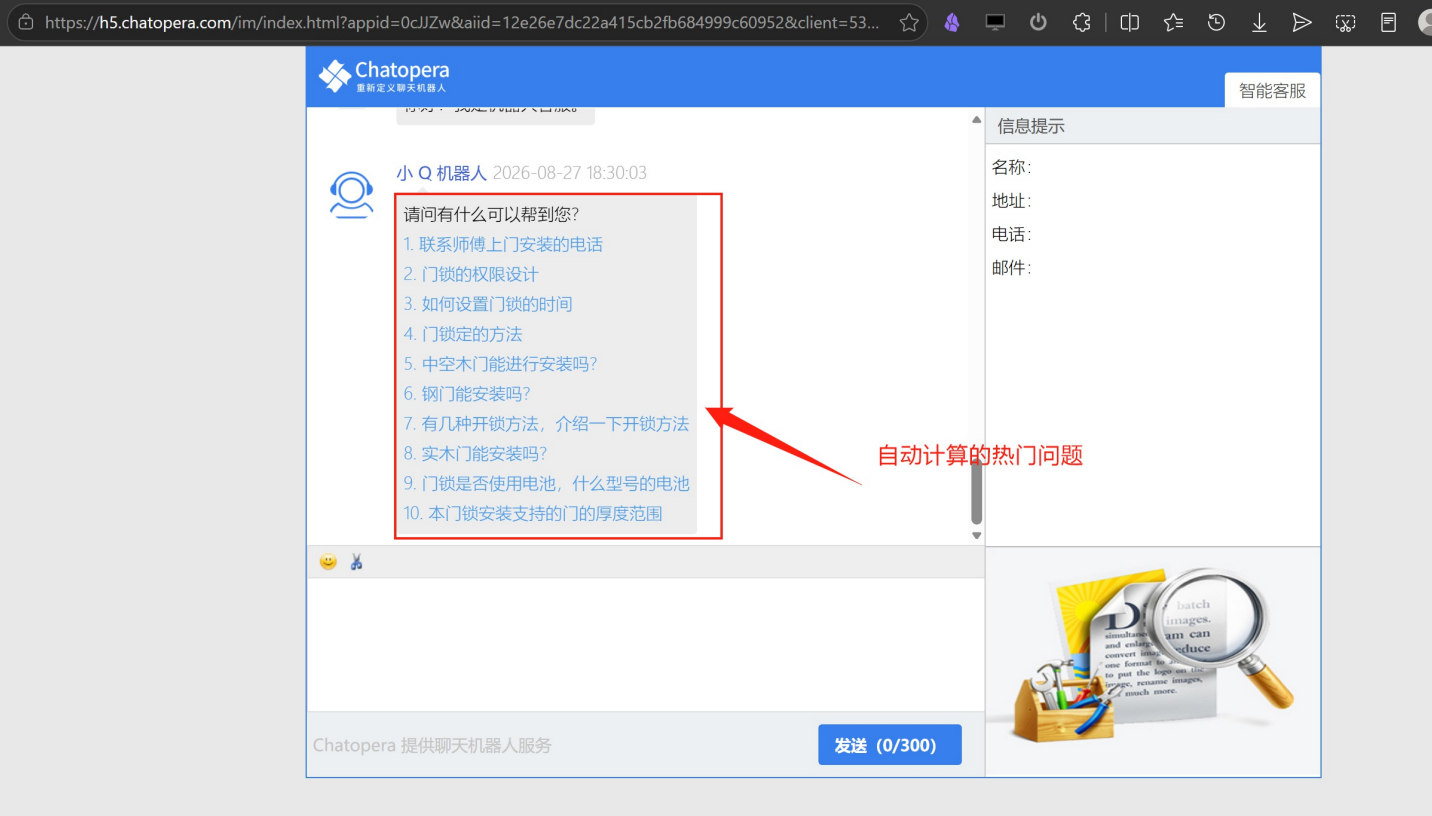
<< 上一步：<3/5> [设置热门问题](#) | >> 下一步：<5/5> [优化打招呼和热门问题列表](#)

<5/5> 优化打招呼 and 热门问题列表

<< 上一步: <4/5> 批量导入 FAQ 知识库, 使用 CDE 和 CLI | 阅读本节内容大约需要 15 min

当您在新手任务中, 遇到任何问题, 欢迎[联系 Chatopera 工程师](#)获得帮助支持~

当我们批量导入了知识库问答对, 再次以访客的身份打开机器人 H5 页面, 我们看到的就是下面的界面。



我们发现, 机器人的打招呼的信息变了, 这是因为, Chatopera 会根据 FAQ 知识库自动识别最常出现的问题, 随着越来越多的访客和机器人对话, 这个热门问题列表会根据统计信息动态的变化, 默认显示最热门的 10 条问答对。

这个默认的打招呼的逻辑, 可能不符合您的应用场景, 比如我们想将问题列表固定, 那么如何做呢?

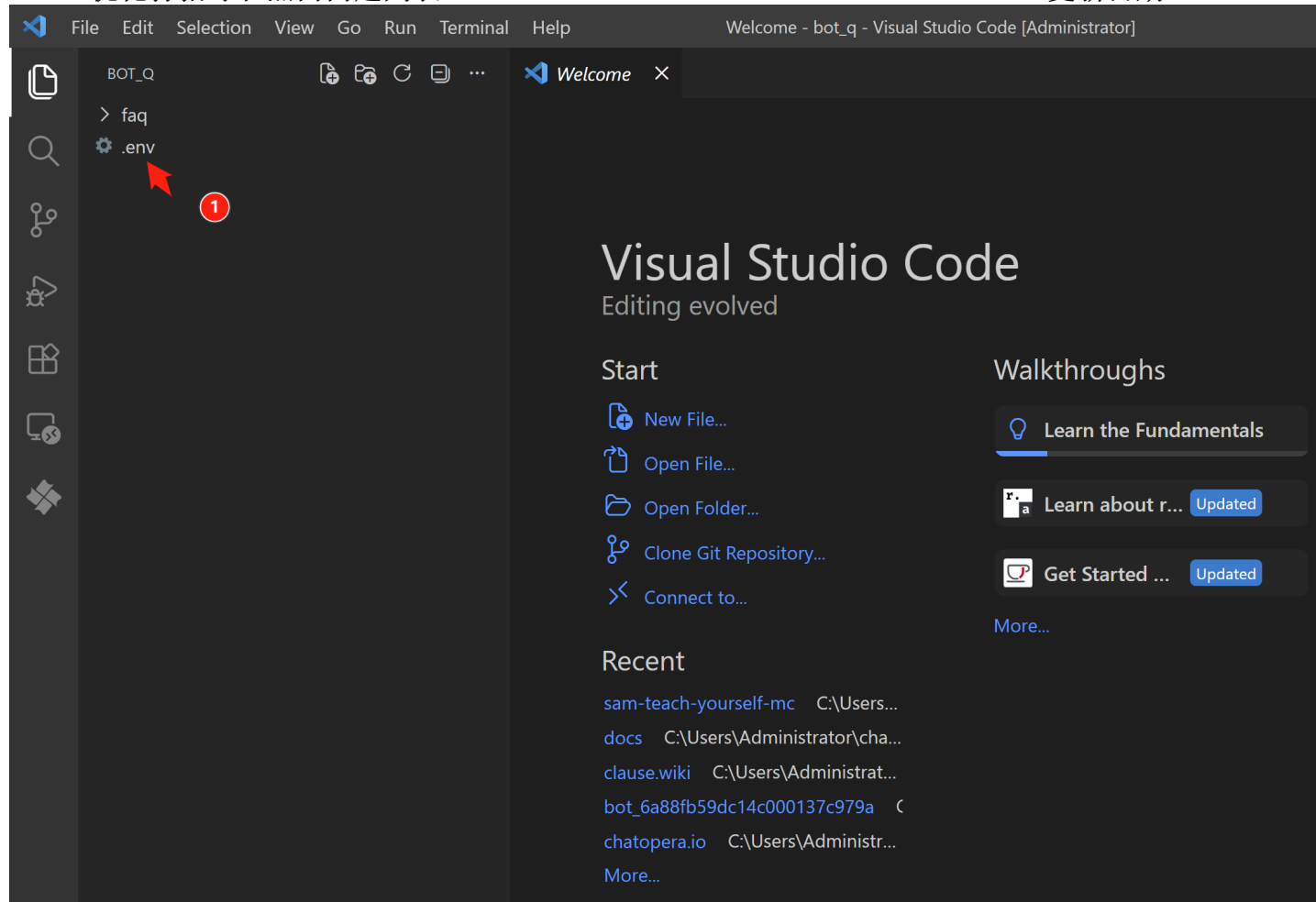
这个过程分为三个步骤:

- 1. 拉取多轮对话
- 2. 查看 greetings 话题脚本
- 3. 修改函数

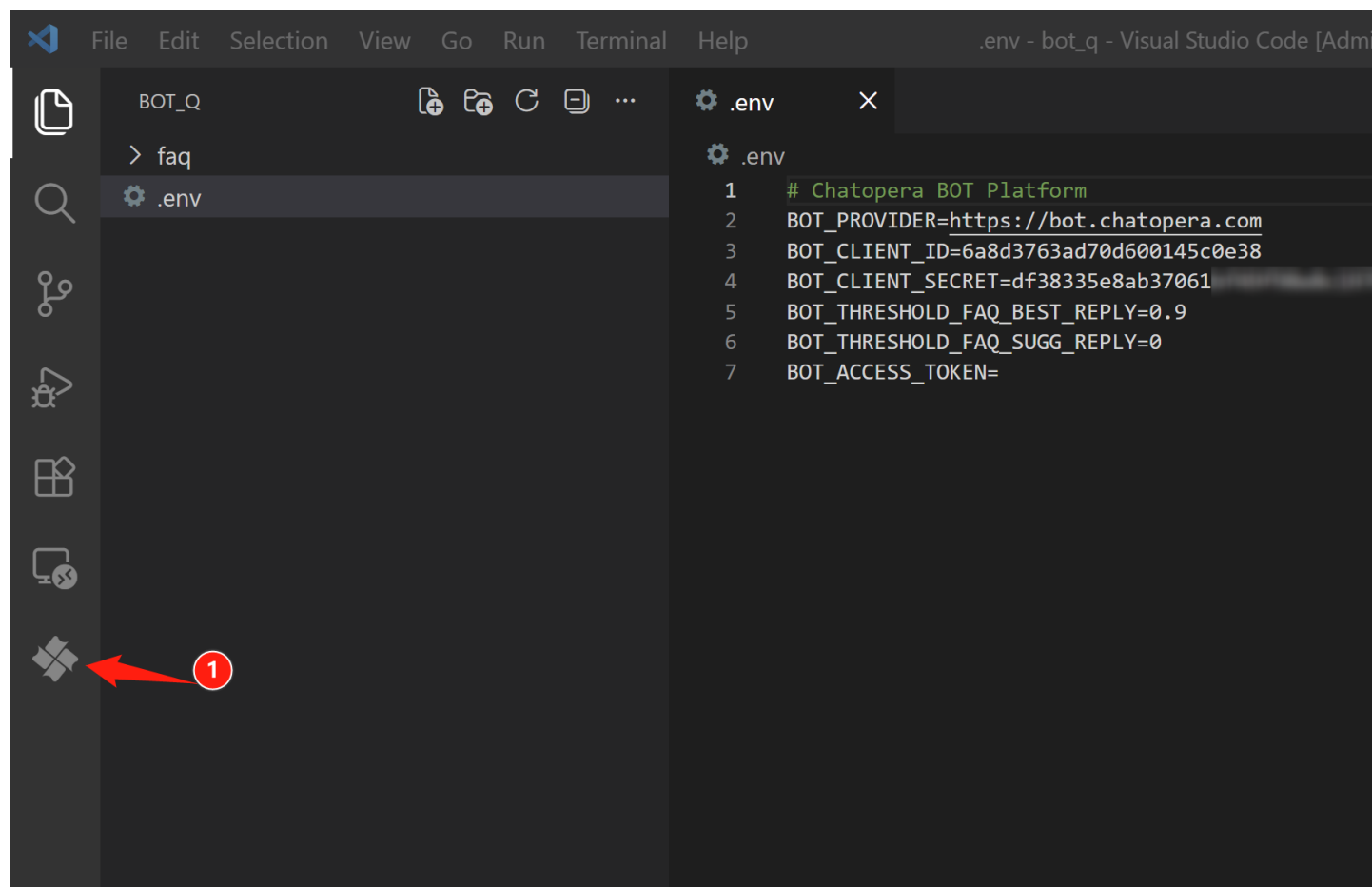
拉取多轮对话

在 VSCode[^install-cde]中打开【机器人根目录】, 确认在根目录下有 .env 文件, CDE 将使用该文件和机器人服务连接。

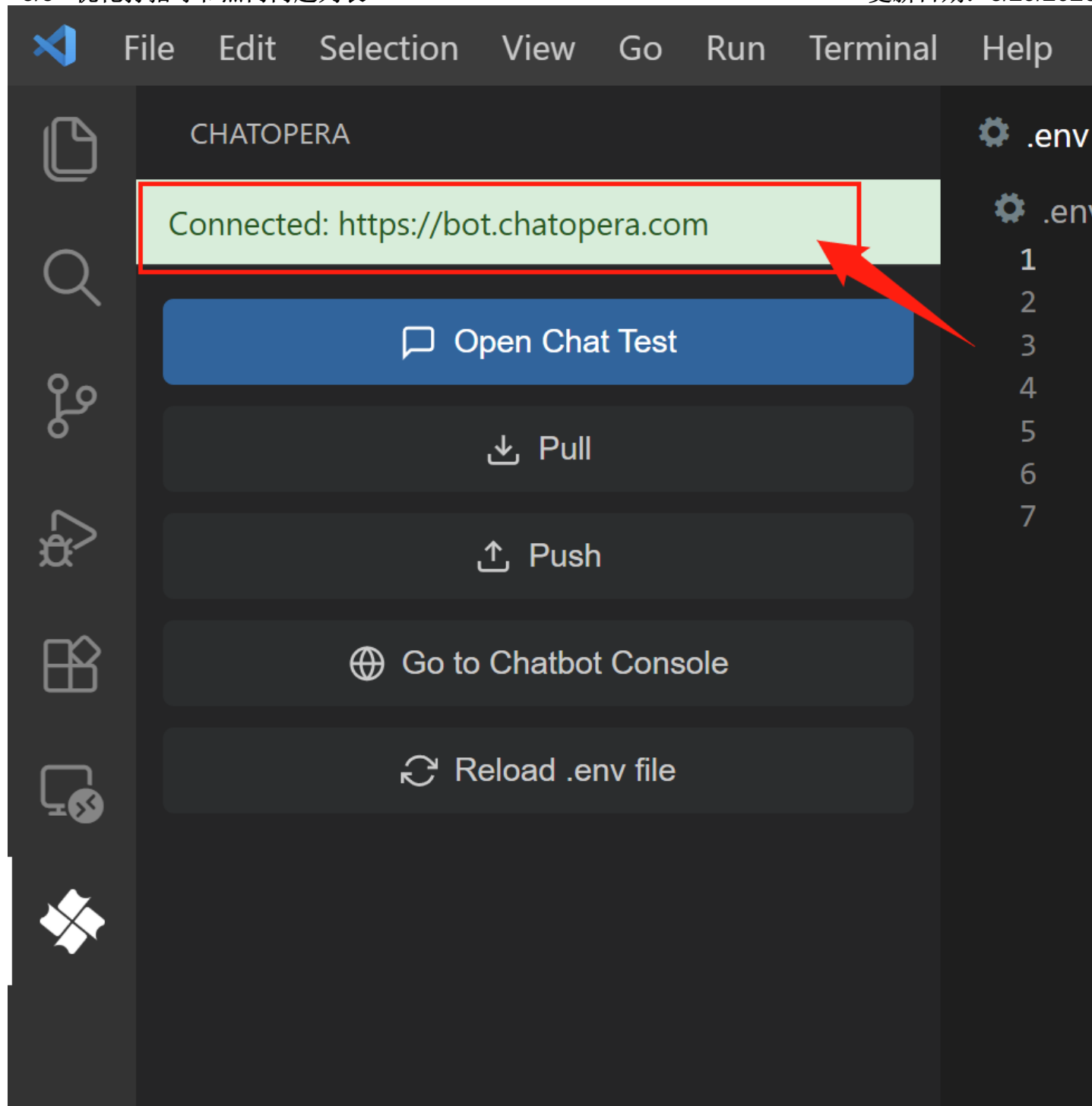
[^install-cde]: 依赖我们已经安装了 [CDE](#)



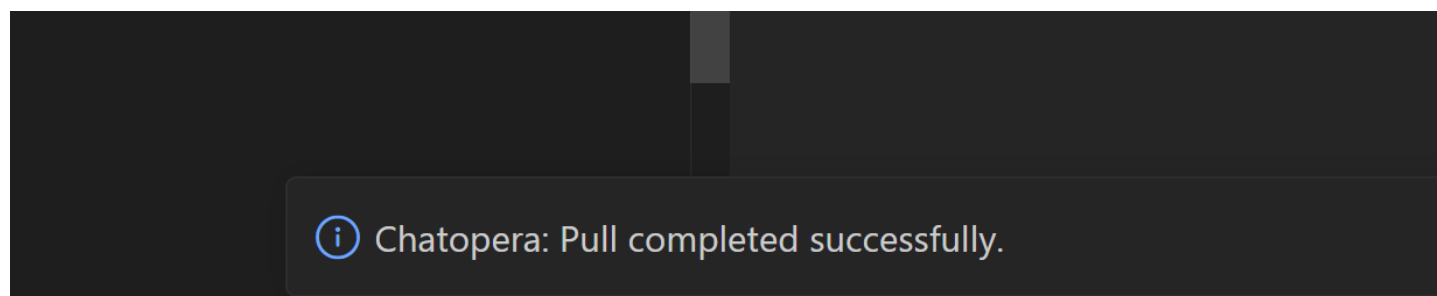
然后，点击左侧菜单中的 Chatopera 图标。



您将看到类似如下的界面。

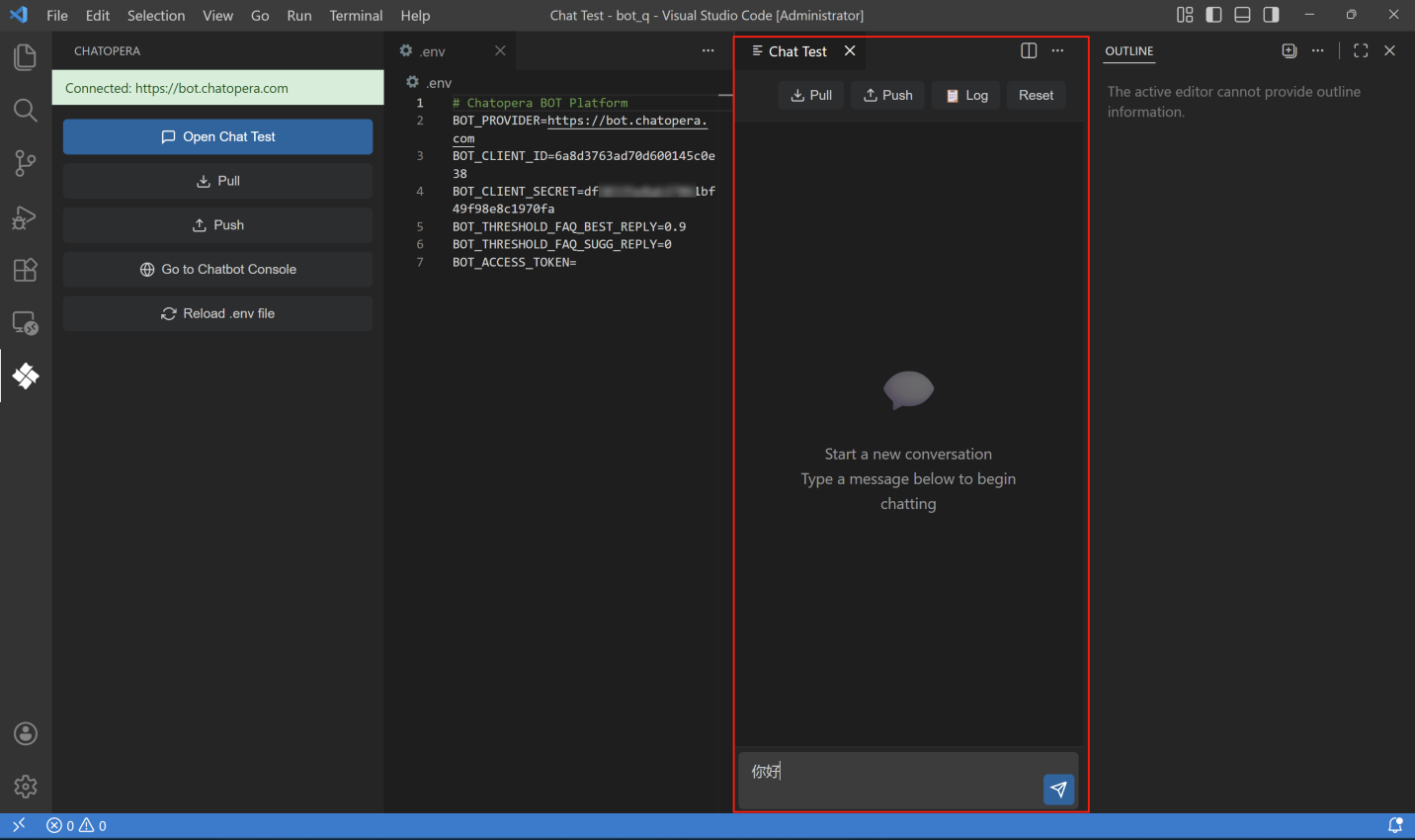


提示连接到了远程的机器人服务，接着点击：pull。这时，CDE 下载了机器人的多轮对话内容，执行完成后，可以看到右下角的通知。

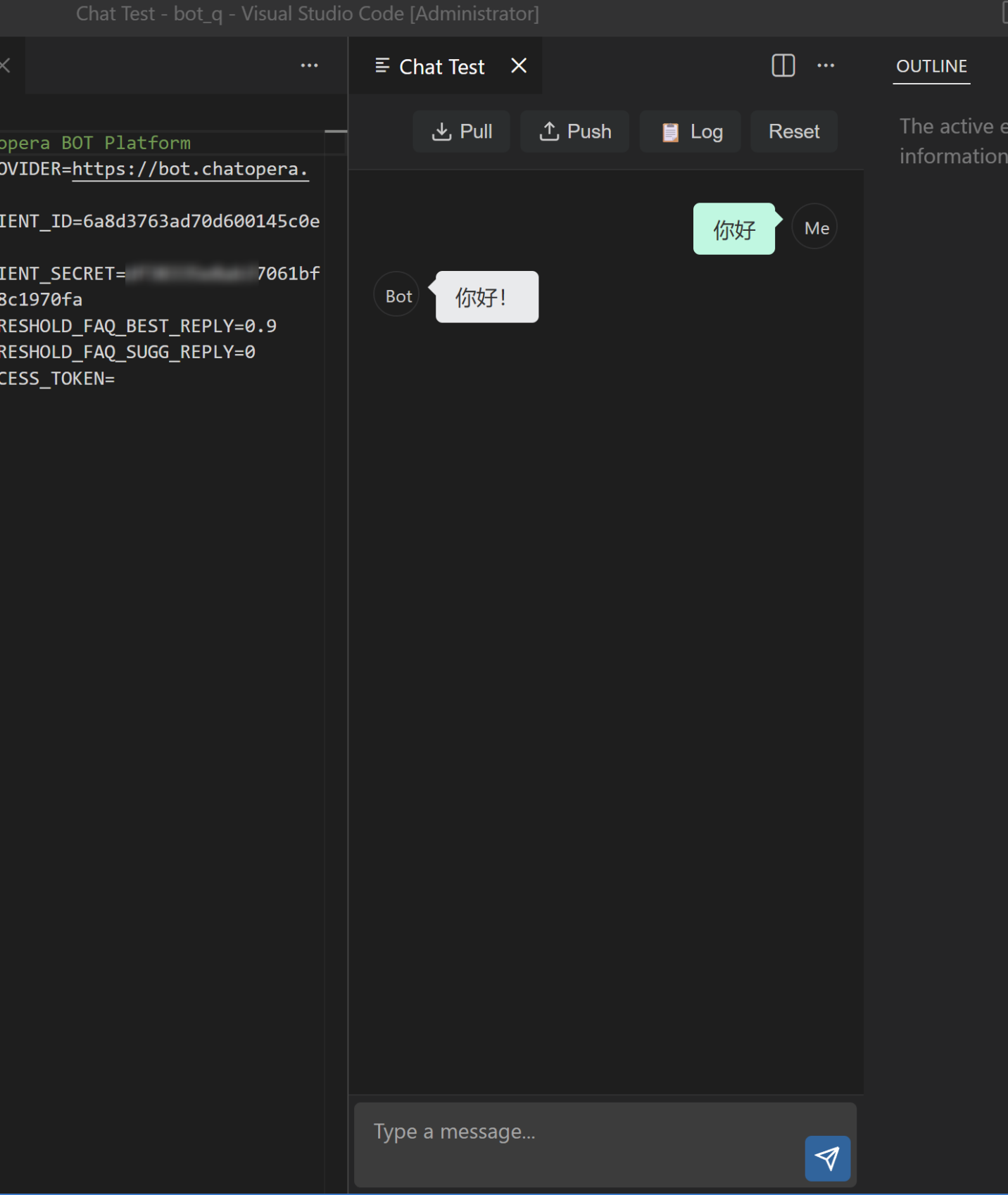


测试对话

测试对话，点击【Open Chat Test】，这时我们会看到出现了对话面板。

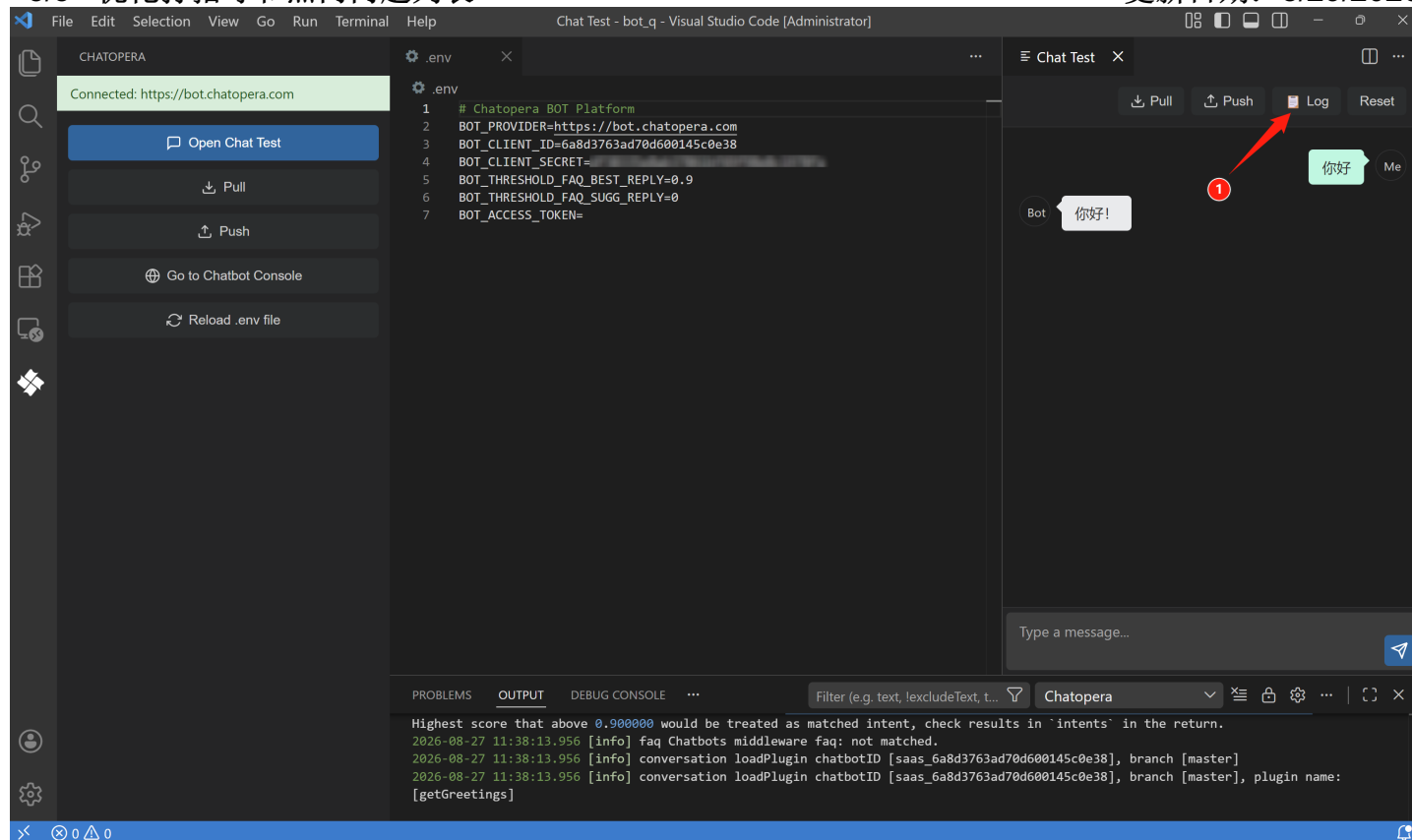


通过发送消息，获得机器人的回复。



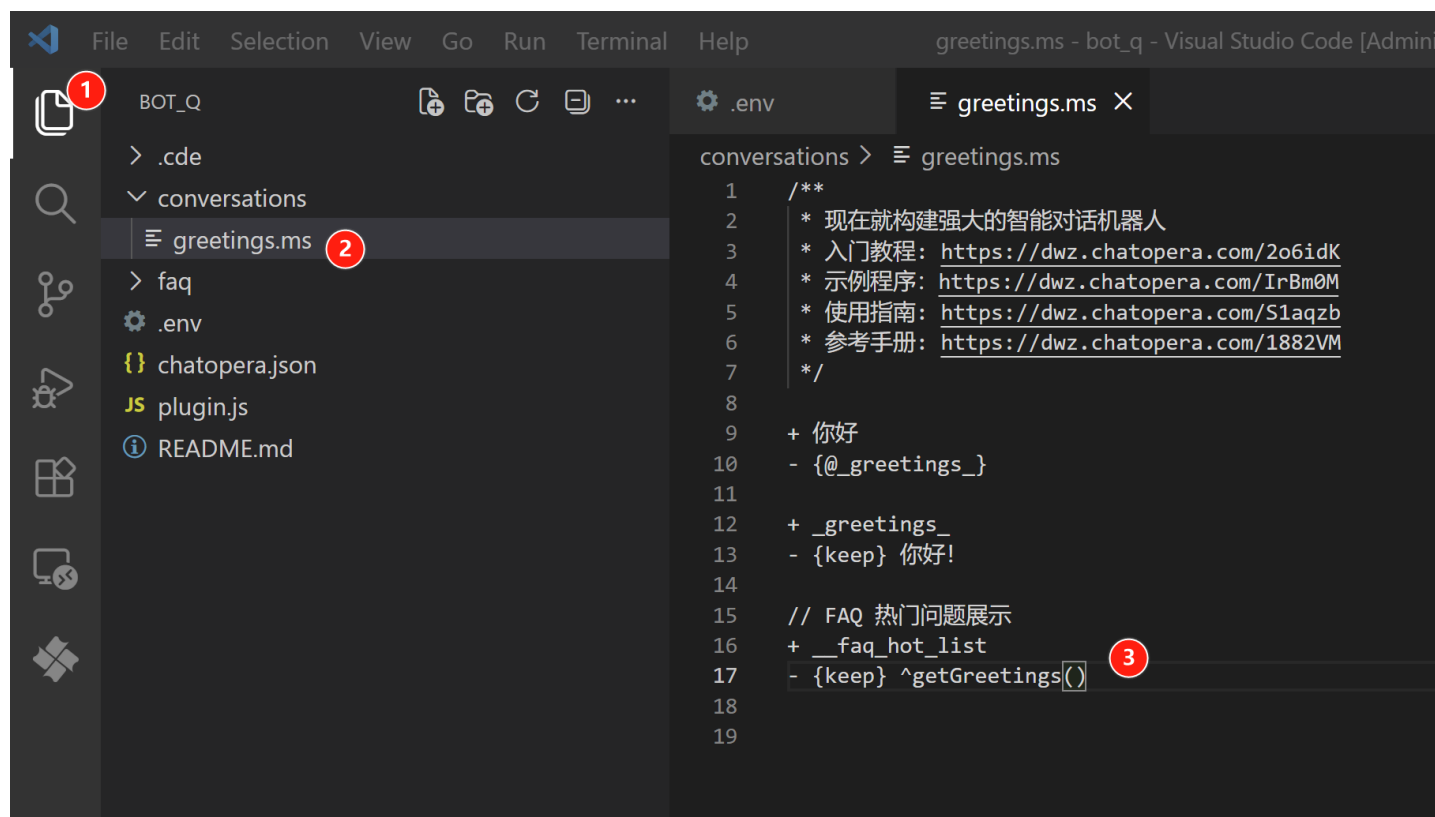
查看调试日志

当我们想要了解机器人的问答的逻辑的时候，可以点击【Log】，获得调试日志。



查看 greetings 话题脚本

这一步，我们在 VSCode 左侧菜单中，打开文件夹浏览，找到 `conversations/greetings.ms` 文件，并打开，如下图。

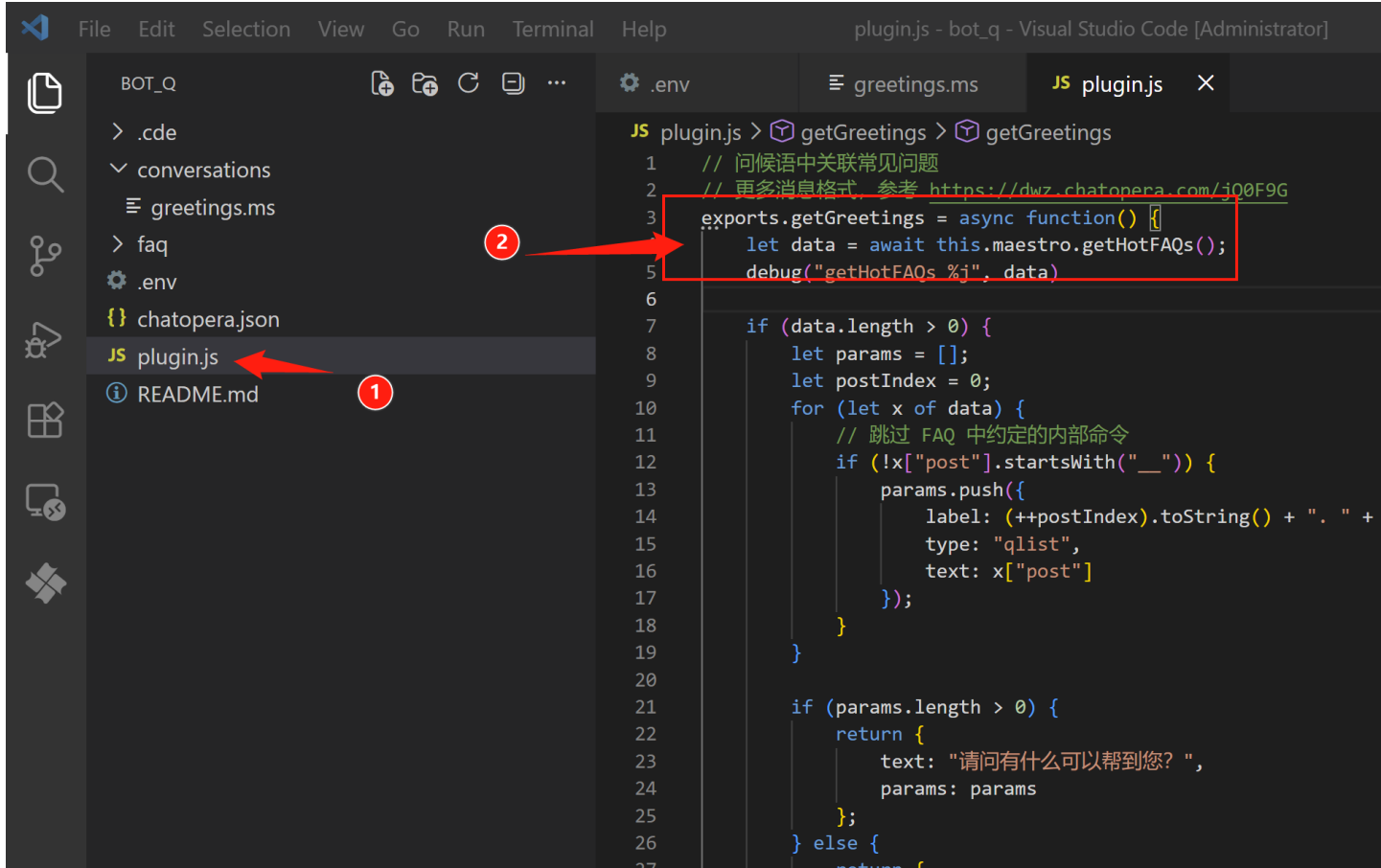


这时，我们可以发现一个叫 `__faq_hot_list` 的匹配器，并且使用了函数 `getGreetings` 的返回值做回复。

修改函数

为了实现修改打招呼的热门问题列表，我们只需要在函数中，修改 `getGreetings` 的返回值，过程如下。

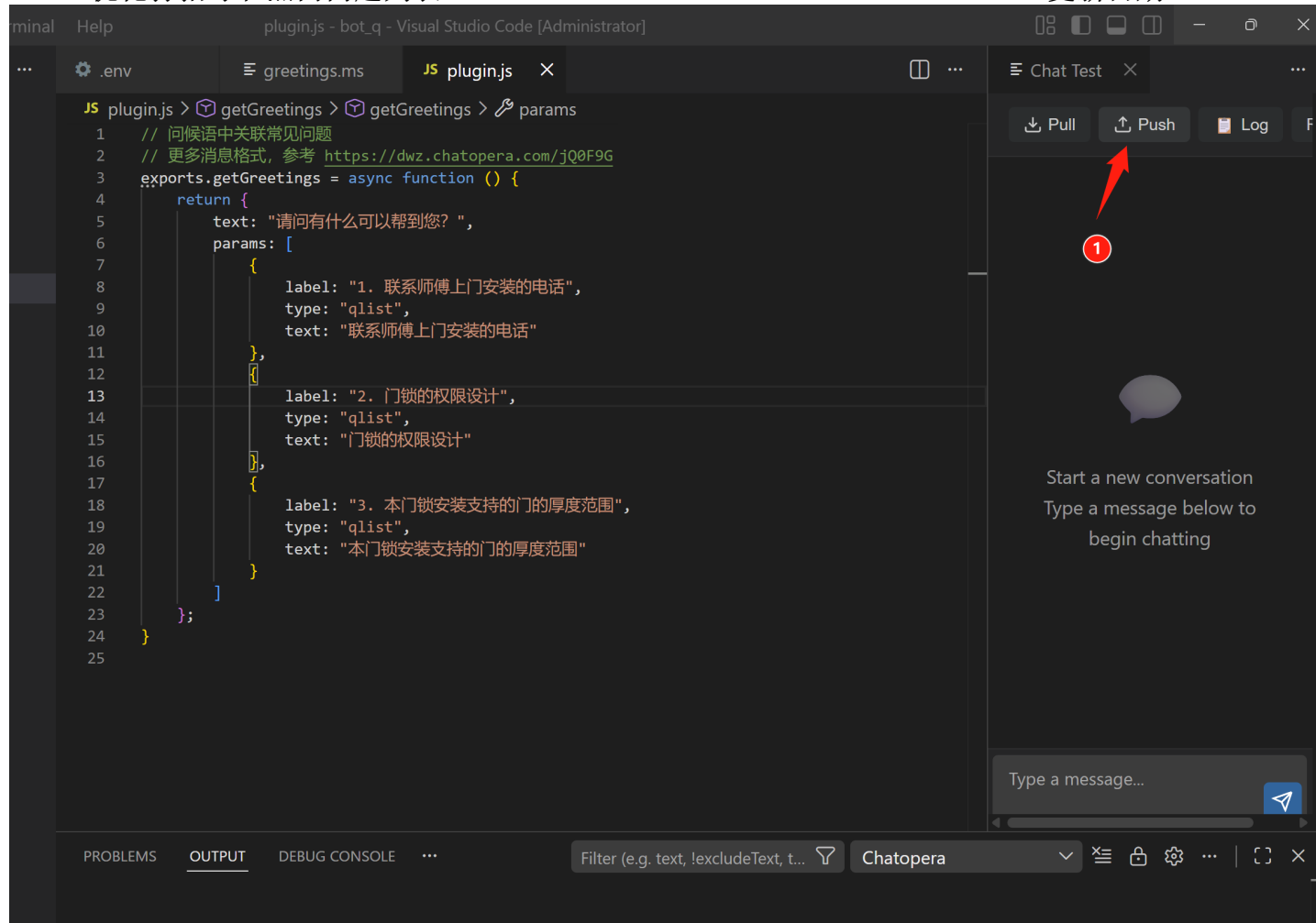
首先，打开 `plugin.js` 文件，找到 `getGreetings` 函数。



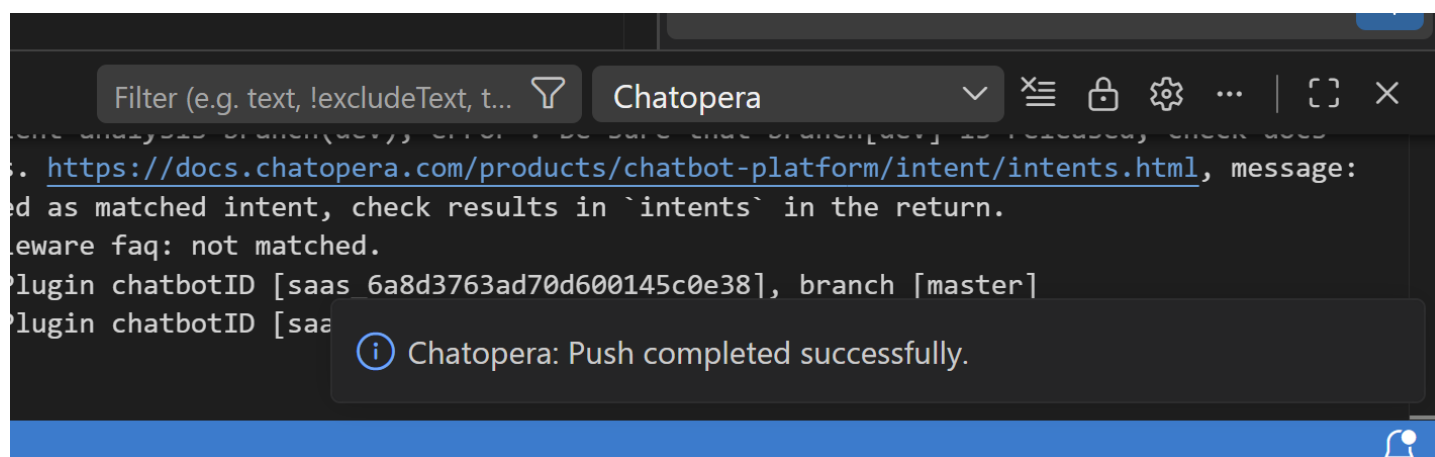
修改 `getGreetings` 的定义，让其内容如下：

```
// 问候语中关联常见问题
// 更多消息格式, 参考 https://dwz.chatopera.com/jQ0F9G
exports.getGreetings = async function () {
  return {
    text: "请问有什么可以帮到您？",
    params: [
      {
        label: "1. 联系师傅上门安装的电话",
        type: "qlist",
        text: "联系师傅上门安装的电话"
      },
      {
        label: "2. 门锁的权限设计",
        type: "qlist",
        text: "门锁的权限设计"
      },
      {
        label: "3. 本门锁安装支持的门的厚度范围",
        type: "qlist",
        text: "本门锁安装支持的门的厚度范围"
      }
    ]
  };
}
```

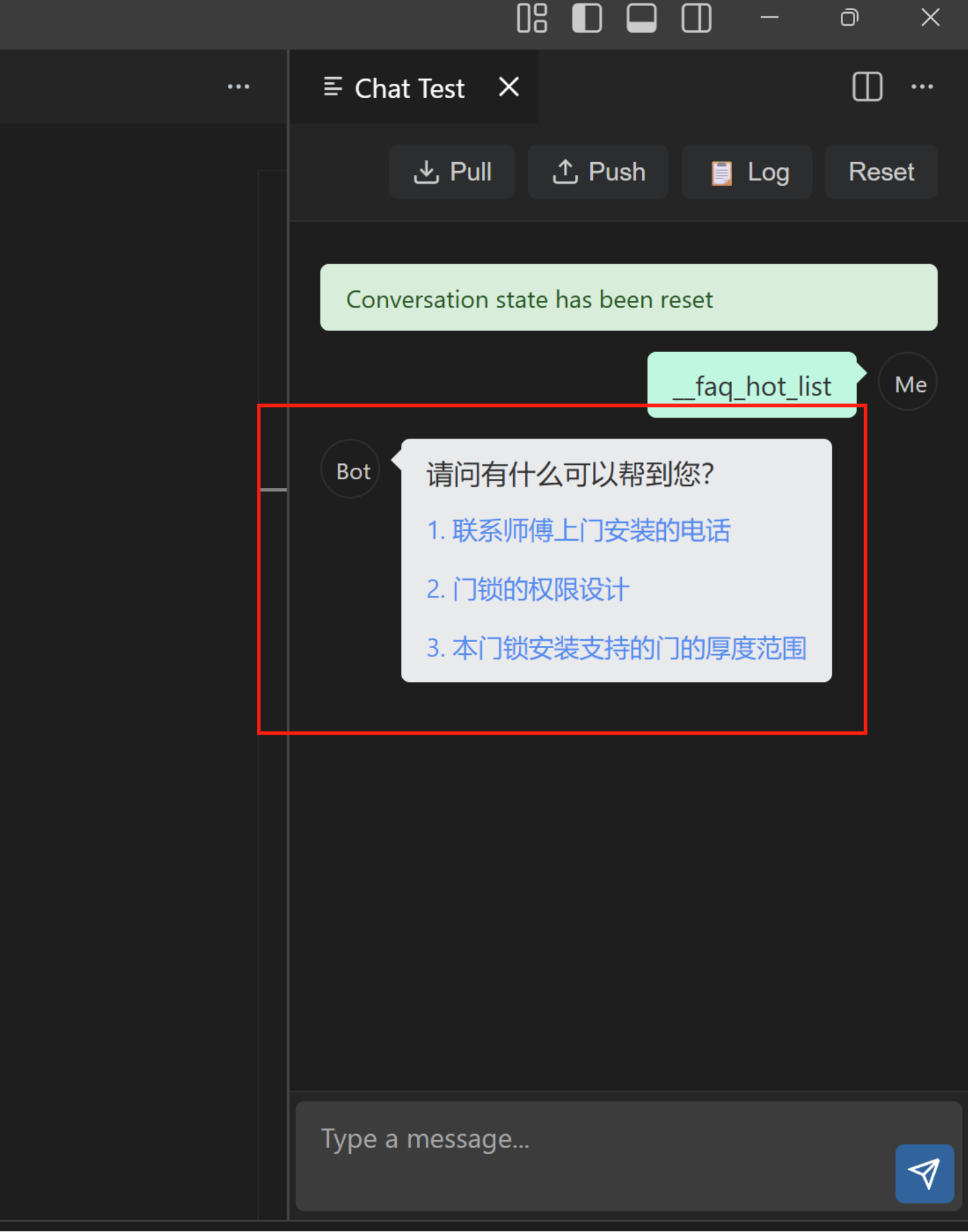
保存，然后进行【push】，如下图点击。



这时，在右下角，我们得到提示：

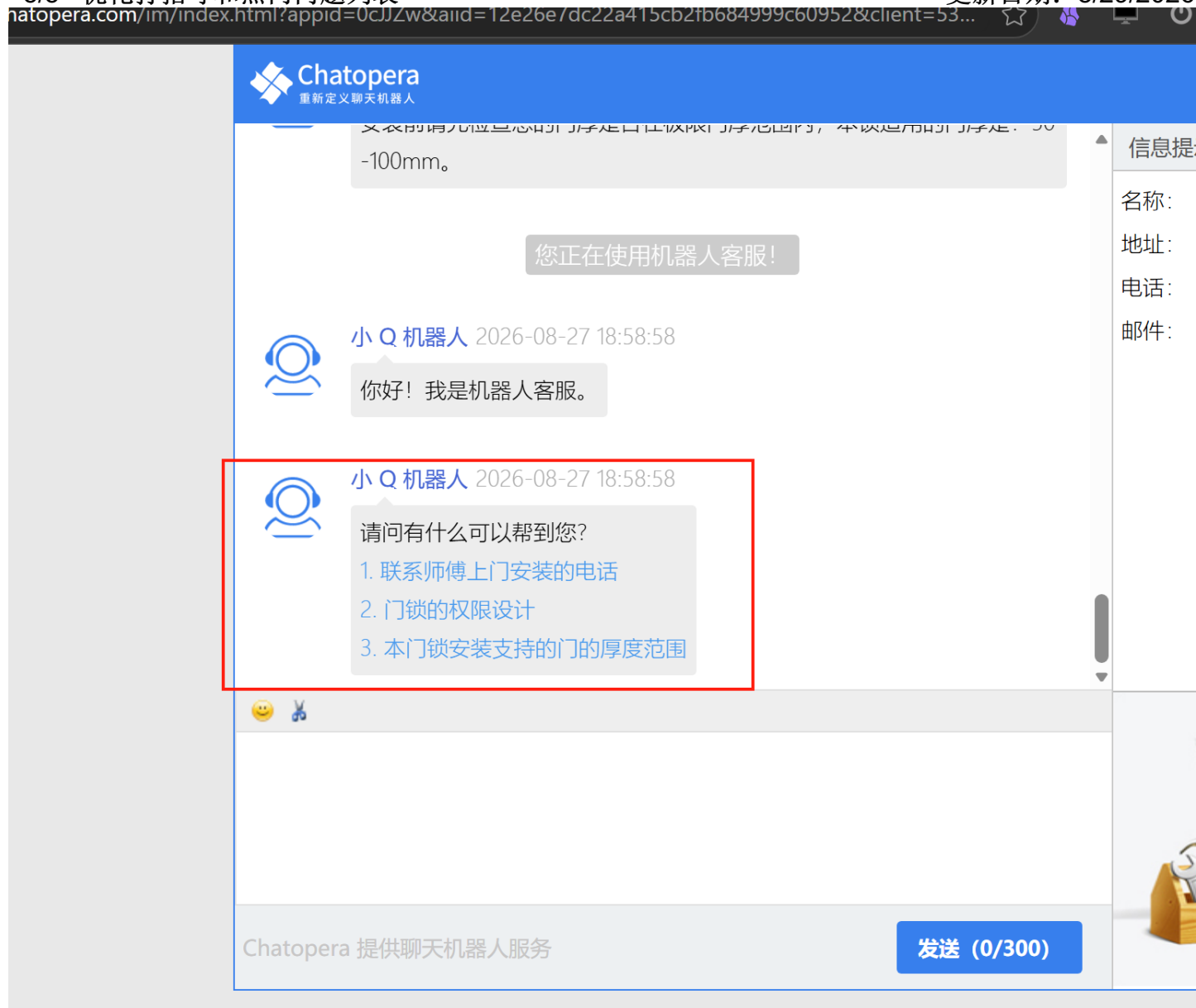


在对话测试窗口，发送 `__faq_hot_list`，看到如下的内容。



说明，我们的欢迎语更新完成。

这时，如果我们再回到访客的角色，刷新 H5 的页面，看到的内容则变成了下图。



入门教程完成

本篇是 Chatopera 云服务入门教程的最后一节，当你看到这里，就可以正式使用 Chatopera 云服务了，衷心的感谢你选择 Chatopera 云服务！

使用进阶

- [使用指南](#)中，是回答您可能问的一些问题
- [参考手册](#)是关于 SDK/API/CLI 的使用
- [背景知识](#)介绍了有关 Chatopera 云服务的工作原理

更多示例程序

继续进阶，请查看更多聊天机器人示例程序 [@Chatopera/chatbot-samples](#) -

- [GitHub](#)
- [Gitee](#)

使用指南

账号

- [注册使用 Chatopera 云服务](#)

RAG 知识库

- [生成 RAG 知识库](#)
- [修改 RAG 知识库提示词模板](#)

FAQ 知识库

- [维护知识库问答对](#)
- [知识库问答及调优测试](#)
- [知识库问答对导入和导出](#)
- [自定义词典增强知识库](#)
- [知识库热门问题查看和导出](#)
- [设置欢迎语热门问题](#)
- [使用函数获得动态答案](#)
- [话术助手快速检索话术](#)
- [集成微信实现转人工客服](#)

意图识别

- [创建和训练意图识别模型](#)
- [发布上线意图识别生产版本](#)

多轮对话

- [开发者工具](#)
- [通配符匹配器](#)
- [使用模糊匹配器](#)
- [使用意图匹配器](#)
- [设置回复](#)
- [使用上下轮钩子](#)
- [使用函数](#)
- [回复富文本内容：设置热门问题、按钮消息、图文消息、列表消息等](#)
- [配置环境变量](#)
- [管理对话状态](#)
- [集成自定义外部对话系统](#)
- [设置定时任务](#)

大语言模型

- [使用大语言模型（LLM）](#)

对话调优

- [对话测试](#)
- [对话历史分析](#)
- [Chatopera CLI 连接多轮对话](#)

渠道

- [H5 聊天控件](#)
- [机器人聊天码](#)
- [春松客服](#)
- [飞书](#)

系统集成

- [SDK](#)
- [CLI 安装和配置](#)
- [CLI 导入和导出对话语料](#)

词典

- [系统词典](#)
- [词汇表词典](#)
- [正则表达式词典](#)

示例程序

- [讲笑话机器人](#)
- [GuessNumberBot](#)
- [预定机票机器人](#)
- [招聘面试机器人](#)
- [冰箱报维修服务机器人](#)

注册使用 Chatopera 云服务

注册

Chatopera 云服务的地址是：<https://bot.chatopera.com>

从浏览器打开后，点击“立即使用”，跳转到登录/注册页面，输入邮箱和密码，使用“回车键”【Enter】提交，即完成。

在登录后，没有做手机号认证的，Chatopera 云服务会主动提示用户绑定手机号，该项目是必须的，未填写手机号的无法使用 Chatopera 云服务。

注意事项：

- 目前，用于邮件通知服务的提供商暂未支持 Gmail 服务，为方便重置密码、获得产品更新信息，请使用其它邮件地址注册。
- 请保证填写正确的邮箱地址用于找回密码，接收系统通知，该邮箱不会泄漏给第三方。
- 用于手机号验证的 SMS 服务仅支持中国大陆运营商，为了保证您的正常注册和使用，请使用相关运营商的手机号。

更新密码

登录后，在右上角看到头像，点击头像，在菜单中点击“个人中心”，来到“个人中心”页面后，可填入新的密码，点击“保存”。

找回密码

进入登录页面 <https://bot.chatopera.com/login>，点击“忘记密码”，根据提示完成。

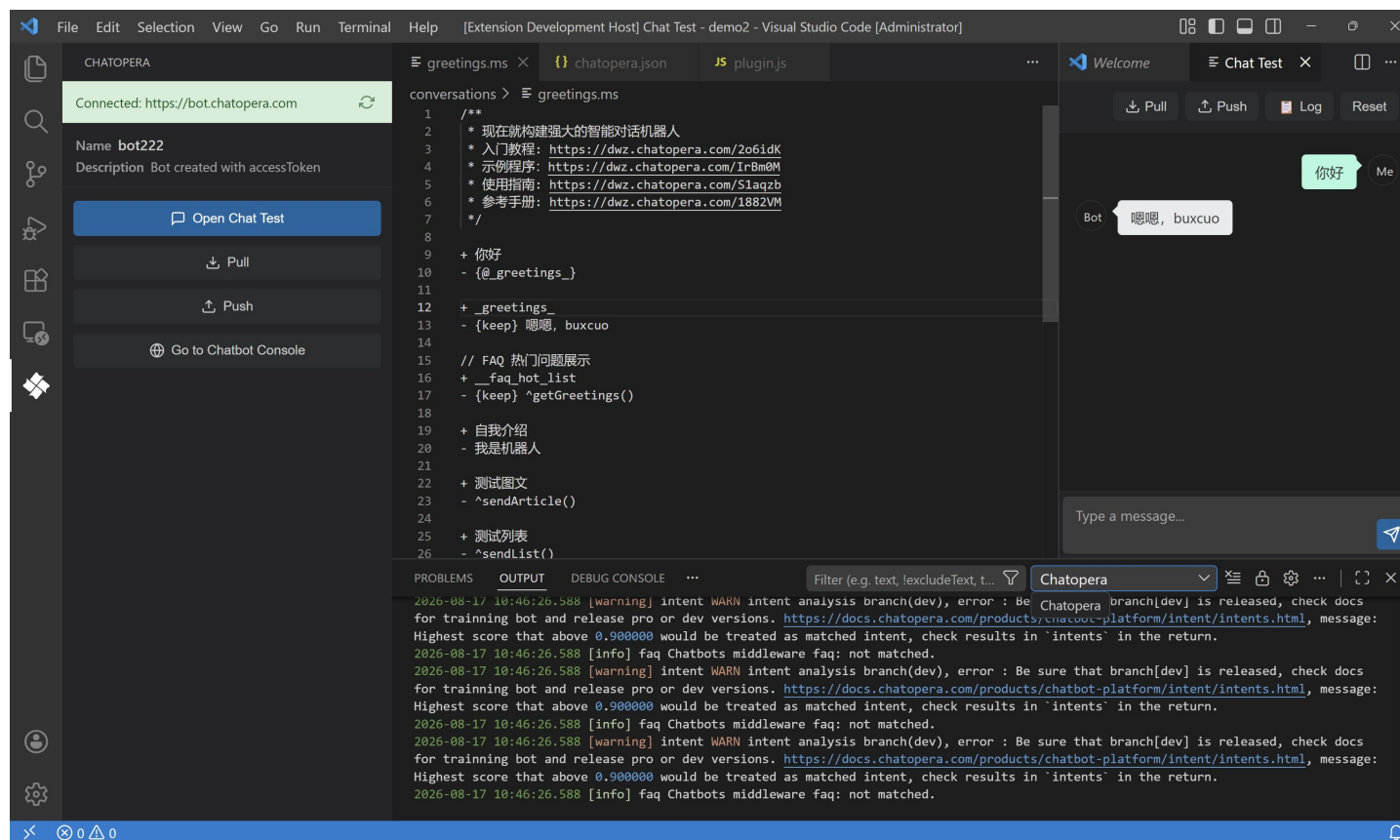
此处会使用邮箱验证账号。

修改头像

登录后，在右上角看到头像，点击头像，在菜单中点击“个人中心”，来到“个人中心”页面后，点击屏幕中间的“头像”，弹出上传头像表单。

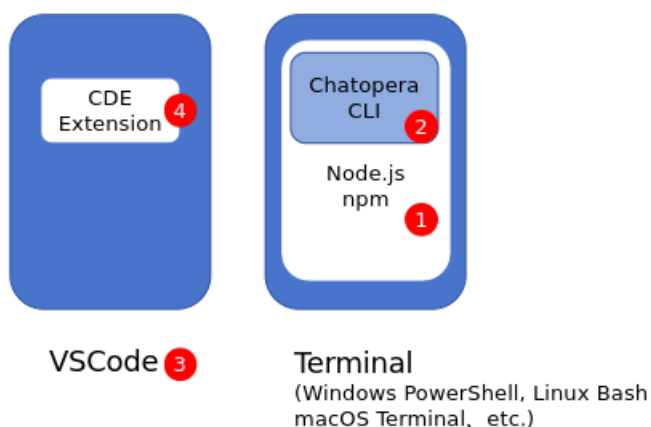
CDE 快速开始

CDE, 即 [Chatbot Development Env](#), 是 Chatopera 云服务面向开发者发布的开发者工具。使用 CDE 可以定制聊天机器人的对话技能, 提升智能化。



安装过程概述

Chatopera Toolchain



- 安装 Node.js: 使用 CDE 需要结合 Chatopera CLI, 安装 CLI 需要 Node.js
- 安装 Chatopera CLI: 安装和配置 Node.js 后, 使用 `npm` 命令在控制台安装
- 安装 VSCode: CDE 是一个 VSCode 插件, 所以需要先安装 VSCode

获得帮助

当您在新手任务中, 遇到任何问题, 欢迎[联系 Chatopera 工程师](#)获得帮助支持~

下载安装包

需要使用 Nodejs v18 或更高版本，在 [Nodejs 官网](#) 下载安装包。Nodejs 安装包支持不同的形式，推荐使用独立安装包。

下载 Node.js®

获得适用于

Windows

 且使用

Docker

 和

npm

 的 Node.js®

v24.19.0 LTS

提示

 Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

1

Docker 对每个操作系统都有特定的安装指导。

2

请参考 <https://docker.com/get-started/> 给出的官方文档

3

4

拉取 Node.js Docker 镜像:

5

docker pull node:24-alpine

6

7

创建 Node.js 容器并启动一个 Shell 会话:

8

docker run -it --rm --entrypoint sh node:24-alpine

9

10

验证 Node.js 版本:

11

node -v # Should print "v24.19.0".

12

13

验证 npm 版本:

14

npm -v # Should print "11.17.0".

PowerShell

Docker 是一个容器化平台。如果遇到任何问题，请访问 [Docker的网站](#)

或者获得适用于

Windows

x64

 平台的 Node.js® 构建。

Windows 安装程序(.msi)

独立文件(.zip)

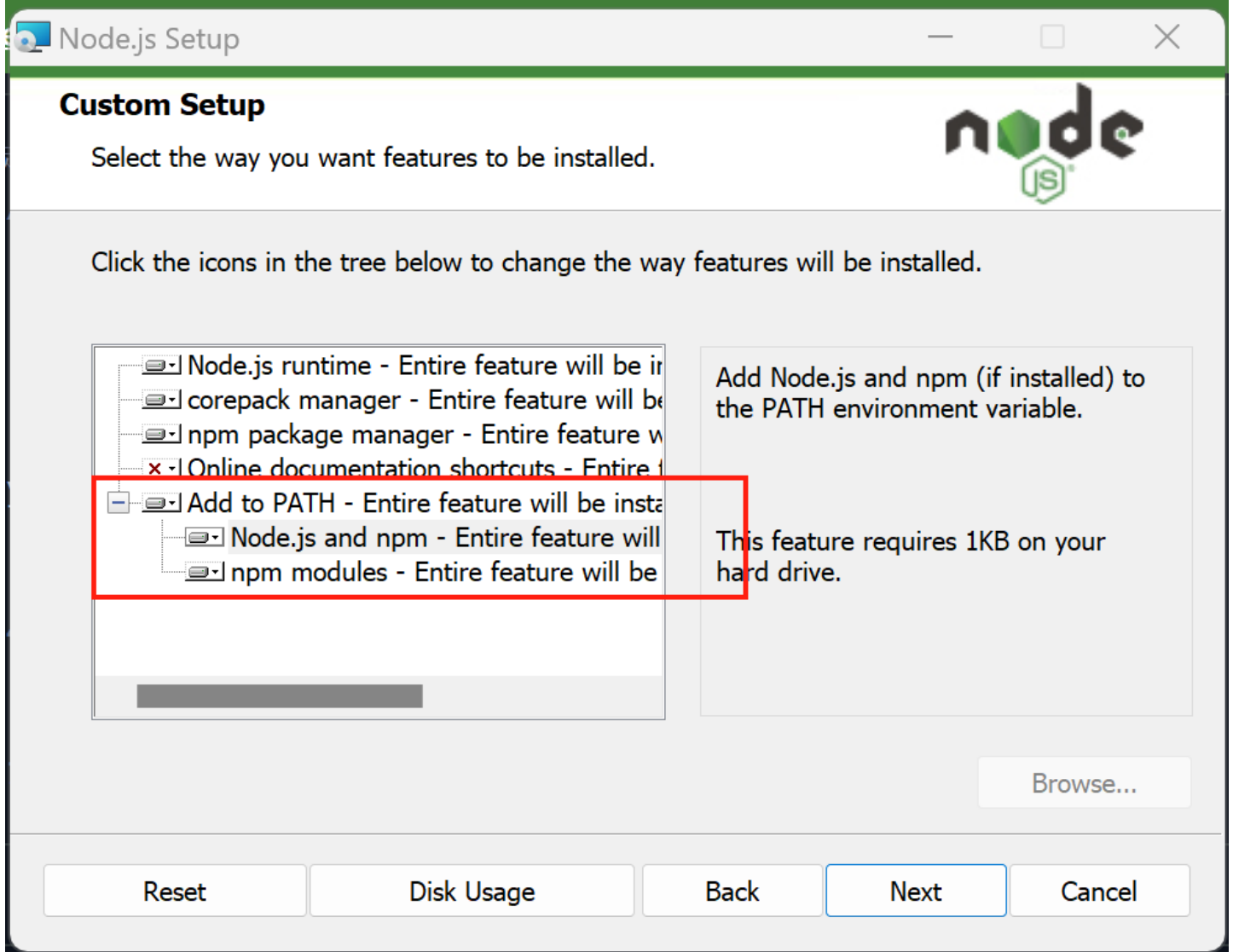
比如，适用于 Windows 平台的 [MSI 安装包](#)。

下载后，打开安装包，执行安装，注意选择设置环境变量。

设置环境变量 Path

在安装过程中，设置如下选项。

将 Node 和 npm 设置到 Path 环境变量。



检查安装

安装完成后，检查是否安装成功。

- Mac/Linux 用户：启动一个命令行控制台，执行 `echo $PATH`，检查 node 的安装路径是否在 `PATH` 中，然后执行 `node --version` 和 `npm --version`，确认安装的版本是否正确。
- Windows 用户：启动一个 CMD Prompt，执行 `echo %Path%` 检查 node 的安装路径是否在 `Path` 中，然后再 CMD 中执行 `node --version` 和 `npm --version`，确认安装的版本是否正确。

```
C:\Users\Administrator>node --version
v24.18.0
```

```
C:\Users\Administrator>npm --version
11.16.0
```

```
C:\Users\Administrator>
```

安装 Chatopera CLI

安装好 Nodejs 后, 在命令行终端执行以下命令:

```
npm install -g @chatopera/sdk
```

```
C:\Users\Administrator>npm install -g @chatopera/sdk
-|
```

上述命令退出后, 执行下面命令测试是否安装成功:

```
bot --help
```

如果能看到下面的输出, 就表明 Chatopera CLI 已经安装成功。

```
C:\Users\Administrator>bot --help
>> load env file C:\Users\Administrator\.env into process.env ...
Usage: bot [options] [command]

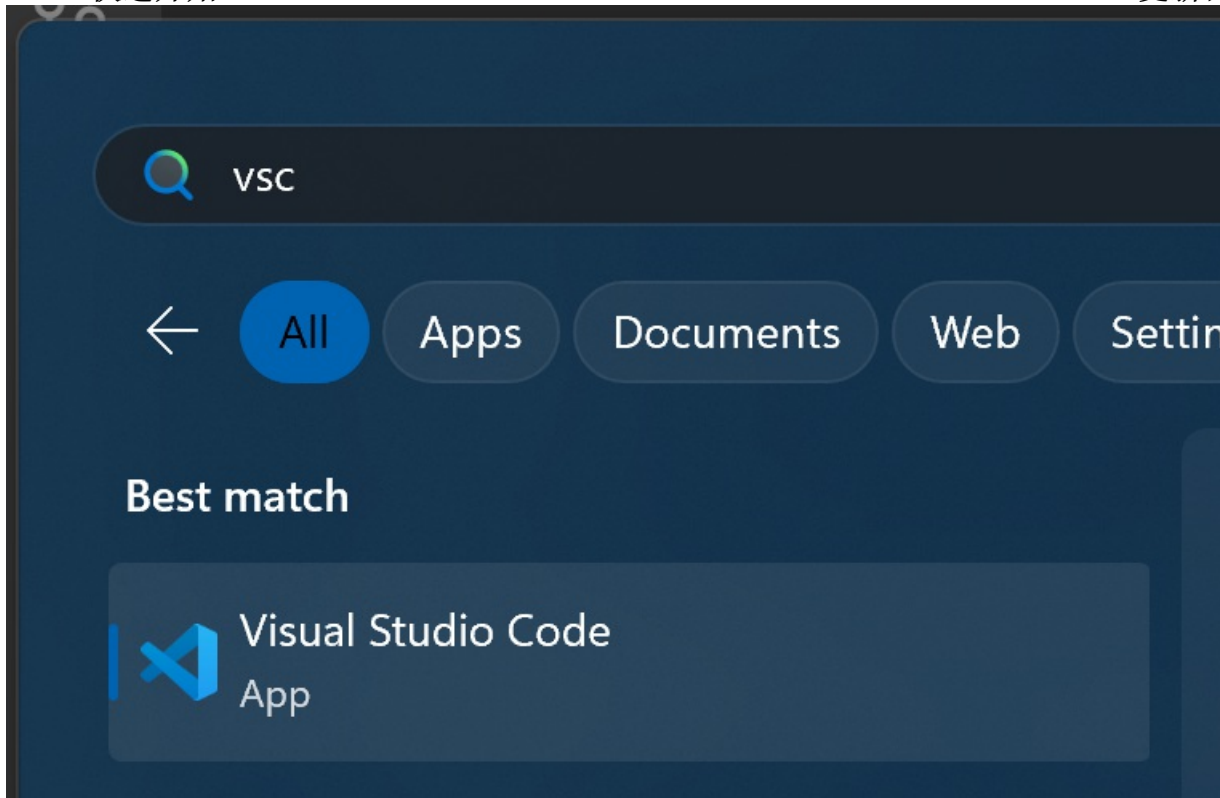
Options:
  -V, --version          output the version number
  -h, --help             display help for command

Commands:
  env [options]          chat with bot via bot#conversation interface, https://dwz.chatopera.com/T0CZ0R
  details [options]     get a bot's detail info, such as name, primaryLanguage
  connect [options]     chat with bot via bot#conversation interface, https://dwz.chatopera.com/SH17N5
  conversation [options] import or export a bot's conversations data
  trace [options]       tail a bot's conversations logging info
  faq [options]         import, export or drop all bot's faqs data
  dicts [options]       sync, import or export a bot's dicts data
  intents [options]     train, import or export a bot's intents data
  rag [options]         RAG Knowledge Base Command Tools
  project [options]     Chatopera Bot Project Management Commands.
  botgen [options]      chat with a chatopera lm for bot generation.
  help [command]        display help for command
```

安装 VSCode

进入 VSCode 官方网站: <https://code.visualstudio.com/Download>

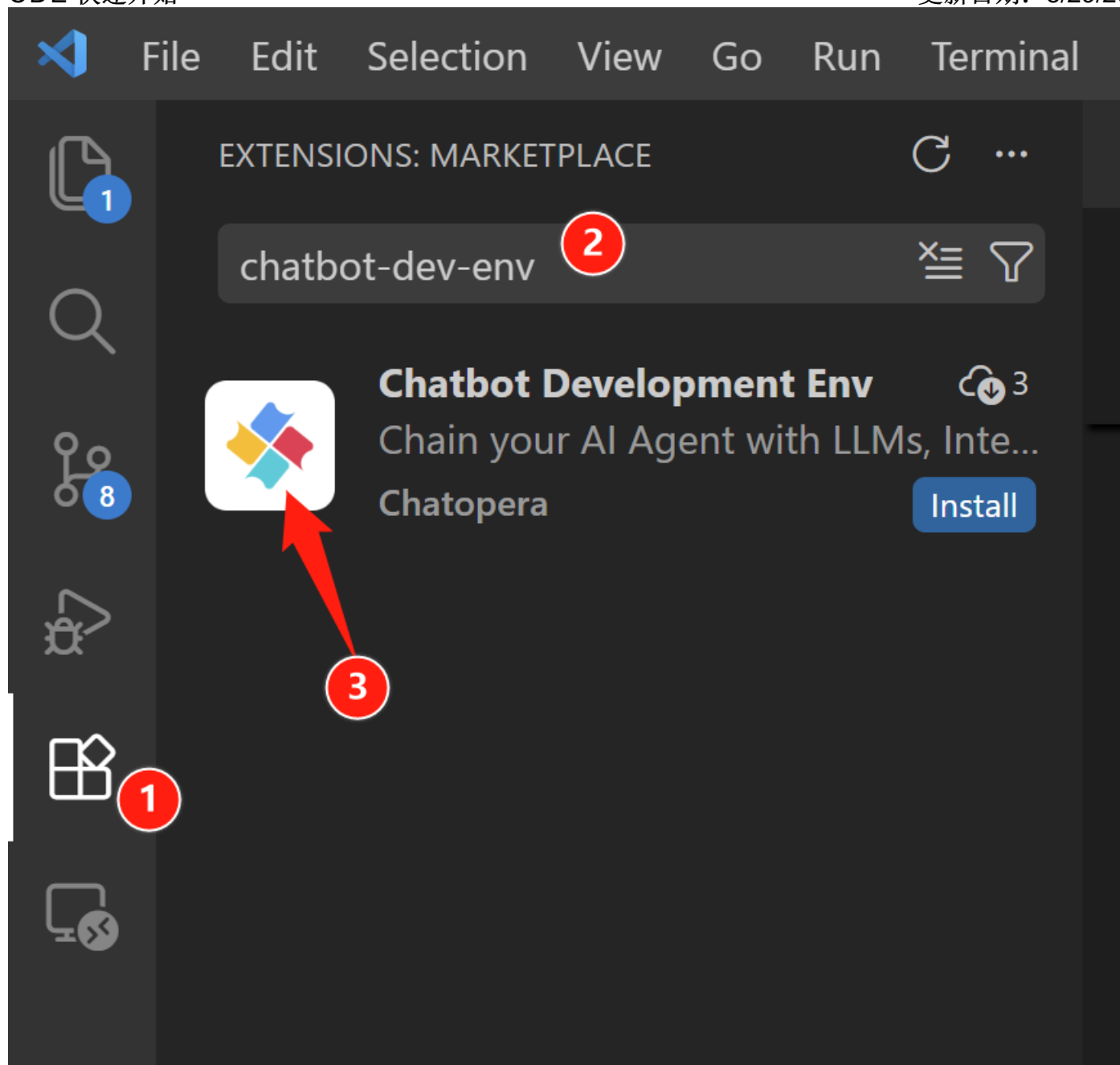
下载您的操作系统的安装包, 然后安装, 安装成功后, 可以在启动中心找到 VSCode。



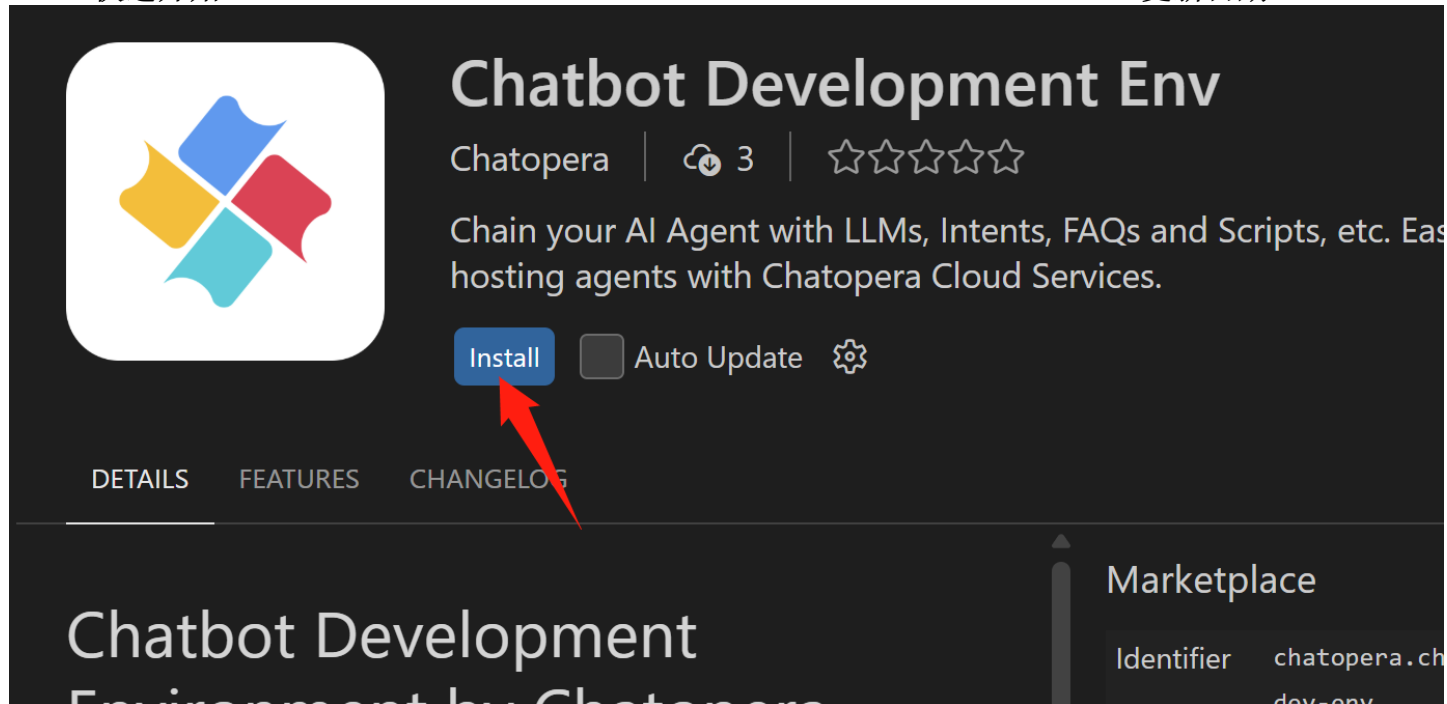
安装 CDE 插件

打开 VSCode, 安装下图:

1. 进入插件管理
2. 搜索 `chatbot-dev-env`
3. 点击 Chatopera 图标, 进入插件详情



4. 点击 `Install` / 安装



Chatbot Development Env

Chatopera | 3 | ☆☆☆☆☆

Chain your AI Agent with LLMs, Intents, FAQs and Scripts, etc. Easy hosting agents with Chatopera Cloud Services.

[Install](#) ☐ Auto Update 

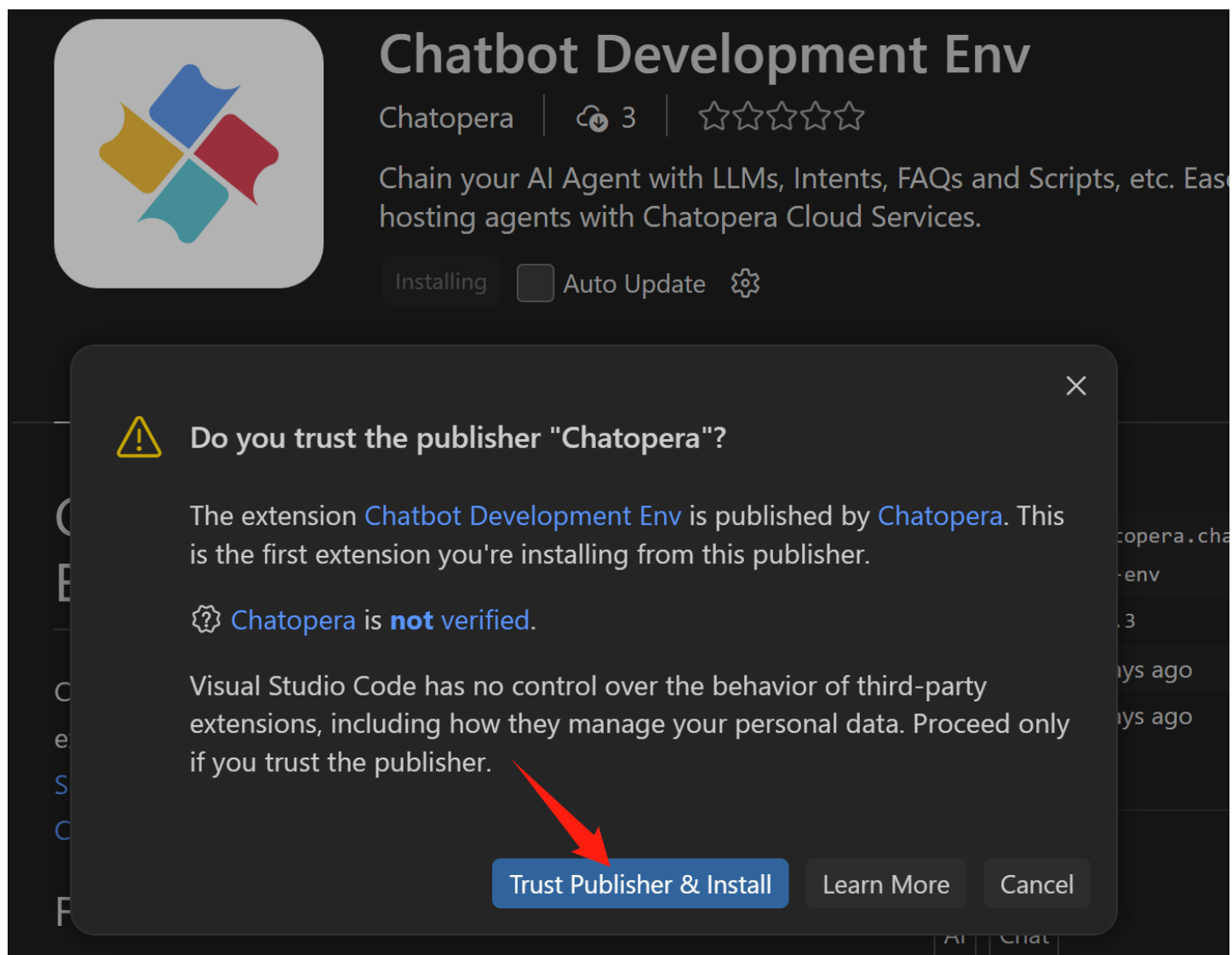
DETAILS FEATURES CHANGELOG

Chatbot Development Environment by Chatopera

Marketplace

Identifier chatopera.chatbot-development-env

5. 点击【信任】



Chatbot Development Env

Chatopera | 3 | ☆☆☆☆☆

Chain your AI Agent with LLMs, Intents, FAQs and Scripts, etc. Easy hosting agents with Chatopera Cloud Services.

[Installing](#) ☐ Auto Update 

Do you trust the publisher "Chatopera"?

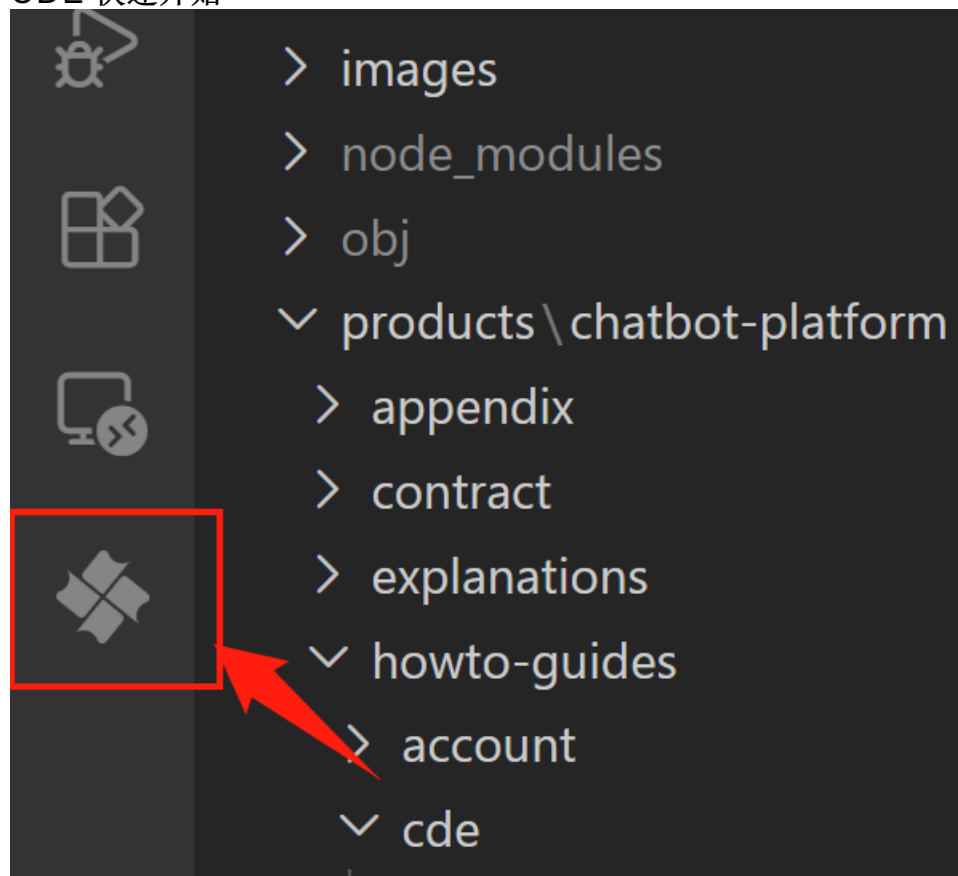
The extension [Chatbot Development Env](#) is published by [Chatopera](#). This is the first extension you're installing from this publisher.

 [Chatopera](#) is **not** verified.

Visual Studio Code has no control over the behavior of third-party extensions, including how they manage your personal data. Proceed only if you trust the publisher.

[Trust Publisher & Install](#) [Learn More](#) [Cancel](#)

安装后，能在左侧查看到 CDE 插件图标。



安装结束。

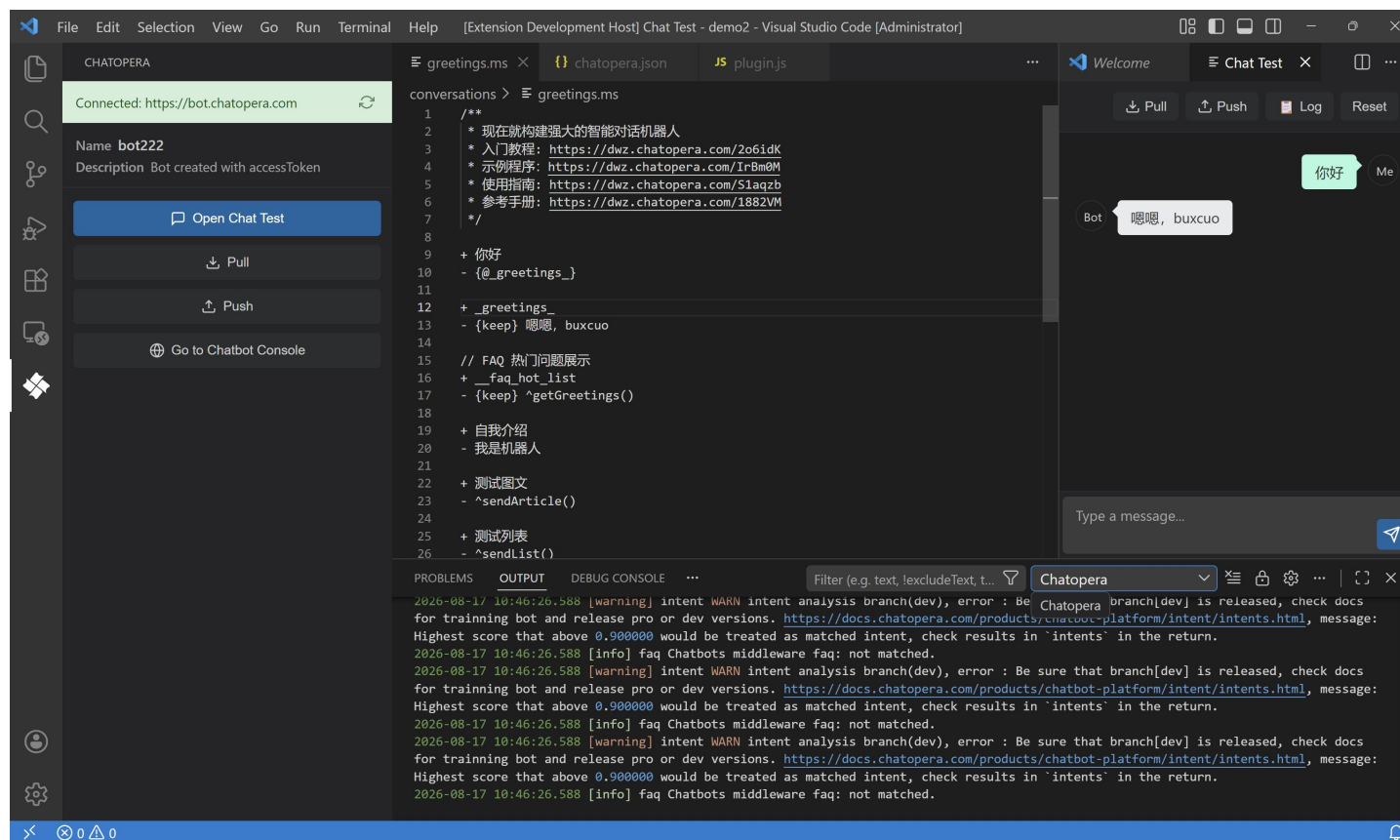
下一步

- CDE 的具体的使用，参考：[入门教程](#)
- 在机器人发布时，可以使用 SDK/API 集成到其他软件中，参考：[Chatopera SDK 和 APIs](#)

CDE Quick Start

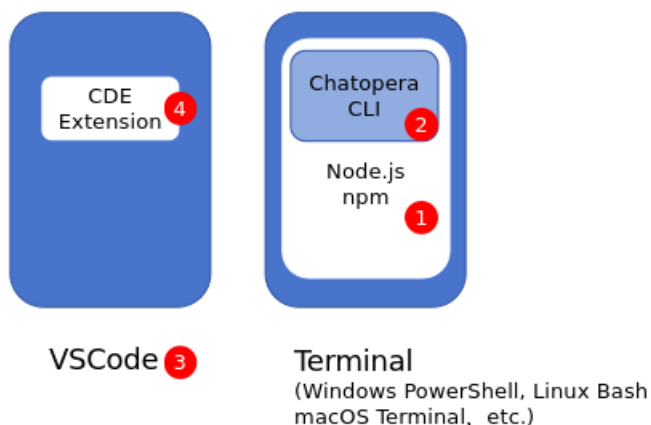
更新日期: 8/29/2026

CDE, namely [Chatbot Development Env](#), is a developer tool released by Chatopera cloud service for developers. Use CDE to customize the conversational skills of the chatbot and improve its intelligence.



Overview of the installation process

Chatopera Toolchain



- Install Node.js: To use CDE, you need to combine it with Chatopera CLI. To install CLI, you need Node.js.
- Install Chatopera CLI: After installing and configuring Node.js, use the `npm` command to install it in the console
- Install VSCode: CDE is a VSCode plug-in, so you need to install VSCode first

Get help

When you encounter any problems during the novice tasks, you are welcome to [contact Chatopera engineers](#) for help and

Install Node.js

Download the installation package

You need to use Node.js v18 or higher, download the installation package from [Nodejs official website](https://nodejs.org/en/). Node.js installation packages support different forms, and it is recommended to use independent installation packages.

下载 Node.js®

获得适用于 Windows 且使用 Docker 和 npm 的 Node.js® v24.19.0 LTS

提示 Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker 对每个操作系统都有特定的安装指导。
2 # 请参考 https://docker.com/get-started/ 给出的官方文档
3
4 # 拉取 Node.js Docker 镜像:
5 docker pull node:24-alpine
6
7 # 创建 Node.js 容器并启动一个 Shell 会话:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # 验证 Node.js 版本:
11 node -v # Should print "v24.19.0".
12
13 # 验证 npm 版本:
14 npm -v # Should print "11.17.0".
```

PowerShell

Docker 是一个容器化平台。如果遇到任何问题, 请访问 [Docker的网站](#)

或者获得适用于 Windows x64 平台的 Node.js® 构建。

Windows 安装程序(.msi) 独立文件(.zip)

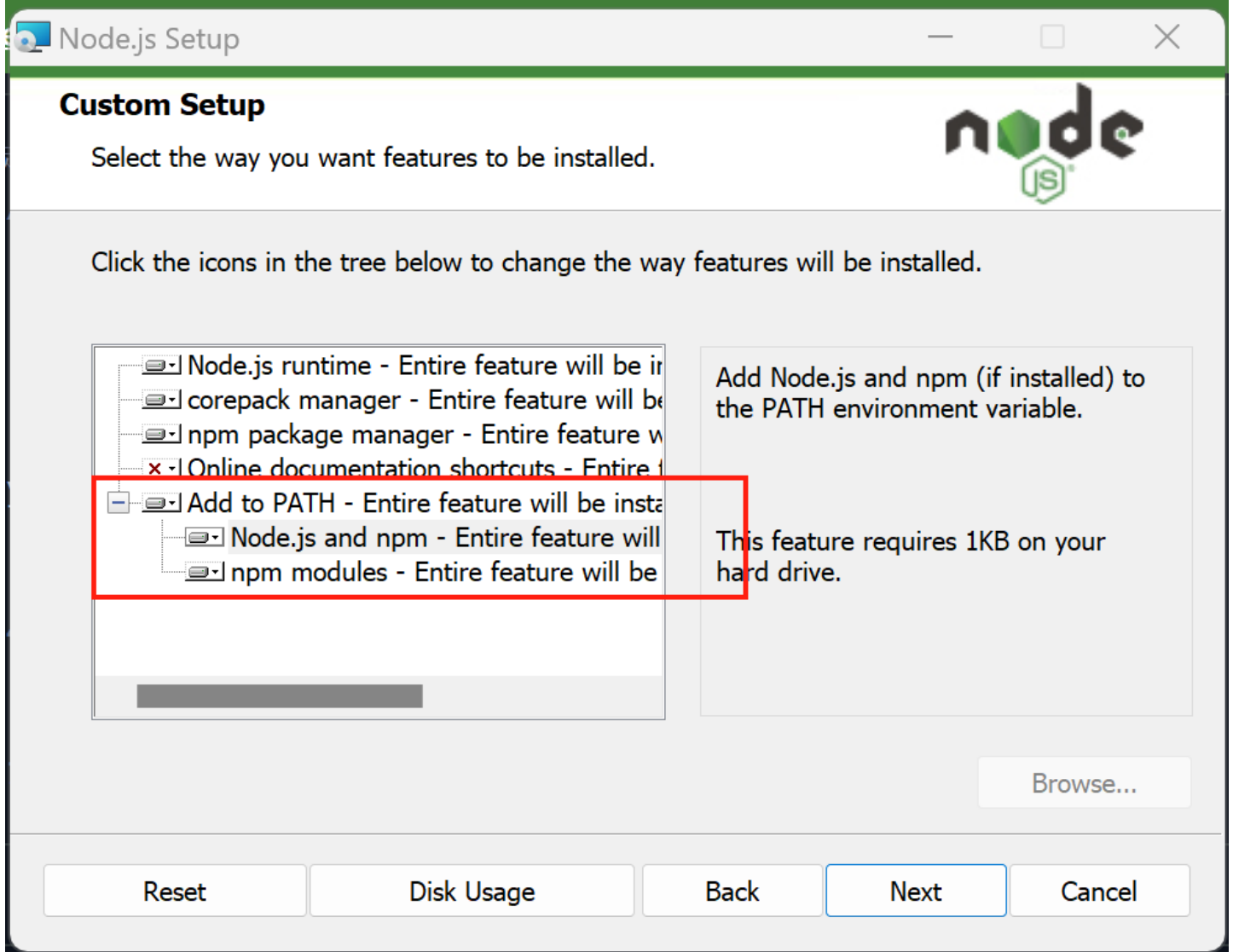
For example, [MSI installation package](#) for Windows platform.

After downloading, open the installation package, perform the installation, and pay attention to setting environment variables.

Set environment variable Path

During the installation process, set the following options.

Set Node and npm to the Path environment variable.



Check installation

After the installation is complete, check whether the installation was successful.

- Mac/Linux users: Start a command line console, execute `echo $PATH`, check whether the installation path of node is in `PATH`, and then execute `node --version` and `npm --version` to confirm whether the installed version is correct.
- Windows users: Start a CMD Prompt, execute `echo %Path%` to check whether the installation path of node is in `Path`, and then execute `node --version` and `npm --version` in CMD to confirm whether the installed version is correct.

```
C:\Users\Administrator>node --version
v24.18.0
```

```
C:\Users\Administrator>npm --version
11.16.0
```

```
C:\Users\Administrator>
```

Install Chatopera CLI

After installing Nodejs, execute the following command in the command line terminal:

```
npm install -g @chatopera/sdk
```

```
C:\Users\Administrator>npm install -g @chatopera/sdk
-|
```

After the above command exits, execute the following command to test whether the installation is successful:

```
bot --help
```

If you can see the following output, it means that the Chaopera CLI has been installed successfully.

```
C:\Users\Administrator>bot --help
>> load env file C:\Users\Administrator\.env into process.env ...
Usage: bot [options] [command]

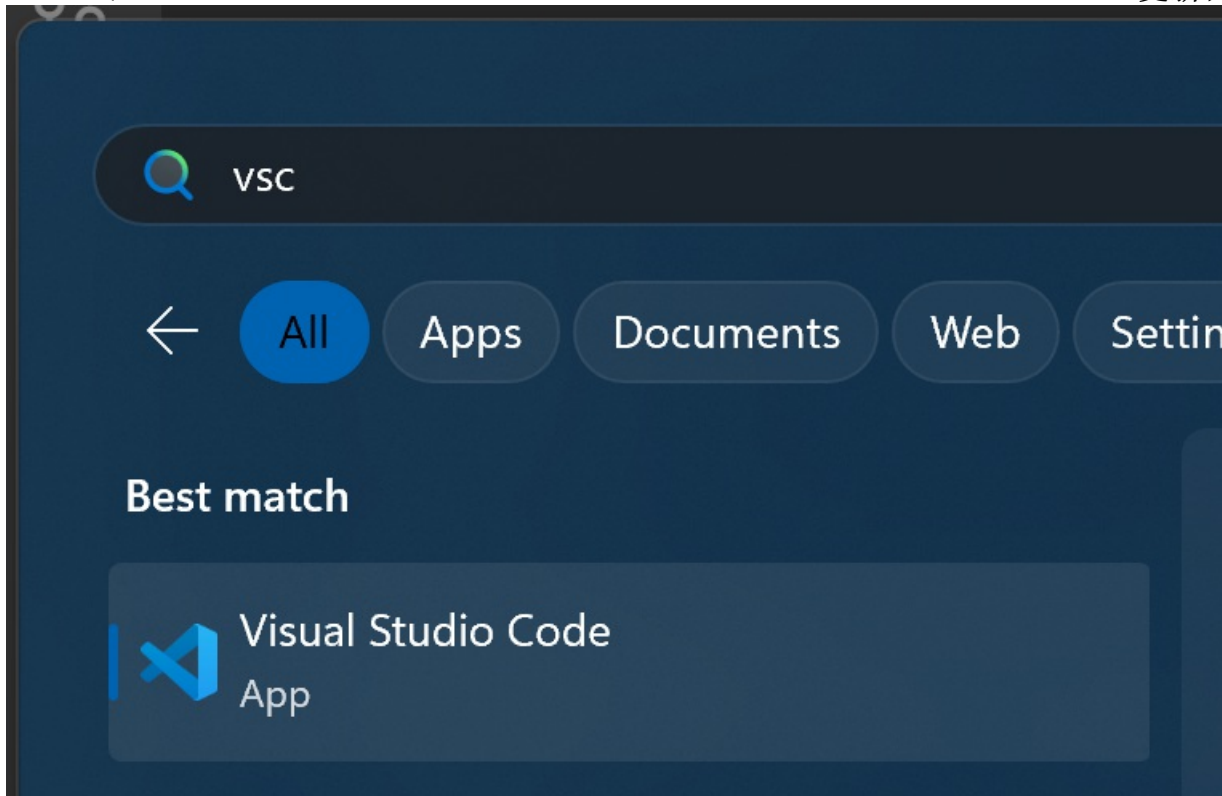
Options:
  -V, --version          output the version number
  -h, --help             display help for command

Commands:
  env [options]          chat with bot via bot#conversation interface, https://dwz.chatopera.com/T0CZ0R
  details [options]      get a bot's detail info, such as name, primaryLanguage
  connect [options]      chat with bot via bot#conversation interface, https://dwz.chatopera.com/SH17N5
  conversation [options] import or export a bot's conversations data
  trace [options]        tail a bot's conversations logging info
  faq [options]          import, export or drop all bot's faqs data
  dicts [options]        sync, import or export a bot's dicts data
  intents [options]      train, import or export a bot's intents data
  rag [options]          RAG Knowledge Base Command Tools
  project [options]      Chatopera Bot Project Management Commands.
  botgen [options]       chat with a chatopera lm for bot generation.
  help [command]         display help for command
```

Install VSCode

Enter the VSCode official website: <https://code.visualstudio.com/Download>

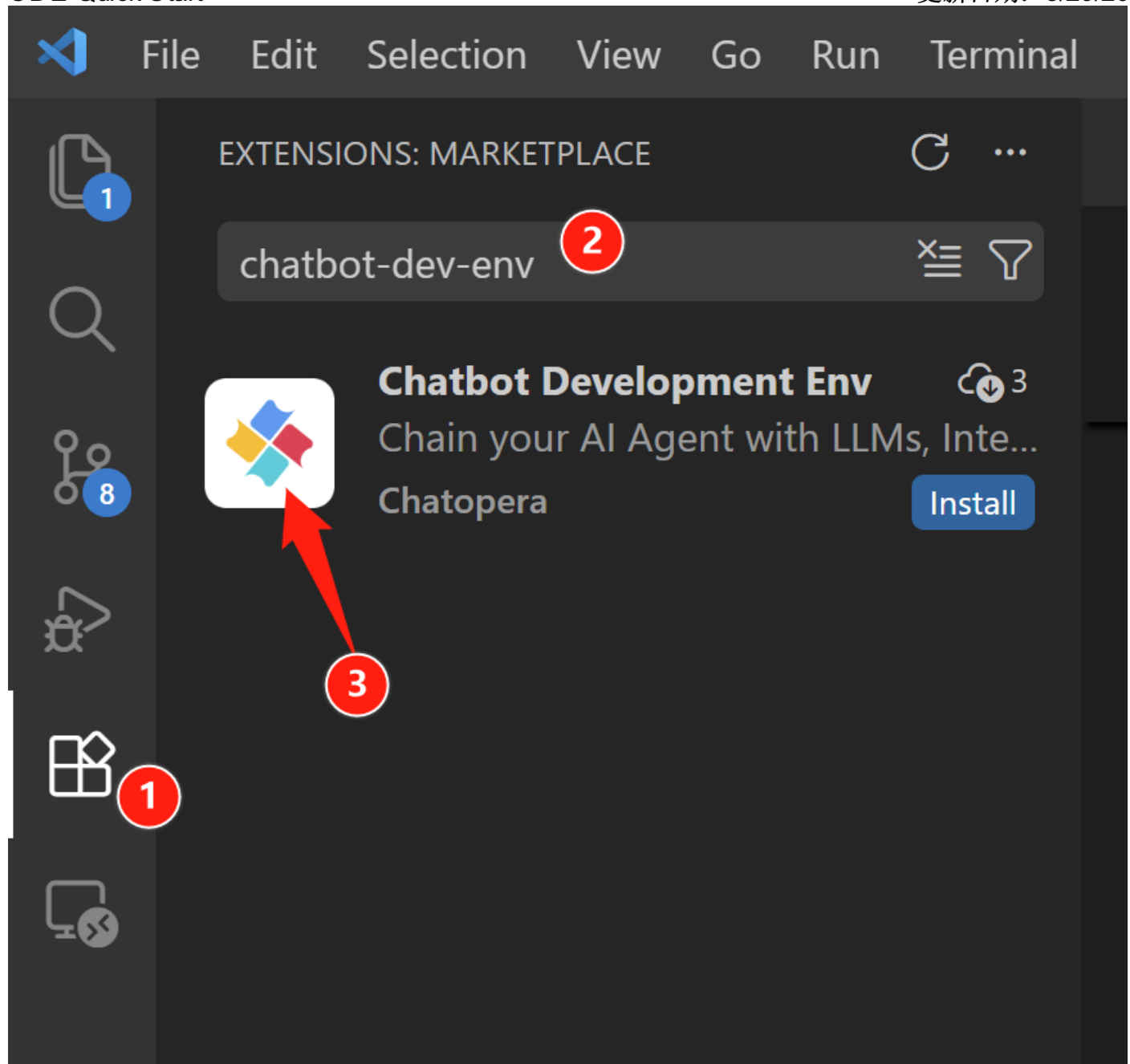
Download the installation package for your operating system and install it. After successful installation, you can find VSCode in the Start Center.



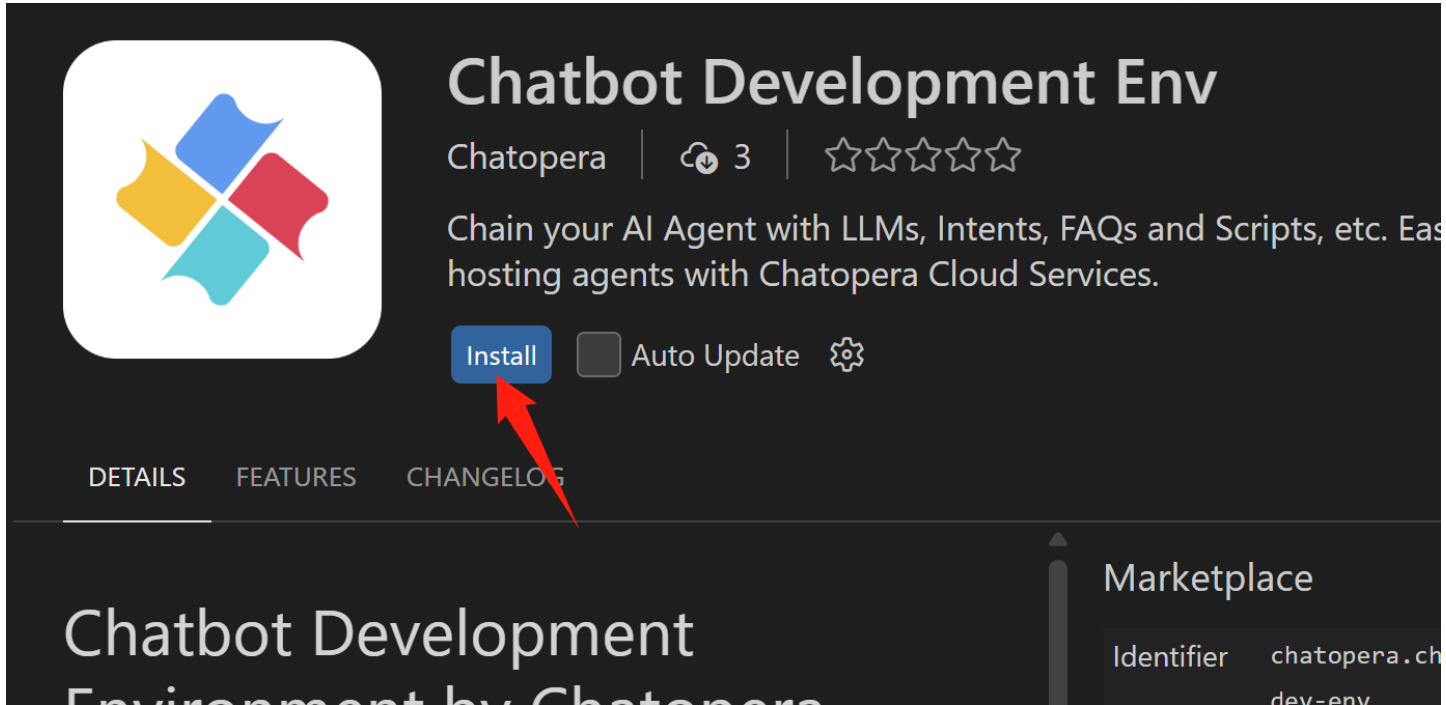
Install CDE plug-in

Open VSCode and install the following image:

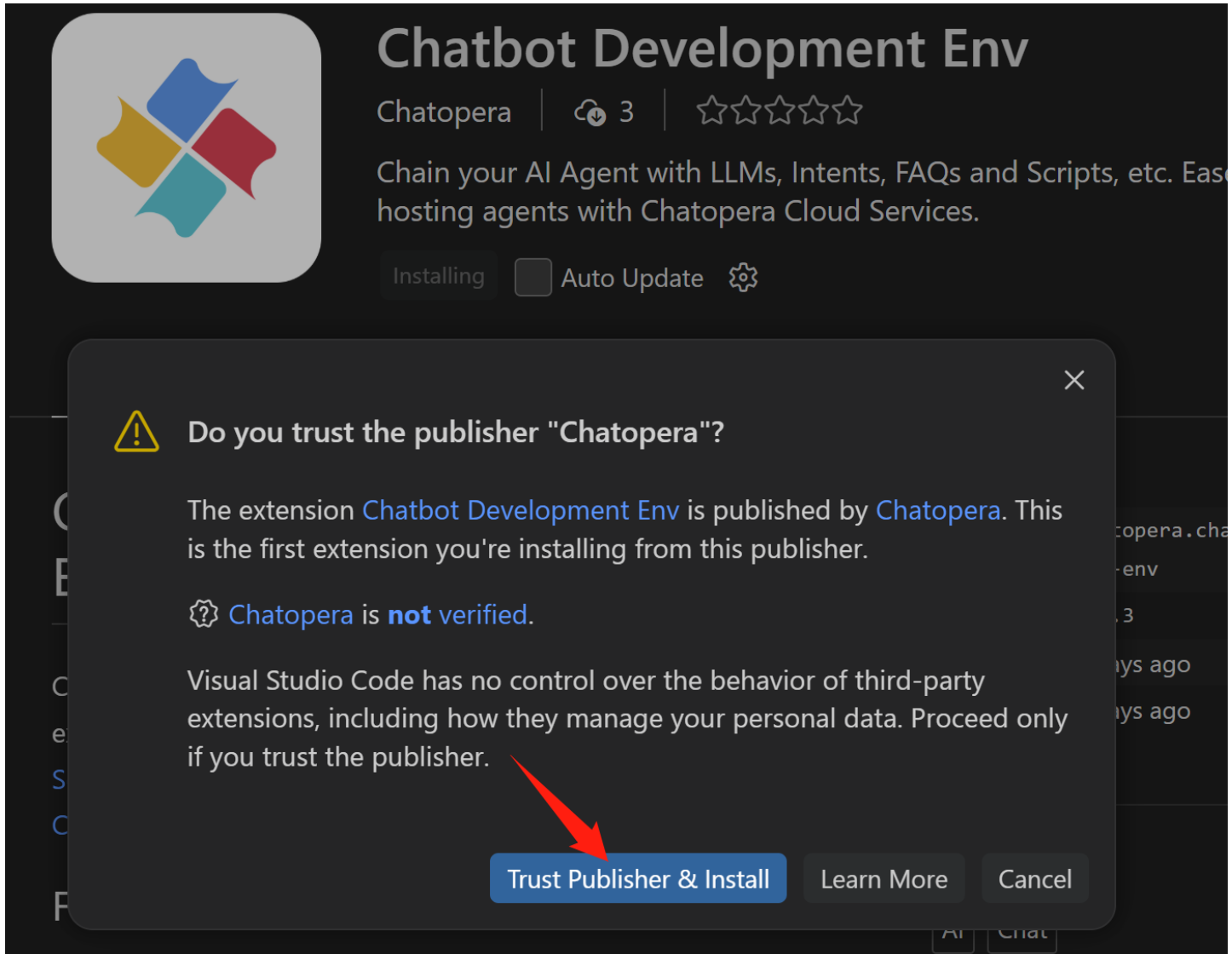
1. Enter plug-in management
2. Search for `chatbot-dev-env`
3. Click the Chatopera icon to enter the plug-in details



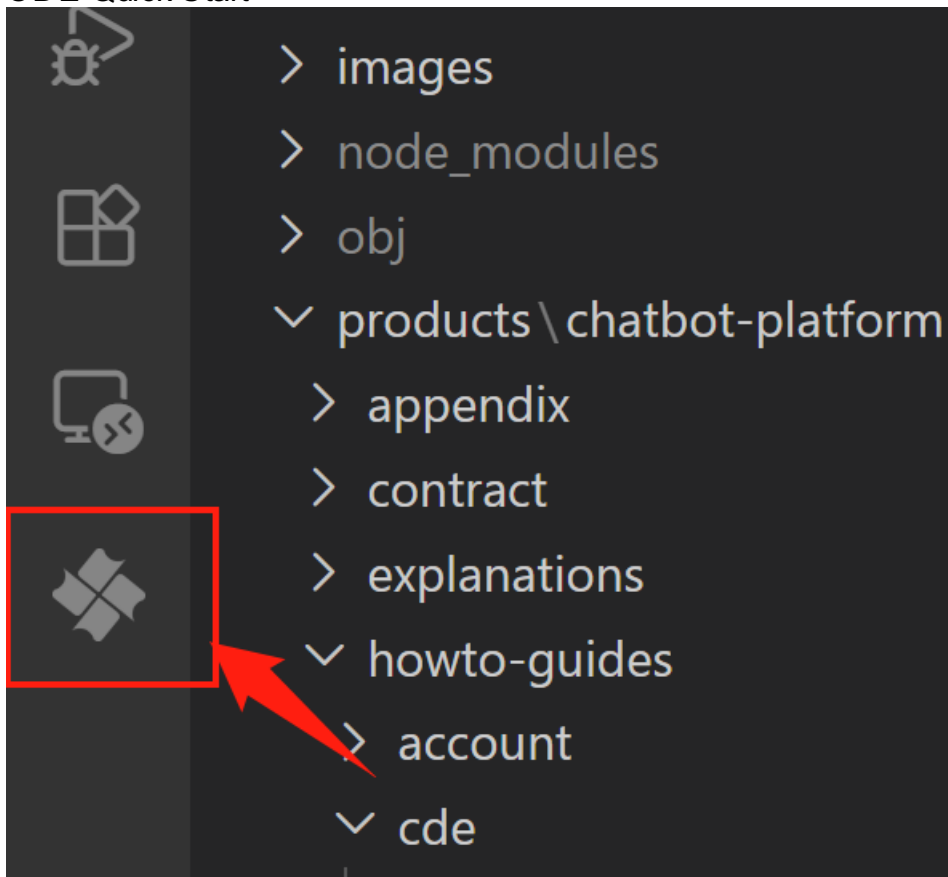
4. Click `Install` / `Install`



5. Click [Trust]



After installation, you can see the CDE plug-in icon on the left.



The installation is completed.

Next step

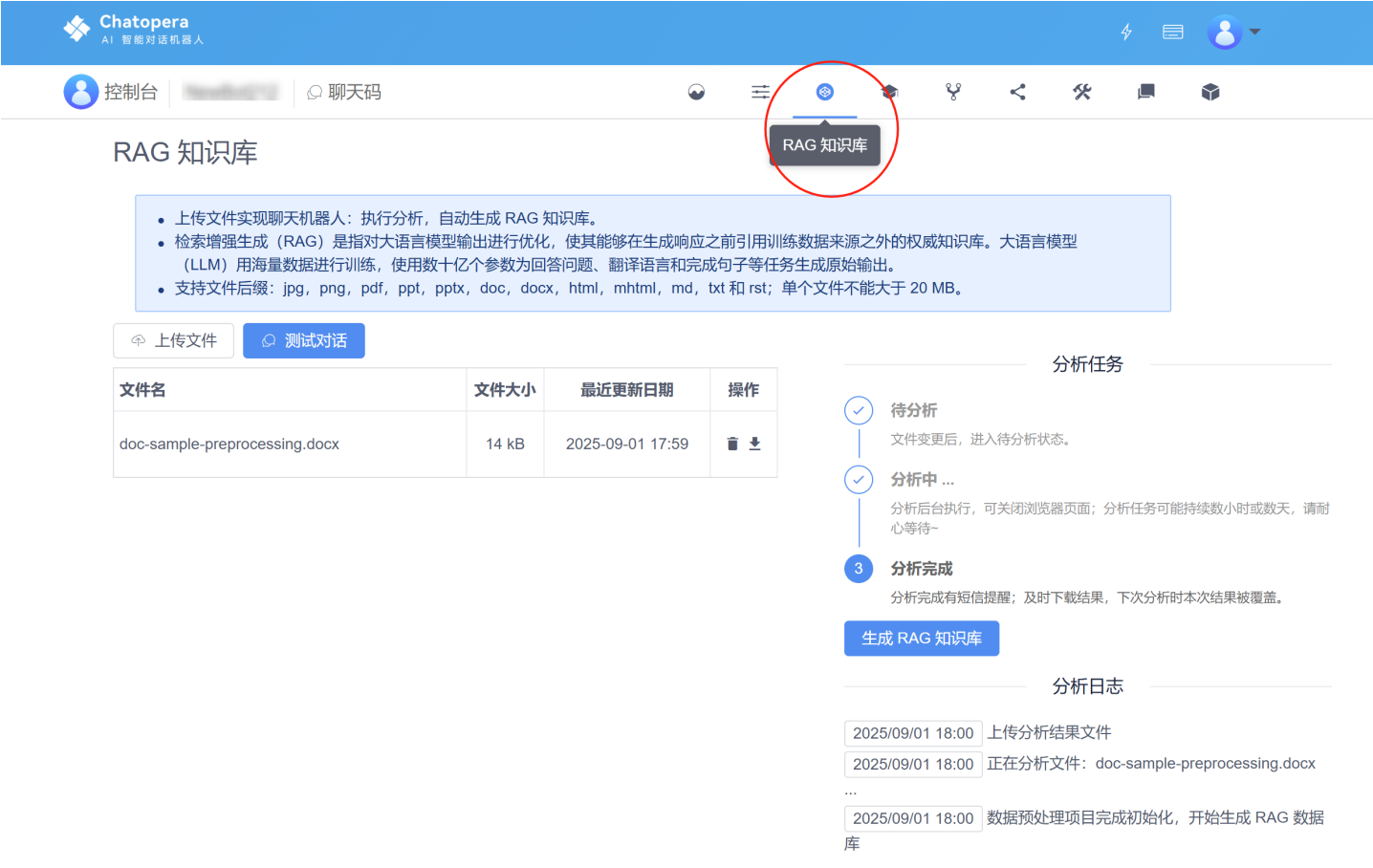
- For specific use of CDE, refer to: [Introductory Tutorial](#)
- When the Chatbot is released, it can be integrated into other software using SDK/API, reference: [Chatopera SDK and APIs](#)

RAG 检索增强

RAG (Retrieval-Augmented Generation) ，检索增强服务，是利用大语言模型的语义理解能力，处理用户上传的文档，生成回复内容的服务。

Chatopera 云服务内置了 RAG 服务，支持用户上传多种格式的文件，生成 RAG 知识库问答。

在创建好的机器人中，进入【RAG 知识库】，并上传文件，生成知识库。



在聊天机器人的多轮对话中，会调用 RAG 知识库，并利用其产生的回答回复给访客。具体过程参考：[多轮对话的检索](#)。

SDK / API 使用

在机器人的【RAG 知识库】上传文件，并生成 RAG 知识库后，机器人的[多轮对话检索](#)就会从 RAG 中，生成答案。

机器人开发者同时也可以调用 RAG 知识库检索，单独的使用 RAG 检索服务，比如，通过 Chatopera NodeJS SDK, 下面位示例代码：

```
const chatbot = new Chatbot(clientId, clientSecret, botProvider);
let method = "post";
let path = "/rag/query";
let payload = {
  query: "海南有几个机场"
};
let resp = await chatbot.command(method, path, payload);
```

详细示例代码：<https://github.com/chatopera/chatopera-nodejs-sdk/blob/master/test/rag.test.js>

更多 API 使用介绍，参考 [对话检索](#)。

修改 RAG 知识库提示词模板

提示词模板

每个机器人，都可以独立的设置提示词模板。提示词是 Chatopera 机器人在和 LLM 大语言模型交互时，发送的文本内容。

提示词模板，是将机器人角色、答案约束、关联文档等融合成为提示词的模板。比如，默认的中文的提示词模板是：

1. 使用下面的上下文信息回答末尾的问题

2. 如果你不知道，就说 "I_DONT_KNOW"，不要胡说。

3. 让答案尽量简洁，只用3、4个句子回答问题。

上下文信息：

{context}

问题：{question}

答案：

其中，有两个固定的占位符：

- `{context}` - RAG 召回的关联文档分片。Chatopera 机器人平台 RAG 模块，将上传的文档按照每 300 个字切分为一个分片；
- `{question}` - 对话用户发送的文本。

修改提示词模板

作为 Chatopera 机器人开发者，可以修改提示词模板，在 RAG 模块中，有修改【提示词模板】的按钮。

 控制台

 阿Q

 聊天码



RAG 知识库

• 上传文件实现聊天机器人：执行分析，自动生成 RAG 知识库。

• 检索增强生成（RAG）是指对大语言模型输出进行优化，使其能够在生成响应时（LLM）用海量数据进行训练，使用数十亿个参数为回答问题、翻译语言和完成

• 支持文件后缀：jpg, png, pdf, ppt, pptx, doc, docx, html, mhtml, md, t

📁 上传文件

✍️ 提示词模板

💬 测试对话

文件名	文件大小	最近更新日期
暂无数据		

在修改提示词时，有以下注意事项：

- 在模板中，必须包含 `{context}` 和 `{question}`

58/290

https://bot.chatopera.com

- 在模板中，比如包含 如果你不知道，就说"I_DONT_KNOW"，不要胡说。 这个约定，因为 Chatopera 机器人平台将会使用这个字符串判断机器人是否找到答案；在没有找到答案的时候，会在机器人的对话技能中，继续挖掘答案。



在了解了注意事项后，您可以灵活的设计提示词模板，比如：

假设你是一个小说家，擅长写科幻小说，请根据以下要求写一篇新的小说

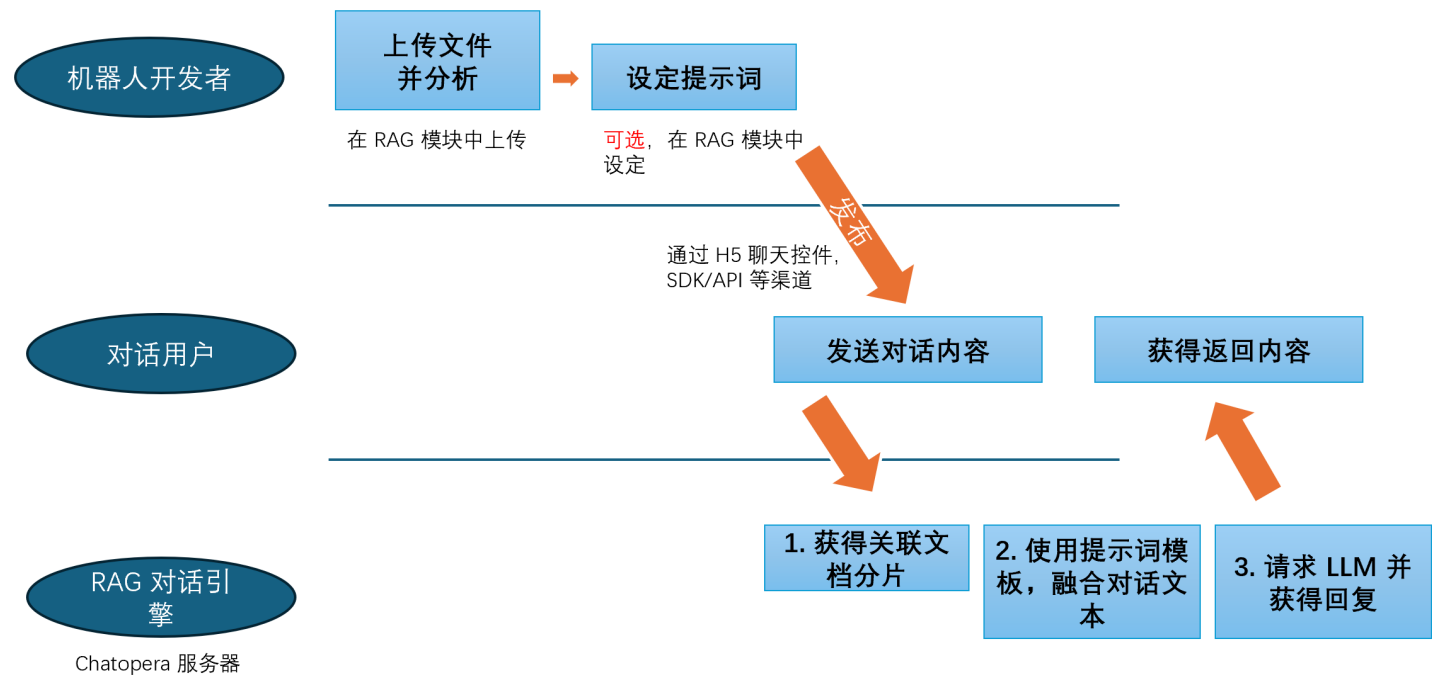
1. 如果你不知道，就说"I_DONT_KNOW"，不要胡说。

2. 行文风格，尽量模仿鲁迅

上下文信息：
{context}
小说情节概述： {question}
答案：

RAG 模块工作模式

为了进一步的帮助您使用 Chatopera 机器人平台 RAG 服务，以下是 Chatopera RAG 模块的工作原理：



知识库问答对维护

能回答问题的前提要了解**问题**是什么。比如用户问：土豆多少钱一斤？如果知识库中没有这个问题，机器人就无法回复给用户，我们需要把这个用户的**输入**添加到知识库中，下次用户再问这个问题，机器人就可以把答案回复给用户。**问题**就是预期用户会发送的文字，机器人使用**答案**回复。

问题、相似问题和答案组成了**问答对**。建立问答对时，支持添加多个**相似问题** - 不同的用户说法不同，但都是同一个意思，相似问也被称为**扩展问**，二者表达同一个概念。相似问题和**自定义词典**大大加强了问答对适应用户各种问法的能力，自定义词典支持维护近义词。下面进行详细介绍。

新建问答对

在知识库页面点击「+」

新建问题

问题 *

选择分类

请选择

相似问题

+ 增加

答案

答1 ×

增加

文本

HTML

图文

确定

关闭

在机器人管理面板中，还有其它地方可以创建问答对，比如历史记录页面点击「+」创建问答对。这些创建或修改问答对的快捷方式提升了维护知识库的便捷性。

设定答案

问答对支持多答案

针对用户业务的特点，每个问答对中，可以包含 1-10 个答案。多答案的设定，对于**业务系统集成**知识库更友好。这是因为，业务系统将保持更大的灵活性，选择最合适的策略推送给来访者。

“业务系统”是一个宽泛的概念，因为知识库是通用的，比如电商、教育等行业都可以使用，在 Chatopera 机器人平台被来访者访问到前，通常会有一个集成渠道或处理业务逻辑的软件，这个软件统一的称之为“业务系统”。

比如，电商类的问题，有一个鞋帽类产品介绍，包含了多个图片，那么知识库中关联多个图文消息，每个图文消息可以每隔 3 秒钟发送一个，消费者收到消息时，对缩略图都形成印象。

又比如，客户端涉及**答案 1**推送给微信渠道用户，**答案 2**推送给微博渠道用户，多答案的设定，可以满足更多复杂需求。

以上举例只是为了说明多答案给应用知识库带来的灵活性，同时，知识库问答对支持多种答案类型。

纯文本

以一段文本内容作为答案。步骤：新建问题页面点击「增加」下拉框，选择「文本」

富文本

使用富文本编辑器，最终答案将保存为 HTML 格式。

步骤：新建问题页面点击「增加」下拉框，选择「HTML」

新建问题

问题 *

西红柿多少钱一斤

选择分类

分类一级

相似问题

+ 增加

番茄多少钱一斤

答案

答1

答2 ×

增加 ▾

B I U A A 14px

1. 西红柿5元一斤

文本

HTML

图文

确定

关闭

图文消息

由标题、描述、图片和超链接组成的答案，适合展示为缩略图并配合文字，来访者点击后打开超链接。

62/290

https://bot.chatopera.com

新建问题

问题 *

西红柿多少钱一斤

选择分类

分类一级

相似问题

+ 增加

番茄多少钱一斤

×

答案

答1

答2

答3 ×

增加 ▾

标题 *

文本

HTML

图文

URL *

描述

确定

关闭

机器人置信度

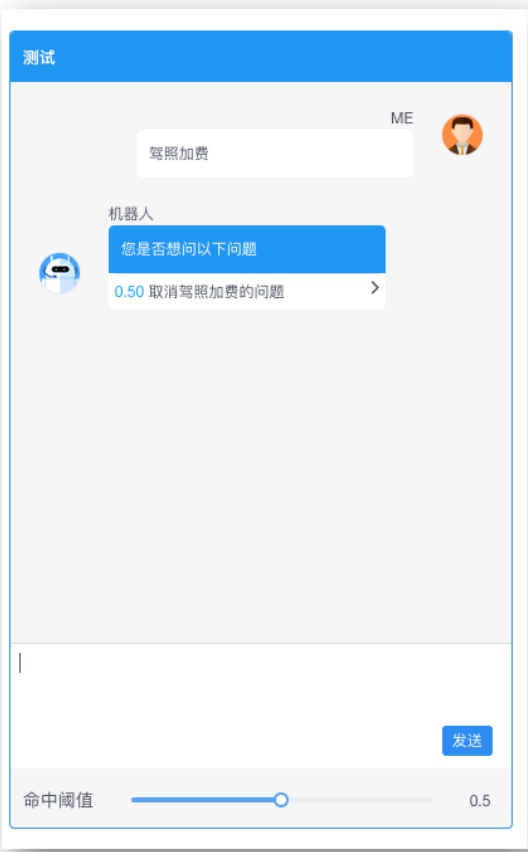
设置问答对后，知识库能回答哪些问题了呢？如果用户问法不一样，机器人的“理解”能力怎么样呢？

置信度

机器人在检索知识库时，有一个判断来访者输入的问题和知识库中哪个问答对最匹配的时候，使用“置信度”为衡量标准。置信度score（也称为命中域值、相似度），它的值在[0~1]区间，值越大代表机器人越确定回复的准确性。

最佳回复和建议回复

当置信度高于某数值时，机器人就认为置信度最高的答案匹配上了来访者的问题，可以将该问答对的答案回复给来访者，该数值称为**最佳回复阈值**；当没有问答对的相似度高于最佳回复阈值，但是有一些问答对可以作为建议回复发送给来访者，展示为“您是想咨询一下问题吗”，然后列一个列表，请来访者选择，那么可以进入这个列表的问答对，需要高于某数值，该数值称为**建议回复阈值**。



对知识库的评测，常常用一系列测试问题查询返回结果，进而计算机器人回答的准确率评价检索系统。调试结果可以指导合理设置最佳回复阈值和建议回复阈值，这两个值对于业务系统集成知识库具有很重要的意义。

在熟悉了置信度的概念以后，就可以开始调试知识库了。

知识库问答及调优测试

测试知识库问答

在机器人管理面板，通过测试对话页，测试知识库。

步骤：测试对话页面输入问题，查看机器人的回复

测试对话

知识库

意图识别

多轮对话

测试

ME

西红柿多少钱一斤

机器人

您是否想问以下问题

1.00 西红柿多少钱一斤

0.95 西红柿

发送

命中阈值 0.5

```
[
  {
    "id": "AXNXDVd8zLtU6n9zKj7C",
    "score": 1,
    "post": "西红柿多少钱一斤",
    "replies": [
      {
        "rtype": "plain",
        "enabled": true,
        "content": "5元一斤"
      }
    ]
  },
  {
    "id": "AXNXDdQdzLtU6n9zKj7D",
    "score": 0.95,
    "post": "西红柿",
    "replies": [
      {
        "rtype": "plain",
        "enabled": true,
        "content": "5元"
      }
    ]
  }
]
```

快速编辑问答对

如果机器人回复的问题命中阈值不高，可以直接点击问题，进行编辑

测试对话

知识库

意图识别

多轮对话

测试

ME

西红柿多少钱一斤

机器人

您是否想问以下问题

1.00 西红柿多少钱一斤

0.95 西红柿

编辑问答对

发送

如果有大量而频繁的评价知识库准确率的需求，可以使用 [系统集成/SDK](#) 实现。

知识库问答对导入和导出

导入和导出是对知识库的数据进行批量管理，导出的数据可以导入到其它机器人知识库中，导出的内容包括知识库分类信息、标准问、相似问和答案和启用状态。

应用场景

1) 备份机器人知识库； 2) 新建机器人快速导入。

本文以 Chatopera 机器人管理控制台为工具进行说明，如果想通过更自动化的方式，参考[使用 CLI 导入和导出对话语料](#)。

批量导出

步骤：知识库页面点击「导出」

+

问题

选择分类：

请选择

搜索

批量导入

导出

下载示例数据

分类管理

问题	是否启用	操作
语文好的进来！ 所属分类: 分类一级	启用	编辑 删除
阜阳卫校怎么走？坐几路公交？阜阳市阜临路511号 所属分类: 分类一级	启用	编辑 删除
黑道圣徒2怎么取消武器的秘籍 所属分类: 分类一级	启用	编辑 删除

文件导出格式为 json 格式，json 是一种节省空间、灵活的、方便用文本编辑器处理和代码处理的流行格式。

批量导入

注意事项：

- 导入的知识库仅支持 json 格式，内容定义参考[示例数据](#)
- 点击「下载示例数据」之后，可以对示例数据进行编辑保存，再导入到知识库
- 导入时可设置问题的状态，问题状态为「禁用」在测试知识库时检索不到

导入

选择文件

100条.json

请添加格式为json

状态	问题
启用	一次泡5根苦丁茶多吗
启用	阜阳卫校怎么走？坐几路公交？阜阳市阜临路511号
禁用	语文好的进来！
禁用	黑道圣徒2怎么取消武器的秘籍
启用	想给宝宝取个名字
启用	围墙围成面积为216平方米的一成矩形的土地，在正中用一堵墙隔成两块，问土地的长和宽选多大，材料最省？
启用	翻译英语

上传

取消

自定义词典增强知识库

近义词是另一种增加机器人智能化水平的手段。比如：

创建知识库问答对，标准问：“土豆多少钱一斤”。不添加相似问题。此时，用户问：“马铃薯多少钱一斤”。

这个时候机器人处理“马铃薯多少钱一斤”时，知识库匹配不到置信度很高的问题。

那么，怎样才能让机器人的回复与“马铃薯多少钱一斤”一样的答案呢？因为它们可是同一个意思啊。

这种情况下就需要配置近义词，我们找到包含「土豆」词条的自定义词典，为土豆添加近义词「马铃薯」，然后再去测试知识库，机器人的回复与“马铃薯多少钱一斤”一样的答案。

自定义词典定义

自定义词典就是用户添加自己的词典，支持创建：词汇表和正则表达式两种类型的词典。自定义词典通常用来管理近义词，新词，专有名词。

提示：知识库中应用自定义词典只支持词汇表类型的自定义词典。

创建自定义词典

创建自定义词典步骤：词典管理页面点击「新建自定义词典」词典类型选择「词汇表」

新建自定义词典

×

词典标识名一经确定后将无法进行修改。

★ 词典标识名

词典标识名为字母、数字、下划线的组合，1-32位，如：Boc

i

★ 词典类型

词汇表

▼

取消

确定

增加词条

添加词条步骤：自定义词典列表点击「编辑」进入添加词条页面

- 开发者可以根据业务需要为自己创建的自定义词典添加多个词条。
- 一个词条包含至少一个标准词和不限个数的同义词，同义词会被识别为标准词。
- 词条用于知识库和意图识别

添加词条

请使用分号：分隔词项，第一个词项将作为标准词，后面的词项将作为同义词

添加

按Enter键快速添加

词条已变更，为保证词条生效

更新知识库索引

训练意图识别模型

输入词项进行搜索

标准词	同义词	操作
西红柿	番茄	编辑 删除

<

1

>

保存

取消

知识库同步自定义词典

在保存了自定义词典后，知识库并没有立即应用变更的自定义词典，近义词并没有立刻生效。只有在自定义词典管理页面或知识库管理页面，点击“更新知识库索引”后，知识库才会同步自定义词典，近义词才会生效。

步骤：知识库页面点击「更新知识库」



验证近义词

“番茄多少钱一斤”有结果了！

用户的问题中包含有「西红柿」的同义词「番茄」，机器人可以识别到。

测试对话

知识库意图识别多轮对话

测试

ME

番茄多少钱一斤

机器人

您是否想问以下问题

1.00 西红柿多少钱一斤

0.95 西红柿

发送

命中阈值 0.5

```
[
  {
    "id": "AXNXDVd8zLtU6n9zKj7C",
    "score": 1,
    "post": "西红柿多少钱一斤",
    "replies": [
      {
        "rtype": "plain",
        "enabled": true,
        "content": "5元一斤"
      }
    ]
  },
  {
    "id": "AXNXDdQdzLtU6n9zKj7D",
    "score": 0.95,
    "post": "西红柿",
    "replies": [
      {
        "rtype": "plain",
        "enabled": true,
        "content": "5元"
      }
    ]
  }
]
```

近义词，新词，专有名词都可以通过自定义词典进行维护，维护好自定义词典对机器人智能化水平有着至关重要的作用。

热门问题评分是知识库中的问答对的相对热门程度。问题的热门度是一个分数，分数值代表某一个问答对作为回复结果出现在客户端的次数，可能是作为最佳答案，也可能是作为建议答案。

查看问答对热门度

知识库

下载热门问题

72/290

设置欢迎语热门问题

欢迎语热门问题，就是进入 Chatopera H5 网页渠道或机扫一扫聊天码以后，进入访客对话窗口后，机器人发送的打招呼信息包含的热门问题列表。



使用多轮对话设计器，可以绑定欢迎语中的热门问题，有两种形式：

- 自动关联热门问题
- 手动关联热门问题

完成入门教程

阅读和理解以下内容，需要您先完成入门教程：[进入链接](#)。

自动关联热门问题

在机器人平台的知识库，每个问答对会根据被检索出现的次数计算热度，被检索出的次数越高，热度越高。

为了支持在欢迎语中，展示热门问题列表，可以用下面的【脚本】和【函数】，根据知识库问题热度，动态的获取。

- 脚本：在脚本中，添加如下内容

```
// FAQ 热门问题展示
+ __faq_hot_list
- {keep} ^getGreetings()
```

- 函数：在函数中，添加如下内容

```
// 问候语中关联常见问题
// 更多消息格式，参考 https://dwz.chatopera.com/jQ0F9G
exports.getGreetings = async function() {
  let data = await this.maestro.getHotFAQs();
  debug("getHotFAQs %j", data)

  if (data.length > 0) {
    let params = [];
    let postIndex = 0;
    for (let x of data) {
      // 跳过 FAQ 中约定的内部命令
      if (!x["post"].startsWith("__")) {
        params.push({
          label: (++postIndex).toString() + ". " + x["post"],
          type: "qlist",
          text: x["post"]
        });
      }
    }

    if (params.length > 0) {
      return {
        text: "请问有什么可以帮到您？",
        params: params
      };
    } else {
      return {
        string: ""
      };
    }
  } else {
    return {
      string: ""
    };
  }
}
```

在多轮对话设计器中保存，并进行测试。

手动关联热门问题

手动设置热门问题作为欢迎语，就是通过编辑【脚本】和【函数】，固定欢迎语中的热门问题。

首先，在多轮对话设计器中，打开刚刚添加的机器人。



然后点击， greetings 话题对应的【编辑】按钮。



接下来，点击【函数】，进入函数编辑窗口。

Chatopera 多轮对话设计器

dit Window Help

Bot123 话题: greetings ▾

新建话题 环境变量 绑定信

多轮对话 返回值

对话

脚本 函数

```
1 // 问候语中关联常见问题
2 // 更多消息格式, 参考 https://dwz.cl
3 exports.getGreetings = async function() {
4   let data = await this.maestro.getHotFAQs();
5   debug("getHotFAQs %j", data);
6
7   if (data.length > 0) {
8     let params = [];
9     let postIndex = 0;
10    for (let x of data) {
11      // 跳过 FAQ 中约定的内容
12      if (!x["post"].startsWith("FAQ:")) {
13        params.push({
14          label: (++postIndex).toString(),
15          type: "q1list",
16          text: x["post"]
17        });
18      }
19    }
20
21    if (params.length > 0) {
22      return {
23        text: "请问有什么可
```

鼠标左键点击函数区域，使光标作用域函数区域，如下图：



已启用话题

回退到历史版本

脚本

函数

```
1 // 问候语中关联常见问题
2 // 更多消息格式, 参考 https://dwz.chatopera.com/jQ0F9G
3 exports.getGreetings = async function() {
4     let data = await this.maestro.getHotFAQs();
5     debug("getHotFAQs %j", data)
6
7     if (data.length > 0) {
8         let params = [];
9         let postIndex = 0;
10        for (let x of data) {
11            // 跳过 FAQ 中约定的内部命令
12            if (!x["post"].startsWith("__")) {
13                params.push({
14                    label: (++postIndex).toString() + ".
15                    type: "qlist",
16                    text: x["post"]
17                });
18            }
19        }
20
21        if (params.length > 0) {
22            return {
23                text: "请问有什么可以帮到您? ",
24                params: params
25            };
26        }
27    }
28 }
```

然后进行全选: 1) Windows 执行 Ctrl + A; 2) Mac 执行 Command + A。

全选后删除: 使用键盘的 Delete 或 Backspace 键。这样, 就清空了函数中的内容, 类似如下:

新建话题

环境变量

绑定信息

上



已启用话题

回

清空

脚本

函数

1

下一步，复制下面的函数到函数编辑窗口。

```
// 问候语中关联常见问题
// 更多消息格式，参考 https://dwz.chatopera.com/jQ0F9G
exports.getGreetings = async function() {
  let data = [
    "自定义1",
    "自定义2",
    "自定义3",
  ];

  if (data.length > 0) {
    let params = [];
    let postIndex = 0;
    for (let x of data) {
      params.push({
        // 展示的文本
        label: (++postIndex).toString() + ". " + x,
        type: "qlist",
        // 点击后，发送给机器人的文本
        text: x
      });
    }

    if (params.length > 0) {
      return {
        text: "请问有什么可以帮到您？",
        params: params
      };
    } else {
      return {
        string: ""
      };
    }
  } else {
    return {
      string: ""
    };
  }
}
```

按照需要，调整 `data` 数组元素即可设置固定的欢迎语热门问题。

比如，希望固定为问题：1. 发送课程资料；2. 发送课程价格。那么，`data` 设置为：

```
let data = [
  "发送课程资料",
  "发送课程价格",
];
```

点击【保存】，类似下图：

新建话题

环境变量

绑定信息

上传

下载

管理控制台

帮助

已启用话题

回退到历史版本

保存

脚本

函数

Me

1 // 问候语中关联常见问题

2 // 更多消息格式, 参考 <https://dwz.chatopera.com/jQ0F9G>

3 exports.getGreetings = async function() {

4 let data = [

5 "自定义1",

6 "自定义2",

7 "自定义3",

8];

9

10 if (data.length > 0) {

11 let params = [];

12 let postIndex = 0;

13 for (let x of data) {

14 params.push({

15 // 展示的文本

16 label: (++postIndex).toString() + ". " + x,

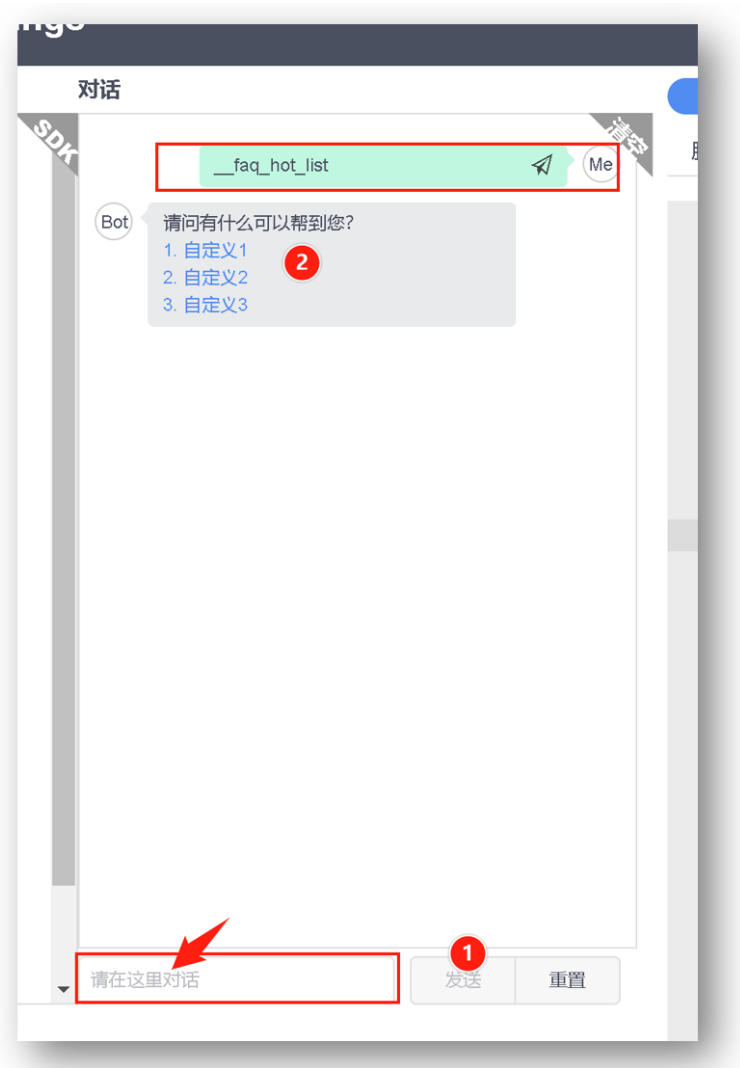
17 type: "qlist",

18 // 点击后, 发送给机器人的文本

19 text: x

这时, 在中间的测试对话窗口, 发送:

__faq_hot_list



这时，设置完成。

使用函数获得动态答案

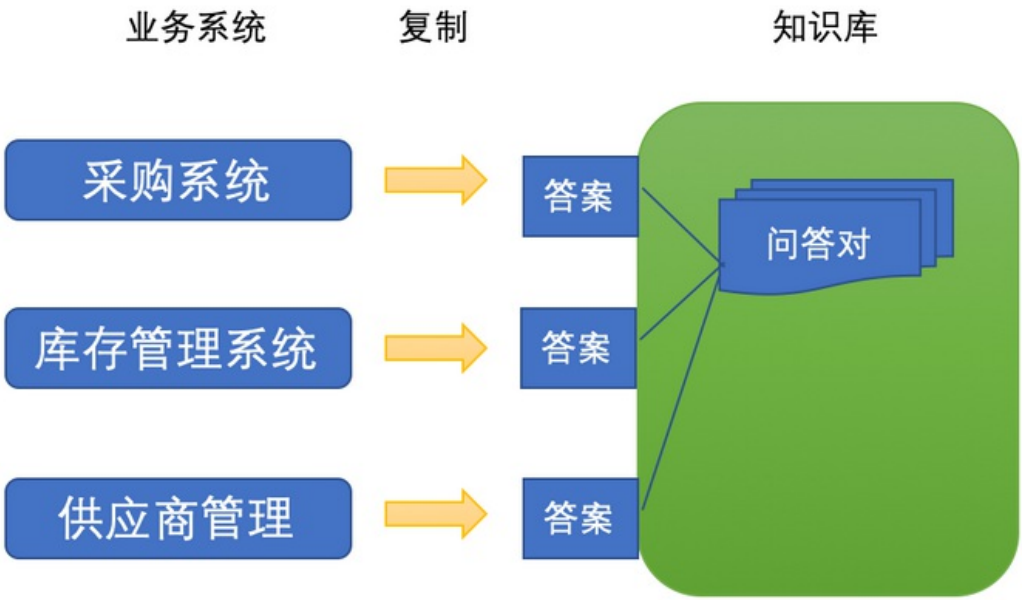
目标

通过使用动态答案，联合多轮对话与知识库的整体使用，使机器人更智能。

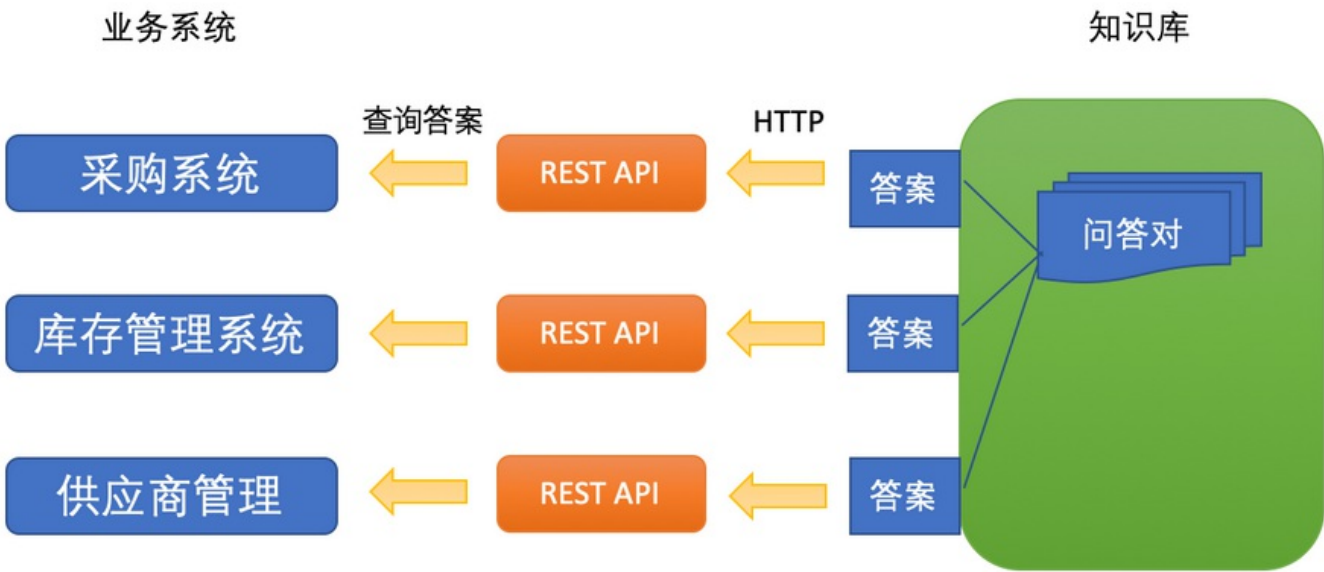
问题描述

在智能对话机器人中，知识库管理可能涉及到从业务系统获得答案，实时的同步答案一个是造成延迟，一个是成本高：在多个地方维护。比如，在业务场景中，有三个子业务系统，分别是【采购系统】/【库存管理系统】/【供应商管理】。每个系统都有一些常见问题，比如采购部门的负责人及联系方式，库存管理系统的负责人及联系方式，这些信息在不同子系统的某个网页上，或者通过 API 调用可以查询到。

在传统方案中，知识库的答案通过手动复制同步，而且这个过程几乎都需要人工完成。



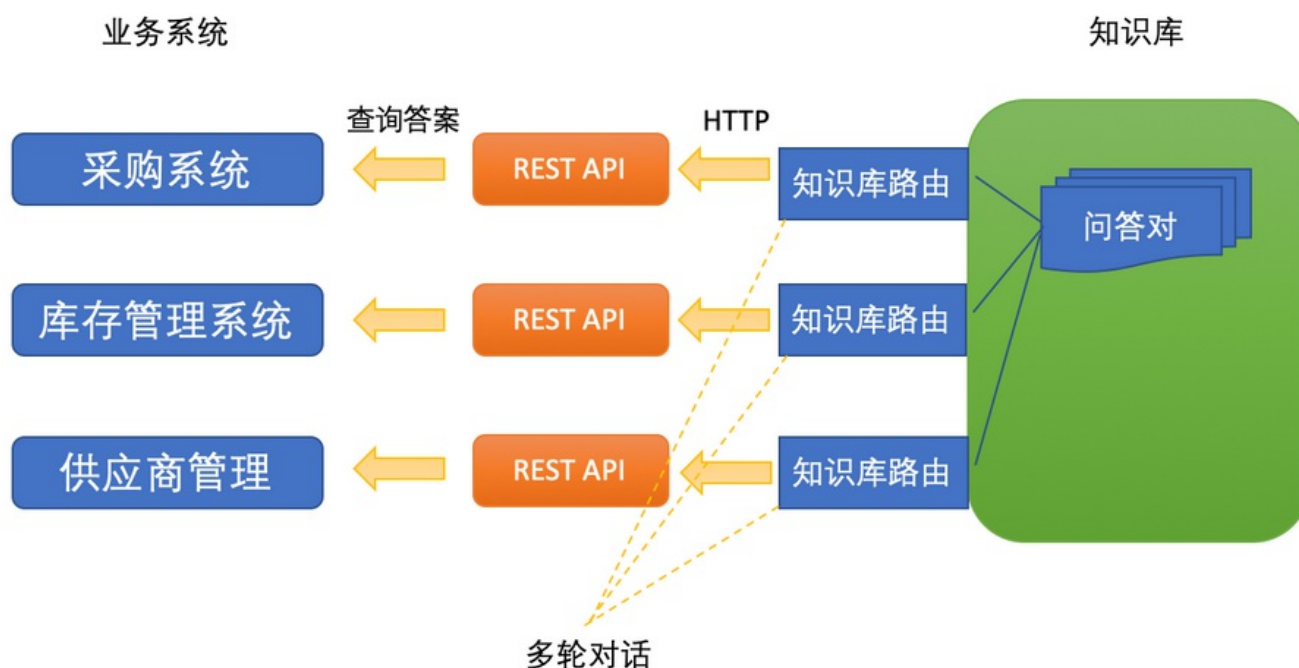
但是，因为有 HTTP API 存在，这个过程可以更简单。一个可行的方案如下：



目前，Chatopera 机器人平台支持这样的使用场景，接下来具体说明。

解决方案

在 [Chatopera 机器人平台](#) 管理控制台上 BOT 开发者可以维护知识库的不同类型的答案，除了录入固定的答案，Chatopera 机器人平台知识库也支持通过调用[多轮对话的话题](#)来从多轮对话获得答案。在多轮对话中，支持自定义 JavaScript 函数，就使得知识库设置动态抓取的答案可行，称之为：知识库路由。



知识库路由

在知识库的答案或多轮对话的函数中设置回复时，可以用 **routeDirectReply** 来指定一个话题和匹配器获得答案。

语法

将知识库答案按照如下语法设置：

```
routeDirectReply#["TOPIC_NAME", "TOPIC_GAMBIT_ID"]
```

TOPIC_NAME: [对话话题](#)

TOPIC_GAMBIT_ID: [匹配器](#)

使用示例

下面我们用一个示例介绍这个功能的使用。

1) 知识库问题 -

采购部门的负责人联系方式是什么？

2) 知识库答案 -

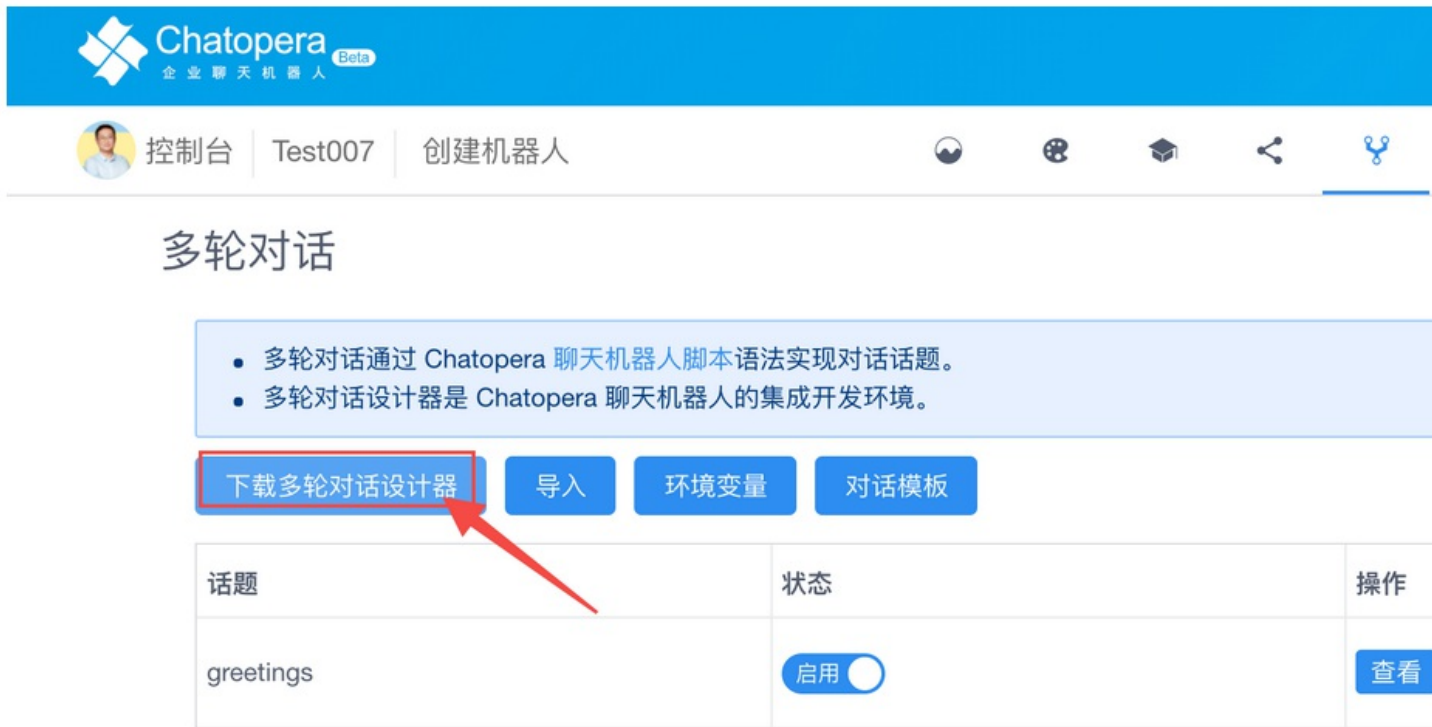
```
routeDirectReply#["__depart_purch_incharge", "__contact_info"]
```

在控制台设置如下 -



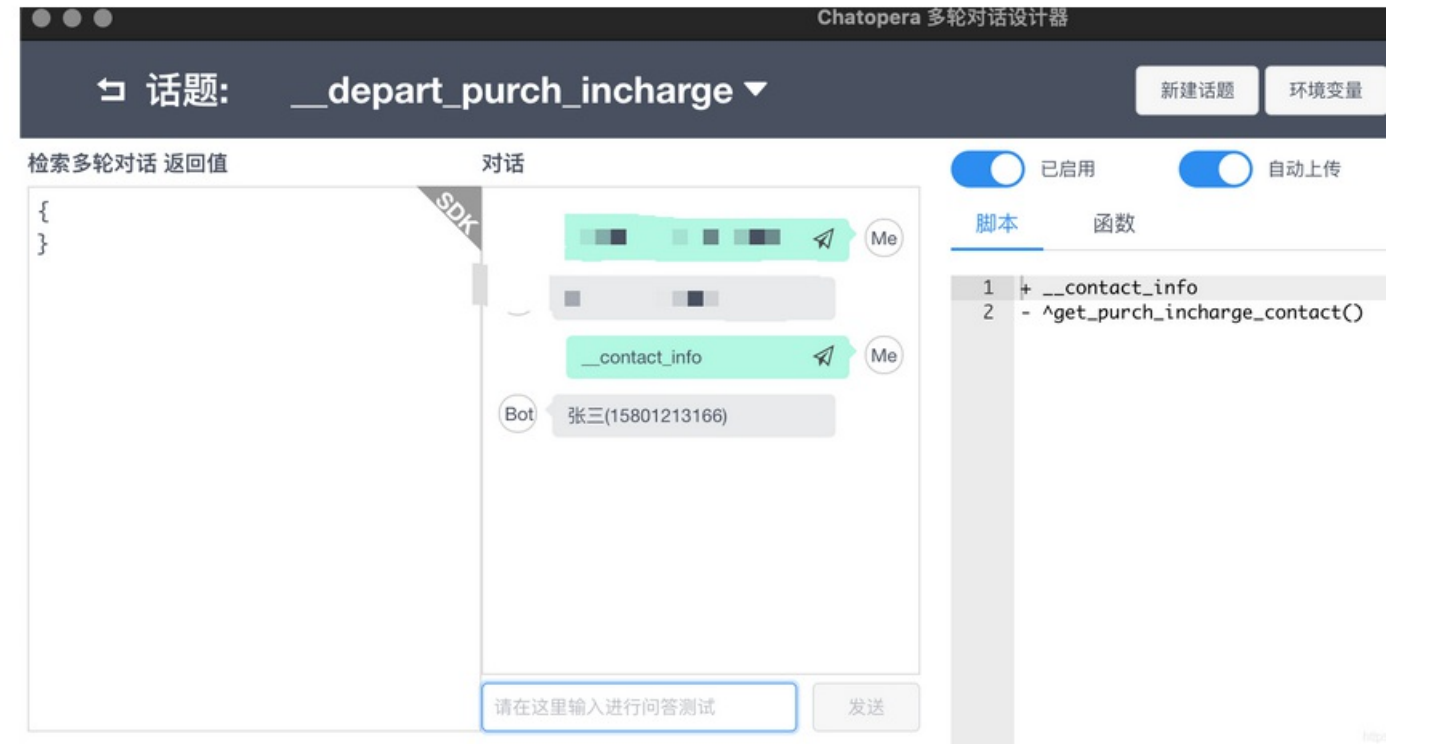
3) 在多轮对话设计器中，设置机器人的回复内容

在机器人管理控制台，打开【多轮对话】版块，点击【下载多轮对话设计器】。



安装后，启动应用并通过机器人设置页面的 clientId 和 secret 导入机器人。创建新的技能：__depart_purch_incharge 在新的技能中，添加用户说法：

```
+ __contact_info
- ^get_purch_incharge_contact()
```

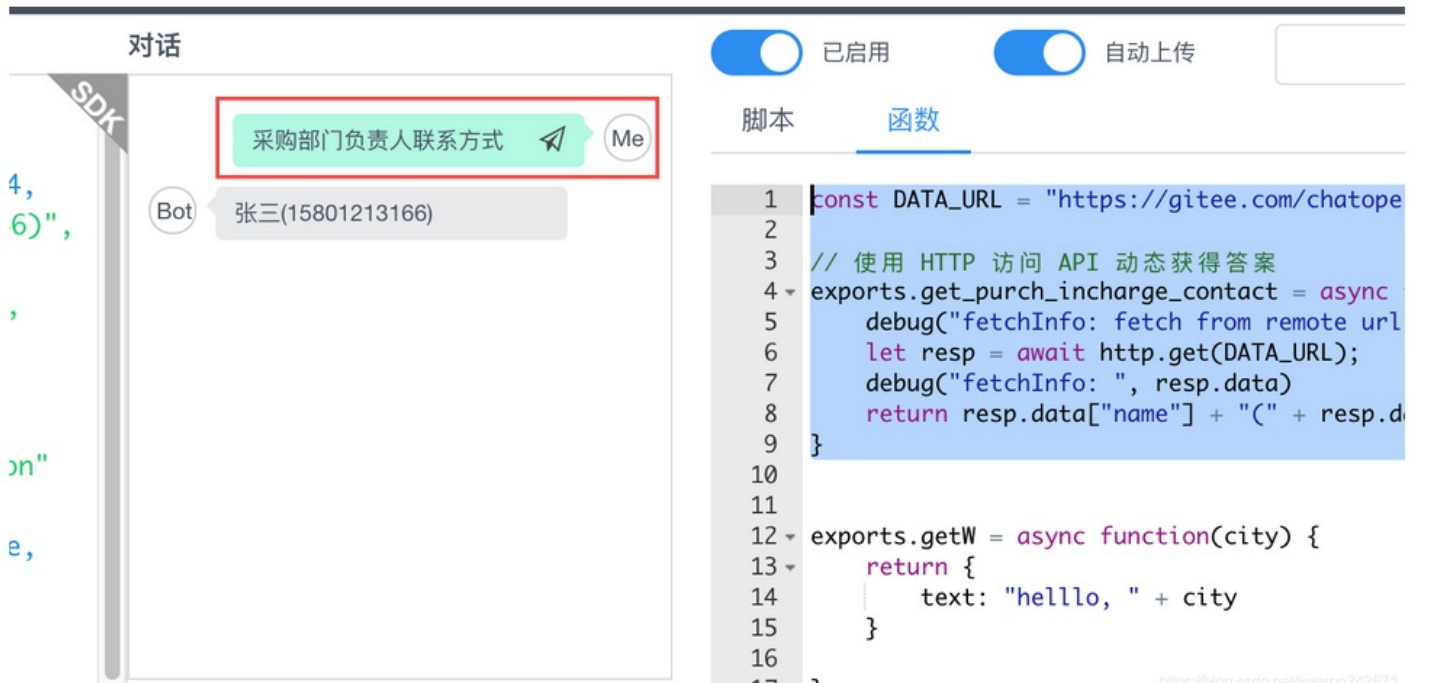



在函数中，添加新的函数。

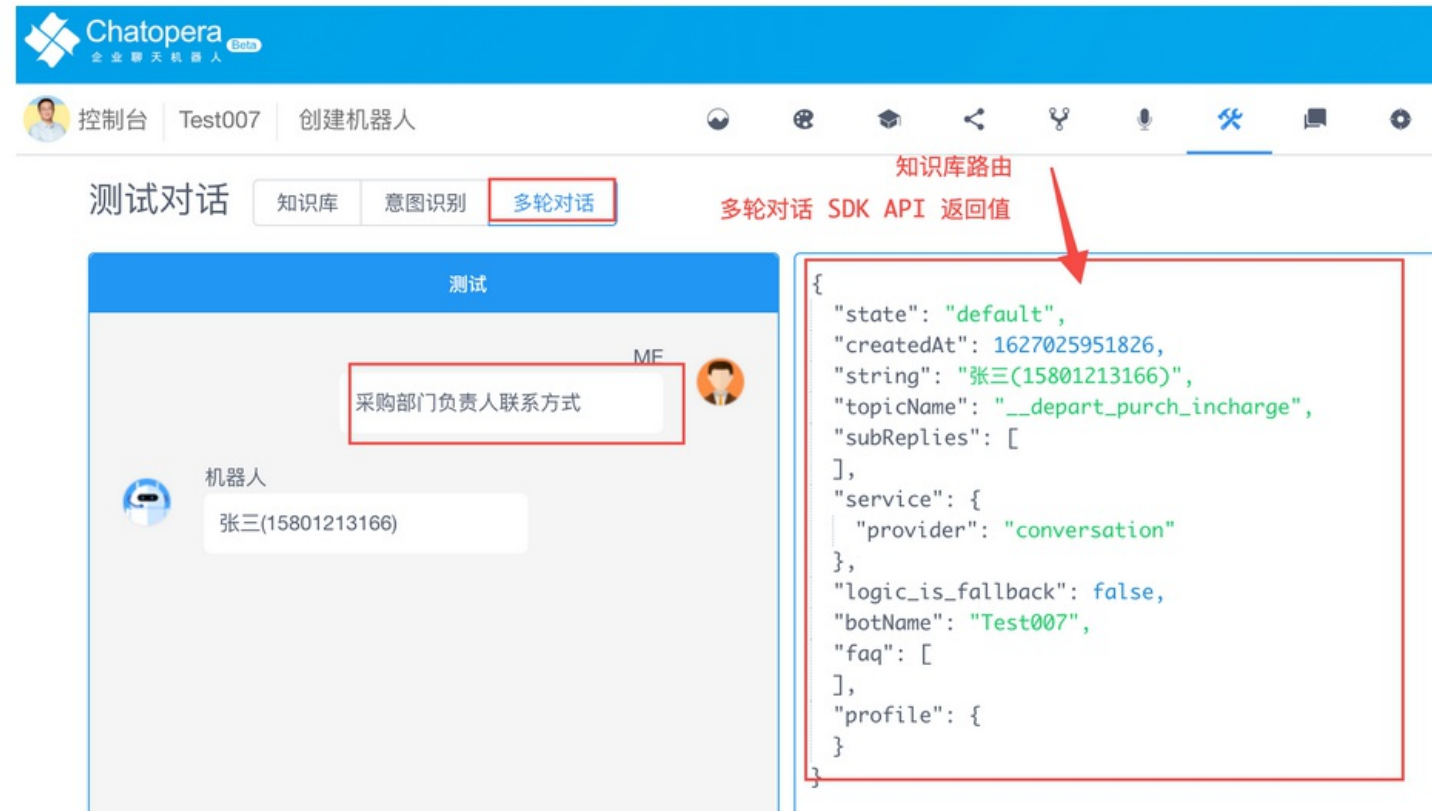
```
const DATA_URL = "https://gitee.com/chatopera/chatbot-samples/raw/master/assets/demo-contact.json"

// 使用 HTTP 访问 API 动态获得答案
exports.get_purch_incharge_contact = async function() {
  debug("fetchInfo: fetch from remote url %s ...", DATA_URL);
  let resp = await http.get(DATA_URL);
  debug("fetchInfo: ", resp.data)
  return resp.data["name"] + "(" + resp.data["phone"] + ")";
}
```

在多轮对话设计器中进行测试：



4) 在机器人控制台进行测试



下一步，就可以接入到业务系统。在返回结果中，除了使用文本的形式，还可以支持自定义的数据结构，返回结果支持展示为卡片、图文、按钮、列表的形式，详细参考[链接](#)。

高级进阶

使用用户输入作为触发器

在一些情况下，按照用户的输入进入多轮对话匹配器更方便，那么在设置路由答案时，使用如下方法。

当 `TOPIC_GAMBIT_ID` 的值为 `$ctx.textMessage$` 时，则使用当前对话的用户输入，在 `TOPIC_NAME` 中进行检索。

举例

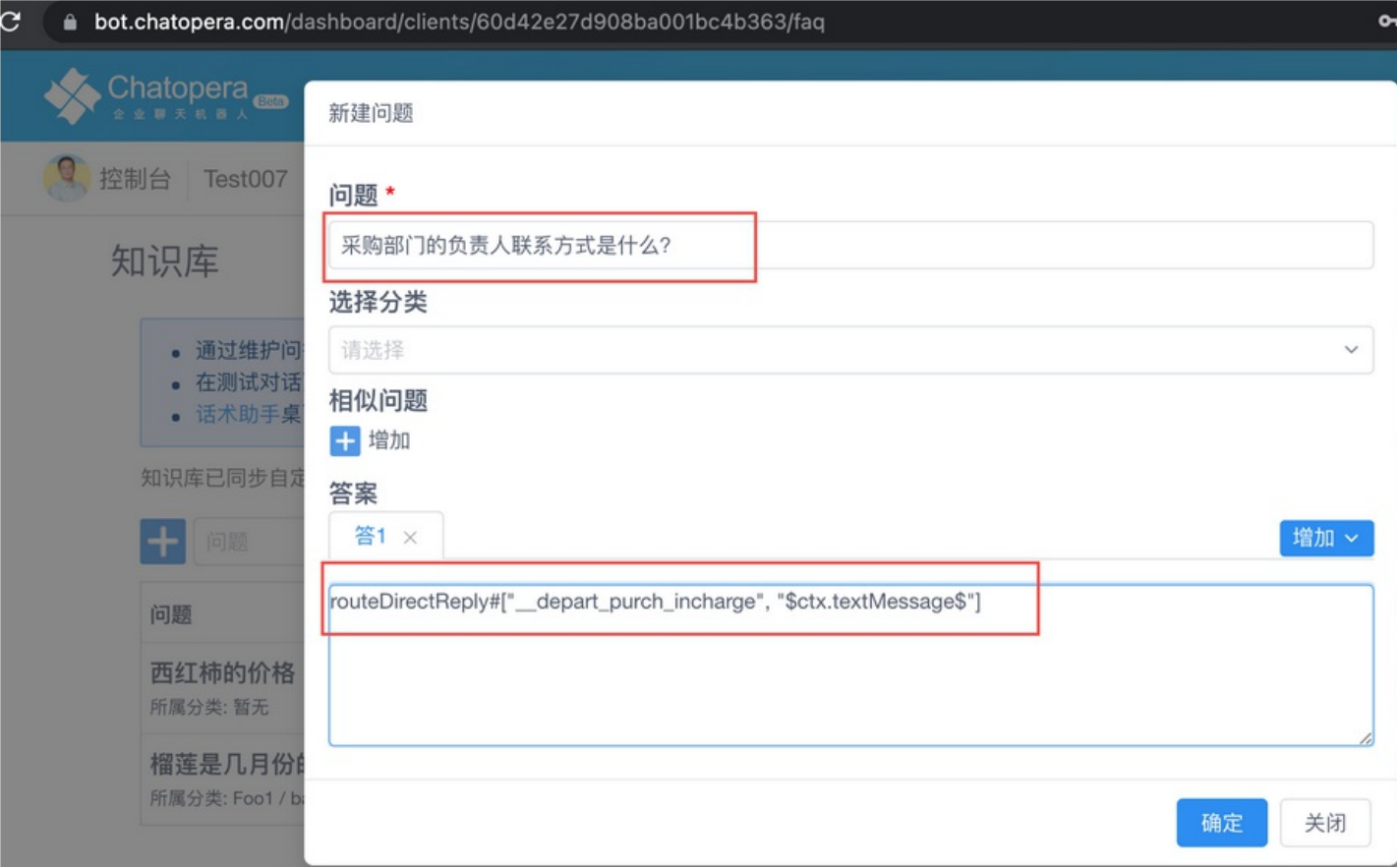
1) 知识库问题 -

采购部门的负责人联系方式是什么？

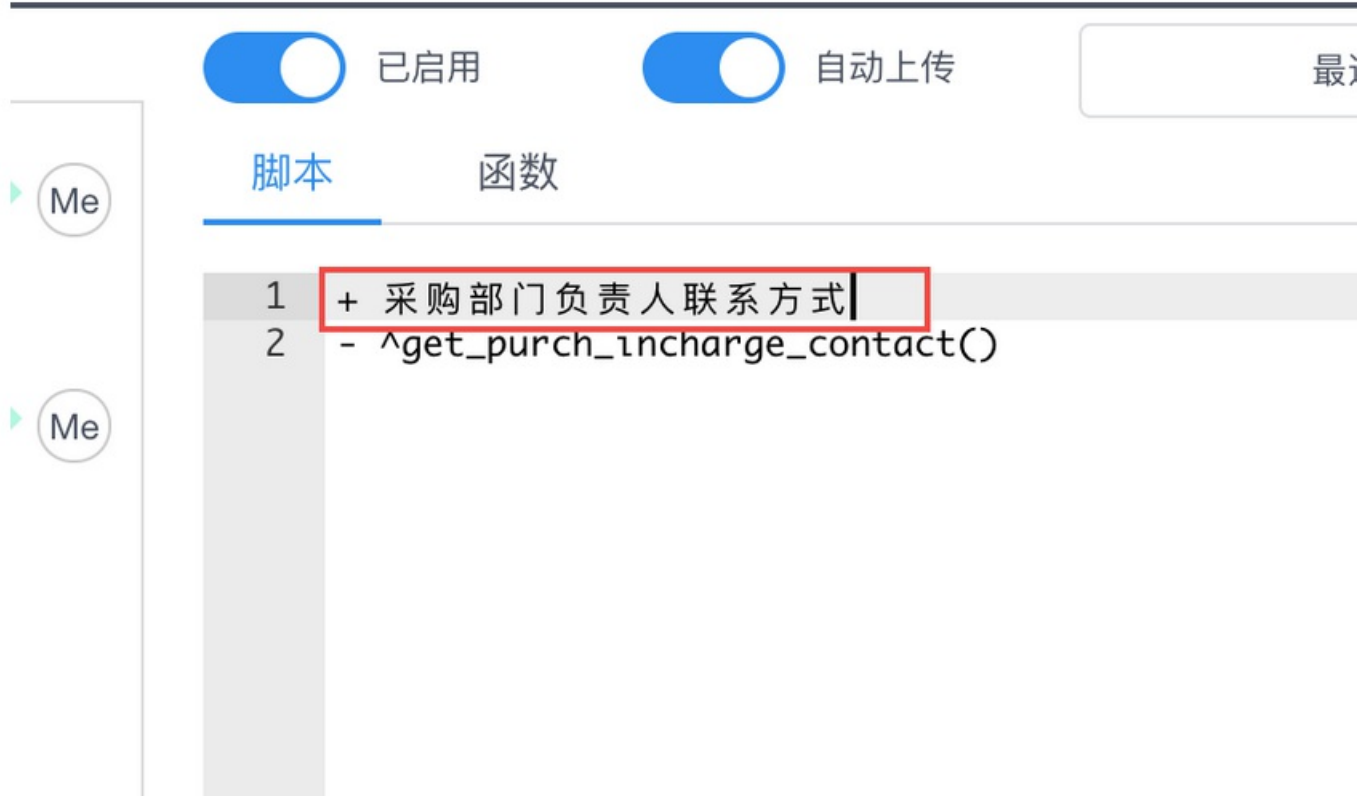
2) 知识库答案 -

routeDirectReply#["__depart_purch_incharge", "\$ctx.textMessage\$"]

在控制台设置如下 -



那么，在多轮对话设计器中，这个规则可以更新为：



快速从机器人知识库检索的输入法！

关键词：快捷语、网络聊天快捷语、网络销售快捷语、知识库快捷、企业输入法、定制知识库、聊天助手、营销话术快捷语、营销话术助手、客服话术助手



Chatopera 知识库话术助手（简称“话术助手”）是面向企业的客户服务人员发布的一款从知识库检索建议回复的桌面软件。长期以来，客户服务占据着企业运营的重要位置，因为回复不及时、不标准和不准确都会给企业造成损失，对于常见问题集的管理，主要使用的工具是 Excel，客服人员想要快速回复客户消息非常困难，同时也有不容易维护的问题，话术助手就是专门用于解决这个问题而开发的。

产品亮点

- 易于配合其他应用，通过复制面板和粘贴面板工作；
- 话术助手工作条支持输入关键词检索，提升话术、问答对检索的便捷性；
- 悬浮窗口，占用很小空间；
- 设计简洁，和桌面背景百搭。

使用教程

使用浏览器打开链接，观看视频：<https://dwz.chatopera.com/A76i8H>

下载和安装

提示：目前，话术助手只支持Windows客户端。

下载地址：

版本	操作系统	下载
v1.5	Windows 7 或更高版本	链接

在下载后，浏览器可能提示“这种类型的文件可能会损坏您的计算机”或者“Windows Defender SmartScreen 已组织启动一个未识别的应用。”，这时请点击“保留”及“更多信息”进行下一步。



配置



双击图标，启动话术助手，看到如下配置界面。

小助手

邮箱

应用ID

密钥

保存

[帮助文档](#)

版本号V1.0

Copyright © 2018 北京华夏春松科技有限公司版权所有

“邮箱”一栏填写有效的电子邮箱地址，该邮箱属于从该 PC 客户端使用 话术助手的人员，比如客服坐席。客服主管可以从 Chatopera 机器人平台的对话历史中看到该邮箱访问机器人知识库的使用情况。

- 获取应用 ID 和密钥

提示：进入 Chatopera 云服务，机器人管理控制台，设置页面，获得 Client ID 和 Secret。

安装完成后，可以在启动菜单和桌面看到“话术助手”的快捷方式。

已经具备了应用 ID 和密钥的用户跳过此步。作为团队组长/企业主管等角色使用或作为个人用途，可以通过注册Chatopera 云服务，然后创建聊天机器人获得。

Chatopera Beta

企业聊天机器人

控制台

小松

+ 创建机器人

Client Id

5c11f1256f80090017b44401

Secret

.....

刷新

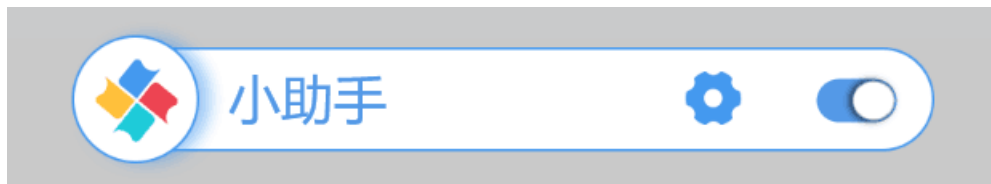
显示

这三项都属于必填项，填写好后点击“保存”。

使用 话术助手获得智能建议回复，有两种形式：1) 利用系统粘贴板；2) 在话术助手工具条里输入关键词。

复制到系统粘贴板

保存设置后，话术助手是置顶在桌面的，在话术助手的最右侧是切换*启用*和*停用*状态的按钮。



工作时，您可能处于不同的聊天软件中，甚至是文档中，都可以通过复制文本获得建议回复。比如，下面是在 QQ 群中，快速回复一个问题，就把这个问题复制到粘贴板，通常是通过【Ctrl + C】完成。

这时，话术助手就从机器人的知识库中查询相似问题，并按照*问题相似度*展开一个智能建议回复列表。点击一个相似问题，就将该问题答案复制到粘贴板，再粘贴该答案到聊天窗口，通常通过【Ctrl + V】完成。

所以，这个过程可以简单的描述为*复制*->*点击*->*粘贴*。

输入关键词

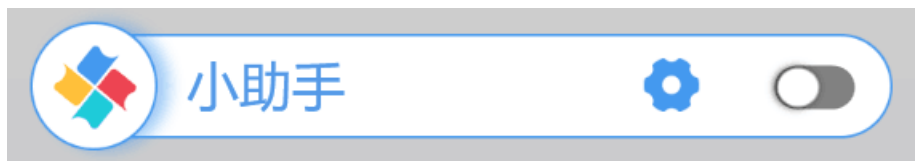
保存设置后，话术助手工具条内显示“请输入搜索关键字”，在此录入文本后，通过回车键或右侧放大镜都可以提交搜索。获得建议回复。



点击取得的答案，答案内容被复制找系统粘贴板，然后可以再粘贴到其它地方，通过【Ctrl + V】完成。

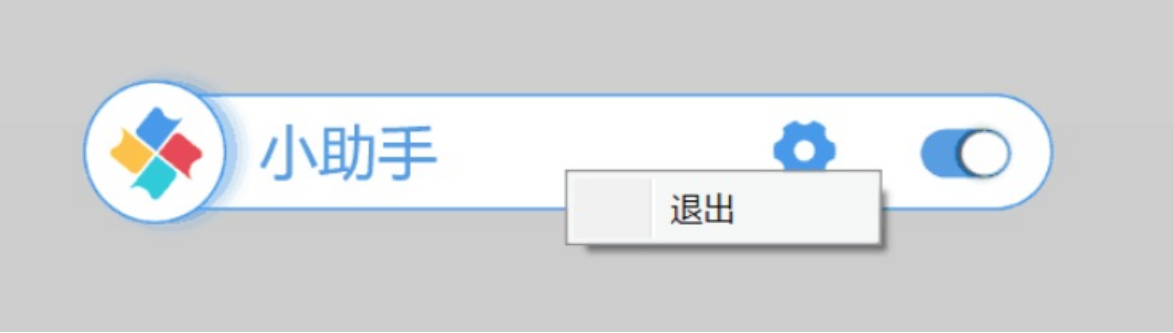
临时停用

话术助手处于*启用*状态时，每次执行文本复制操作都会做智能建议回复的查询，被复制内容会被发送到远程服务，为了保护用户隐私，您可以临时停用话术助手，这时并不执行智能建议回复，您的隐私数据也就得到保护。在任何状态，任何理由下，Chatopera 不会在未取得用户允许的情况下，获取或利用用户的隐私数据。



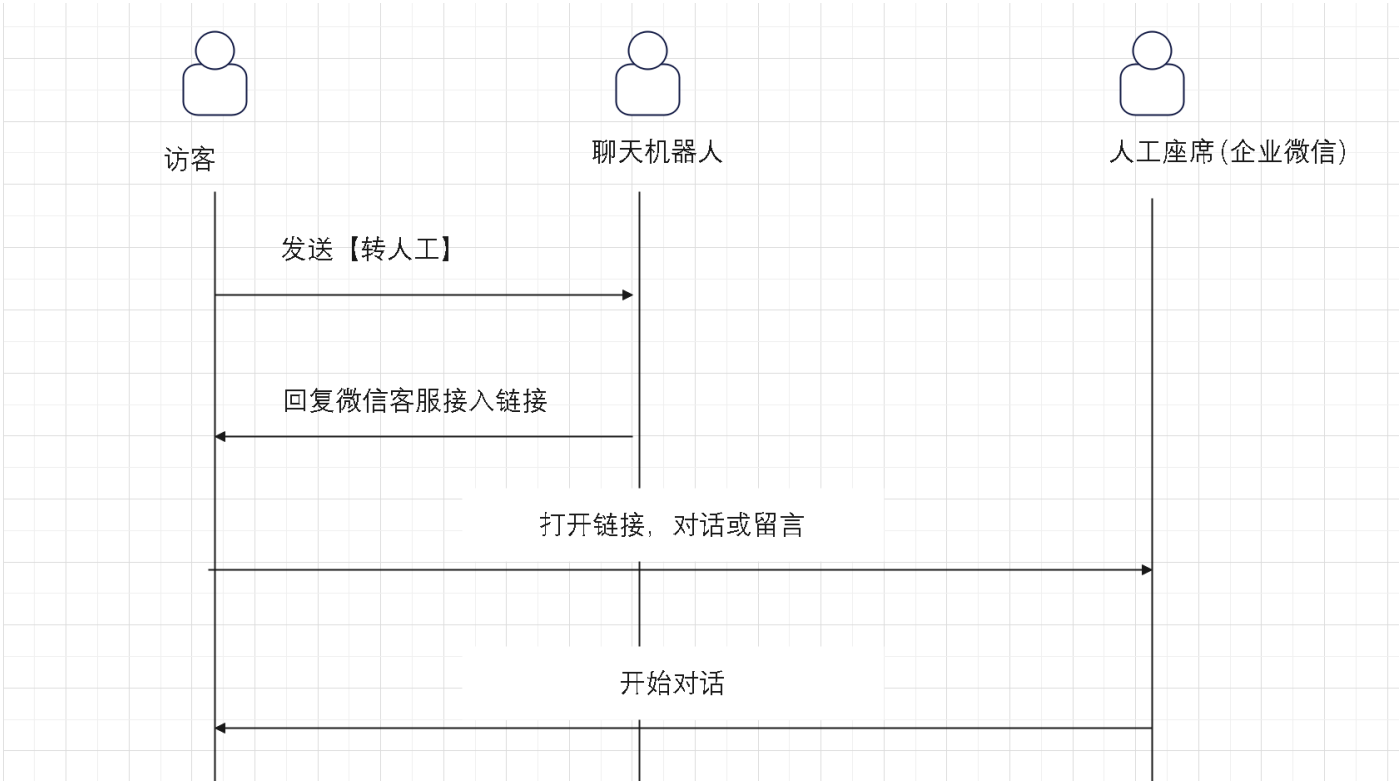
退出

彻底关闭话术助手，在话术助手的悬浮区域右键，在弹出的菜单中点击“退出”。



集成微信实现转人工客服

访客在和聊天机器人对话的过程中，可能需要和人工座席对话，以此咨询机器人解答不了的问题。这时候，访客会习惯性的发送“转人工”，“人工座席”等表达这个诉求，以下内容介绍一个快速、低成本的基于微信的方案：使用企业微信的微信客服完成人工座席咨询。



该方案具备以下优势：

- 低成本，使用企业微信的免费版本即可
- 符合用户习惯，访客使用微信，人工座席使用企业微信
- 移动客户端友好，访客和企业服务人员都可以使用微信
- 人工座席可以在 48 小时内连续反馈到访客
- 支持工作时间设置，非工作时间可以留言
- 支持分配到多座席，忙时按状态自动分配
- 支持发送语音、图片等

以下为具体的配置过程。

注册企业微信

打开链接 <https://work.weixin.qq.com/>，点击【立即注册】。

连接创造价值

🕒 致敬每一个在自己岗位上勇敢担当的人

立即注册

设置企业微信应用

登录企业微信: <https://work.weixin.qq.com>

然后, 打开链接: https://work.weixin.qq.com/wework_admin/frame#apps

看到如下的界面, 找到【微信客服】并点击。



设置微信客服

在上一步，进入了【微信客服】，接着点击【创建账号】

[« 返回](#)[应用详情](#)



微信客服

可以在微信内、外各个场景中接入微信客服，用户发起咨询，配置接待人员即可回复消息，还有聊天工具栏、升级服务等工具帮助企业做好服务。 [API](#)

可见范围  [修改](#)

客服账号

[创建账号](#)

Q 搜索

	Chatopera 客户顾问 + 添加账号	姓名: 第一客服 11:00-18:00 在线
	Chatopera 在线客服 + 添加账号	姓名: 在线客服 11:00-18:00 在线
	客服账号 + 添加账号	姓名: 客服账号 11:00-18:00 在线

接入场景

在新的页面，设置账号信息 -

名称：自定义，比如

接待方式：修改为【仅人工接待】

接待人员：选择企业微信中的联系人，或您自己。接待时间、接待分流等其它设置按照您的业务需要设定。

接待方式 人工接待 [修改](#)

人工接待配置

接待人员

 [修改](#)

接待时间

周一至周日 00:00-次日00:00 [修改](#)

接待分流

优先上次接待人员 [修改](#)

单人最多同时接待

20位客户 [修改](#)

转接

仅可转接给接待人员 [修改](#)

点击【创建】。

获得微信客服链接

完成上一步后，自动回到了微信客服管理页面，找到刚刚创建的客服账号，点击进入如下。

[返回](#)

应用详情



微信客服

可以在微信内、外各个场景中接入微信客服，用户发起咨询，配置接待人员即可回复消息，还有聊天工具栏、升级服务等工具帮助企业做好服务。 [API](#)

已启用

☒

可见范围

 [修改](#)

客服账号

[创建账号](#)

 豆丁 @华夏春松

，周一至周日 00:00-次日00:00 接待

[>](#)

此时，可见微信客服的【接入链接】，复制接入链接。



豆丁

展示的视频号

配置

接入链接

https://work.weixin.qq.com/...

复制链接

生成二维码

接待方式

人工接待 修改

人工接待配置

接待人员



修改

接待时间

周一至周日 00:00-次日00:00

修改

接待分流

优先上次接待人员

修改

单人最多同时接待

20位客户

修改

转接

仅可转接给接待人员

修改

设置机器人转人工问答对

进入 Chatopera 机器人知识库，创建问答对。



知识库

- 通过维护问答对的形式提供检索服务，使用标准问、相似问和词典管理等。
- 在测试对话页面，调试知识库，通过 SDK 接口等形式访问知识库，更多 [详细介绍](#)。
- 使用 Chatopera CLI 进行自动化知识库导入、导出操作，参考 [详细介绍](#)。
- 话术助手桌面软件支持查询知识库，获得智能建议回复。

知识库已同步自定义词典信息

+

问题

选择分类: 请选择

搜索

批量导入

导出

下载

分类管理

问题	是否启用	热门度①	操作
营业时间 所属分类: 暂无	启用	1	编辑 删除

问答对的表单设置如下：

Chatopera

重新定义聊天机器人

控制台

知识库

通过维护问题

在测试对话框

使用 Chatopera

话术助手桌面

知识库已同步自定

+

问题

分类管理

问题

营业时间

所属分类: 暂无

新建问题

问题 *

转人工

选择分类

请选择

相似问题

+

增加

人工客服

答案

答2 ×

增加

标题 *

URL *

描述

- 问题: 转人工
- 相似问题: 人工客服
- 答案: 1) 选择增加图文消息答案, 2) 删除 答案1, 3) 此时界面上仍保留了 答案2, 设置 答案2 的图文消息
- 标题: 人工座席
- URL: 即接入链接, 类似 `https://work.weixin.qq.com/kfid/xxxx`
- 描述: 使用微信与人工座席对话
- 上传缩略图: 在问答对的右侧有上传图标, 点击上传图标, 上传图文消息的图片。

保存, 接下来, 访客就可以与您定义的人工座席通过微信对话了。



创建和训练意图识别模型

意图和词典的关系

进入机器人意图管理页面，新建意图，意图的编辑包括用户说法和槽位。在添加意图的槽位时需要配置词典，包括新建自定义词典或引用系统词典，参考[词典的管理文档](#)。

需要强调的是，在词表词典创建后，默认是没有词条的，这种情况会导致训练失败；相似的，正则表达式词典也需要有表达式定义，否则会导致机器人训练失败。在遇到训练失败时，通过错误提示消息进行解决，在保存词典，保存意图时，会进行训练。

意图命名规则

- 意图标识名为字母、数字、下划线的组合，1-32 位，如：RailTypes
- 同一个机器人下的意图标识名不能重复
- 意图标识名一经确认后无法进行修改

创建：

在意图管理页面点击新建自定义词典，在弹出的对话框里输入正确的自定义词典名称，点击确定。与新建自定义词典一样

编辑意图和槽位

添加槽位：添加槽位时可以选择应用系统词典或自定义词典，可以设置是否必填和追问，必填的话必须设置追问。

 Chatopera
企业聊天机器人

控制台 | ceshi | + 创建机器人



← eat 测试

用户说法

输入用户可能的说法, 用{}使用槽位, 例如: 我想预订{CityName}的机票

添加 按Enter键可以快速添加

用户说法	操作
我想吃饭	删除
我要吃{food}	删除
我饿了	删除

槽位

槽位名称 time 词典 @TIME 必填 ☒

追问 什么时候

添加

槽位名称	词典	操作
taset	taset	删除
food	food	删除

@TIME

food

num

taset

引用系统词典

@TIME

@LOC

您可以 [新建词典](#) 或 [引用系统词典](#)

☒ 你想吃啥

保存 取消

删除槽位: 对于不需要的槽位可以进行删除, 点击删除按钮即可。

修改槽位: 对于需要修改的槽位, 直接进行编辑和修改即可。

槽位

槽位名称 例如：CityName

词典 请选择

必填

追问 例如：请问是哪个城市

添加

槽位名称	词典	必填	追问	操作
taset	taset	☒	有啥口味要求	删除
food	自定义词典 food num taset	☒	你想吃啥	删除
time	引用系统词典 @TIME @LOC	☒	什么时候	删除

保存

取消

添加说法：对于用户的说法，可以添加没有槽位的说法，也可以用{}关联槽位，把槽名称放到{}里面即可，一个说法可以绑定多个槽位。

← eat

测试

用户说法

输入用户可能的说法，用{}使用槽位，例如：我想预订{CityName}的机票

添加

按Enter键可以快速添加

用户说法	操作
我想吃饭	删除
我要吃{food}	删除
我饿了	删除

槽位

槽位名称

例如：CityName

词典

请选择

必填

追问

例如：请问是哪个城市

添加

槽位名称	词典	必填	追问	操作
taset	taset		有啥口味要求	删除
food	food		你想吃啥	删除
time	@TIME		什么时候	删除

保存

取消

删除说法：点击删除按钮即可。

修改说法：直接进行编辑和修改即可。

保存意图

在编辑完所有说法和槽位之后，需要进行保存。保存并且训练成功有提示。成功就可以到测试对话页面进行测试。

102/290

<https://bot.chatopera.com>

← eat

测试

用户说法 ①

输入用户可能的说法，用{}使用槽位，例如：我想预订{CityName}的机票

添加

按Enter键可以快速添加

用户说法	操作
我想吃饭	删除
我要吃{food}	删除
我饿了	删除

槽位 ①

槽位名称 ① 例如：CityName

词典 ① 请选择

必填 ① ☐

追问 ① 例如：请问是哪个城市

添加

槽位名称	词典	必填	追问	操作
taset	taset	☒	有啥口味要求	删除
food	food	☒	你想吃啥	删除
time	@TIME	☒	什么时候	删除

保存

取消

✔ 训练成功，可进行测试

测试和发布机器人

保存机器人词典，意图等信息后，Chatopera 云服务会提示开始重新训练机器人，训练会有成功或失败两种结果，训练会持续一段时间，时间长短取决于意图、词表、槽位的数量；完成训练后，代表机器人测试版本已经用最新的数据更新，在测试页面，导航至"意图识别"，和测试版本进行对话。

发布上线意图识别生产版本

调试版本

每次保存并且训练成功后，都会自动更新调试版本。但是此时不该急于发布上线，需要多次在测试对话页面进行测试，确保无误之后，再发布上线。

测试：在测试对话页面与机器人进行对话，在右侧会把对应的意图、槽位以及槽位值之类的信息展示出来。

Chatopera

企业聊天机器人

Beta

控制台

ceshi

+ 创建机器人

测试对话

多轮对话

知识库

意图识别

测试

我要吃面条

机器人

有啥口味要求

ME

加辣

机器人

什么时候

ME

晚上5点

会话已完成，请建立新的会话

机器人

请问您需要什么帮助

刷新

发送

回复信息

意图名称: eat

序号	槽位	值	是否必填	绑定词典
1	taset	加辣	是	taset
2	food	面条	是	food
3	time	晚上5点	是	@TIME

生产版本

测试无误后便可在【意图>>发布管理】模块对调试版本进行发布上线，发布后的线上版本可用于实际使用，线上的集成通过集成页面获取 SDK 和使用文档。

← 版本管理

- 每个机器人包含调试和生产两个版本。
- 每次机器人训练通过后，将自动更新调试版本。
- 调试版本发布上线后，将覆盖当前生产版本，生产版本用于实际的集成应用。

调试版本

版本 ID	修改时间	操作
16d0b9d3a4e	2019-09-07 20:05:17	发布上线

生产版本

版本 ID	上线时间
暂无筛选结果	

使用大语言模型（LLM）

Chatopera 云服务支持在知识库、多轮对话和意图识别均没有命中回复的时候，请求大语言模型（LLM）以进一步的获得答案。Chatopera 的大语言模型底层服务使用百度文心一言，实现方式和 ChatGPT 类似。

应用场景



- 回答事实性问题，比如“珠穆朗玛峰的海拔是多少？”
- 回答生活、天气、出行信息，比如“今天北京的天气怎么样？”，“北京四环现在拥堵吗”，“东坡肘子的做法”
- 艺术创作，比如“作一首七言绝句”

注意事项

- Chatopera 云服务用户可以选择针对机器人开启或关闭大语言模型的检索；
- 在 Chatopera 的检索算法中，大语言模型排在兜底回复之前，具体参考[多轮对话的检索](#)；
- 大语言模型会消耗时间，并且因为其返回值较慢（大约 5s ~ 20s），造成了对话的延迟；
- 推荐优先通过 [Chatopera H5 聊天控件](#)进行机器人的发布，在 Chatopera H5 中优化了交互体验以适应返回值较慢的问题。

开启配置

以下为不同情况下，让机器人支持大语言模型检索的方法。

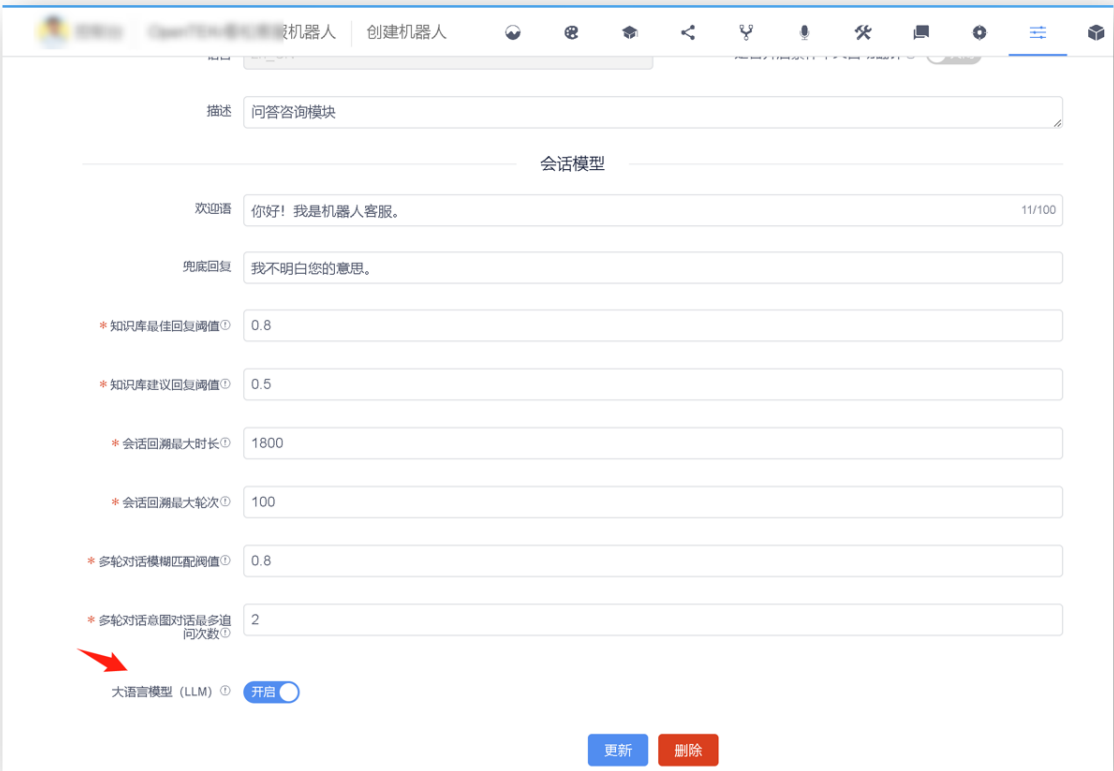
新建聊天机器人

登录 Chatopera 云服务后，在首页点击【创建机器人】，确认表单中【大语言模型（LLM）】是开启状态。继续填写表单，点击【确认】。



配置已有聊天机器人

进入聊天机器人配置页面，在【会话模型】中，找到【大语言模型（LLM）】，设置开关状态至【开启】。点击【更新】。



关闭配置

进入聊天机器人配置页面，在【会话模型】中，找到【大语言模型（LLM）】，设置开关状态至【关闭】。点击【更新】。

+ [今天] 晚上吃什么

- 烤鸭

通配符匹配器 / Star Gambit

为了让 **匹配器** 能适应复杂的需求，Chatopera 机器人平台使用通配符规则，通配符既能让规则具有更好的匹配能力，也能让[回复和函数中使用不同通配符的值，即通配符取值](#)，基于通配符的匹配器称为通配符匹配器，也称为 **Star Gambit**。

注意：下面的某些通配符左右带有空格，这些空格是必须的。

全能匹配

全能匹配通配符会匹配零到无穷个字符、单词。

(*)

此处的输入也会被系统捕获或者存储。

+ (*) 你好 (*)

- 欢迎光临

对话效果

匹配：客服你好

匹配：你好

匹配：你好吗

固定长度通配符

如果你知道你想要的字符长度，使用固定长度通配符。

*n

其中 n 代表长度。

此处的通配符可以被系统捕获，而且可以在回答中使用。

+ 早安 *2

- 早安

对话效果

匹配：早安北京

不匹配：早安乌鲁木齐

可变长度通配符

***~n**

如果只想匹配一些字符，可控长度的通配符是个不错的选择。n 代表你想匹配的最大长度。

+ 早安 *~4

- 早安

对话效果

匹配：早安

匹配：早安北京

匹配：早安哈尔滨

匹配：早安乌鲁木齐

不匹配：早安君士坦丁堡

区间通配符

如果想匹配一个确定的区间，比如 2 到 4 个字符之间，区间通配符绝对可以满足需要。

***(最短-最长)**

此通配符可以被系统捕获和用在回复中。

+ 早安 *(2-4)

- 早安

对话效果

匹配：早安北京

匹配：早安乌鲁木齐

不匹配：早安

词性通配符

在匹配器中，在某个位置使用词性来匹配一个词汇集合，比如名称、动词等。被匹配到的词，可以使用通配符取值在回复中使用。

比如：

+ <noun>是个好地方

- 嗯

+ <noun>很好吃

- ^echoYummy(<cap>)

```
exports.echoYummy = async function(cap1) {  
  return "那就多吃点" + cap1  
}
```

对话效果

匹配：香港是个好地方
匹配：蛋糕很好吃

如果在一句话中存在多个要匹配的词性，使用 `<noun1>`，`<noun2>` 进行标记，就是增加序号。

支持的词性

词性	词性匹配器	含义	支持语言
名词 / Noun	<code><noun></code> , <code><noun1></code> , <code><noun2></code> ... <code><nounN></code> , <code><nouns></code>	组织机构，地名，人名等	机器人所有语言
形容词 / Adjective	<code><adjective></code> , <code><adjective1></code> , <code><adjective2></code> ... <code><adjectiveN></code> , <code><adjectives></code>	比如：美好，顺利，难忘	机器人所有语言
动词 / Verb	<code><verb></code> , <code><verb1></code> , <code><verb2></code> ... <code><verbN></code> , <code><verbs></code>	比如：跑步，游泳	机器人所有语言
副词 / Adverb	<code><adverb></code> , <code><adverb1></code> , <code><adverb2></code> ... <code><adverbN></code> , <code><adverbs></code>	比如：急忙，快速	机器人所有语言
代词 / Pronoun	<code><pronoun></code> , <code><pronoun1></code> , <code><pronoun2></code> ... <code><pronounN></code> , <code><pronouns></code>	比如：我，他，她	机器人所有语言
人名 / Names	<code><name></code> , <code><name1></code> , <code><name2></code> ... <code><nameN></code> , <code><names></code>	比如：张三，李四	简体中文，繁体中文，英文

更多例子

+ `<noun>`是个好地方
- 嗯

+ `<noun>`很好吃
- `^echoYummy(<cap1>)`

+ 今天是`<adjective>` (*) 一天
- 美好！

+ `<verb>`对健康有好处
- 喜欢`<cap1>`

+ `<adverb>`地躲避
- 机智

+ `<pronoun>`在张望什么
- 等人

+ `<name>`有什么爱好
- 旅游

```
exports.echoYummy = async function(cap1) {  
  return "那就多吃点" + cap1  
}
```

必选项

必选项用在你有一系列可选项，但是必须有一个被匹配。

([候选-1](#)|[候选-2](#)|...|[候选-n](#))

+ 早安(北京|上海|天津)

- 早安

匹配：早安北京

不匹配：早安西安

不匹配：早安

可选项

可选项用来确定一些额外的内容。

[可能值-1|可能值-2]

+ 早安 [美丽的] 北京

- 早安

匹配：早安北京

匹配：早安美丽的北京

不匹配：早安热闹的北京

注意：此处 [] 前后的空格不可省略。

下一步

- 通配符取值：在回复中使用匹配到的值
- 模糊匹配器：容错能力更强和智能的匹配器
- 意图匹配器：借助意图识别模块，轻松实现任务型对话

使用模糊匹配器

+ \${商场几点开始营业|商场几点开门}
- 营业时间：8:00 AM – 11:00 PM

模糊匹配器 / Like Gambit

也称为 Like Gambit，使用容错能力更强的语法定义规则。

- [模糊匹配器的设计初衷和背景知识](#)

模糊匹配器的功能

模糊匹配器的主要特点是支持设定多个句子，输入和句子进行匹配，根据阈值判定是否匹配。有默认阈值，Chatopera 机器人平台可以在机器人管理控制台设置页面，调整该阈值，也可以在每个匹配器上设定单独的阈值；该阈值的取值需要介于 0 和 1 之间，越高代表越严格的匹配，越相似。

基于模糊匹配器，匹配器的容错能力更强，调试更简单；但是模糊匹配器不考虑位置的因素，这一点与通配符匹配器较为不同。

模糊匹配器和通配符匹配器各有优劣，Chatopera 机器人平台开发者在看过不同的匹配器示例后，根据需要灵活选择。

接下来介绍模糊匹配器的详细使用。

模糊匹配器的语法

形式 1

```
+ ${threshold}{句子1|句子2|...|句子N}  
- 回复
```

句子可以设定 1 或多个，多个句子时使用 "|" 进行分隔。threshold 的取值范围是 $(0,1)$ 。

比如：

```
+ ${0.7}{商场几点开门|商场营业时间}  
- 营业时间：早上八点半至晚上十一点
```

形式 2

```
+ ${句子1|句子2|...|句子N}  
- 回复
```

比如：

+ \${商场几点开门|商场营业时间}
- 营业时间：早上八点半至晚上十一点

在这种形式下，省略了形式 1 里的阈值 threshold，那么在计算输入和 句子1，...， 句子N 的是否匹配时，使用的阈值是机器人控制台【设置页面】设定的值【多轮对话模糊匹配阈值】，默认为 0.8。

控制台BOT202108-2创建机器人

Client Id611b9a96d1164f001b9c095bSecret.....复制刷新显示

* 机器人名称BOT202108-2

* 语言en_US

描述请描述机器人

会话模型

欢迎语My super power is talk.
兜底回复I don't understand.
会话回溯最大时长1800会话回溯最大轮次10多轮对话模糊匹配阈值0.9

更新删除

多轮对话中检索回复时，使用模糊匹配器（Like Gambit）时，默认的置信度，和输入文本的相似度超过置信度的规则视为匹配；在模糊匹配器中可覆盖默认值，比如 \${0.7}{句子1|句子2}

匹配器及回复复用

模糊匹配器受机器人记忆模型限制，如果需要在在一个记忆周期内一直生效，可以使用 {keep} 标记，比如：

+ \${商场几点开门|商场营业时间}
- {keep} 营业时间：早上八点半至晚上十一点

详细介绍，参考[文档](#)。

使用调试

对于一条模糊匹配器规则，究竟要适应多少种问法？容错是保守一点，还是要非常灵活？这需要在调试机器人时，Chatopera 机器人平台用户，自行判断。在多轮对话设计器内，可以增加模糊匹配器里的句子，或者适当的调节日阈值 threshold。

调试过程

- 1) 开启自动上传；
- 2) 撰写模糊匹配规则，设定答案；
- 3) 保存；
- 4) 在对话中，发送测试文本；
- 5) 查看日志，得到匹配调试信息。

如下图：

113/290

<https://bot.chatopera.com>

Chatopera 多轮对话设计器

话题: greetings

新建话题 环境变量 绑定信息 上传 下载 帮助

检索多轮对话 返回值

SDK

对话

已启用 自动上传 最近 保存

脚本 函数

1. 开启自动上传

2. 撰写规则

3. 测试对话

4. 查看日志, 得到调试信息

```
{
  "state": "default",
  "createdAt": 1629199666950,
  "string": "9点半",
  "topicName": "greetings",
  "subReplies": [
  ],
  "service": {
    "provider": "conversation"
  },
  "logic_is_fallback": false,
  "botName": "BOT202108-1",
  "faq": [
  ],
  "profile": {
  }
}
```

请在这里输入进行问答测试 发送

日志

```
[saas_611b9829d1164f001b9c095a], branch [master], requirePlugin
2021-08-17 19:27:46 INFO conversation loadPlugin chatbotID
[saas_611b9829d1164f001b9c095a], branch [master]
2021-08-17 19:27:46 INFO conversation loadPlugin chatbotID
[saas_611b9829d1164f001b9c095a], branch [master], requirePlugin customPlugins {}
2021-08-17 19:27:46 DEBUG conversation [DoesMatch] like gambit(threshold 0.7): ${0.7}{商场几点开门} -matched-> 商场开门时间
2021-08-17 19:27:46 DEBUG conversation [DoesMatch] like gambit(threshold 0.7): ${0.7}{商场几点开门} -similarity-> 1. score(0.71029) : 商场 几 点 开 门
```

调试信息

关于模糊匹配器在调试时, 得到的调试信息, 进一步介绍。

每个模糊匹配器, 都会和输入进行比较, 计算相似度, 打印的日志类似:

```
2021-08-18 13:52:52 DEBUG conversation [DoesMatch] like gambit(threshold 0.8): ${商场几点开门} -
similarity-> 1. score(1) : 商场 几 点 开 门
```

其中, `DoesMatch` 是服务端计算该相似度的函数名, `like gambit(threshold)` 表示这个匹配器是模糊匹配, 并输出本次的阈值, 后面就是匹配器内容, `-similarity->` 后的输出是每个句子和输入内容的相似度, `score()` 内就是相似度, 分数后面是句子的分词结果; 多个句子时, 使用 "|" 分隔。

比如:

```
2021-08-18 13:57:18 DEBUG conversation [DoesMatch] like gambit(threshold 0.8): ${商场几点开门|商场开门时间}
-similarity-> 1. score(1) : 商场 几 点 开 门 | 2. score(0.71029) : 商场 开 门 时 间
```

以上句子之间增加了序号, 按照分数由大到小进行排列。

如果模糊匹配器的一个句子与输入之间相似度大于阈值, 那么就判定为匹配, 输出日志类似如下:

```
2021-08-18 13:57:18 DEBUG conversation [DoesMatch] like gambit(threshold 0.8): ${商场几点开门|商场开门时间}
-matched-> 商场几点开门
```

模糊匹配器取值

如果匹配上模糊匹配器, 回复的函数中, 可以获得哪些信息, 进一步的让回复内容更加智能?

模糊匹配器上, 函数中利用输入的自然语言处理信息办法: 使用 `this.message` 内包含的信息进行分析。

假如有这样的脚本和函数:

```
/**
 * 模糊匹配器示例脚本
 */

+ ${商场几点开门|商场开门时间}
- ^getOpenTime()

exports.getOpenTime = async function() {
  debug("getOpenTime words %j", this.message.words) // this.message.words 是 JSONArray
  debug("getOpenTime tags %j", this.message.tags) // this.message.tags 是 JSONArray
  return "九点半"
}
```

那么，就可以在 this.message.words 中得到分词的信息，在 this.message.tags 得到相对应位置的词性。

比如，如果输入是“商场的开门时间”，那么相应的 words 和 tags 如下：

```
getOpenTime words: ["商场","的","开门","时间"]
getOpenTime tags: ["nis","ude1","vi","n"]
```

所以，“商场”的词性是“nis”，“的”的词性是“ude1”。那么，“nis”和“ude1”又代表什么呢？它们是词性标识。

在 Chatopera 机器人平台，不同语言使用词性标识因为语言本身的原因，不是很一致。

语言	词性标注集
中文（简体中文 <code>zh_CN</code> ，繁体中文 <code>zh_TW</code> ）	文档
英文 / en_US / English	文档
日语 / Japanese	文档

得到匹配信息，再结合 Chatopera 机器人平台内其它的内置函数库，可以实现更为强大的对话能力！

通过提供不同形式的匹配器，适合智能对话机器人开发者在不同场景下因地制宜的选择构建规则的方法。

接下来

- [有关更多关于 `this.message` 的使用介绍](#)
- [理解多轮对话工作机制](#)
- [检索多轮对话：使用多轮对话接口进行系统集成](#)
- [通配符匹配器：使用语法建立规则](#)
- [意图匹配器：借助意图识别模块，轻松实现任务型对话](#)

```
intent book_cab
- ^orderCab()
- {x} ^loseOrderCab()
```

意图识别匹配器 / Intent Gambit

也称为 Intent Gambit，在 多轮对话脚本 中使用 [意图识别模块](#) 实现任务型对话。

- 意图匹配器的设计初衷和背景知识

意图匹配器语法

语法（[注意语法中存在的空格](#)）：

```
intent INTENT_NAME
- ^succHandlerFn()
- {x} ^loseHandlerFn()
```

在意图匹配器中，开始必须是 "intent"，用来标志后面的内容是面向意图匹配器。

intent INTENT_NAME 就是意图匹配器，INTENT_NAME 是意图识别模块中，意图的名字，大小写不敏感，这意味着 book_cab 和 BOOK_CAB 是一样的。



INTENT_NAME / 意图名字

然后，在下一行，使用 "-" 开始设定回复，因为需要处理意图识别信息，而该信息是传递给函数的，所以，回复设置为自定义的函数。在本文后面的部分，会详细说明在函数中取值。

在意图匹配器下，有两种类型的回复：

- 1) **成功函数**：识别到意图，并在对话交互中得到所有的必须的槽位信息，调用“`成功函数`”；
- 2) **失败函数**：识别到意图，但是追问交互中，在限定的追问次数下，没有得到必须的槽位信息，在这种情况下，会使用对话用户最后的检索，从知识库和对话脚本中，继续检索，如果得到答案，则回复该答案；如果没有得到答案，则调用“`失败函数`”。

“失败函数”有一个特殊的标记：`{x}`。即一对大括号内写入小写字母 `x`。**失败函数是选填的**，如果没有定义则会回复兜底回复。在失败函数内，也是可以取得当前意图识别的信息。

举例

```
intent book_cab
- ^orderCab()
- {x} ^loseOrderCab()
```

以上，`orderCab` 是成功函数，`loseOrderCab` 是失败函数。

`成功函数` 和 `失败函数` 的执行时刻的更多信息，参考 [多轮对话工作机制](#)。

以上就是意图匹配器的脚本语法，接下来介绍函数中，使用意图识别信息、配置意图识别分支等更高级功能点。

意图会话生命周期

当一个意图被识别到，对话进入意图识别对话，就会创建一个**意图会话**。意图会话，在下面四种情况下会失效：

空闲时间太久而过期

对话用户在【会话回溯最大时长】时间下未发送请求，意图会话因为**空闲时间太久而过期**；

即【会话回溯最大时长】，当对话用户，超过一段时间不和机器人交互，那么会话状态被清空，包括意图会话。同样，进入 BOT 管理控制台的设置页面调整该值，默认为半个小时。

新识别到的意图覆盖意图会话

对话从意图识别模块识别新的意图，**就是又匹配上了新的意图**，旧的会话周期被覆盖；

意图识别调试分支重新训练

意图匹配器，通过在多轮对话脚本中的环境变量 `@SYS_INTENT_BRANCH` 的值来指定集成**意图识别的版本**，即调试版本或生产版本，二者的值分别是 `pro` 和 `dev`，默认为 `dev`。

调试版本，顾名思义，是开发测试时使用，每次重新训练，其对应的意图会话都会被删除，在使用意图匹配器时，就会造成多轮对话在上下文处理上的失败。所以，每次意图识别重新训练，如果多轮对话集成的是调试分支，则意图会话的逻辑就失败了。

因为意图匹配器默认就是使用调试分支，所以，对于生产环境的 BOT，有如下建议：

- 1) 发布训练好的意图识别调试版本为生产版本，进入机器人意图管理页面，点击【版本管理】，点击【发布上线】。



2) 在 多轮对话设计器 或 Chatopera 机器人平台管理控制台，进入 环境变量设置。

修改 `@SYS_INTENT_BRANCH` 的值为 `pro`。

```
@SYS_INTENT_BRANCH=pro
```

这时，因为使用了意图识别生产版本，不会因为调试分支的变动而影响意图会话。

函数内关闭意图会话

在 多轮对话函数 中，包括 成功函数、失败函数 或 其它的匹配到的脚本函数，使用 `this.intent.drop = true` 设置后，当前的意图会话被关闭。

设置 `this.intent.drop = true`，就是放弃当前意图信息。即包括函数成功处理了意图，或放弃当前意图会话信息。采用这个设计的原因是，意图会话的信息处理，这项权利尽量的保留给 BOT 开发者，在函数中实现，可以满足更多业务需求，让 BOT 更智能。

`this.intent.drop = true` 的调用很重要，如不调用会导致后续的意图识别不起作用，使用详情参考：[如何在函数中，关闭或放弃正在进行的意图会话。](#)

使用函数完成对话

意图识别可以在 CDE 中使用，最新版本[下载地址链接](#)。

编辑意图识别对话窗口如下：

话题: greetings

新建话题 环境变量 绑定

检索多轮对话 返回值

对话

SDK

```
{
  "state": "default",
  "string": "目的地是哪里?",
  "botName": "dev1078",
  "logic_is_unexpected": false,
  "logic_is_fallback": false,
  "service": {
    "provider": "intent",
    "intent": {
      "branch": "dev",
      "state": "proactive",
      "entities": [
        {
          "name": "originLoc",
          "val": "深圳",
          "requires": true,
          "dictname": "@LOC"
        }
      ]
    }
  }
}
```

我想打车

Bot 我不明白您的意思。

我想打车

Bot 您想从哪里出发?

深圳

Bot 什么时间出发?

明天下午

Bot 目的地是哪里?

请在这里输入进行问答测试

发送

已启用话题

脚本 函数

```
1 /**
2  * 意图识别对话示例
3  * https://github.com/chatopera/chatbot
4  */
5
6
7
8  intent book_cab
9  - ^orderCab()
10 - {x} ^loseOrderCab()
```

日志

```
branch [master] exist in database, but not syncd with memory, load it into memory
2021-09-01 19:42:51 INFO conversation loadPlugin chatbotID
[saas_611325bed908ba001bc659da], branch [master], plugin name: [loseOrderCab]
2021-09-01 19:42:51 INFO conversation loadPlugin chatbotID
[saas_611325bed908ba001bc659da], branch [master], plugin name: [orderCab]
2021-09-01 19:42:51 DEBUG conversation [doesGambitMatch] intent gambit(threshold-0.9)
matched: book_cab(1) | branch dev
2021-09-01 19:42:55 DEBUG conversation [query] conversation request { "fromUserId":
"superadmin", "textMessage": "深圳" }
```

在脚本和函数编辑区域，使用语法匹配意图识别和处理意图识别信息；对话窗口进行测试，在日志中查看日志和 `debug` 信息；在 SDK 中查看返回值 JSON 数据。

意图匹配器取值

如上文，有一个例子：

```
intent book_cab
- ^orderCab()
- {x} ^loseOrderCab()
```

当对话用户的发送文本匹配上了“book_cab”中的说法，比如“我想打车”。多轮对话即启用【book_cab】内的对话流程，意图识别对话在成功获得该意图的必须的槽位信息，或超过了设置的最大追问次数，则会进入多轮对话函数，前者会调用【成功函数】，后者会调用【失败函数】。前文，做过相关描述。那么，成功函数和失败函数的内，如何获得意图识别对话的信息呢？

成功函数

成功函数，比如上文例子中的 `orderCab` 不需要在对话脚本中设定任何参数，函数执行的时候，会将意图识别信息，加入该函数的 `this` 命名空间下，使用 `this.intent` 读取。

`this.intent` 是一个 JSON 数据，数据格式如下：

```
{
  "name": "{{INTENT_NAME}}",
  "topicName": "{{TOPIC_NAME}}",
  "branch": "{{INTENT_BRANCH}}",
  "entities": [
    {
      "name": "{{ENTITY_NAME}}",
      "val": "{{ENTITY_VALUE}}",
      "requires": true,
      "dictname": "{{DICT_NAME}}"
    },
  ],
  "state": "{{STATE}}"
}
```

INTENT_NAME: 意图识别的名字

TOPIC_NAME: 对话脚本话题的名字

INTENT_BRANCH: 集成意图识别的版本的分支, 默认为 `dev`, 如果使用意图识别生产版本分支, 在首轮对话中设置环境变量 `@SYS_INTENT_BRANCH` 为 `pro`

ENTITY_NAME, *ENTITY_VALUE*, *DICT_NAME*: 该意图下的关键的参数, 槽位信息; `requires` 为 `true` 时, 代表该槽位是必填的

STATE: 状态, 包括 `resovled`, `proactive`, `losed` 三个状态, 分别代表意图全部必填参数识别完成, 正在追问参数信息和追问超过最大次数

除了增加了 `this.intent`, 其它成功函数的使用和多轮对话函数是一致的。比如, 设定回复:

```
// 回复只有文本
exports.orderCab = async function() {
  debug("[orderCab] intent %s", JSON.stringify(this.intent));
  // 处理下单信息
  // ...
  this.intent.drop = true; // 成功下单, 设置当前意图会话过期
  return "下单成功, 订单编号: SE111."; // 设定回复
}

// 回复包含自定义业务字段
exports.orderCabExt = async function() {
  debug("[orderCab] intent %s", JSON.stringify(this.intent));
  // 处理下单信息
  // ...
  this.intent.drop = true; // 成功下单, 设置当前意图会话过期
  return {text: "下单成功, 订单编号: SE111."; // 设定回复,
    params: { YOUR_KEY: YOUR_VALUE }} // 自定义业务字段
}
```

`this.intent` 的一个实际数据示例:

```
{
  "name": "book_cab",
  "topicName": "greetings",
  "branch": "dev",
  "entities": [
    {
      "name": "originLoc",
      "val": "天津",
      "requires": true,
      "dictname": "@LOC"
    },
    {
      "name": "date",
      "val": "明天下午四点",
      "requires": true,
      "dictname": "@TIME"
    },
    {
      "name": "destLoc",
      "val": "北京",
      "requires": true,
      "dictname": "@LOC"
    }
  ],
  "state": "resolved"
}
```

失败函数

失败函数中，`this.intent` 的信息和使用，和成功函数是一致的，这是调用的实际不一致，同时它是可选的。

快速读取 intent.entities

默认情况下，`intent.entities` 是一个数组，如果需要用槽位的名称去取值，要便利，不方便，使用以下方法快速使用 `intent.entities` 生成一个 JSON Object，然后使用槽位名称取值。

```
let entities = _.keyBy(this.intent.entities, 'name');
```

比如，有一个槽位名字为 `date`，则接下来获得这个槽位的值：

```
let dateRawString = entities["date"]["val"];
```

转化相对时间为绝对时间

加入一个日期槽位 `dateRawString` 值为“明天下午 5 点”，那么如何从这个字符串提取出时间“2021-09-03 17:00”呢？

```
let extractedDates = await this.maestro.extractTime(dateRawString);
debug("extracted date", extractedDates);
if(extractedDates.length > 0){
  // 目标时间就是: extractedDates[0] '2021/09/03 17:00'
  debug("Target date", extractedDates[0]);
}
```

更多关于 `extractTime` 的介绍[参考链接](#)。

保存变量信息到意图会话

函数执行的时候，如果有意图会话存在，就会被加载到函数的 `this.intent` 中。在两个或多个连续的多轮对话中，可以通过 `this.intent.extras` (JSON Object) 来保存变量。该信息会和该意图会话周期一致。比如：

```
/**
 * 提取时间实体
 */
async function extractTimeEntity(maestro, entities, property) {
  let dates = await maestro.extractTime(entities[property]["val"], "YYYY年MM月DD日 HH:mm");
  return dates.length > 0 ? dates[0] : "";
}

exports.handleAirplaneTicketOrder = async function() {

  debug("[handleAirplaneTicketOrder] this.intent", JSON.stringify(this.intent))

  let entities = _.keyBy(this.intent.entities, 'name');
  // 获得信息
  let date = await extractTimeEntity(this.maestro, entities, "date");
  // 保存到 this.intent.extras
  this.intent.extras = {
    date: date
  }

  ...
}
```

`handleAirplaneTicketOrder` 函数保存了一个变量到 `this.intent.extras` 中，稍后另外一次对话时，调用了另外一个函数 `placeAirplaneTicketOrder`，就可以直接用这个变量。

```
exports.placeAirplaneTicketOrder = async function() {
  // 直接取值
  // this.intent.extras.date

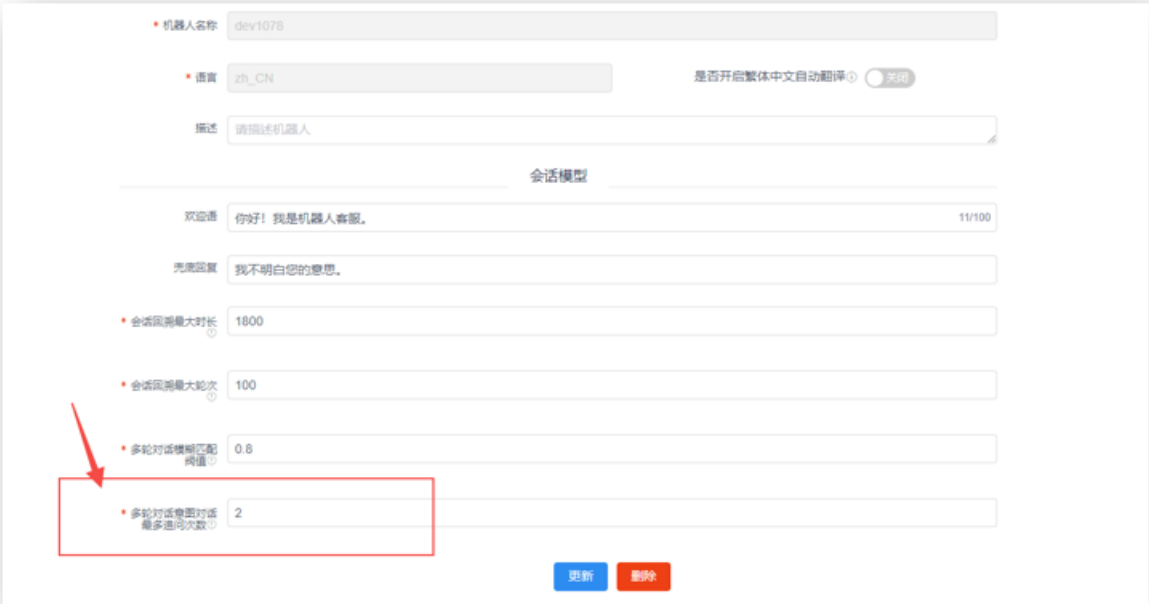
  ...
}
```

设置最大追问次数

匹配意图识别对话之后，机器人会根据槽位信息情况，必填的槽位是否都获取到，进行追问，这个追问可以设定一个最大次数，当达到最多的追问时，机器人会按照顺序检索回复内容：知识库->对话脚本->失败函数（见上文“失败函数”的定义）。

对于每个机器人，BOT 开发者可以自定义设置，进入 BOT 设置页面，找到【[多轮对话意图对话最多追问次数](#)】，默认为 2。

在后续的多轮对话中，如果意图对话还没有过期，则依旧会从意图对话中检索。如果追问的信息被匹配到，则做意图对话；否则将不会追问，而是以(知识库->对话脚本->失败函数)这个顺序检索回复。



设置集成意图识别分支

在意图识别模块，包括两个分支：调试分支；生产分支。生产分支，就是指发布到生产环境，实际上线的 BOT 服务。

设计两个分支，主要是考虑到，一个机器人的意图识别数据可以不断的优化，那么每个阶段优化好以后再选择上线到生产分支，这样在调试和优化的时候不影响生产分支的对话。

该设置为，在多轮对话设计器或 BOT 管理控制台的多轮对话页面，设置【环境变量】来调整，默认为调试分支，设置参数为：`@SYS_INTENT_BRANCH`，使用 `dev` 代表集成调试分支，使用 `pro` 代表集成生产分支；默认为调试分支。

升级生产环境的多轮对话脚本

通过多轮对话设计器，或 Chatopera 机器人平台的机器人多轮对话管理页面，上传多轮对话脚本，因为刷新的缘故，正在进行的意图会话失效，这会对话用户造成体验上的困扰，所以，对于生产环境的更新，建议在业务低分时间进行！

其它使用说明

SDK 返回意图识别信息

当对话用户，有匹配到意图，正在进行意图识别的对话时，使用多轮对话检索 API，返回值中，`service.provider` 的值是 `intent`，并且 `service.intent` 是当前意图信息。示例数据如下：

```
{
  "string": "您想从哪里出发?",
  "topicName": "greetings",
  "subReplies": [],
  "service": {
    "provider": "intent",
    "intent": {
      "name": "book_cab",
      "threshold": 0.9,
      "branch": "dev",
      "state": "proactive",
      "entities": [
        {
          "name": "originLoc",
          "val": "",
          "requires": true,
          "dictname": "@LOC"
        },
        {
          "name": "date",
          "val": "",
          "requires": true,
          "dictname": "@TIME"
        },
        {
          "name": "destLoc",
          "val": "",
          "requires": true,
          "dictname": "@LOC"
        }
      ]
    }
  },
  "logic_is_unexpected": false,
  "logic_is_fallback": false,
  "botName": "dev1078",
  "faq": [],
  "profile": {}
}
```

可见, `service.intent` 的数据与函数中的 `this.intent` 是一致的。

在回复中跳转到指定意图

假设在多轮对话脚本中, 已经设定了一个意图的意图匹配器。然后在多轮对话中, 还可这样开始该意图的对话: 使用 `topicRedirect` 切换话题。

文本回复

```
+ ${0.5}{自定义的文本}
- ^topicRedirect("intents", "book_airplane_ticket", true)
```

注意: 此处 `topicRedirect` 的第三个参数设置为 `true`, 代表目标匹配器是一个意图匹配器。

函数回复

文本中, 定义了函数

```
+ ${0.5}{自定义的文本}
- ^handleXXFn()
```

函数 `handleXXFn`:


```
exports.handleXXFn = async function() {  
  // do your magic  
  return "^topicRedirect(\" intents\", \"book_airplane_ticket\", true)"  
}
```

接下来

- [函数](#): 使用 `http` 模块请求外部系统 API 服务
- [理解多轮对话工作机制](#)
- [查看示例程序: 预约机票](#)
- [通配符匹配器](#): 使用语法建立规则
- [模糊匹配器](#): 容错能力更强和智能的匹配器

设置回复

在**匹配器**中，我们已经学到了怎么添加一个回答。事实上你可以添加任意数量的回答。这里还有一些高级功能可以帮助你完成更多的任务。

文本形式

最直接的设置文本答案的方法。

```
+ 在吗
- 你好，在的
```

如果添加了多个回答，系统会从中随机挑选一个作为回复, 然后丢掉这个回答。

```
+ 在吗
- 亲，在的
- 亲，有什么需要帮助
- 你好，请问遇到什么问题了吗？
```

所谓丢掉这个答案，是指机器人针对同一个用户，在【会话回溯最大时长】内再次匹配上该**匹配器**时，选择回复时，不考虑使用过的回复。

```
+ 在吗
- {keep} 亲，在的
- 亲，有什么需要帮助
- 你好，请问遇到什么问题了吗？
```

模糊匹配器受机器人记忆模型限制，如果需要在在一个记忆周期内一直生效，可以使用 `{keep}` 标记，详细介绍，参考[文档](#)。

也可以在匹配器前添加`*{keep}*`，就不用在每个回答中都添加了

```
+ {keep} 在吗
- 亲，在的
- 亲，有什么需要帮助
- 你好，请问遇到什么问题了吗？
```

如果回答很长，可以通过“^”分割以方便可读性。可以通过“\n”实现换行

```
+ 在吗
- 你好，这里是客服中心，\n
^ 请问遇到什么问题了吗？
```

它等价于

```
+ 在吗
- 你好，这里是客服中心，请问遇到什么问题了吗？
```

引用回复

有些时候，在问答对中重用一些回复能使编写脚本效率更高，这时可以定义一个问答对，并在脚本其它位置引用它。

```
+ 在吗
- {@__greeting__} 请问有什么能帮助您？

+ __greeting__
- 亲，在的。
- 你好，客服小美为您服务
- 亲亲，稍等，客服马上就到
```

引用的方式就是 `{@匹配器}`，匹配器中的下划线不是必须的，上面例子中的匹配器 `__greeting__` 的命名是为了增强脚本的可读性。

将一些常用的回复以约定的匹配器命名方式命名，不但方便复用回复，使脚本可读性好，方便维护，而且更方便使用[上下轮钩子](#)实现多轮对话。

```
+ 在吗
- {@__greeting__}

+ (*) 问路 (*)
% {@__greeting__}
- 请使用地图APP

+ (*) 打车 (*)
% {@__greeting__}
- 出租车在门口挥手示意停车

+ (*)
% {@__greeting__}
- {@__greeting__}

+ __greeting__
- 亲，在的。请问有什么能帮助您？
- 你好，客服小美为您服务
- 亲亲，稍等，客服马上就到
```

通配符取值

通配符取值是针对[通配符匹配器](#)的，匹配成功后，可以读取匹配到的词汇。在回答中需要使用输入中的通配符值，这时可以使用 `<cap>` 达到目的。

```
+ 我是 *~3
- 你好，<cap>
```

```
匹配：小明比小红高
回答：你确定小明比小红高吗？
```

如果用户输入，“我是张三”，那么系统将回复“你好，张三”，当有多个槽时，可以使用多个 `<cap>`。

```
+ *2 比 *2 高
- 你确定<cap1>比<cap2>高吗？
```

在对话中，我们有时候会需要以前的通配符值，看一下下面这个例子：

```
+ 我叫 *~3
- 你好，<cap1>

+ 你猜我叫什么？
% 你好，<cap1>
- 你刚说了，你叫<p1cap1>
```

<pNcapM>代表了以前的通配符。其中 N 代表在在对话中之前的问答， M 代表捕获的位移。

此处，上下轮钩子，即%开头的句子，代表匹配器“你猜我叫什么？”只服务于“你好，<cap1>”作为回复时，它会被优先匹配。上一轮对话的回复通过(%)的方式指定了接下来的逻辑，形成多轮对话。

关于上下文钩子的[详细介绍文档](#)。

提示： 1) +/ %/- 前的空格不是必须的，在多层对话中，空格可以增强脚本的可读性，比如使用上下轮钩子中的段首缩进。 2) 通配符取值适用于通配符匹配器，其它形式的匹配器，有不同的值传递方式，参考它们的详细说明。

函数

函数就是通过写 JavaScript 脚本，处理回复，在脚本中，可以实现各种逻辑，做外部系统集成等。

```
+ 我叫 *~3
- ^getGreetings(<cap1>)
```

那么，在多层对话设计器内，通过 `exports.getGreetings = async function(){ // do your magic }` 来定制化回复内容。

函数的详细使用[参考文档](#)。

接下来

- [上下轮钩子](#)
- [函数](#)

使用上下轮钩子

上下轮钩子的语法

在实际应用中，和机器人聊天时，很可能要通过上下轮钩子完成一个任务。我们用(%)来定位之前回复，声明新的匹配器，(%)后的内容是和某个回复内容一样的字符串，这里的(%)称之为“上下轮钩子”。

```
+ (*)
- 您身高多少

+ *(3-5)
% 您身高多少
- 我的身高也是<cap1>
```

让我们一起看看这个例子：

1. 当用户输入任何文字，我们用通用通配符触发回答，然后系统回复“您身高多少”。
2. 当用户继续输入时，系统会先从历史中查看之前的回复中是否有对应的上下文，在这里指的是“% 您身高多少”
3. 最后，如果用户输入 3 到 5 个字符，系统匹配匹配器“+ *(3-5)”，并且回复“我也是<cap>”。“<cap>”代表的就是用户输入的内容。

理解钩子在多轮对话中的作用

“访客发送给机器人，机器人回答一次”，就是一轮对话。其中，访客发送给机器人的文本，就是加号后面的规则，机器人因此进行匹配。匹配上了，就通过减号后面的生成答案。

那么，如果有多个匹配器(+ 加号定义的规则)，那么选择哪些进行匹配呢？默认就是都匹配。但是如果上一轮对话中，可以限定下一轮的匹配器集合，这样就仅在这个集合中寻找匹配器。如何限定呢？就是定义上下轮对话的钩子。这个钩子就通过 % 上一轮回复的字符串 确定。

```
// 预约机票
```

```
intent {keep} book_airplane_ticket  
- ^handleAirplaneTicketOrder()
```

上一轮回复

```
+ ${0.6}{没错, 出票吧 | 出票}
```

```
钩子 % ^handleAirplaneTicketOrder()  
- {keep} ^placeAirplaneTicketOrder()
```

```
+ ${0.6}{不预约了}
```

```
% ^handleAirplaneTicketOrder()  
- {keep} ^cancelAirplanTicketReservation()
```

```
+ ${0.6}{信息有误, 重新预约}
```

```
% ^handleAirplaneTicketOrder()
```

通过钩子关联的下一轮回复的匹配器集合

使用函数

函数是一个强大而有趣的设计。在回复中，可以使用函数来获取整条消息对象，用户对象或者其它资源，比如数据库。Chatopera 机器人平台的函数，就是内置的 JavaScript / Node.js 运行环境，基于安全和性能的考虑，这个 JavaScript 运行时是经过特殊处理的 Node.js 执行时。

函数定义

把通配符值当做变量传给函数，例如下面这个例子：

```
+ 我的用户名是 *(2-10)
- ^getUserAccount(<cap>)
```

所以，调用函数的方式就是使用“^”。在函数的编辑窗口中，可以这样定义：

```
exports.getUserAccount = function(account, cb) {
  cb(null, "对不起，系统没有找到" + account);
}
```

函数的声明中，参数列表首先是通配符的值，可以传多个，然后最后一个参数始终是回调函数(cb)，cb 的参数列表为 (error, text)，text 作为文本添加到回复中，error 是指处理中发生异常。

函数的定义也同样支持 async/await 语法，例子如下：

```
exports.getUserAccount = async function(account) {
  return "对不起，系统没有找到" + account;
}
```

使用 async/await 时，需要抛出异常时，通过 throw new Error("ERROR MESSAGE") 完成。

在系统集成时，业务系统的需求千差万别，为了灵活的支持各种需求，在函数中也可以自定义返回值。

```
exports.getUserAccount = function(account, cb) {
  cb(null, {
    text: "对不起，系统没有找到" + account,
    params: {
      resetPasswd: "http://example.com/reset-account"
    }
  });
}
```

其中，params 可定义为 [] 或 {} 对象，该用法同样适用于 async/await 函数。

```
exports.getUserAccount = async function(account) {
  return {
    text: "对不起，系统没有找到" + account,
    params: {
      resetPasswd: "http://example.com/reset-account"
    }
  }
}
```

在系统集成/多轮对话检索时，返回值 data 内将增加 params 属性，与以上设定的值相同。

复合函数

在回复中，可以添加任意多的函数，比如

```
+ ...  
- 联合 ^callFunction1() 和 ^callFunction2(<cap1>)
```

嵌套函数

在函数的回调函数中，函数名会被解析成对应的函数，所以放心的在回复中添加任意合法的函数，比如在脚本中这样写：

```
+ ...  
- ^nestedAFunction()
```

然后，在函数中，定义如下：

```
exports.nestedAFunction = function(cb) {  
  cb(null, "张三 ^nestedBFunction()");  
}  
exports.nestedBFunction = function(cb) {  
  cb(null, "和李四");  
}
```

下一步

- [内置函数库](#)
- [函数返回值](#)

使用富文本消息类型，提升了用户体验，这些回复的设定，需要借助 CDE。

- CDE 的[安装链接](#)
- 快速入门 [CDE 的使用](#)

简而言之，就是使用函数回复约定的数据格式，在客户端会“渲染”成不同的样式。

热门问题问候语



在默认情况下，机器人的问候语是在 Chatopera 云服务的机器人属性页面进行设置，该设置只支持纯文本。

控制台 | 小松 | + 创建机器人

Client Id

5c11f1256f80090017b44401

Secret

.....

机器人名称

小松

描述

回答春松客服产品相关问题

语言

zh_CN

欢迎词

你好！我是机器人客服。

兜底回复

我不明白您的意思。

更新

删除

访客登录进入客服聊天窗口往往是有一些问题要咨询的，所以更为合理的用户体验是将一些常见问题作为导航用途显示在问候语

下面, 比如这样:



具体的使用方法

具体的使用方法是, 找到 greetings 话题, 或创建一个新的话题, 设置脚本和函数:

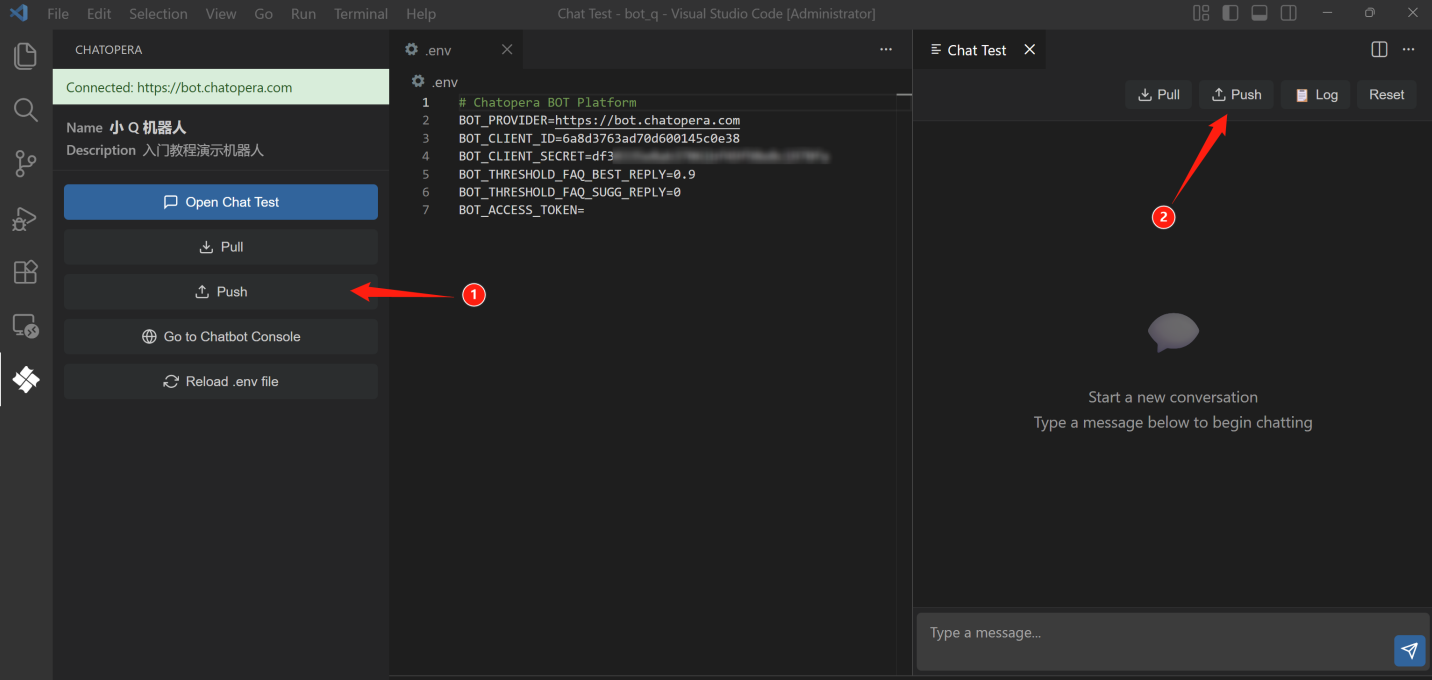
- 脚本: 在脚本中, 定义函数返回。

```
// FAQ Hotlist
+ __faq_hot_list
- {keep} ^get_greetings()
```

- 函数: 函数返回一种约定的数据格式。

```
// 问候语中关联常见问题
exports.get_greetings = async function() {
  return {
    text: "请问有什么可以帮到您?",
    params: [{
      label: "1. 产品列表",
      type: "qlist",
      text: "产品列表"
    }, {
      label: "2. 当季热销产品",
      type: "qlist",
      text: "当季热销产品"
    }, {
      label: "3. 退换货咨询",
      type: "qlist",
      text: "退换货咨询"
    }
  ]
};
}
```

然后, 保存, 使用【Push】推送给机器人。



解释说明

该规则 `__faq_hot_list` 将保证在访客和机器人连接成功后， 机器人发送函数 `get_greetings` 返回的内容。此处 `__faq_hot_list` 是固定的， `get_greetings` 函数名和下面的技能函数名保持一致便可。

`__faq_hot_list` 是春松客服与 [Chatopera 机器人平台](#)之间约定的一个钩子。

`{keep}` 的作用是机器人记忆中，可重复的使用一个回复，参考[文档](#)。

按钮消息

另外一种情况是机器人回答一个选择列表， 访客点选按钮进行选择。



- 脚本

```
+ (*) 产品列表 (*)
- ^get_products()
```

该规则将保证在访客的问题中有 `产品列表` 时， 发送函数 `get_products` 返回的内容。

- 函数

```
// 按钮选择消息
exports.get_products = async function() {
  return {
    text: "您对下面哪个产品感兴趣",
    params: [{
      label: "上衣",
      type: "button",
      text: "介绍一下上衣"
    },
    {
      label: "服装",
      type: "button",
      text: "介绍一下鞋帽"
    }
  ]
}
```

在脚本中，只需要设定 `params` 中的内容作为选型，其他保持默认。

图文消息

Chatopera 云服务还支持的一类消息是图文消息，通过是分享一个链接，展示链接的图片和简介。



- 脚本

在脚本中，定义函数返回。

```
+ 介绍一下上衣
- ^get_shangyi()
```

该规则将保证在访客的问题中有 介绍一下上衣时，发送函数 `get_shangyi` 返回的内容。

- 函数

```
// 图文消息
exports.get_shangyi = async function() {
  return {
    text: "{CLEAR} 图文消息",
    params: [{
      type: 'card',
      title: "秋冬上衣优选",
      thumbnail:
        "https://ss0.bdstatic.com/70cFuHSh_Q1YnxGkpoWK1HF6hhy/it/u=277597165,2202071991&fm=15&gp=0.jpg",
      summary: "秋冬上衣优选秋冬上衣优选秋冬上衣优选秋冬上衣优选秋冬上衣优选秋冬上衣优选秋冬上衣优选",
      hyperlink: "https://www.1688.com/huo/B6F9CDAFD7B0C5AEBAA2CDE2CCD7BCD3BAF1C9CFD2C2C7EFB6AC.html"
    }]
  }
}
```

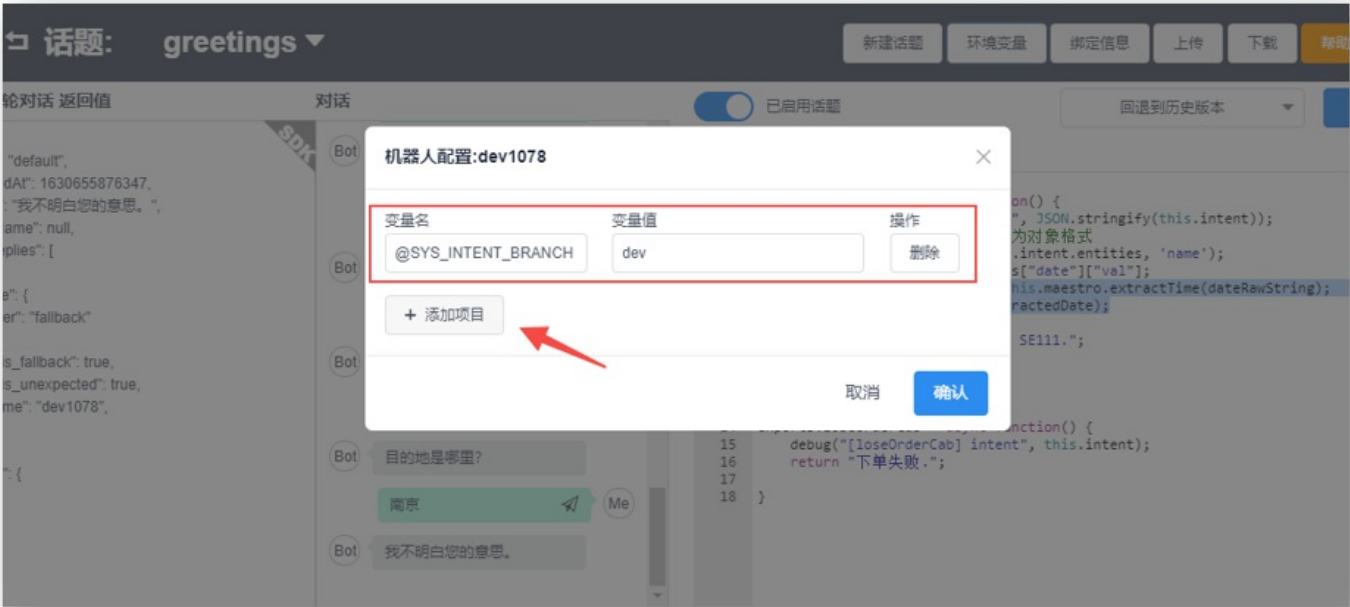
其中，title代表大标题，thumbnail是图片，summary是简介，hyperlink是点击图文消息后，跳转的链接，这几项是用户可以自定义的，其它项就使用默认值，比如type的值必须是card。

在掌握了知识库、脚本语法和函数后，用户可以支持各种客服场景。

多轮对话支持配置环境变量，在开发过程中，设定对话可配置的信息。

设置环境变量

在多轮对话设计器配置环境变量。

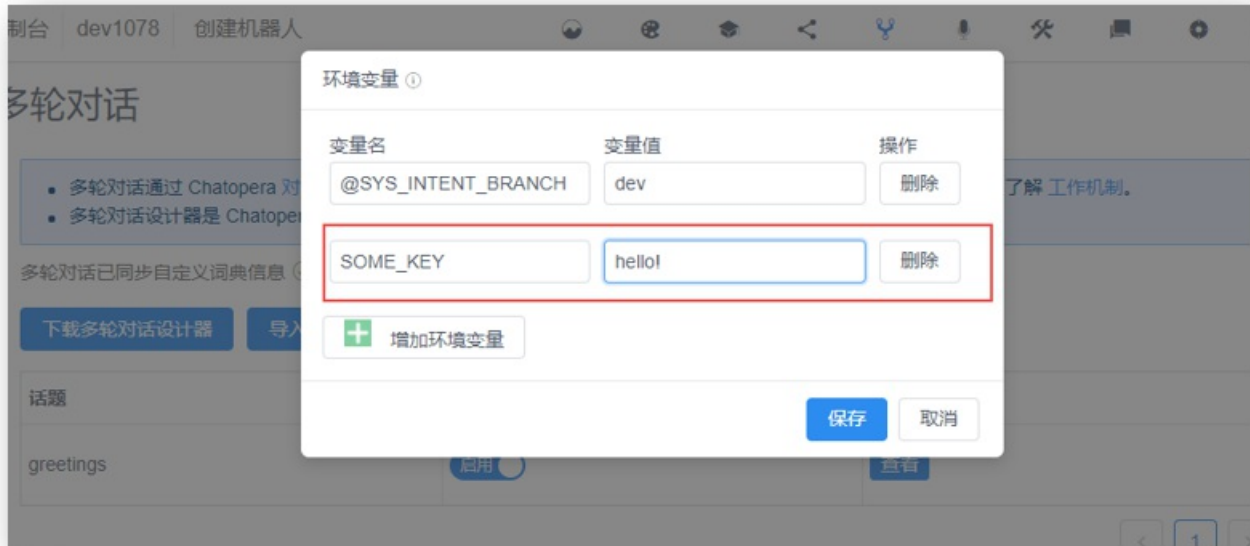


在函数中，使用全局变量 `config` 从环境变量读取值。

比如：

```
exports.someFunction = async function(){
  return config["SOME_KEY"];
}
```

这样就方便了分发多轮对话，从多轮对话设计器中导出对话应用 c66，Chatopera 机器人平台用户可以将该 c66 文件导入另外一个机器人，并使用环境变量配置属性。



在 多轮对话 中，也存在系统环境变量，这些变量属于 Chatopera 机器人平台约定的，以 "@" 开头，其值可以被 Chatopera 机器人平台用户修改，但它们往往存在默认值和合法值的约束。

系统环境变量	默认值	取值说明	描述
@SYS_INTENT_BRANCH	dev	dev 或 pro	多轮对话集成意图识别的分支，dev 代表调试分支，pro 代表生产分支

管理对话状态

在 多轮对话 中，来访者和机器人对话的过程可以想像成来访者使用地图游览，每个时刻，来访者的位置都可以映射到地图上。地图有一定的行走路径，来访者下一步所能选择的方向是由当前状态提供的。

现在，来访者与机器人对话的状态会保持 30 分钟，比如来访者想要点餐，但是说了一句“我要点晚餐”，然后半个小时没有继续发送信息，那么等来访者再次发送信息时，机器人已经不记得上文了。

上下文状态和机器人记忆，都是让机器人有更强的对话处理能力：通过上下轮钩子和通配符取值，设计对话流程，满足各种定制化业务。

在实际应用中，对话状态按照匹配器、上下轮钩子和时间作用改变外，还有哪些办法影响对话状态呢？以下介绍几种方法，影响对话状态。

匹配器及回复复用

在机器人检索多轮对话时，会考虑到 30 分钟内的 100 轮对话（此处 30 分钟和 100 轮分别对应机器人设置界面参数：会话回溯最大时长，会话回溯最大轮次），我们称这样的 一个限定条件是机器人的记忆。假如一个对话如下：

```
+ 你好
- 你好！
- 欢迎光临！
```

它使用了通配符匹配器，有两个回复：“你好！”和“欢迎光临！”。假设在记忆周期内，发送两次“你好”给机器人，得到的回复分别是“你好！”和“欢迎光临！”。这时，再次发送“你好”，机器人则会回复兜底回复。

这是因为，机器人对“你好”的回复都消耗尽了。如果想让机器人在记忆周期内，保证可以重复的使用某一个回复，就需要加上 `{keep}` 标记，使用方法如下。

通配符匹配器

```
+ 你好
- {keep} 你好！
- 欢迎光临！
```

也可以在匹配器前，使用 `{keep}`，这样就相当于在每个回复前都加了 `{keep}`，比如：

```
+ {keep} 你好
- 你好！
- 欢迎光临！
```

当匹配器的有多个回复时，会按照记忆过滤，再从后选中随机选择一个作为回复，如果在回复上使用了 `{keep}` 则起到了：保证该匹配器在记忆中重复使用，标记该回复作为默认回复。

注意: 此处 `{` 和 `}` 是半角符号。

模糊匹配器

模糊匹配器中，`{keep}` 的使用也有两种形式，形式与通配符匹配器一致。

```
+ {keep} ${0.6}{喜欢夏天还是冬天}
- 都喜欢
```

或者

```
+ ${0.6}{喜欢夏天还是冬天}
- {keep} 都喜欢
```


意图匹配器在 `intent` 后添加 `{keep}`，注意 `{keep}` 前后有空格。

```
// 预约机票
intent {keep} book_airplane_ticket
- ^handleAirplaneTicketOrder()
```

意图匹配器中，使用 `{keep}` 只有一种形式。

清除状态

所谓清除状态，就是在处理完某次回复，不再需要用户处于当前对话状态，而恢复默认对话状态。

解决办法：设置回复的文本以 `{CLEAR}` 开头。比如：

```
+ 再见
- {CLEAR} 感谢您选择我们，期待再次光临！
```

`{CLEAR}` 可以添加在 回复 或 函数返回值 中。在 Chatopera 机器人平台返回给来访者时，文本内容会去掉这个前缀。

这个方法很实用，尤其是在全能匹配器，对话可能进入死循环，因为所有的输入都被全能匹配拦截，那么在对应的回复中使用 `{CLEAR}` 就达到了下一次对话进入默认状态，不优先匹配全能匹配器的目的。

切换话题

在多轮对话中，开发者可以定义多个对话主题，对话主题的名字是字母组成的字符串，使用回复在不同对话主题间跳转。

跳转到通配符匹配器或模糊匹配器

```
+ 你好
- ^topicRedirect("greetings","你好")
```

`topicRedirect` 是内置的函数，第一个参数是目标主题名字，第二个参数是目标匹配器。

跳转到意图匹配器

在函数中，跳转到指定的意图匹配器有一点特殊，参考[使用说明](#)。

知识库路由

在知识库的答案或多轮对话的函数中设置回复时，可以用 `routeDirectReply` 来检索一个指定的话题和匹配器，直接路由到多轮对话的主题和匹配器。

```
routeDirectReply#["TOPIC_NAME", "TOPIC_GAMBIT_ID" [,INHERIT_PARAMS]]
```

`TOPIC_NAME`: 话题名称

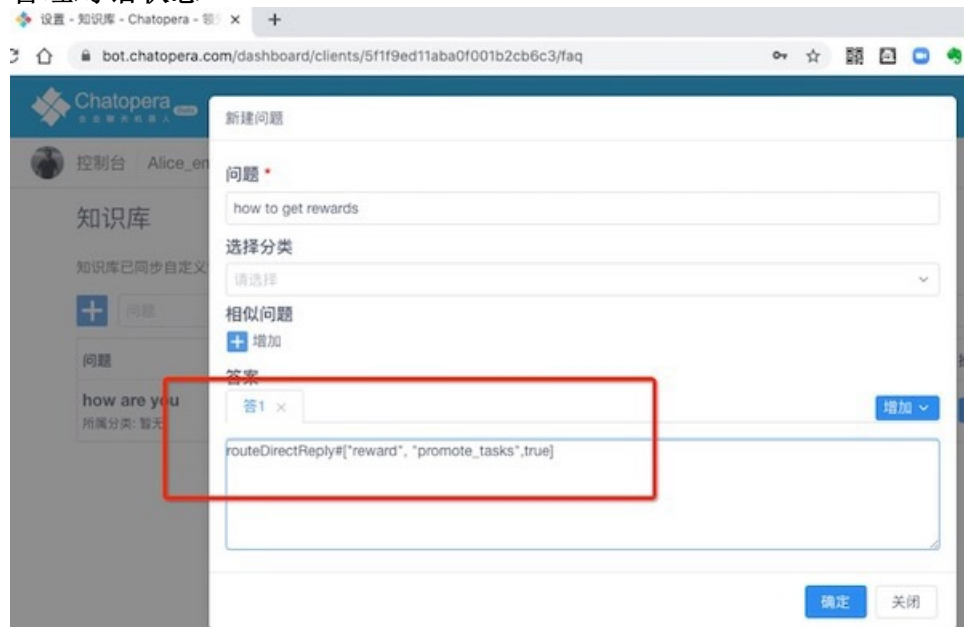
`TOPIC_GAMBIT_ID`: 匹配器

其中，`INHERIT_PARAMS` 是可选参数，决定当前对话取得的 `params` 是否覆盖接下来对话的 `params`，值为 `[true|false]`，默认为 `false`。

另外，当 `TOPIC_GAMBIT_ID` 的值为 `$ctx.textMessage$` 时，则使用当前对话的用户输入，在 `TOPIC_NAME` 中进行检索。

比如

```
routeDirectReply#["class_001_pre", "__C1PRE_GAMBIT_003",true]
```



提示: **routeDirectReply**需要设定为知识库问答对里的第一个答案, 答案类型为 纯文本 `plain`。

概述

作为用户，我想在 Chatopera 云服务内集成外部的对话系统，外部对话系统包括但不限于：

- 大语言模型
- 其它知识库

一个外部的系统，可以视为：一个接受对话请求的 API。

- API 有约定 HTTP Method
- Request Params 或 BODY
- Response 结果，比如 JSON 格式

解决方案概述

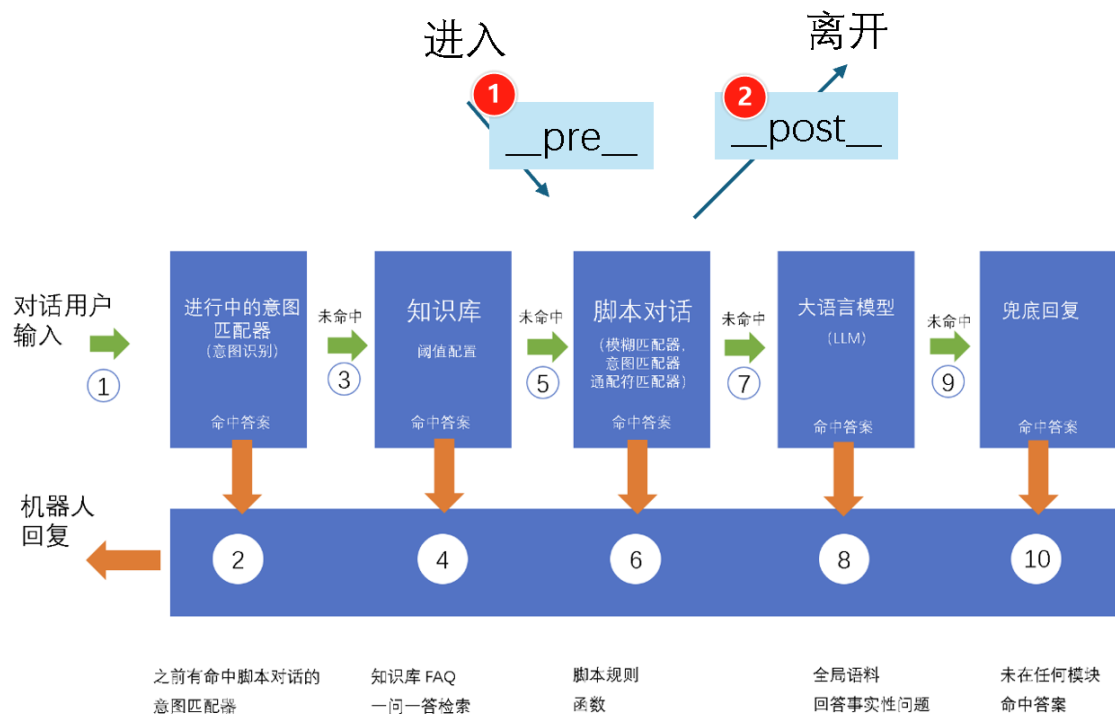
以下为不同方案，可以选择其中一种。

多轮对话脚本 Hooks

使用该方案，需要您了解 JavaScript，下面也提供相关示例代码，任何编程经验都可以很快入门。

首先，了解 Chatopera 云服务的工作原理，尤其是检索的顺序，参考文档 [《多轮对话的检索》](#)。

其次，当前面没有得到答案，对话引擎进入多轮对话脚本检索，在检索中，话题也是排序的。排在最后的话题是 `__post__`，检索有下图。



利用 `__post__` 可以实现集成自定义的外部的对话系统。

使用 `__post__` 脚本实现自定义外部对话系统集成

参考文档《[安装开发者工具](#)》。

导入机器人

- 打开多轮对话设计器，点击【添加】
- 在浏览器中，进入机器人设置页面，获得 ClientID 和 secret
- 在多轮对话设计器【添加】表单中，填入 ClientID 和 secret

打开话题 greetings

在多轮对话设计器中，打开新添加的机器人。在机器人话题列表页面，点击编辑。



新建话题



话题名称，填写 ，点击确认。



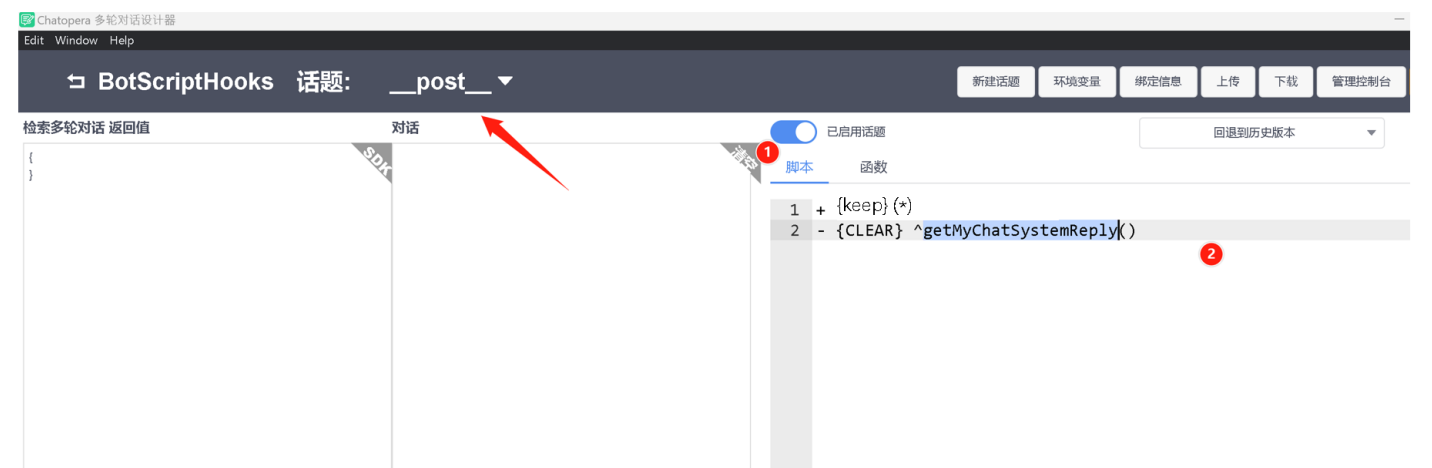
此处话题名称必须是 `__post__`。

编辑话题

编辑 `__post__` 话题

```
+ {keep} (*)
- {CLEAR} ^getMyChatSystemReply()
```

此处，`getMyChatSystemReply` 是函数名，可以自定义，比如改成 `getAnyChatSystemReply`。



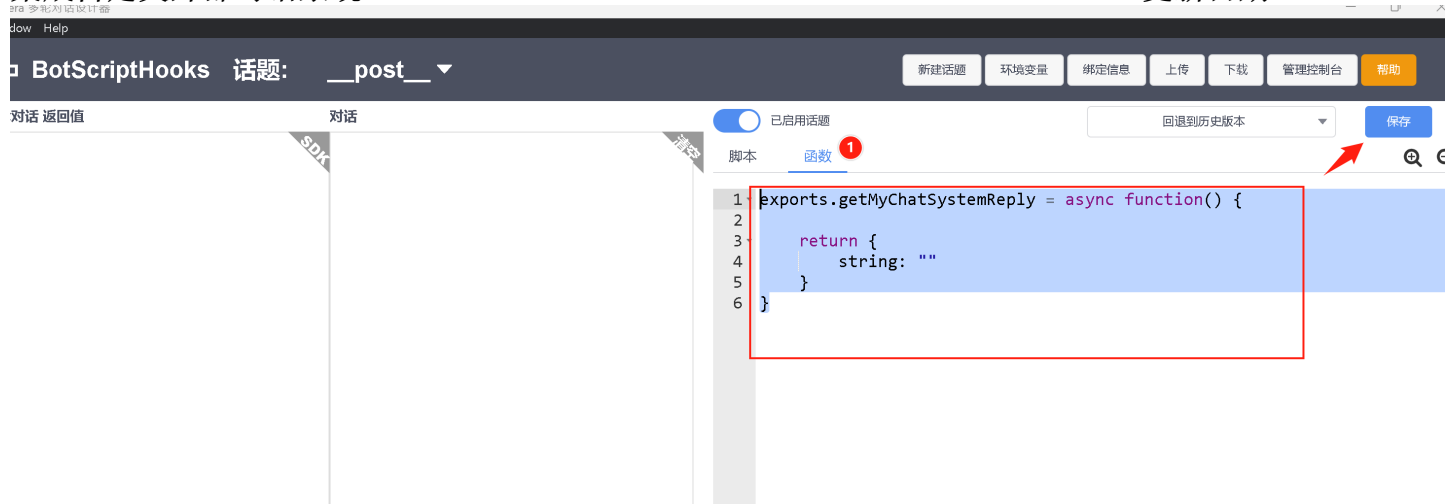
编辑函数

进入函数编辑页面。

首先，定义函数体。

```
exports.getMyChatSystemReply = async function() {
  return {
    string: ""
  }
}
```

点击保存。



其中，返回值中 "string" 字段就是机器人回复的文本，如果设定为空字符串，则代表没有检索到答案，多轮对话检索将进入 LLM 大语言模型（如果开启）或兜底回复。

下面，实现和外部对话系统集成关键环节：使用 `http` 库发送请求，处理回复。

更新函数 `getMyChatSystemReply`，内容如下：

```
exports.getMyChatSystemReply = async function() {
  debug("[getMyChatSystemReply] %s %s", this.user.id, this.message.clean);
  // this.user.id: 当前对话访客的唯一标识
  // this.message.clean: 当前对话访客的请求文本

  // 使用 http 库请求 API
  let resp = await http.get("https://www.chatopera.com/files/botplt/miscs/dummy.chatsystem.json");
  debug("[getMyChatSystemReply] http response %j", resp.data);

  // 使用 resp 进一步的处理逻辑
  let reply = "";
  if (resp.data.reply) {
    reply = resp.data.reply;
  }

  return {
    string: reply
  }
}
```

保存，然后进行测试。

话题: __post__ ▼

新建话题

对话

已启用话题

脚本

函数

SDK

测试123

Bot 测试123通过, 一切正常, 请保持。(这是根据“测试123”以及字数限制给出的简短回答, 希望满足你的需求。)

测试123

Bot 测试123成功, 结果无误, 继续前行!(根据“测试123”的指令和30字的限制, 给出了简要且积极的回答。)

测试123

Bot 测试123成功, 一切正常, 结果符合预期。加油!(30字以内的回答, 简洁明了地表达了测试成功的意思。)

测试123

Bot 测试123成功, 结果准确, 一切顺利。(这个回答在30字以内, 简洁明了地表达了测试成功的意义。)

1 测试123

Bot 今天是个好日子 2

```
1 exports.getMyChatSyst
2
3   debug("[getMyChat
4
5   // 使用 http 库请
6   let resp = await
7
8   // 使用 resp 进一
9   let reply = "";
10  if (resp.data.rep
11      reply = resp.
12  }
13
14  return {
15      string: reply
16  }
17 }
```

最终, 得到答案是【今天是个好日子】, 是因为 http response data 的内容如下:

```
{
  "req": "",
  "reply": "今天是个好日子"
}
```

相关链接

- 脚本对话检索顺序, <https://dwz.chatopera.com/7C91Oz>
- 脚本中 keep 的作用: 让该规则永久有效
- 脚本中 clear 的作用: 清楚状态, 下一轮对话依旧走全技能域的检索
- 更多 http 库使用的例子, <https://dwz.chatopera.com/0P46fh>
- 更多函数多轮对话语法, <https://dwz.chatopera.com/1882VM>

设置定时任务

应用场景

在 多轮对话 中，执行了 函数 A，然后想在若干时间后执行 函数 B。那么则可以使用 `setTimeout` 函数实现。

使用说明

比如有如下脚本：

```
+ 你好
- {keep} ^fnA()
```

`fnA` 是一个函数，在 函数 中定义如下：

```
async function B() {
  debug("Exec B")
}

exports.fnA = async () => {

  setTimeout(async () => {
    debug("Run fnB");
    await B()
  }, 5000)

  return "hello"
}
```

其中，`setTimeout` 的使用就是执行计时任务，`5000` 代表 5 秒后执行函数体：

```
async () => {
  debug("Run fnB");
  await B()
}
```

执行结果：

2022-07-09 11:05:51	DEBUG	conversation	call fnA result: hello
2022-07-09 11:05:51	DEBUG	conversation	call fnA:
2022-07-09 11:05:55	DEBUG	conversation	Run fnB
2022-07-09 11:05:55	DEBUG	conversation	Exec B

关于 `setTimeout` 函数的进一步介绍，参考 [文档](#)。

测试页面

对于不同方式实现对话，都需要验证机器人的对话是否符合预期，Chatopera 机器人平台提供统一一个模块完成这个任务，导航进入【机器人详情>>测试页面】。

Chatopera Beta

企业聊天机器人

控制台 | dev002 | + 创建机器人

测试对话

多轮对话 | 知识库 | 意图识别

测试

ME

人脸识别

机器人

我不明白您的意思。

ME

人脸识别

机器人

您是否想问以下问题

0.73 人脸识别失败

命中阈值 0.5

```
[
  {
    "id": "AW70N5Q907BxWDoZ_hDM",
    "score": 0.7317734512205795,
    "post": "人脸识别失败",
    "reply": "1111",
    "reply_plain_text": "1111"
  },
  {
    "id": "AW70N71307BxWDoZ_hDN",
    "score": 0.34730097313826713,
    "post": "身份证识别",
    "reply": "2222",
    "reply_plain_text": "2222"
  },
  {

```

在该页面下，也需要选择对话实现方式，根据对话方式不同，对话过程中，右侧的信息展示形式也是不同的，使用测试对话页面，也提升用户实现机器人的开发效率。

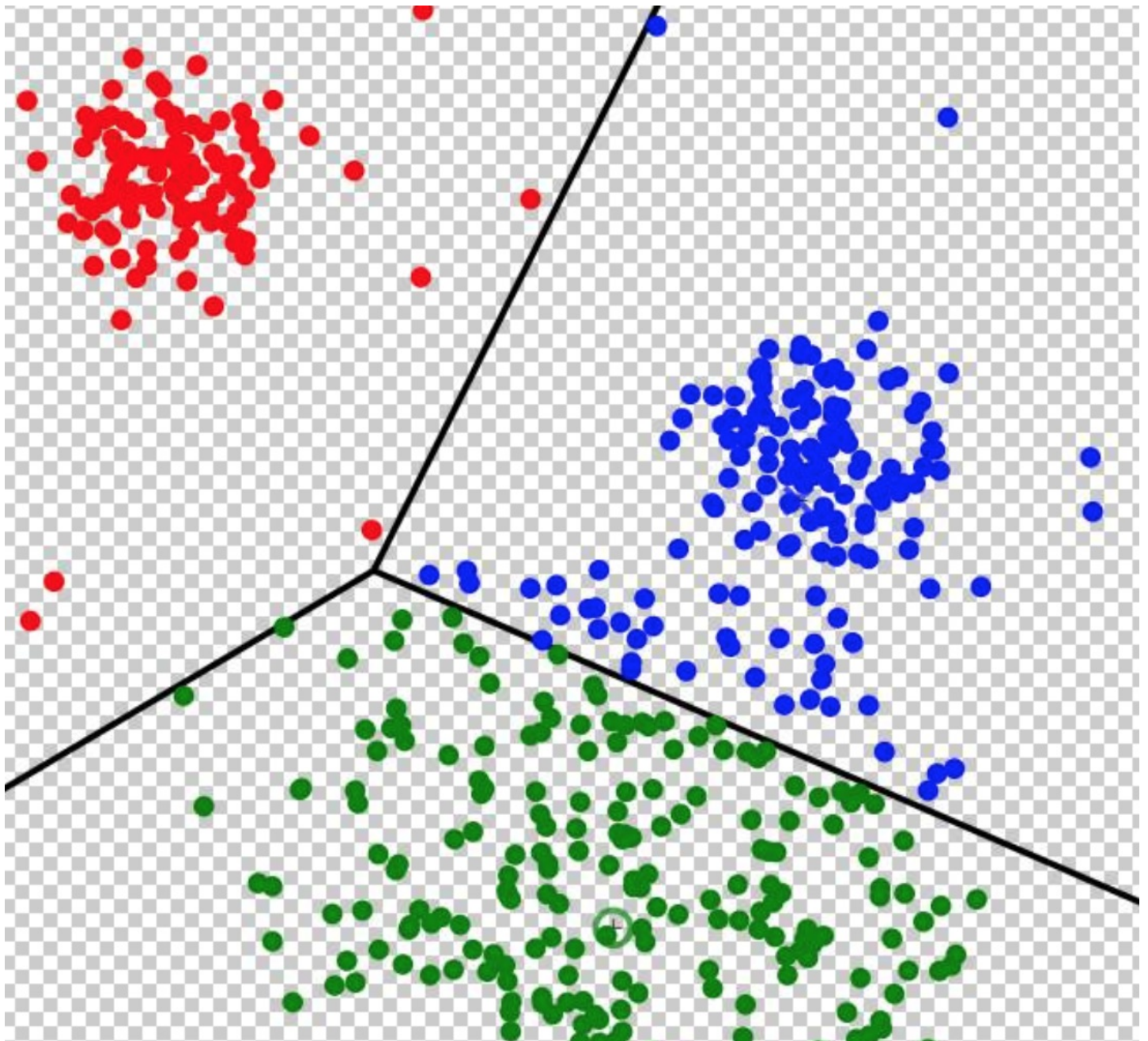
对话历史分析

概述

使用机器学习分析对话历史。**Chatopera 机器人平台聚类分析**是将多个文本按照相似度分类，分成一个个聚簇。每个聚簇中任何两个问题都是相似的。聚类分析在运营聊天机器人、提升智能化水平起着关键作用。聚类分析的结果可以用来：

- 1) 优化知识库：发现新知识，新问题或已有问题的相似问；
- 2) 优化多轮对话脚本：发现对话流程，话术，业务流程需求；
- 3) 优化意图识别：新的意图，意图中的参数；
- 4) 优化商业智能：发现新需求，访客痛点，业务流程改造；
- 5) 衡量用户活跃度，分析用户激活、留存、客户服务等环节的问题。

在聊天机器人上线后，可能未必达到及格的程度。比如知识库命中率比较低，这时候，就要经常的使用聚类分析功能，优化知识库，常用方案是每天上午执行过去 24 小时或 3 天的对话历史，然后下载聚类结果，进行机器人优化。



对话历史分析更新日期：8/29/2026

聚类分析，有如图，不同颜色的零散的点，使用线段尽量将相同颜色的点汇聚在一个区域，聚类分析算法就是寻找最佳的线段。聚类分析算法使用非监督的机器学习算法，不保证聚类结果的准确无误，通常是聚类后方便进一步的人工标注工作，此处恕不赘述细节。

使用须知

- 1) 聚类分析只针对对话历史记录产生的数据，包括知识库检索 API 和多轮对话检索 API；
- 2) 执行聚类分析任务是免费的。

提交聚类任务

在机器人详情页面，在导航菜单中打开“聚类分析”页面。 点击“创建任务”。



在弹出的对话框中选择聚类针对的时间范围：开始日期距离现在远，结束日期距离现在近。聚类任务分析的数据就是这个时间段内机器人和访客的对话历史数据。

使用快捷方式创建聚类时间范围：今天，过去 24 小时，过去 3 天等。

下载聚类结果

提交聚类结果后，自动回到聚类分析页面，聚类分析页面的主要内容是聚类任务的列表，包括每个聚类任务的创建日期，聚类时间范围，状态。

- 聚类分析是将多个文本按照相似度归类，分成一个个聚簇。每个聚簇中任何两个问题都是相似的。
- 聚类分析常用来发现新知识，以及衡量用户在客服中的咨询情况分析，更多内容参考 [文档中心](#)。

创建任务

创建时间	数据开始日期	数据截止日期	状态	操作
2020-10-28 15:17:36	2020-07-28 00:00:00	2020-10-28 23:59:00	执行中	刷新
2020-10-28 11:05:12	2020-07-28 00:00:00	2020-10-28 23:59:00	已完成	下载结果
2020-10-28 10:25:05	2020-10-27 10:24:00	2020-10-28 10:24:00	异常结束	下载结果

状态分为：执行中、异常结束、执行完成。

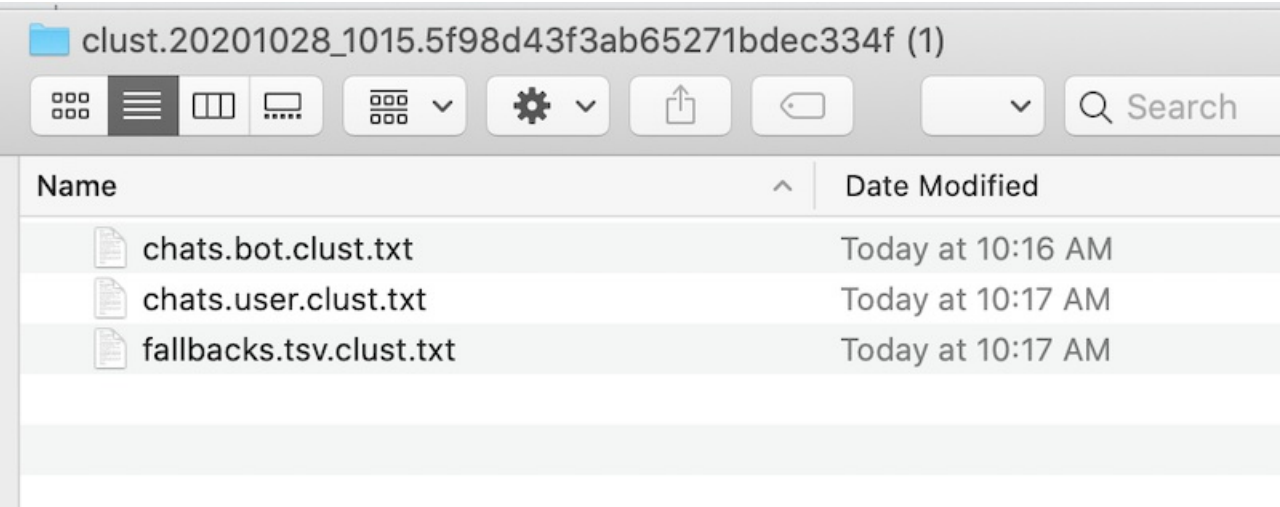
针对执行完成的聚类任务，在操作中可以下载结果，点击后浏览器将下载 Zip 压缩包，建议用户安装 [7-Zip 软件](#)。

执行中的聚类任务，可以通过点击操作中的刷新按钮更新状态信息。

对于异常结束，会有提示，通常是由于聚类时间范围内，没有发生对话历史记录的原因。

查看聚类结果

将上一步得到的压缩包解压，解压文件内包含如下文件：



Chatopera 机器人平台提供聚类输出包括：

- 1) 访客发送内容的聚类结果，文件：chats.user.tsv.clust.txt；
- 2) BOT 发送内容的聚类结果，文件：chats.bot.tsv.clust.txt；
- 3) BOT 回复为兜底回复时，用户的发送内容的聚类结果，文件：fallbacks.tsv.clust.txt；
- 4) 访客点击没有帮助时，发送的问题，聚类结果，文件：negative.tsv.clust.txt。

每个文件中，包含了聚簇、该聚簇关键词和聚簇内成员等信息。

如果所选聚类时间范围包含数据量太少，也会在聚类结果中提示，因为在这种情况下，没有聚类的价值，通过肉眼就可以完成对话分析。

聚类通常针对的是大规模数据，成千上万、乃至数十万数百万的对话历史记录。

CLI 连接多轮对话

Chatopera CLI 是命令行终端工具，[参考安装文档](#)。在测试 Chatopera BOT 多轮对话时，可以使用 CLI 连接到机器人的多轮对话服务，进行测试验证，使用 CLI 可以：

- 提升自动化任务
- 查看接口融合的返回值
- 快速验证对话技能

以下的代码是在命令行终端中运行，比如 Windows CMD、PowerShell、Git Bash、Linux Bash 等 Shell 环境。

以下的代码使用 CLI 版本是 [@chatopera/sdk 2.8.12](#)。

- 检查 Chatopera CLI 版本：`bot --version`
- 升级执行命令 `npm install -g @chatopera/sdk`

初始化

```
mkdir YOUR_FOLDER_NAME # 自定义一个目录
cd YOUR_FOLDER_NAME
bot env
```

运行后的日志如下：

```
$ bot env
.env file[/mnt/c/Users/Administrator/chatopera/cli/.../ts/.env] is generated.
ra.com/products/chatbot-platform/howto-guides/cli-install-config.html
```

此时在目录下生成了配置文件 `.env`：

```
$ cat .env
# Chatopera Cloud Service
BOT_PROVIDER=https://bot.chatopera.com
BOT_CLIENT_ID=
BOT_CLIENT_SECRET=
BOT_THRESHOLD_FAQ_BEST_REPLY=0.8
BOT_THRESHOLD_FAQ_SUGG_REPLY=0.6
BOT_ACCESS_TOKEN=
```

设置 `.env` 里的值：

值	描述
BOT_PROVIDER	Chatopera BOT 平台地址，默认为 http://bot.chatopera.com/
BOT_CLIENT_ID	机器人的 ClientID，在机器人控制台的设置界面获得
BOT_CLIENT_SECRET	机器人的 secret，在机器人控制台的设置界面获得
BOT_THRESHOLD_FAQ_BEST_REPLY	机器人知识库最佳回复阈值，默认为 0.8

值	描述
BOT_THRESHOLD_FAQ_SUGG_REPLY	机器人知识库建议回复阈值，默认为 0

机器人控制台的设置界面：

设置

基本信息

Client Id

Secret

复制

刷新

显示

* 机器人名称

入门教程测试

* 语言

zh_CN

是否开启繁体中文自动翻译①

关闭

描述

请描述机器人

会话模型

连接

在完成配置后，在 `.env` 文件所在文件夹下，执行下面命令，进行对话：

```
bot connect
```

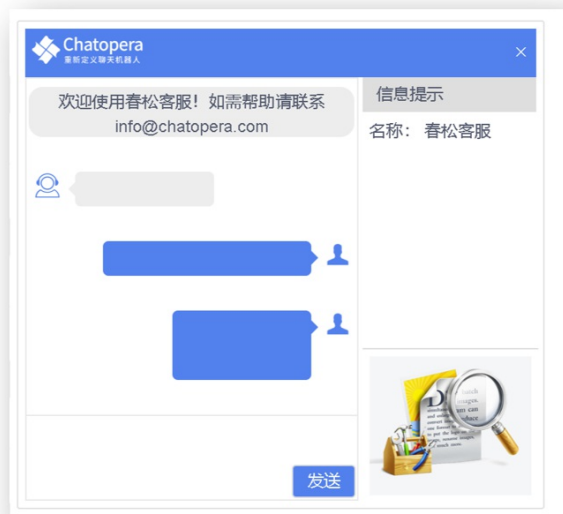
发送问题：

```
(Administrator@chatopera:~/cli) [ /c/Users/Administrator/chatopera/cli ] Fri Ap
$ bot connect
dirpath /mnt/c/Users/Administrator/chatopera/cli/saas_62e7...
>> load env file /mnt/c/Users/Administrator/chatopera/cli/saas_.../.env ...
2023-04-21 12:48:12 INFO >> connect to https://bot.chatopera.com, clientId 62e756889fb7..., secret
2023-04-21 12:48:12 INFO [connect] FAQ Best Reply Threshold 0.8, Suggest Reply Threshold 0
? Text 服务
2023-04-21 12:48:15 INFO {
  "rc": 0,
  "data": {
    "state": "default",
    "createdAt": 1682052492200,
    "string": "我不明白您的意思。",
    "topicName": null,
    "subReplies": [],
    "service": {
      "provider": "fallback",
      "faqBestReplyThreshold": 0.8,
      "faqSuggReplyThreshold": 0
    },
    "logic_is_fallback": true,
  },
}
```

在回复中，展示返回值 JSON 数据。

H5 聊天控件

Chatopera 云服务支持将聊天机器人立即发布到网页，这项功能就是 H5 聊天控件，具体功能和使用见下。



电脑客户端网页



手机客户端网页

支持网页聊天窗口

网页聊天窗口是 Chatopera 云服务为每个机器人提供的一个 URL 地址，打开该地址可以直接作为访客和机器人进行对话。

- 支持通过 URL 超链接打开一个网页聊天窗口，与机器人进行会话
- 打开的网页聊天窗口支持手机客户端和电脑客户端，自适应布局
- 打开的网页聊天窗口支持自定义形象、欢迎语、品牌标识
- 支持电脑客户端的网页中，自定义介绍信息、广告图片

支持网页聊天控件

H5 聊天控件另外一项能力是支持在已有的网页中添加浮动的按钮，点击按钮即跳转到网页聊天窗口。

- 支持自定义聊天控件在网页中的位置
- 支持延时加载聊天控件
- 支持自定义聊天控件中的文本
- 支持配置聊天控件的颜色
- 支持邀请功能及自定义邀请展示的信息



网页聊天控件



主动邀请

管理入口

以上信息, 可通过【机器人管理控制台-系统集成】找到 H5 聊天控件的信息及配置。

企业应用

企业应用是已经集成了 Chatopera 机器人平台的企业软件, 用户只需要配置即可使用。

		
接入设置 使用说明	使用说明 私有部署 定制开发	演示示例 私有部署 定制开发

文字链代码

将文字链代码 URL 发送给访客:

- 通过 IM 工具发送, 比如微信
- 将该 URL 使用工具生成二维码, 然后发送该二维码
- 将该 URL 作为超链接, 比如网页 HTML 超链接、微信公众号菜单的按钮的超链接、微信公众号文章阅读原文的地址

使用 H5 聊天控件的形式有很多, 只要访客在浏览器中打开【文字链代码】的 URL 即可与你的聊天机器人对话。

消息格式

H5 聊天控件中, 支持丰富的交互形式, 见 [函数返回值](#)。

机器人聊天码

如何快速的让消费者与聊天机器人对话，在客户服务中，提升自动化业务流程？

在线下，最好的方式莫过于消费者可以扫码，然后立即与聊天机器人对话。Chatopera 机器人聊天码，就是用来满足这一需求的。

获得聊天码

作为 Chatopera 云服务用户，您可以在机器人的管理控制台获得聊天码，在机器人导航菜单中，点击【机器人聊天码】即可。



现在，您就可以将聊天码打印粘贴了，是不是很简单？

配置聊天框属性

Chatopera 机器人聊天码，就是 Chatopera H5 聊天控件的 URL 地址的二维码。因此，聊天框的属性，可以使用机器人 Chatopera H5 聊天控件的配置进行设置。

详情[参考链接](#)。

本节以[Nodejs SDK](#) 为例子演示使用流程，更多 SDK 下载参考[链接](#)。

基础知识

[Nodejs](#) 是 JavaScript 运行时环境，面向服务器端应用开发，底层使用 Google V8 引擎。[Nodejs](#) 尤其被前端开发者偏爱，因为它让前端开发者以“熟悉”的方式开发后端应用。[Nodejs](#) 的出现一度降低了开发成本，并且成为“快应用”开发趋势出现，[Nodejs](#) 包管理工具 [Npm 站点](#) 是开源领域最大的包管理服务。不同语言的 SDK 使用细节大同小异，因为它们都是调用[Chatopera 机器人平台](#)的[RestAPIs](#)，这些 RestAPIs 是标准一致的。每种语言的 SDK 使用流程按照顺序包括：下载 SDK，实例化[Chatbot](#)类为对象，请求接口和处理返回结果。

获取 *ClientId* 和 *Secret*

[SDK](#) 中每个机器人实例需要通过*ClientId*和*Secret*初始化，这两个字段是认证和授权用途。打开机器人【设置】页面，拷贝*ClientId*和*Secret*。

控制台 IntentBot 创建机器人

设置

基本信息

Client Id: 613aff95d908ba001bc70dfc Secret: 复制 刷新 显示

* 机器人名称: IntentBot

* 语言: zh_CN 是否开启繁体中文自动翻译: 关闭

描述: 请描述机器人

会话模型

欢迎语: 你好! 我是机器人客服。 11/100

兜底回复: 我不明白您的意思。

安装 SDK

```
npm install @chatopera/sdk --save
```

实例化 [Chatbot](#) 类为对象

```
var Chatbot = require("@chatopera/sdk").Chatbot;  
var chatbot = new Chatbot(clientId, secret [, serviceProvider]);
```

参数说明

NAME	TYPE	REQUIRED	DESCRIPTION
clientId	string	☐	在 机器人控制台/机器人/设置 中获取
secret	string	☐	获取办法同上
serviceProvider	string	☐	Chatopera 机器人平台地址， 当使用 Chatopera 云服务时，该值为 https://bot.chatopera.com ，也是默认值

提示： 参数列表中，写在 `[]` 内的部分是选填参数，如果不填写使用默认值，下同。

调用接口示例

得到 `Chatbot` 实例后，怎么样请求接口服务呢？假设对该机器人的基本信息感兴趣，获取基本信息方式如下：

```
var response = await chatbot.command("POST", "/faq/query", {
  query: "不锈钢板现在是什么价格",
  fromUserId: "sdktest1",
});
console.log("response: ", response)
```

或者获取 `Promise` 返回

```
chatbot.command("GET", "/").then(
  (response) => {
    console.log("机器人名称: ", response.data.name);
  },
  (err) => {}
);
```

此处，不深入探讨 `await` 和 `Promise` 的相关知识，它们是和 JavaScript 语言相关的内容。在这个例子中，我们请读者注意，给定一个机器人类的实例，再请求 API 服务是多么的简单，`Chatbot#command` 接口提供了一系列的方法，也是下文给您详细介绍的重点。

示例程序

示例程序代码库: <https://github.com/chatopera/webchat>

通过系统集成的示例程序，快速掌握 SDK 使用，尤其是对话检索 API；系统集成示例程序也可以用来调试和检查机器人，在和业务系统集成前测试相关接口。与 Chatopera 机器人平台的测试页面不同，系统集成示例程序更侧重上线前，对 SDK 相关接口的测试。

在系统集成示例程序中，使用 Chatopera 机器人平台地址，clientId 和 secret 立刻连接聊天机器人，开始对话。

功能：

- 提供对话页面，方便系统集成测试
- 使用 Bot Provider 地址，clientId 和 secret 连接机器人
- 实现 Dialogue Management: 融合意图识别检索、多轮对话检索和知识库检索
- [appjs](#) 使用 [Chatopera Node.js SDK](#)，可作为系统集成参考

启动应用

本示例程序提供 Docker 容器镜像，使用 Docker 启动服务进行体验

```
docker run -it --rm -p 8668:8668 chatopera/webchat:develop
```

Chatopera Test Client

Service Provider

https://bot.chatopera.com

Client ID

Client Secret

Username

tc

Chat

CLI 安装和配置

Chatopera CLI 是连接 Chatopera 机器人平台，管理和维护资源的工具，包括一些常用的命令，辅助开发者实现和管理对话机器人。尤其是在有自动化或批量管理的需要时。

安装

Bash

因为使用过程中，依赖于命令终端界面，在 Linux 或 Mac 上，有 Bash（或兼容）环境。

对于 Windows 用户，请安装 [Git Bash for Windows](#)，安装完成后，在桌面右键，就可以点击【Git Bash】进入命令终端界面。

本文以下内容，涉及 Shell 命令，都假设命令终端界面是 Bash Shell 环境。

Node.js

Chatopera CLI 依赖于 [Nodejs v10+](#) 环境，使用 `npm` 进行安装（`npm` 是 `Nodejs` 安装完成后得到的 CLI 工具）。

- Windows 上安装 Node.js 及配置环境变量[参考文档](#)
- Linux 上安装 Node.js [参考文档](#)

Chatopera CLI

有了 Node.js 环境，在命令行终端，执行如下命令：

```
npm install -g @chatopera/sdk
```

在 Windows 上，安装过程输出类似的日志：

```
C:\Users\Administrator\AppData\Roaming\npm\bot -> C:\Users\Administrator\AppData\Roaming\npm\node_modules\@chatopera\sdk\bin\bot.js
+ @chatopera/sdk@2.7.1
added 147 packages from 104 contributors in 36.389s
```

检查安装是否正确，执行：

```
$ bot --version
2.x # 得到类似输出，代表安装正确，2.x 为当时最新的 Chatopera CLI 版本
```

如果上述命令 `bot --version` 执行提示错误，检查环境变量 `PATH` 路径，比如 `C:\Users\Administrator\AppData\Roaming\npm` 是否在 `PATH` 内。

配置参数

Chatopera CLI 命令行工具支持读取文件配置变量，比如 `provider`，`clientid` 等常用的变量。

参数设定优先级: 命令行参数 > `.env` 文件 > 环境变量

映射配置项	命令行参数	环境变量	备注
clientid, 机器人 ID	<code>-c, --clientid [value]</code>	<code>BOT_CLIENT_ID</code>	无默认值，必填
secret, 机器人密钥	<code>-s, --clientsecret [value]</code>	<code>BOT_CLIENT_SECRET</code>	无默认值，必填

映射配置项	命令行参数	环境变量	备注
provider, Chatopera 机器人平台地址	<code>-p, --provider [value]</code>	<code>BOT_PROVIDER</code>	默认值, https://bot.chatopera.com
accessToken, 访客设置密钥	无	<code>BOT_ACCESS_TOKEN</code>	保留值, 无默认值
faqBest, FAQ best reply threshold	<code>-fb, --faq-best [value]</code>	<code>BOT_THRESHOLD_FAQ_BEST_REPLY</code>	FAQ 知识库最佳回复阈值, 默认值 0.8
faqSugg, FAQ suggest reply threshold	<code>-fs, --faq-sugg [value]</code>	<code>BOT_THRESHOLD_FAQ_SUGG_REPLY</code>	FAQ 建议回复阈值, 默认值 0.6

其中, `.env` 文件例子如:

```
# Chatopera Cloud Service
BOT_PROVIDER=https://bot.chatopera.com
BOT_CLIENT_ID=
BOT_CLIENT_SECRET=
BOT_ACCESS_TOKEN=
```

`.env` 文件存储的也是环境变量值。 `bot` 命令会沿当前执行命令的路径(`pwd`), 寻找 `.env` 文件。

比如, 在 `/Users/chatopera/chatopera-nodejs-sdk` 下执行 `bot` 命令, 那么, `.env` 文件按照以下顺序进行查找, 一旦查找到就加载为配置, 并退出查找。

```
/Users/chatopera/chatopera-nodejs-sdk/.env
/Users/chatopera/.env
/Users/.env
/.env
```

下一步

- [命令行界面 \(CLI\) 参考手册: 帮助、使用命令参考手册](#)
- [使用 CLI 导入和导出对话语料](#)

CLI 导入和导出对话语料

Chatopera CLI 支持的功能就是对不同对话语料的数据导入和导出；外加一些其他命令，辅助自动化管理机器人。

准备工作

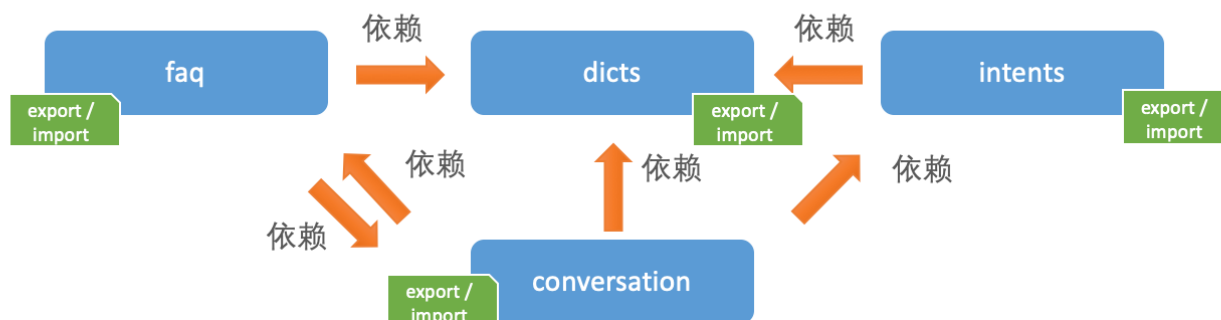
- 安装 Chatopera CLI，参考文档 [Chatopera CLI](#);
- 配置 Chatopera CLI，使用 `.env` 文件，参考文档 [Chatopera CLI](#)。

为简化说明，以下各示例配置使用了 `.env` 文件，因为略去了从命令行传入的一些参数。

导入和导出命令规范

针对于不同模块的导入和导出命令，CLI 命令是遵循同一个规范的。

```
bot [dicts|faq|intents|conversation] --action [import|export] --filepath {{FILE_PATH}}
```



如果 dicts 后导入，则需要执行 ``bot dicts -a sync`` 同步到其它模块。

如果是导出（`export`），没有加入 `--filepath` 部分，则会默认保存文件到当前工作目录（`cwd` 目录）。

导出对话语料

对话语料指词典、知识库、意图识别和多轮对话的内容，它们包括了机器人的所有对话技能。

删除文件

```
rm -rf bot.dicts.json bot.faqs.json bot.intents.json bot.conversations.c66
```

执行导出，以下命令并无顺序依赖关系

```
bot dicts -a export -f bot.dicts.json && \
bot faq -a export -f bot.faqs.json && \
bot intents -a export -f bot.intents.json && \
bot conversation -a export -f bot.conversations.c66
```

导入对话语料

将到处的对话语料上传给指定的机器人（比如通过 `.env` 设置）。

以下命令有顺序依赖关系

```
bot dicts -a import -f bot.dicts.json && \  
bot faq -a import -f bot.faq.json && \  
bot intents -a import -f bot.intents.json && \  
bot conversation -a import -f bot.conversations.c66
```

这样，目标机器人就具备从之前导出的机器人的技能。

CLI 命令详细介绍

更多命令介绍，参考文档 [Chatopera CLI](#)。

现有系统词典

系统词典可以直接引用，无需配置词条。

已有的系统词典包括：地名(@LOC)，人名(@PER)、时间(@TIME)、组织机构(@ORG)和任何字符串(@ANY)。在引用系统词典页面中可以看到

 Chatopera
企业聊天机器人

Beta

 控制台 | ceshi | + 创建机器人



← 引用系统词典

- 系统词典可以直接引用，无需配置词条。
- 被引用了的系统词典可以在槽位配置时使用。

词典标识名	词典中文名	词典示例	操作
@PER	人名	郭德纲;于谦	引用
@TIME	时间	今天;下午一时	引用
@LOC	地名	北京市;东京	引用
@ORG	组织机构	北京华夏春松科技有限公司	引用

共 4 条

<

1

>

各个系统词典的作用

名称	描述	示例
@LOC	地名	北京市、东京
@PER	人名	郭德纲、于谦
@TIME	时间	今天、下午一时
@ORG	组织机构	北京华夏春松科技有限公司
@ANY	任何字符串	请求发送的任何字符串

引用与取消引用

对于系统词典可以引用和取消引用。被引用了的系统词典可以在槽位配置时使用

← 引用系统词典

- 系统词典可以直接引用，无需配置词条。
- 被引用了的系统词典可以在槽位配置时使用。

词典标识名	词典中文名	词典示例	操作
@PER	人名	郭德纲;于谦	引用
@TIME	时间	今天;下午一时	引用
@LOC	地名	北京市;东京	取消引用
@ORG	组织机构	北京华夏春松科技有限公司	引用

共 4 条

<

1

>

词汇表词典就是管理词条，一个词汇是否属于某词汇表词典，就是检查该词汇是否属于该词典的词条集合。

创建

命名规则:

- 词典标识名为字母、数字、下划线的组合，1-32 位，如：RailTypes
- 同一个机器人下的词典标识名不能重复
- 词典标识名一经确认后无法进行修改

在词典管理页面点击新建自定义词典，在弹出的对话框里输入正确的自定义词典名称，词典类型选择“词汇表”，点击确定

 Chatopera
企业聊天机器人

 控制台 | ceshi2 | + 创建机器人



词典管理

- 词典包括自定义词典和系统词典。自定义词典是开发者自己创建的词典；系统词典是平台提供给开发者使用的常用词典。
- 词典主要用于意图中槽位的识别和填充，不同机器人的自定义词典不能共享。

新建自定义词典

引用系统词典

系统词典

以下是已经引用的系统词典列表

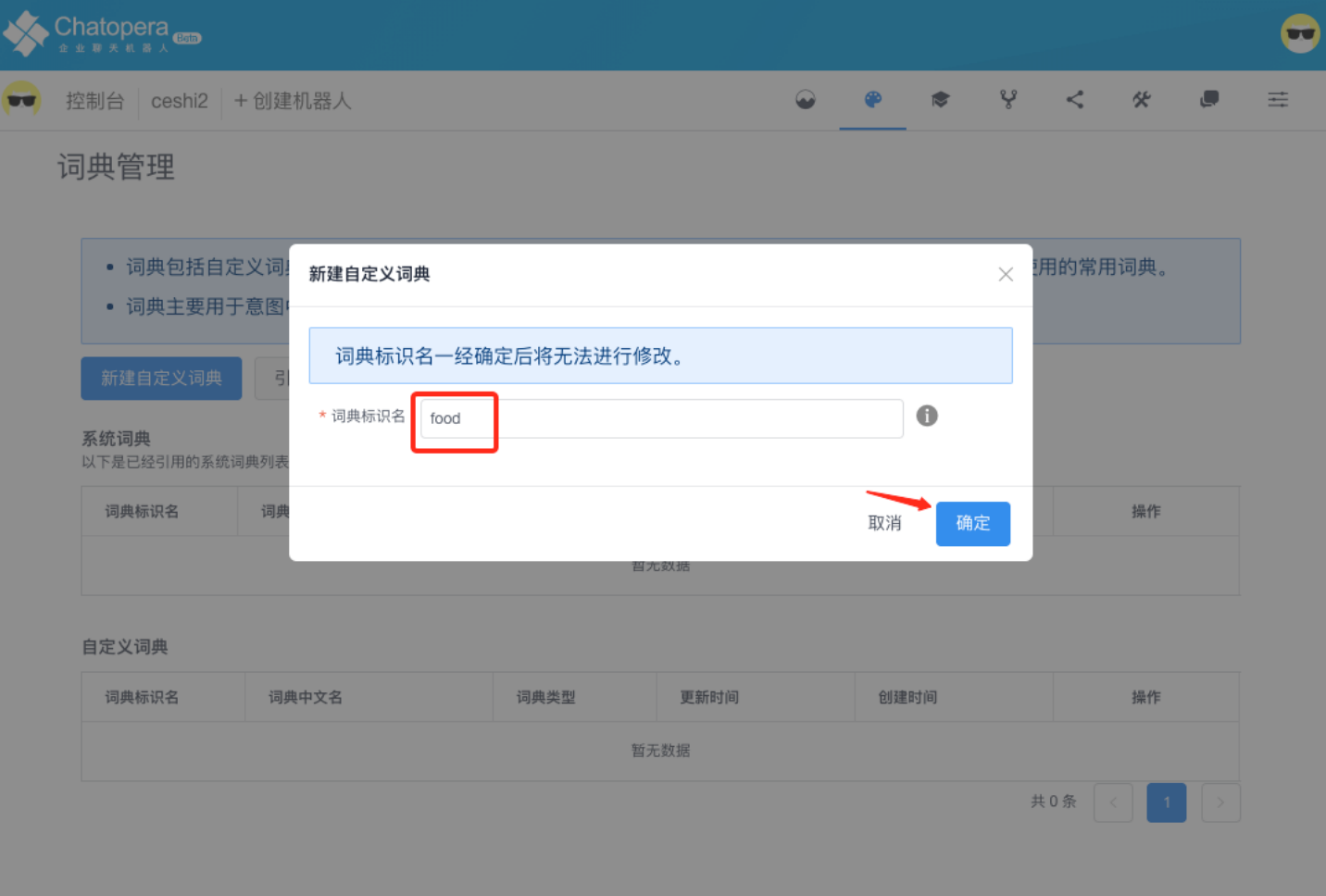
词典标识名	词典中文名	词典类型	词典示例	更新时间	创建时间	操作
暂无数据						

自定义词典

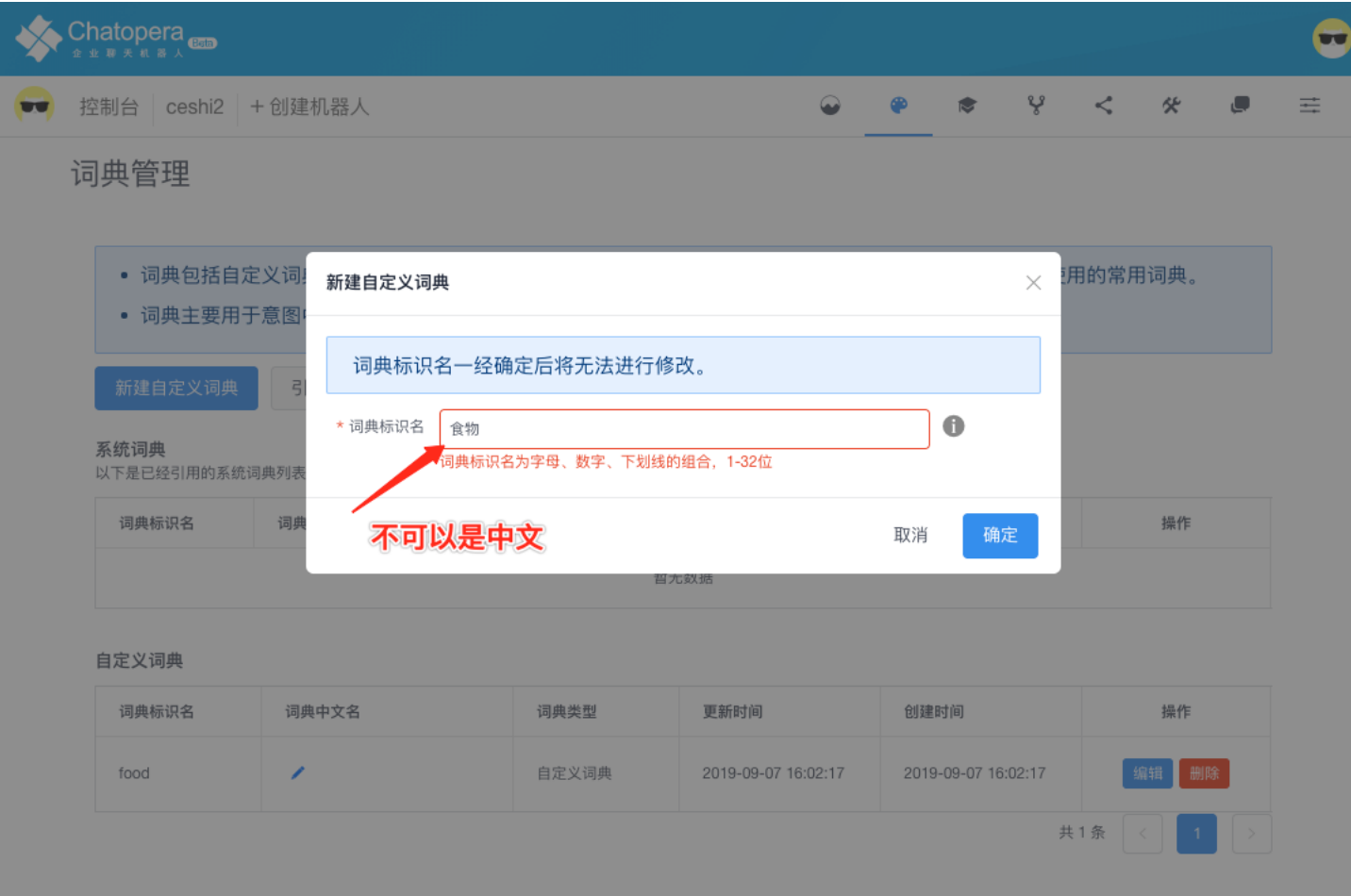
词典标识名	词典中文名	词典类型	更新时间	创建时间	操作
暂无数据					

共 0 条

1



如果创建的自定义词典名称不符合要求，会有相应的提示



对于已经创建的自定义词典可以修改中文名

 Chatopera Beta
企业聊天机器人



 控制台

ceshi

+ 创建机器人



词典管理

• 词典包括自定义词典和系统词典。自定义词典是开发者自己创建的词典；系统词典是平台提供给开发者使用的常用词典。

• 词典主要用于意图中槽位的识别和填充，不同机器人的自定义词典不能共享。

新建自定义词典

引用系统词典

系统词典

以下是已经引用的系统词典列表

词典标识名	词典中文名	词典类型	词典示例	更新时间	创建时间	操作
暂无数据						

自定义词典

词典标识名	词典中文名	词典类型	更新时间	创建时间	操作
num		自定义词典	2019-09-07 16:08:27	2019-09-07 16:08:27	编辑 删除
taset		自定义词典	2019-09-06 23:14:30	2019-09-06 23:14:30	编辑 删除
food	食物 保存 取消	自定义词典	2019-09-06 23:13:15	2019-09-06 23:13:15	编辑 删除

共 3 条

<

1

>

删除

对于不需要的自定义词典点击删除按钮即可

词典管理

- 词典包括自定义词典和系统词典。自定义词典是开发者自己创建的词典；系统词典是平台提供给开发者使用的常用词典。
- 词典主要用于意图中槽位的识别和填充，不同机器人的自定义词典不能共享。

新建自定义词典

引用系统词典

系统词典

以下是已经引用的系统词典列表

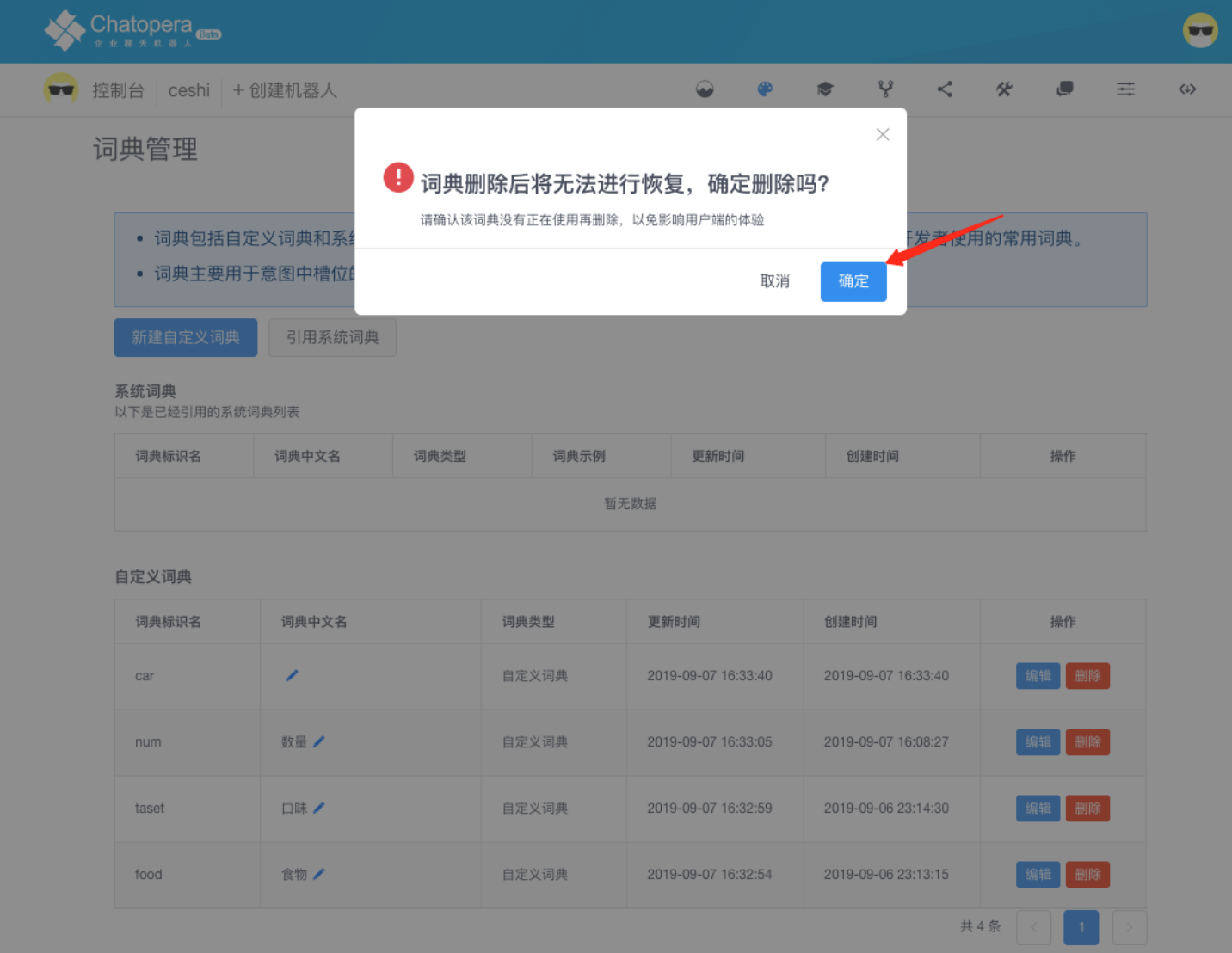
词典标识名	词典中文名	词典类型	词典示例	更新时间	创建时间	操作
暂无数据						

自定义词典

词典标识名	词典中文名	词典类型	更新时间	创建时间	操作
car		自定义词典	2019-09-07 16:33:40	2019-09-07 16:33:40	编辑 删除
num	数量 	自定义词典	2019-09-07 16:33:05	2019-09-07 16:08:27	编辑 删除
taset	口味 	自定义词典	2019-09-07 16:32:59	2019-09-06 23:14:30	编辑 删除
food	食物 	自定义词典	2019-09-07 16:32:54	2019-09-06 23:13:15	编辑 删除

共 4 条

<1>



对于已经绑定了槽位的自定义词典不可删除

 Chatopera
企业聊天机器人

该词典已经绑定槽位，不可删除



 控制台 | ceshi | + 创建机器人



词典管理

- 词典包括自定义词典和系统词典。自定义词典是开发者自己创建的词典；系统词典是平台提供给开发者使用的常用词典。
- 词典主要用于意图中槽位的识别和填充，不同机器人的自定义词典不能共享。

新建自定义词典

引用系统词典

系统词典
以下是已经引用的系统词典列表

词典标识名	词典中文名	词典类型	词典示例	更新时间	创建时间	操作
暂无数据						

自定义词典

词典标识名	词典中文名	词典类型	更新时间	创建时间	操作
num	数量 	自定义词典	2019-09-07 16:33:05	2019-09-07 16:08:27	编辑 删除
taset	口味 	自定义词典	2019-09-07 16:32:59	2019-09-06 23:14:30	编辑 删除
food	食物 	自定义词典	2019-09-07 16:32:54	2019-09-06 23:13:15	编辑 删除

共 3 条

<

1

>

添加词条

请使用分号；分隔词项，第一个词项将作为标准词，后面的词项将作为同义词。

词条的增删改查

添加词条：编辑完成后点击添加或者回车。

← food

- 开发者可以根据业务需要为自己创建的自定义词典添加多个词条。
- 一个词条包含至少一个标准词和不限个数的同义词，同义词会被识别为标准词。

添加词条

标准词: 饺子 × 同义词: 水饺 ×

添加 按Enter键快速添加

输入词条进行搜索

标准词 ①	同义词 ①	操作
米饭		编辑 删除
面条		编辑 删除
馒头		编辑 删除

<

1

>

保存 取消

删除词条：点击删除按钮即可。

修改词条：一个词条的标准词一旦确定不可修改，但是同义词可以修改，点击编辑按钮，出现输入框，可以对同义词进行修改。

← food

- 开发者可以根据业务需要为自己创建的自定义词典添加多个词条。
- 一个词条包含至少一个标准词和不限个数的同义词，同义词会被识别为标准词。

添加词条

请使用分号：分隔词项，第一个词项将作为标准词，后面的词项将作为同义词

添加按Enter键快速添加

输入词项进行搜索

标准词	同义词	操作
饺子	水饺	编辑删除
米饭	大米 ×米 ×大米饭 × 请使用分号：分隔多个同义词	保存取消
面条		编辑删除
馒头		编辑删除

<

1

>

保存取消

查找词条：对于词条较多对词典来说，可以快速搜索词条。在搜索框输入需要搜索的词条之后点击放大镜或者点击回车键即可搜索。搜索完成后搜索框中出现一个清除按钮，点击之后清除本次搜索。

← food

- 开发者可以根据业务需要为自己创建的自定义词典添加多个词条。
- 一个词条包含至少一个标准词和不限个数的同义词，同义词会被识别为标准词。

添加词条

请使用分号：分隔词项，第一个词项将作为标准词，后面的词项将作为同义词

添加 按Enter键快速添加

米

标准词 ①	同义词 ①	操作
米饭	大米;米;大米饭	编辑 删除
		< 1 >

保存 取消

保存

在编辑完所有词条之后，需要进行保存，否则变更不生效。比如本页词条有变更的情况下进行翻页或者搜索，都需要先进行保存。保存成功后，会重新训练机器人。

← food

- 开发者可以根据业务需要为自己创建的自定义词典添加多个词条。
- 一个词条包含至少一个标准词和不限个数的同义词，同义词会被识别为标准词。

添加词条

请使用分号；分隔词项，第一个词项将作为标准词，后面的词项将作为同义词

添加

按Enter键快速添加

输入词项进行搜索



标准词 ①	同义词 ②	操作
饺子	水饺	编辑删除
米饭	大米;米;大米饭	编辑删除
面条		编辑删除
馒头		编辑删除

<

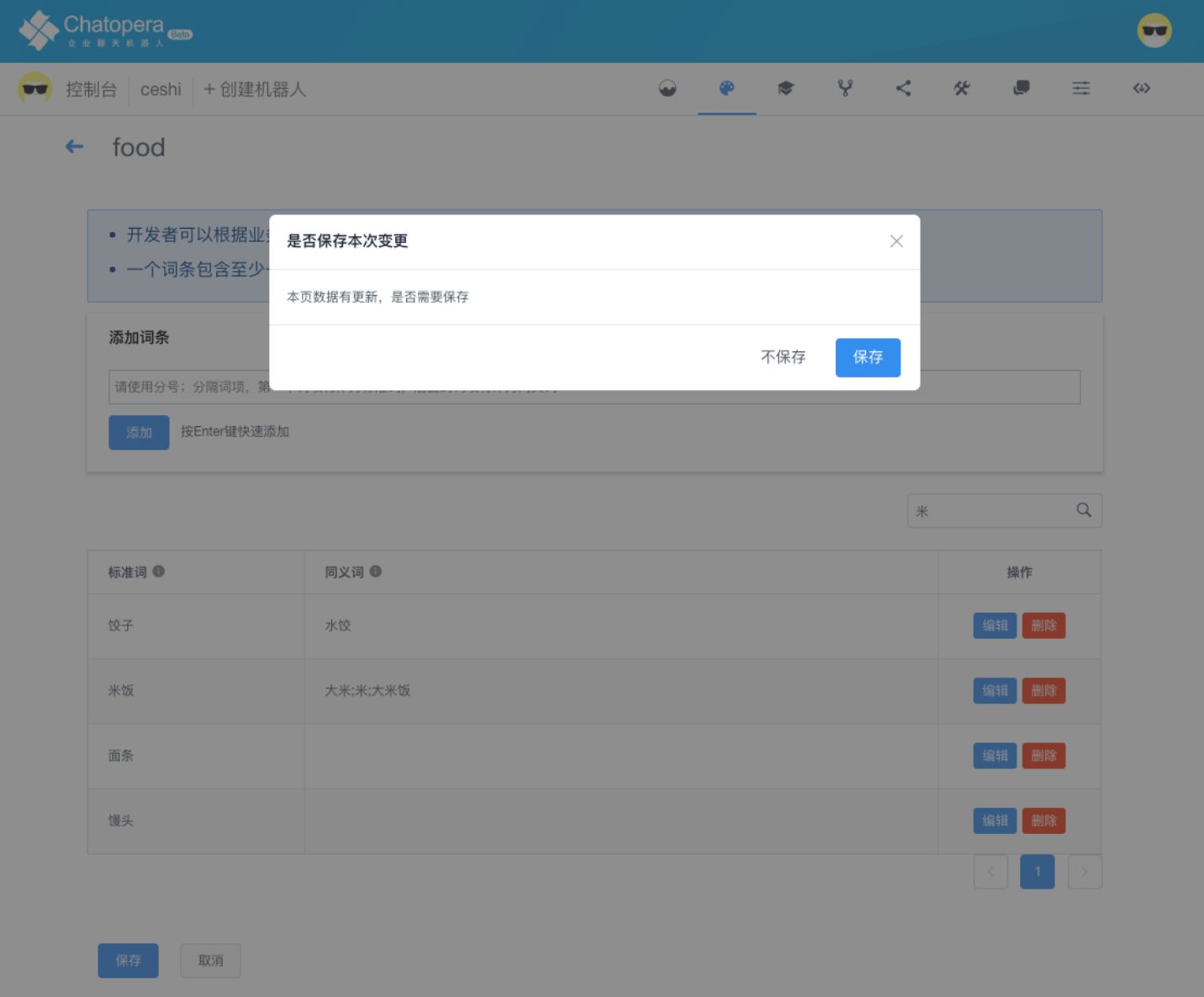
1

>

保存中

取消

保存成功，正在训练



维护

为了用户体验更好，需要对自定义词典进行维护与管理

创建时：在创建词典添加词条时，尽可能多的，全面的添加不同的词条和同义词，便于在槽位中被识别出来（如果用户说的词条在该自定义词典中没有，就会识别不出来）

使用时：在历史消息中可以查看所有机器人与用户的对话，如果发现用户说的词条在该自定义词典中没有，应该立即加上。

正则表达式词典是通过正则表达式的形式约束一个词汇的集合，正则表达式是以一定语法作为规则的语法。

创建

命名规则：

- 词典标识名为字母、数字、下划线的组合，1-32 位，如：RailTypes
- 同一个机器人下的词典标识名不能重复
- 词典标识名一经确认后无法进行修改

在词典管理页面点击新建自定义词典，在弹出的对话框里输入正确的自定义词典名称，词典类型选择"正则表达式"，点击确定

Chatopera

企业聊天机器人

控制台

ceshi2

+ 创建机器人

词典管理

词典包括自定义词典和系统词典。自定义词典是开发者自己创建的词典；系统词典是平台提供给开发者使用的常用词典。

词典主要用于意图中槽位的识别和填充，不同机器人的自定义词典不能共享。

新建自定义词典

引用系统词典

系统词典

以下是已经引用的系统词典列表

词典标识名	词典中文名	词典类型	词典示例	更新时间	创建时间	操作
暂无数据						

自定义词典

词典标识名	词典中文名	词典类型	更新时间	创建时间	操作
暂无数据					

共 0 条

<

1

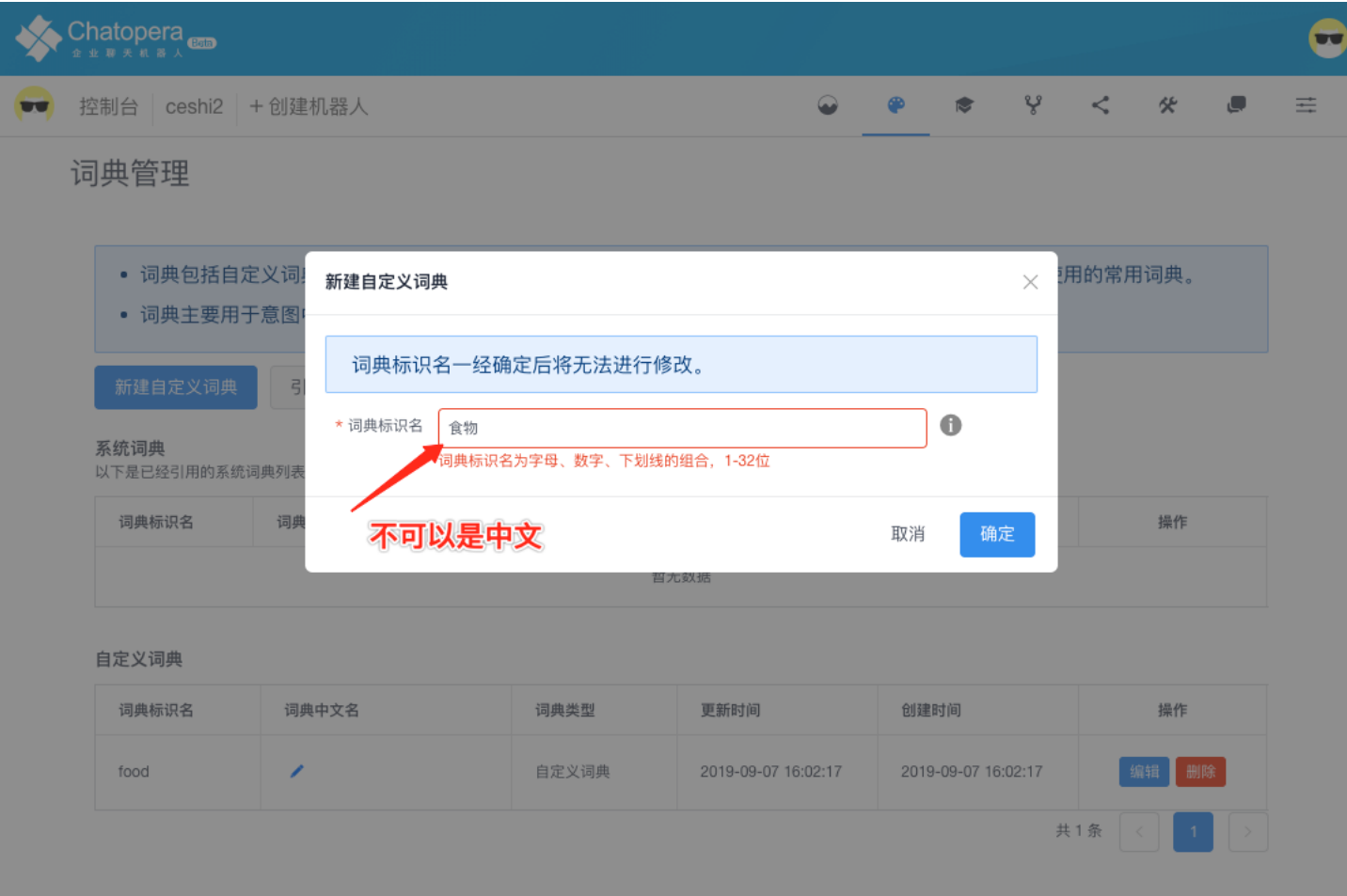
>

178/290

https://bot.chatopera.com



如果创建的自定义词典名称不符合要求，会有相应的提示



编辑

Chatopera 机器人平台中，正则表达式词典支持的表达式规范为PCRE 标准。

添加正则表达式定义，进入【词典管理>>自定义词典】列表，选择正则表达式词典，点击"编辑"。

← IDNumber

- 开发者可以根据业务需要为自己创建的自定义词典添加正则表达式。
- 添加的正则表达式应符合规范

添加表达式

\w[-\w.+]*@[A-Za-z0-9](-A-Za-z0-9)+\.[A-Za-z]{2,14}

添加按Enter键快速添加

正则表达式

\w[-\w.+]*@[A-Za-z0-9](-A-Za-z0-9)+\.[A-Za-z]{2,14}

操作

编辑删除

保存✔ 训练成功，可进行测试重置

在"添加表达式"项目下，写入表达式，点击"添加"，然后点击"保存"。保存成功后会重新训练机器人，如果遇到错误会弹出提示信息，对于没有意图或意图中没有说法的情况，训练机器人会失败，这时候保存正则表达式词典是成功的，该错误可忽略，后续在添加了意图和说法后，就正常了。

正则表达式定义支持多个表达式，在使用正则表达式词典时，会按照正则表达式列表的顺序进行匹配，如果匹配到词汇，就跳过其它表达式，否则继续匹配，直到遍历表达式列表。

保存成功后，还需要确认表达式是能按照预期工作，这个验证的过程通过【验证表达式】完成，该项目在表达式列表的下面。

正则表达式

操作

\w[-\w.+]*@[A-Za-z0-9](-A-Za-z0-9)+\.[A-Za-z]{2,14}

编辑删除

保存✔ 训练成功，可进行测试重置

验证表达式

能发电子发票到我的邮箱吗？foo@bar.com

是否匹配：是

能发电子发票到我的邮箱吗？foo@bar.com

匹配表达式：\w[-\w.+]*@[A-Za-z0-9](-A-Za-z0-9)+\.[A-Za-z]{2,14}

验证

验证表达式过程就是输入一段测试文本，点击"验证"，验证结果将显示在下面。

常用正则表达式

正则表达式是一个易学难精的技能，一方面是介绍材料众多，另一方面是网络上有很多写好的常用的表达式。以下定义三个常用的表达式。

项目	表达式
手机号	(?:\+?86)?1(?:3\d{3} 5[^4\D]\d{2} 8\d{3} 7[^0129\D](?:<=4)(?:0\d 1[0-2] 9\d) \d{2}) 9[189]\d{2} 66\d{2})\d{6}
邮箱	\w[-\w.+]*@([A-Za-z0-9][-A-Za-z0-9]+\.)+[A-Za-z]{2,14}
身份证号	\d{15}(\d\d[0-9xX])?
QQ 号	[1-9][0-9]{4,}

正则表达式的基础知识参考[链接](#)，更多关于正则表达式材料在网络上自行查找，相关 **Perl** 正则表达式或 **PCRE** 正则表达式的适用于 **Chatopera** 机器人平台的正则表达式词典。

表达式注意事项

上文提到了 Chatopera 机器人平台的正则表达式词典是基于 C++ Boost Regex 库的，并使用 PCRE 规范，实际上和 PCRE 有细微出入，以下内容进行介绍。

- 1. 如果在 `[]` 之间需要包含 `-`，并且不代表区间，而是 `-` 代表本身，那么需要将 `-` 放在末尾或开头，否则会报错；
- 2. 在表达式使用中只取第一个匹配的值，如果一个表达式能匹配到输入语句的多个值，其它的值被忽略。

PCRE

PCRE 是“Perl-compatible regular expressions”的缩写，广泛的被各种编程语言支持，虽然在不同语言支持时，会有微小的不同，实际使用中仍可参考 PCRE 的说明，Chatopera 机器人平台使用 C++ 开发，底层的正则表达式接口使用[Boost Regex 库](#)

示例程序

使用示例程序, [参考文档](#)。

- [讲笑话机器人](#)
- [GuessNumberBot](#)
- [预定机票机器人](#)
- [招聘面试机器人](#)
- [冰箱报修服务机器人](#)

更多示例程序, [进入 projects ...](#)

参考手册

- [术语](#)
- [通配符匹配器](#)
- [内置函数库](#)
- [函数返回值](#)
- [命令行工具（CLI）](#)
- [SDK](#)
- [常见问题](#)

在多轮对话实现的过程中，解决的是一个复杂问题：能够让机器能分析和执行的自然语言的聊天过程。为此，需要定义一些专用的术语和概念。

多轮对话

在多轮对话实现的过程中，解决的是一个复杂问题：能够让机器能分析和执行的自然语言的聊天过程。为此，不得不定义了一些专用的术语和概念。

多轮对话设计器

Conversation Designer，根据需求撰写对话脚本，对话函数的软件工具。

对话、话题

Topic / Topic Name，这几个词代表同一个概念：由一些脚本组成的、满足预期对话能力的单元。机器人的对话能力由多个对话主题组成，对话主题是对话脚本模块化管理的形式。

输入

Input，用户向聊天机器人发送的消息的文字形式。

匹配器

Gambit，匹配用户输入文字的规则，包括以下三种：

- 通配符匹配器 / Star Gambit，写法规则，约束字符串集合
- 模糊匹配器 / Like Gambit，使用机器学习匹配用户输入和匹配器包括的成员之间的相似度，如果相似度大于阈值则认为匹配，在机器人控制台或语法规则内设置阈值
- 意图匹配器 / Intent Gambit，调用意图识别，在成功获得槽位信息或超过最大追问次数后，调用回复

回复

Reply，机器人回复用户输入的文字。

上次回复

Last Reply，作为机器人给来访者回复的最近一次的内容。

上下轮钩子

Conversation Hooks，通过上次回复，链接匹配器，形成多轮对话。

函数

Function，可以从脚本中接受输入，并通过 JavaScript 执行任务返回结果的代码。

对话状态

Conversation State，多轮对话是按照状态机的模型设计的，对话状态就是当前对话处于状态机的什么状态。

对话应用

Conversational App，c66 文件，从多轮对话设计器导出的文件。

对话模板

Chatbot Sample，聊天机器人示例程序，开源地址，<https://github.com/chatopera/chatbot-samples>。



意图识别

词典

机器人的词汇表

- 自定义词典：通过词汇表，正则表达式来设定词汇集合，表达实体概念
- 系统词典：Chatopera 机器人平台内置地名、组织机构名、人名和时间等

意图

一个对话任务的最小单元

- 说法：表达同样意图的不同说法，可包含槽位标识
- 槽位：在意图中，完成用户任务需要的信息

调试

从设计到实现机器人的技能

- 训练机器人：建立词汇表，意图，为意图添加槽位和说法
- 测试对话：在测试对话页面，测试意图识别，优化训练内容

版本

每次训练都会得到新的意图识别模型，系统创建新的版本

- 调试版本：调试中的机器人版本
- 生产版本：训练好的机器人模型，满足上线使用需求，从调试版本发布而成

会话

也称为 session，和用户的单个意图关联的对话；在系统集成时，会话生命周期，部分阶段由开发者管理。

- 请求对话，需要先创建会话，会话会绑定 0-1 个任务：刚开始不知道用户意图，当确定用户意图后，该 session 就只和这个意图相关。

聊天

发送访客的请求，获得机器人的回复

- 通过访客的 ID，session id，文本消息和机器人进行聊天

会话 和 聊天 都是与上线集成相关，详细参考 [系统集成](#)。

其它术语

对话语料

指机器人的词典内容、意图识别内容、知识库问答对和多轮对话脚本、函数和环境变量，以上内容的集合。对话语料就是机器人的对话能力依赖的所有原料。

内置函数库

在开发中, 进一步提供系统灵活性, 在 `函数` 定义中, 系统预置了以下几个工具对象, 方便聊天机器人开发者实现各种功能的聊天机器人。以下内容假设读者已经掌握了 `JavaScript` 编程语言。

要点提示

注意 `this` 的使用

一些 builtin 的函数只在 `exports` 的函数内可用, 注意在这些函数前的 `this`, 它们是必须的。使用 `this` 前缀的变量代表该变量依赖于在函数内部的 `this` 对象。比如:

```
exports.customFn = async function(){
  debug(this.user.id); // this.user is available here inside an exports function
}
```

JavaScript 运行时

目前, Chatopera 机器人平台为函数提供的 JavaScript 运行时是 `node:10.21.0`, 发布于 2020-06-02。一些微小的 [最新的 Nodejs](#) 的 LTS 上的功能可能并不支持, `v10.21.0` 目前是应用最多的 LTS 版本, 更稳定和兼容广泛。

函数库

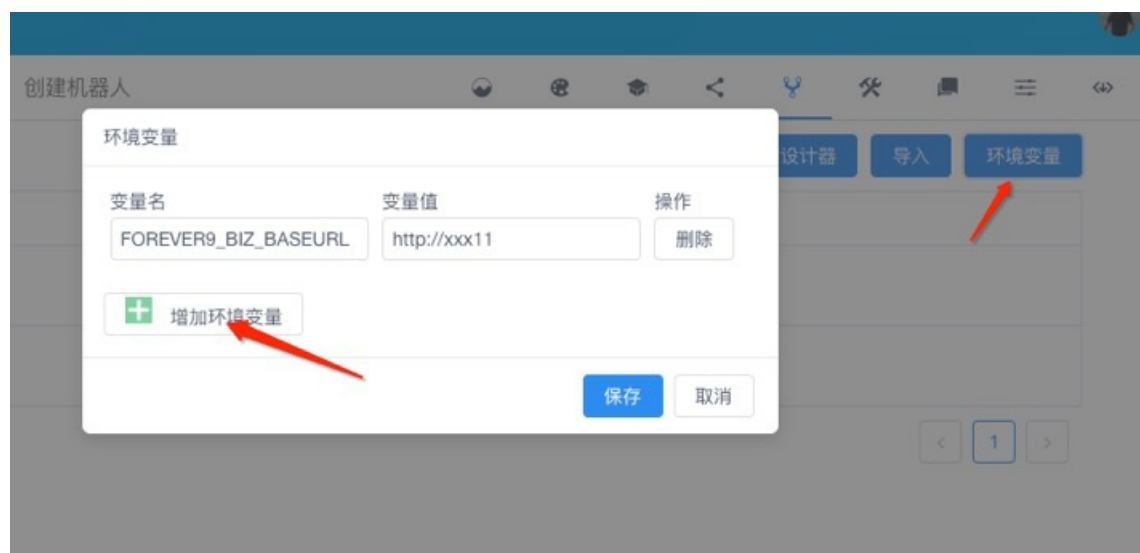
- [basics](#)
- [message](#)
- [user](#)
- [maestro](#)
- [3rd-party](#)

basics

基本函数库。

config

获取环境变量，环境变量在多轮对话设计器和聊天机器人多轮对话控制台都可以定义，目的是在设计阶段和运行阶段，多轮对话使用不同的配置。



`config` 作为函数中的全局常量，不需要用户定义，不支持改变该值，只能作为读取用途，并且 `config` 只是键值对，值只是 `string` 类型。

```
exports.print_key_value = function(key, cb){
  // 通过通配符获得key，查看其在环境变量中对应值
  cb(null, "Ok, value is " + config[key]);
}
```

- `config` 更多使用示例代码: <https://github.com/chatopera/chatbot-samples/search?q=config>

`环境变量` 常用来配置一些生产环境对应的信息。

http

用于在函数内部，通过 HTTP 协议集成外部系统。

`http` 作为函数中的全局常量，不需要用户定义。

```
http.get(url[, config])
http.delete(url[, config])
http.head(url[, config])
http.options(url[, config])
http.post(url[, data[, config]])
http.put(url[, data[, config]])
http.patch(url[, data[, config]])
```

Chatopera 机器人平台中，函数内置 `http` 常量使用 `axios` 包实例化，`http` 即 `axios`。

- `http` 更多使用示例代码: <https://github.com/chatopera/chatbot-samples/search?q=http>
- `axios` 详细使用文档: <https://github.com/axios/axios>

打印调试日志。

`debug` 作为函数中的全局常量，不需要用户定义。

```
debug("hello")
debug("hello %s", stringVar)
debug("hello %s, %j", stringVar, jsonVar)
debug("hello %s, %o", stringVar, objVar)
```

- `debug` 更多使用示例代码: <https://github.com/chatopera/chatbot-samples/search?q=debug>

—

`lodash` 是一个一致性、模块化、高性能的 JavaScript 实用工具库。Lodash 通过降低 `array`、`number`、`objects`、`string` 等等的使用难度从而让 JavaScript 变得更简单。Lodash 的模块化方法 非常适用于：

- 遍历 `array`、`object` 和 `string`
- 对值进行操作和检测
- 创建符合功能的函数

鉴于这些特点，Chatopera 内置函数库增加了全局变量 `_` 来使用 `lodash v4.17.15` 的接口。

比如，可以在函数中这样写将 JSON Array 转化为 JSON Object。

```
var array = [
  { 'dir': 'left', 'code': 97 },
  { 'dir': 'right', 'code': 100 }
];

_.keyBy(array, function(o) {
  return String.fromCharCode(o.code);
});
// => { 'a': { 'dir': 'left', 'code': 97 }, 'd': { 'dir': 'right', 'code': 100 } }

_.keyBy(array, 'dir');
// => { 'left': { 'dir': 'left', 'code': 97 }, 'right': { 'dir': 'right', 'code': 100 } }
```

更多 `lodash` 方法，参考 <https://lodash.com/docs/4.17.15>。

message

`this.message` 是每次用户输入文本经过自然语言处理后的对象，它并不是全局对象，必须在 `exports` 的函数中，使用 `this.message` 来引用。

请求文本

即对话用户发送的文本内容。

```
this.message.original // String, 输入文本的原始内容
this.message.clean // String, 输入文本的改写, 经过词根转化的输入字符串
```

`this.message` 还包括丰富的自然语言处理信息，如下的函数打印 `words` 对应的 `tags` (词性), `lemms` (词根)。

```
exports.getCustomDictnames = async function() {
  debug("words %j, %j, %j", this.message.words, this.message.tags, this.message.lemms);

  let terms = _.zip(this.message.words, this.message.tags, this.message.lemms);

  for (let i in terms) {
    debug("No. %d, word: %s, tag: %s, lemm: %s", i, terms[i][0], terms[i][1], terms[i][2]);
  }

  return {
    text: "hello"
  }
}
```

下面是详细介绍。

词性信息

```
this.message.words // JSONArray, 分词结果
this.message.tags // JSONArray, 对应位置分词的词性
this.message.entities // JSONArray, 输入文本包含的命名实体
this.message.names // JSONArray, 输入文本包含的人名
this.message.nouns // JSONArray, 输入文本包含的名词
this.message.adverbs // JSONArray, 输入文本包含的副词
this.message.verbs // JSONArray, 输入文本包含的动词
this.message.adjectives // JSONArray, 输入文本包含的形容词
this.message.pronouns // JSONArray, 输入文本包含的指示代词
```

关于 `this.message.tags` 包含了所有的分词对应的词性，可以用来做更多的判断，不同语言的词性集是不同的，参考 [Chatopera 多轮对话 Message 语言词性标注](#)。

词根信息

词根是一个词的标准词，在[词典管理](#)中，通过词汇表词典，可以定义标准词及其关联的近义词，那么 `this.message.lemms` 会存储 `words` 对应的标准词。

```
this.message.lemms // String, 输入文本的改写, 经过词根转化的输入字符串
```

当一个词汇(`word`) 不在自定义词汇表词典，或本身就是标准词时，在 `this.message.lemms` 里的对应信息就是自身。

使用详细介绍[参考链接](#)。

自定义信息

在多线程对话检索接口中，传入自定义的信息 `extras`。

```
this.message.extras
```

传入方法参考，[多线程对话检索接口](#)。

user

在函数中获得对话用户的信息。

this.user.id

当前对话用户标识，在 [多轮对话检索](#) 传入的 `userId` 信息。

在函数中，获取该唯一标识信息：

```
this.user.id
```

比如：

```
exports.myUserId = function(cb){
  cb(null, {
    text: "Your id is " + this.user.id
  })
}
```

this.user.history

当前用户与机器人对话的历史

```
this.user.history // 类型：数组
```

其中，每个元素内容如下：

```
{
  "input" : {
    "timestamp" : ISODate("2020-07-28T17:06:01.458Z"),
    "original" : "hello",
    "extras": {}
  },
  "reply" : {
    "createdAt" : 1595955961672.0,
    "string" : "#in-params#",
    "topicName" : "greetings",
    "clearConversation" : false,
    "props" : {
      "params" : [
        {
          "hyperlink" : "http://",
          "thumbnail" : "http://xx.png",
          "summary" : "描述",
          "title" : "标题",
          "type" : "card"
        }
      ]
    }
  },
}
```

其中，*original*就是来访者的输入，*reply*是上一轮对话中机器人的回复。*extras* 是通过多轮对话接口传入的自定义信息，传入方法参考，[多轮对话检索接口](#)。

历史对话按照降序排列，即最近发生的对话在上面，最多存储 100 轮历史对话数据。

maestro

`maestro` 是管理对话状态存储和自然语言处理的高级接口，它并不是全局对象，必须在 `exports` 的函数中，使用 `this.maestro` 来引用。

注意事项:

1. 对于返回值是 `Promise` 的接口，使用 `try catch`, `await` 进行调用可读性更好。
2. 该对象不是全局函数，作用域在函数内部，使用时依赖于函数内 `this` 变量。

User

对话用户信息相关，此部分为 `maestro` 内实现的，和对话用户相关的还有 [user](#) 库。

profile

设置用户画像

```
await this.maestro.profile(userId, property, value) 返回值 Promise
```

userId: 用户唯一标识

property: 用户的属性

value: 属性值

该接口用于持久化一个用户的信息，此处 *userId* 可以使用当前用户的信息 `this.user.id`；`property` 和 `value` 是根据业务灵活定义。

该信息被持久化到数据库里，该接口的目的是设置用户画像，比如用户对一个产品是否有兴趣。

用户画像可以通过[系统集成/用户管理/获取用户画像信息 API](#)获得。

KeyValue Pairs

用于管理持久化的信息，通过键值对存取。

set

存储键值对，支持过期时间，过期时间以秒为单位。VALUE 可以是时间，字符串，数字。

该命令也可以用于更新键值对，或者更新 EXPIRES 时间，让该信息不过期。

```
this.maestro.set(KEY, VALUE [, EXPIRES]) 返回值: Promise
```

ttd

获得键值对有效时间，键值对是存储在 Redis 服务中，如果设定键值对时同时对 `EXPIRES` 赋值，那么到达过期时刻，该键值对将被删除。使用 `ttd` 接口获得一个键值对有效存在时间，返回值是有效的秒数，如果返回值为 `-1` 则代表该值不存在。

```
this.maestro.ttd(KEY) 返回值: Promise(number)
```

incrby

增加键的值，对于 Number 类型的键，增加一定值

```
this.maestro.incrby(KEY, NUMBER) 返回值: Promise
```

获得一个键的值

```
this.maestro.get(KEY) 返回值: Promise
```

del

删除一个键和其值

```
this.maestro.del(KEY) 返回值: Promise
```

Extract information

信息提取

extractNumber

获得一句文本中的阿拉伯数字，比如 来访者提到“一二三”，“123”等数字，会被提取为一个转化为阿拉伯数字的数组。

```
let numbers = await this.maestro.extractNumber(cap1);
```

返回值: `numbers` 是处理后的数字数组，识别的文本中，可以包含多个数字；未获得数字时，数组长度为 0。比如

```
let numbers = await this.maestro.extractNumber("一二三和123? ");  
// numbers [ '123', '123' ]
```

extractTime

获得绝对时间，比如 来访者提到“明天”，“后天”，“下周一”等相对时间，会被计算出正确的绝对时间，时区为北京时间。

```
let dates = await this.maestro.extractTime(cap1[, format, timezone]);
```

其中，`format` 格式参考[momentjs#format](#)，如果设定 `format` 的值为 `long`，返回值数组中数据格式为数字，类似于使用 JavaScript `(new Date()).getTime()` 获得的数据。

`timezone` 值设定参考[momentjs#timezone](#)。

`format` 默认值为 `YYYY/MM/DD HH:mm`，`timezone` 默认值为 `Asia/Shanghai`。

返回值: `dates` 是处理后的时间数组，识别的文本中，可以包含多个时间；可以具体到分钟，格式化时，未得到的时间被设置为 `0`。

比如

```
let dates = await this.maestro.extractTime("明天和后天是几月几号?", "YYYY年MM月DD日", "Asia/Shanghai");  
// dates [ '2021年09月02日', '2021年09月03日' ]
```

extractRegexDicts

提取指定文本中，包含的[自定义正则表达式词典](#)信息。

```
let matches = await this.maestro.extractRegexDicts(string);
```

返回值:

```
{
  "词典标识": [
    {
      "val": 匹配值,
      "begin": 开始位置,
      "end": 结束位置,
    }
  ],
  ...
}
```

举例，比如在函数中使用：

```
exports.getCustomRegexDictnames = async function() {
  debug("this.message.original %s", this.message.original)
  let matches = await this.maestro.extractRegexDicts(this.message.original);
  debug("[extractRegexDicts] matches %j", matches);

  return {
    text: "hello "
  }
}
```

返回值：

```
{
  "email": [
    {
      "begin": 18,
      "end": 28,
      "val": "doo@cc.com"
    }
  ]
}
```

查看详细[使用说明](#)。

digest

自动生成文本的摘要，假设文本有“。！？”等分隔句子的符号。那么 `digest` 将会获得其中最关键的句子作为摘要。

```
let texts = await this.maestro.digest("冬天来了。春天还会远吗?", 5);
```

第一个参数文本内容，第二个参数是返回摘要文本的字数长度。

返回值 `texts` 是一个数组，是输入文本中的若干句子，按照重要程度排序。

keywords

提取文本关键词。

```
let words = await this.maestro.keywords("冬天来了。春天还会远吗?", 5);
```

第一个参数文本内容，第二个参数是返回关键词语的数量。返回值 `words` 是一个数组，是输入文本中的若干词语，按照重要程度排序。

FAQs Integrations

在 `maestro` 集成[知识库](#)的相关功能，知识库是通过问答对的形式管理机器人的语料。

在函数中检索该机器人的知识库。

```
let results = await this.maestro.searchFAQs(text, topk);
debug("results faqs %j", results);
```

其中, `text` 是搜索条件, 不能为 null 或空; `topk` 是最多检索返回多少相关信息, 默认为 20。比如,

```
let results = await this.maestro.searchFAQs("黄金的主要产地", 10);
debug("results faq %j", results);
```

返回值 `results` 是一个数组 (JSONArray), 每个成员是一个问答对信息, 数据按照 `score` 降序排序。数据结构如下:

```
[
  {
    "id": "{{DOC_ID}}",
    "score": 1,
    "post": "黄金的主要产地",
    "replies": [
      {
        "rtype": "plain",
        "enabled": true,
        "content": "美国, 圣地亚哥"
      }
    ],
    "categories": [
      "标的"
    ]
  }
]
```

其中, `score` 为相似度 ([0, 1] 区间); `post` 为问答对标准问; `id` 是问答对 ID; `replies` 为答案信息; `categories` 为分类信息。

以下介绍更为详细的使用例子:

```
# 脚本
+ {keep} 查询知识库
- ^searchBotFAQ()

# 函数
exports.searchBotFAQ = async function() {
  let results = await this.maestro.searchFAQs("黄金的主要产地", 1);
  debug("results faq %j", results);

  return {
    text: "知识库返回"
  }
}
```

getFAQList

获得该机器人知识库的问答对列表。


```
let params = {
  query: {{text}},
  limit: 10,
  page: 1,
  categories: []
};
let results = await this.maestro.getFAQList(params);
debug("[getFAQListTest] results faq %j", results);
```

其中, 参数 `params` 中字段的使用如下:

- `params.query` - 字符串, 查询语句, 模糊匹配问题; 非必填, 默认为空;
- `params.limit` - 正整数, 每页返回问答对条数; 非必填, 默认为 10;
- `params.page` - 返回列表的页码, 从 1 开始的正整数; 非必填, 默认为 1;
- `params.categories` - 问答对的分类信息; 非必填, 默认为 `[]`; 填写字符串数组, 数组内元素按照顺序, 从左到右分别是一级分类, 二级分类, 三级分类, 以此类推, 最多支持四级, 比如 `["动物", "海洋动物"]`

返回值 `results` 是一个数组(JSONObject), 如下:

```
{
  "total": 1,
  "current_page": 1,
  "total_page": 1,
  "data": [
    {
      "createtime": "2024-02-01T05:38:05.853Z",
      "post": "虎鲸",
      "categories": [
        "动物",
        "海洋动物"
      ],
      "updatetime": "2024-02-01T06:00:27.684Z",
      "enabled": true,
      "id": "uxQsY40BLFVcfeSHoai6",
      "inquiryscore": 1
    }
  ]
}
```

其中, 字段的含义如下:

- `total` - 符合查询条件的总的记录条数
- `current_page` - 当前数据列表的页码
- `total_page` - 全部符合条件的记录的总页码
- `data` - JSONArray, 问答对概要信息 (`post` 标准问, `categories` 分类, `enabled` 是否启用, `id` docId, `inquiryscore` 问题的热度), docId 可以用于继续检索该问答对的详情, 见 `getFAQDetail` 的说明。

使用举例

`getFAQList` 的使用, 可以结合脚本语法, 下面举一个例子:

```
+ 查询 (*)
- {keep} ^searchFaqTest(<cap1>)
```

```
exports.searchFaqTest = async function(text) {
  let params = {
    query: text,
    limit: 10,
    page: 1,
  };
  let results = await this.maestro.getFAQList(params);
  debug("[searchFaqTest] results faq %j", results);

  return "hello"
}
```

由上, 注意 `<cap1>` 和 `text` 的用法。用户输入时, [通配符匹配值](#)可以传入到查询语句中。

getFAQDetail

根据指定的知识库问答对标识 (docId) 返回问答对详情。

```
exports.searchFaqTest2 = async function() {
  let docId = "foo"; // 字符串
  let result = await this.maestro.getFAQDetail(docId);
  debug("[searchFaqTest] result faq %j", result);
  return "hello"
}
```

其中, `docId` 就是问答对的唯一标识。返回值有两种情况: 1) 该问答对存在; 2) 该问答对不存在。

问答对存在的返回值

获得了详情 (`rc = 0`), `result` 结果类似:

```
{
  "rc": 0,
  "id": "uxQsY40BLFVcfeSHoai6",
  "inquiryscore": 5,
  "post": "虎鲸",
  "categories": [
    "动物",
    "海洋动物"
  ],
  "enabled": true,
  "replies": [
    {
      "rtype": "plain",
      "enabled": true,
      "content": "2121"
    }
  ]
}
```

问答不对存在的返回值

未找到 (`rc = 1`)。

```
{ "rc": 1, "error": "Not found." }
```

getHotFAQs

该函数返回知识库热门问题列表, 按照热门度降序排列, 如果所有热门度分数都是 0, 那么随机返回。

```
let results = await this.maestro.getHotFAQs(topk);
debug("getHotFAQs results %j", results);
```

其中, `topk` 为大于 0 的正整数。 `results` 内容类似:

```
[
  {
    "docId": "0aU-xxx",
    "inquiryscore": "0",
    "post": "{{问题1}}",
    "enabled": true,
    "categories": []
  },
  {
    "docId": "OKU-yyy",
    "inquiryscore": "0",
    "post": "{{问题2}}",
    "enabled": true,
    "categories": []
  }
]
```

其中, `inquiryscore` 为热门度分数, `post` 为问答对中的问题。

Notifications

通知服务。

sendMail

通过邮件服务器发送邮件, 使用 [nodemailer](#) 实现。

```
exports.sendMail = async function(argvs) {

  let mailSettings = {
    service: config['MAIL_SERVICE'],
    auth: {
      user: config['MAIL_ACCOUNT'],
      pass: config['MAIL_PASSWORD']
    }
  };

  let transporter = this.maestro.nodemailer.createTransport(mailSettings);
  // setup e-mail data with unicode symbols
  let mailOptions = {
    from: `HR<${config['MAIL_ACCOUNT']}>`, // sender address
    to: config['MAIL_RECEP'], // list of receivers
    subject: `【应聘】小主, 新增候选人了, 岗位${opening}`, // Subject line
    text: '', // plaintext body
    html: content, // html body
  };
  // send mail with defined transport object
  transporter.sendMail(mailOptions, function(error, info) {
    if (error) {
      debug(error);
    }
  });
};
```

以上 SMTP 邮件服务的配置需要定义在环境变量中, 支持的邮箱服务参考[nodemailer](#), 可配置 QQ 企业邮箱、163 邮箱等。

其他第三方函数库。

Octokit

[Octokit](#) 是 GitHub 官方推出的 RestAPIs Toolkit，可以方便的使用 JavaScript 管理 GitHub 资源，请求服务。

在 Chatopera 机器人平台内置函数库中，包含了 Octokit 类，可以直接在函数中，完成 GitHub Issue 创建，查询等任务，使用举例如下：

```
const octokit = new Octokit({
  auth: "YOUR_GITHUB_PERSONAL_ACCESS_TOKEN"
});
exports.handleOpenGithubIssue = async function() {
  await octokit.request(`POST /repos/chatopera/cskefu/issues`, {
    owner: "chatopera",
    repo: "cskefu",
    title: "YOUR TITLE",
    body: "YOUR BODY",
    labels: ["label1" "label2"]
  });
}
```

其中，`YOUR_GITHUB_PERSONAL_ACCESS_TOKEN` 是每位 GitHub 注册用户自己创建的 Personal Access Token，字符串类似于：`ghp_i75Hmkg1xxx`。

更多关于 `Octokit` 以及 `Octokit#request` 的介绍，[参考链接](#)，更多 `GitHub RestAPIs` 的介绍，[参考链接](#)。

- 在函数中使用 Octokit 更多使用示例代码：<https://github.com/chatopera/chatbot-samples/search?q=Octokit>

函数返回值

函数的返回值，可以用于自定回复消息内容或状态跳转等。在多层对话中，回复除纯文本外，还可以支持多媒体消息，这样用户交互的体验更佳。不同消息的类型还需要渠道能够兼容，或在业务系统中进行适配。已经默认完成适配的渠道或软件：

- [Chatopera H5 聊天控件](#)

列表消息

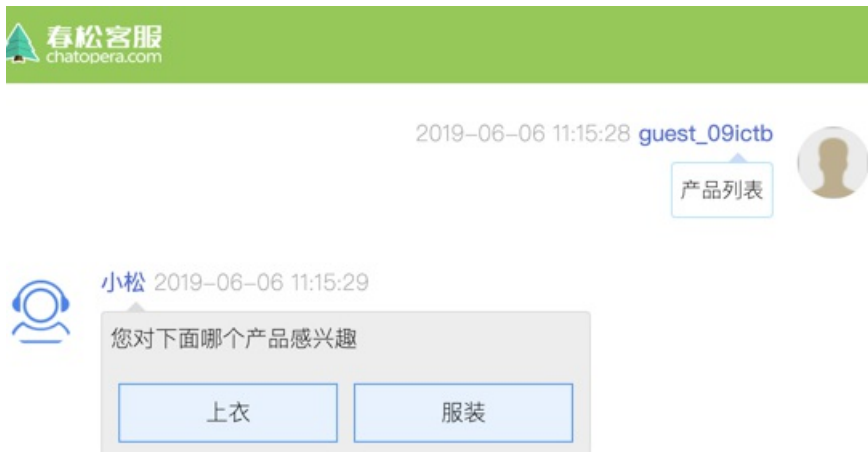


函数返回值中，`params` 数组中每个元素约定如下：

```
// 问候语中关联常见问题
exports.getGreetings = async function() {
  return {
    text: "请问有什么可以帮到您？",
    params: [{
      label: "1. 产品列表",
      type: "qlist",
      text: "产品列表"
    }, {
      label: "2. 当季热销产品",
      type: "qlist",
      text: "当季热销产品"
    }, {
      label: "3. 退换货咨询",
      type: "qlist",
      text: "退换货咨询"
    }
  ]
};
}
```

其中，`label` 是显示文字，`type` 为固定值 `qlist`，`text` 是点击该问题时发送给机器人的文本内容。

按钮消息



```
// 按钮选择消息
exports.get_products = async function() {
  return {
    text: "您对下面哪个产品感兴趣",
    params: [{
      label: "上衣",
      type: "button",
      text: "介绍一下上衣"
    },
    {
      label: "服装",
      type: "button",
      text: "介绍一下鞋帽"
    }
  ]
}
```

类似图文消息，不同的是：1）type 值为 button；2）业务上一般会设定一组按钮消息只能有一个按钮被点击一次。

图文消息



```
// 图文消息
exports.get_shangyi = async function() {
  return {
    text: "{CLEAR} 图文消息",
    params: [{
      type: 'card',
      title: "秋冬上衣优选",
      thumbnail: "https://ss0.bdstatic.com/xx.jpg",
      summary: "秋冬上衣优选秋冬上衣优选秋上",
      hyperlink: "https://www.1688.com/B6AC.html"
    }
  ]
}
```

此处，因为显示图文消息以包含文字，text 需要使用固定值 {CLEAR} 图文消息，type 为固定值 card；title，thumbnail，summary 和 hyperlink 分别是图文消息的标题，缩略图，描述和超链接地址。

提示： {CLEAR} 作为前缀代表机器人本轮清除多轮对话状态。

自定义业务字段

从以上的消息形式的实现，就是借助函数返回值中 params 这个属性，在实现聊天机器人时，函数中可在 params 自定义的业务字段，都会被 SDK/API 返回。

```
// 自定义业务字段
exports.get_shangyi = async function() {
  return {
    text: "以帮助您下单了。",
    params: {
      orderId: "BIL-001"
    }
  }
}
```

通常是在业务系统内，使用业务字段完成更多工作。

命令行界面（CLI）

Chatopera CLI 是连接 Chatopera 机器人平台，管理和维护资源的工具，包括一些常用的命令，辅助开发者实现和管理对话机器人。尤其是在有自动化或批量管理的需要时。CLI 完全基于 [Chatopera Node.js SDK](#)。

- [安装和配置 Chatopera CLI](#)
- [使用 CLI 导入和导出对话语料](#)

命令帮助

打印 CLI 可用命令。

```
bot --help
```

得到类似输出：

```
Usage: bot [options] [command]

Options:
  -V, --version          output the version number
  -h, --help             display help for command

Commands:
  details [options]      get a bot's detail info, such as name, primaryLanguage
  connect [options]      chat with bot via bot#conversation interface
  conversation [options] import or export a bot's conversations data
  trace [options]        tail a bot's conversations logging info
  faq [options]          import or export a bot's faqs data
  dicts [options]        sync, import or export a bot's dicts data
  intents [options]      train, import or export a bot's intents data
  help [command]         display help for command
```

也可以针对一个命令，获得更多帮助提示，比如：

```
bot connect --help
bot trace --help
bot conversation --action export --help
```

获得类似输出：

```
Usage: bot connect [options]

Options:
  -c, --clientid [value]  ClientId of the bot
  -s, --clientsecret [value] Client Secret of the bot, optional, default null
  -u, --username [value]  Username to chat with bot, default: cmdline
  -p, --provider [value]  Chatopera Bot Service URL, optional, default
                          https://bot.chatopera.com
  -fb, --faq-best [value] FAQ best reply threshold, optional, default 0.8
  -fs, --faq-sugg [value] FAQ suggest reply threshold, optional, default
                          0.6
  -h, --help             display help for command
```

配置文件

env

生成配置文件，对生成的配置文件，需要修改，设置键值对。


```
bot env [-fp FOLDER]
```

默认为当前命令执行所在目录。

比如：

```
bot env -fp /c
```

将生成配置文件 `/c/.env`。

词典

词典在知识库、多轮对话、意图识别中都有使用和依赖，在导入知识库文件、多轮对话文件或意图识别文件之前，最好是先导入词典文件，以免为使用带来影响。

export

导出引用的系统词典、所有自定义词典（词汇表词典和正则表达式词典）。

举例：

```
bot dicts --action export --filepath /tmp/bot.dicts.json
```

import

导入引用的系统词典、所有自定义词典（词汇表词典和正则表达式词典）。

举例：

```
bot dicts --action import --filepath /tmp/bot.dicts.json
```

sync

触发同步命令，知识库、意图识别和多轮对话同步最新的近义词词典；此步骤将引起数据改写，生产环境宜业务低峰时间段进行。

```
bot dicts --action sync
```

多轮对话

connect

连接聊天机器人，在命令行终端连接 BOT 并进行对话。

示例：

```
bot connect -c xxx -s xxx -u zhangsan
```

其中，`clientid`和`clientsecret`从每个机器人的设置页面获取，`username`代表用户名，是一个不含空格或特殊符号的字符串，每个用户的唯一标识，`provider`是[Chatopera 机器人平台](#)地址，默认为[Chatopera 云服务](#)。

在对话中，可以使用快捷方式，快速输入。

快捷方式	MAC OSX / WINDOWS
回溯历史	↑ 上箭头；↓ 下箭头

快捷方式	MAC OSX / WINDOWS
打印历史	Shift + → 右箭头
使用索引输入历史，索引根据打印历史获得	输入索引，然后 Ctrl + Shift + Shift + → 右箭头

export

导出多轮对话为 c66 文件。

示例：

```
bot conversation --action export --filepath /tmp/bot.conversations.c66
```

import

导入多轮对话脚本，在命令行终端发布脚本文件到多轮对话中。

示例：

```
bot conversation --action import --filepath /tmp/bot.conversations.c66
```

其中 filepath 为 xx.c66 文件，支持相对路径和绝对路径。

因为多轮对话可能使用了词典，所以宜先导入词典，再导入多轮对话。

trace

打印聊天机器人日志：方便调试多轮对话脚本，实时跟踪服务器端日志，排查问题。

示例：

```
bot trace --log-level DEBUG
```

Log level 可以是 [DEBUG|INFO|WARN|ERROR]。

知识库

export

导出知识库问答对、扩展问和分类信息等，目前 CLI 导入和导出只支持 JSON 格式，欲使用 Excel 从机器人平台浏览器管理界面完成。

举例：

```
bot faq --action export --filepath /tmp/bot.faqs.json
```

import

导入知识库问答对、扩展问和分类信息等。

举例：

```
bot faq --action import --filepath /tmp/bot.faqs.json
```

因为知识库可能使用了词典，所以宜先导入词典，再导入知识库。

意图识别

export

导出意图识别说法、槽位等信息。

```
bot intents --action export --filepath /tmp/bot.intents.json
```

import

导入意图识别说法、槽位等信息。

```
bot intents --action import --filepath /tmp/bot.intents.json
```

因为意图识别可能使用了词典，所以宜先导入词典，再导入意图识别。

导入命令也会自动执行训练意图调试分支，训练完成后，命令退出。

意图识别的导入和导出不会处理生产版本上线信息和操作，需要 Chatopera 机器人平台用户自行维护意图识别模块的生产分支。

train

训练意图识别的调试分支。

```
bot intents --action train
```

已知问题

在 Windows WSL 内调用 SDK，返回异常 invalid timestamp

错误原因：Windows WSL 的时间同步有 BUG，未能和互联网时间同步。

解决方案：执行手动的时间同步。

假设使用的 WSL 发行版本基于 Ubuntu，那么可以执行下面的命令解决。

```
sudo apt-get install ntpdate  
sudo ntpdate time.windows.com
```

开源项目

Chatopera CLI 工具是[开源的](#)，更多使用示例参考：

<https://github.com/chatopera/chatopera-nodejs-sdk/tree/master/scripts>

下一步

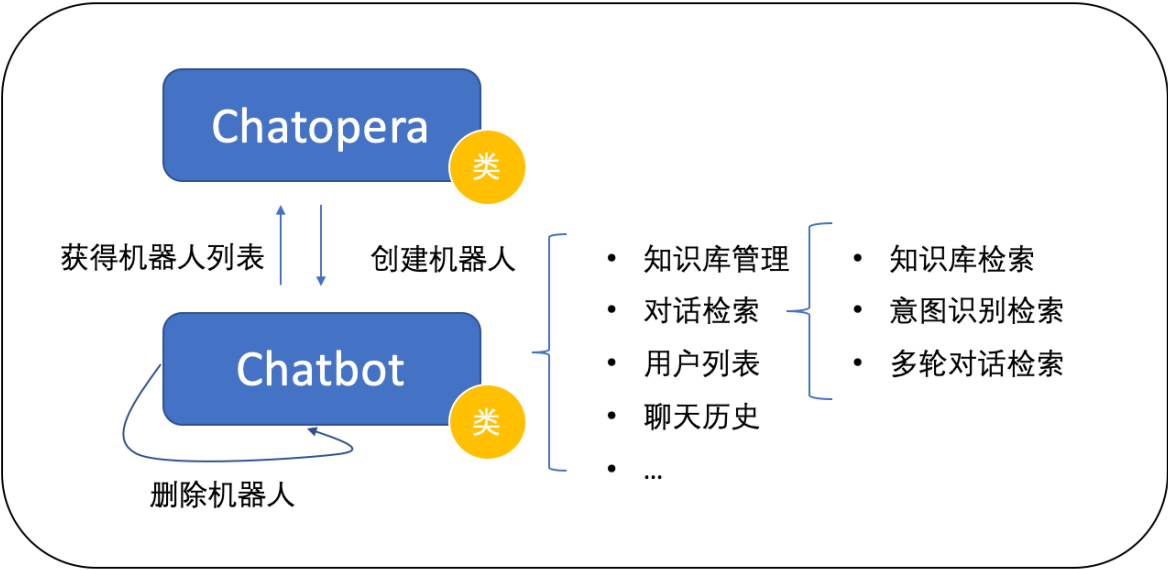
- [使用 CLI 导入和导出对话语料](#)

Chatopera 机器人平台用户可以在不同的软件系统、程序中接入 Chatopera 机器人。SDK 是面向不同编程语言和 Chatopera 机器人平台集成的工具库，包括 Node.js、Go、PHP、Python、Java 等。

能力概述

SDK 能力概述：使用 `Chatopera` 类创建机器人、获得机器人信息；使用 `Chatbot` 类管理某一机器人的资源；`Chatopera` 类是系统账户级别，`Chatbot` 是单个机器人级别。

`Chatopera` 类使用控制台中【访问设置】的 `Personal Access Token` 实例化，`Chatbot` 类使用每个机器人的 `clientId` 和 `secret` 初始化。



SDK

支持多语言

下载安装

SDK 简化了集成的复杂度，某些语言可以通过包管理工具安装。SDK 下载地址参考列表：

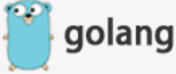
语言	下载地址	使用指南
Python	chatopera-py-sdk	示例程序 / 技术支持
Java	chatopera-java-sdk	示例程序 / 技术支持 / JavaDocs
Go	chatopera-go-sdk	示例程序 / 技术支持
PHP	chatopera-php-sdk	示例程序 / 技术支持
Node.js	chatopera-nodejs-sdk	示例程序 / 技术支持

为了方便开发者调用 SDK，每个 SDK 内均有示例程序或测试程序作为参考。

提示： 以上 SDK 同时支持私有部署的 Chatopera 机器人平台。

SDK

SDK是将聊天机器人以服务的方式接入您的系统的方式。若您在配置中遇到问题，请通过info@chatopera.com 或 查看帮助文档。

 使用说明 示例程序 接口文档	 使用说明 示例程序 接口文档	 使用说明 示例程序 接口文档	 使用说明 示例程序 接口文档
 使用说明 示例程序 接口文档			

如不能满足您使用环境或者语言的 SDK，请[创建工单](#)进行描述，Chatopera 将尽快满足您的需求。

核心类

在每个语言的 SDK 中，均实现两个类：`Chatopera` 类和 `Chatbot` 类。

`Chatopera` 类

`Chatopera`类是与 Chatopera 机器人平台集成的一个高级类，因为 Chatopera 云服务为开发者提供聊天机器人服务，`Chatopera`类的对象就是 Chatopera 云服务中一个注册账户的代理。

通过左侧导航菜单了解使用详情。

`Chatbot` 类

`Chatbot`类是与 Chatopera 云服务集成的一个核心类，因为 Chatopera 云服务为开发者提供聊天机器人服务，`Chatbot`类的对象就是 Chatopera 云服务中一个聊天机器人的代理。

通过左侧导航菜单了解使用详情。

已知问题

在 Windows WSL 内调用 SDK，返回异常 invalid timestamp

错误原因：Windows WSL 的时间同步有 BUG，未能和互联网时间同步。

解决方案：执行手动的时间同步。

假设使用的 WSL 发行版本基于 Ubuntu, 那么可以执行下面的命令解决。

```
sudo apt-get install ntpdate
sudo ntpdate time.windows.com
```

接下来

- [Chatopera 类详细说明](#)

- [Chatbot](#) 类详细说明：对话检索、机器人信息及更新，etc.
- 遇到问题？常看“获得帮助”

构造函数

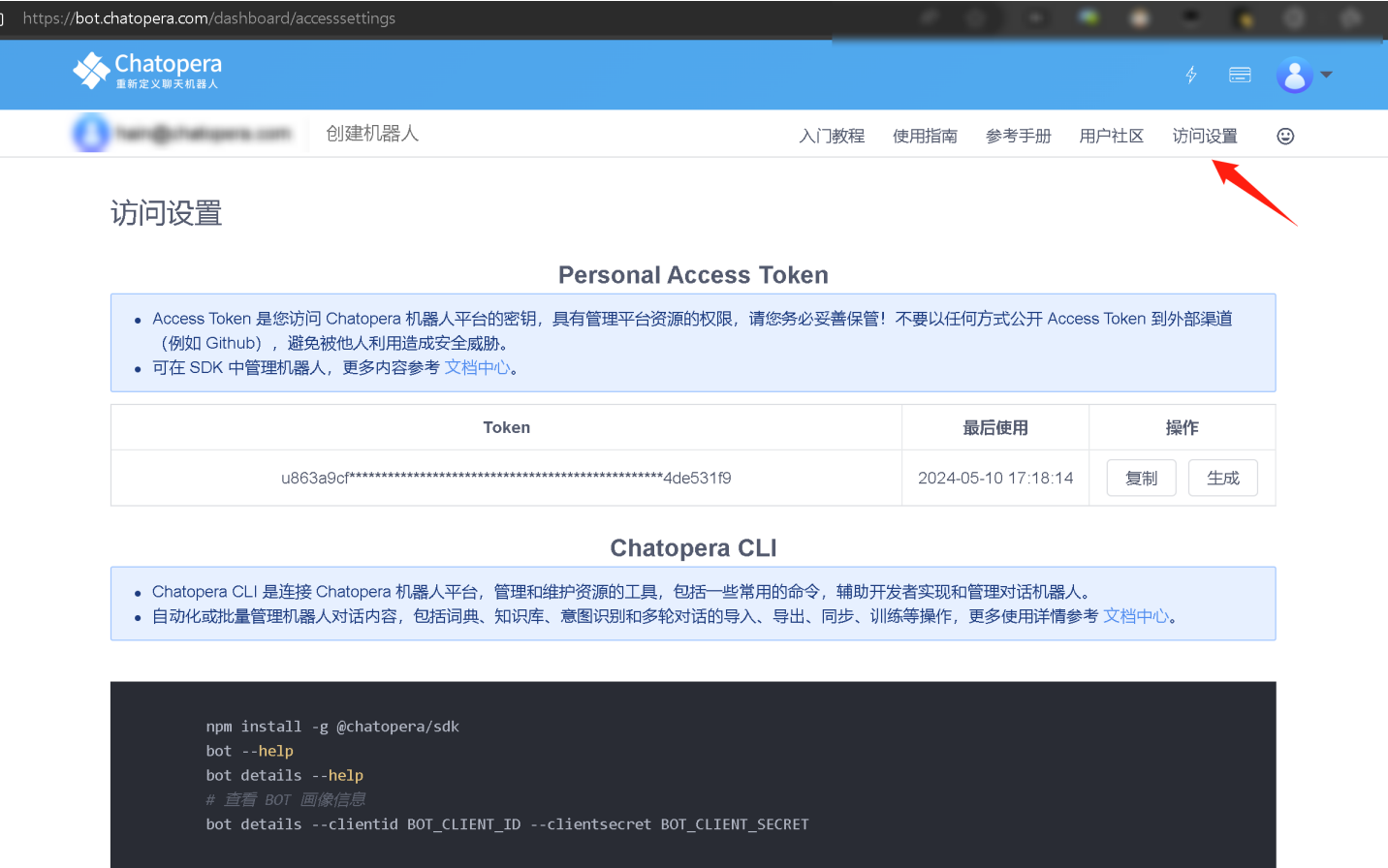
构造函数

```
Chatopera(accessToken [, botProvider])
```

参数说明

NAME	TYPE	REQUIRED	DESCRIPTION
accessToken	string	□	在 机器人控制台/访问设置 中获取，即 <code>Personal Access Token</code>
botProvider	string	□	Chatopera 机器人平台地址， 当使用 Chatopera 云服务时，该值为 https://bot.chatopera.com ，也是默认值

在机器人控制台，打开【访问设置】：



刚刚注册的账户需要点击生成进行初始化；点击复制将 Token 值复制到粘贴板。

注意：**Personal Access Token**(简称：**Access Token**) 是您访问 Chatopera 机器人平台的密钥，具有管理平台资源的权限，请您务必妥善保管！不要以任何方式公开 **Access Token** 到外部渠道（例如 Github），避免被他人利用造成安全威胁。

更多实例化例子

不同语言下，`Chatopera` 类的包名或引用方式不同。

Node.js

```
const { Chatopera } = require('@chatopera/sdk');
...
const chatopera = new Chatopera(accessToken[, botProvider]);
```

Java

```
import com.chatopera.bot.sdk.Chatopera;
...
Chatopera chatopera = new Chatopera(accessToken[, botProvider]);
```

Python

```
from chatopera import Chatopera
co = Chatopera(accessToken[, botProvider])
```

PHP

假设使用[composer](#)作为包管理工具，其它安装方式参考[chatopera-php-sdk](#)。

```
<?php

include_once **DIR** . "/vendor/autoload.php";
$chatopera = new Chatopera\SDK\Chatopera($accessToken[, $botProvider]);
```

Go

```
import (
    "github.com/chatopera/chatopera-go-sdk"
)
...
var admin = chatopera.NewChatopera(accessToken[, botProvider])
```


发送请求

Chatopera 实例的核心接口是 `command`，以下也使用 `Chatopera#command` 来指这个接口，该接口是对 RestAPI Request 的高级封装，内部完成签名认证，**RequestHeaders**和**RequestBody**等处理。

接口规范

```
result = chatopera.command(method, path [, body])
```

提示： result 返回在 Node.js 中使用 `await` 或 `Promise`，参考[快速开始](#)；其它语言直接用 `=` 便可获取。

参数说明

NAME	TYPE	REQUIRED	DESCRIPTION
method	string	□	对于资源的具体操作类型，由 HTTP 动词表示。有效值包括 <code>GET</code> ， <code>POST</code> ， <code>PUT</code> ， <code>DELETE</code> 和 <code>HEAD</code> 等
path	string	□	资源的执行路径，通常包含资源实体名称或唯一标识，也可能在 <code>path</code> 中使用 <code>queryString</code> 传递参数
body	<code>JSON</code> 数据结构	□	<code>body</code> 是请求中的数据，对应 RestAPI 中的 Http Body

`method` 不同动词代表的含义一般如下：

- GET - 从服务器取出一项或多项资源；
- POST - 在服务器创建一个资源；
- PUT - 在服务器更新一个资源；
- DELETE - 在服务器删除一个资源。

还有更多类型的 `method`，没有上述几种常用，在此不进行赘述。

`queryString` 是 URL 的一部分。典型的 URL 看起来像这样: `http://server/resource?foo=A&bar=B`。其中，`foo=A&bar=B`就是 `queryString`，通常用来传递参数，这个例子中包含两个参数：`foo` 值为 `A`；`bar` 值为 `B`。在下文中，`path` 参数中可能包含 `queryString`，形式如 `foo={{var1}}&bar={{var2}}`，需要把 `{{var1}}` 和 `{{var2}}` 替换为实际值。

`body` 数据是 JSON 格式的，不同语言对于 JSON 格式支持方式不同。[JSON](#)是一种轻量级的数据交换格式，描述了使用键值对、数组、字符串、数字、日期和布尔类型等值存储对象。[JSON](#)在不同语言下，等价数据结构如下。

语言	JSON OBJECT	JSON ARRAY
JavaScript	<code>{...}</code>	<code>[...]</code>
Java	<code>org.json.JSONObject</code>	<code>org.json.JSONArray</code>
PHP	基本类型 <code>array</code>	基本类型 <code>array</code>
Python	基本类型 <code>dict</code>	基本类型 <code>list</code>
Go	<code>map[string]interface{}</code>	<code>[]map[string]interface{}</code>

提示： 相对而言，JSON 等价的数据结构，在获取JSON Object的键值或JSON Array的长度和成员时，语法不同，但都易于掌握。在使用时，参考不同 SDK 的[示例程序](#)。

body是否必填以及是JSON Object还是JSON Array，取决于method和path的值，不同method和path的组合对应了不同的接口功能，满足不同需求，下文将介绍满足各种需求的method和path，并各个说明body参数。

返回值

返回值即请求结果，针对接口定义，Chatopera#command的返回值result是JSON Object，并有以下属性。

KEY	TYPE	DESCRIPTION
rc	int	response code，返回码，大于等于 0 的正整型。0代表服务器端按照请求描述，正常返回结果；rc 不等于 0 是代表异常返回。
data	JSON	数据资源。正常返回时，服务器端执行逻辑成功，比如查询时，data就是查询结果。
msg	string	消息，当服务器端执行请求成功，并且不需要返回数据资源时，通过msg代表文本信息，比如提示信息。
error	string	异常消息，当服务器端返回异常时，具体出错信息包含在error中。
total	int	分页，所有数据记录条数。
current_page	int	分页，当前页码，（分页从 1 开始）。
total_page	int	分页，所有页数。

每次请求结果中，rc是必含有的属性，其它属性为可能含有。不同rc的正整数形代表不同的异常，data、status以及分页信息，则因method和path而异，以下进行详细介绍。

提示： 不同语言对返回值可能进行了封装，但是不离其宗，都是基于以上定义，比如 Java SDK 中，定义com.chatopera.bot.sdk.Response作为Chatopera#command接口返回值，Response类提供getRc、getData和toJSON等方法，提升代码可读性。在使用时，参考不同 SDK 的[示例程序](#)。

下文中使用的method，path，body和result等均代表以上介绍的概念。

创建聊天机器人

```
Chatopera#command("POST", "/chatbot", body)
```

示例代码：[Node.js](#) | [Java](#) | [PHP](#) | [Python](#) | [Go](#)

body / JSON Object

```
{
  "name": "小巴",
  "primaryLanguage": "zh_CN",
  "fallback": "请联系客服。",
  "description": "我的超级能力是对话",
  "welcome": "你好，我是机器人小巴巴",
  "trans_zhCN_ZhTw2ZhCn": false
}
```

KEY	TYPE	REQUIRED	DESCRIPTION
name	string	□	机器人名字。
primaryLanguage	string	□	聊天机器人语言，目前支持简体中文（zh_CN）、繁体中文(zh_TW)、英文（en_US）、日语（ja）、泰语（th）。默认为 zh_CN .当使用 zh_CN 时可开启自动识别繁体并翻译(trans_zhCN_ZhTw2ZhCn: true)，默认为 false。
fallback	string	□	兜底回复，当请求机器人对话时，没有得到来自多轮对话、知识库或意图识别回复时，回复此内容。
welcome	string	□	机器人问候语。
description	string	□	机器人描述。

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "clientId": "{{clientId}}",
    "secret": "{{secret}}",
    "name": "小巴",
    "description": "Test",
    "primaryLanguage": "zh_CN",
    "createdAt": "Mon Aug 02 2021 20:35:23 GMT+0800 (CST)"
  }
}
```

clientId: 初始化 Chatbot 类的信息

secret: 初始化 Chatbot 类的信息

获得聊天机器人列表

```
Chatopera#command("GET", "/chatbot?limit={{limit}}&page={{page}}")
```

示例代码：[Node.js](#) | [Java](#)

KEY	TYPE	DEFAULT	DESCRIPTION
limit	int	默认值 20, 选填	返回数据条数
page	int	默认值 1, 选填	返回数据页面索引

result/ JSON Object

```
{
  "rc": 0,
  "total": 3,
  "total_page": 3,
  "data": [
    {
      "clientId": "{{clientId}}",
      "name": "TestBot1627889023922",
      "description": "",
      "primaryLanguage": "zh_CN",
      "createdAt": "Mon Aug 02 2021 15:23:44 GMT+0800 (CST)",
      "secret": "{{secret}}",
    },
    ...
  ]
}
```

total: 所有数据条数

total_page: 所有页面数

获得聊天机器人密钥

```
Chatopera#command("GET", "/secret/{{clientId}}")
```

示例代码：[Java](#)

path

KEY	TYPE	DEFAULT	DESCRIPTION
clientId	string	无，必填	机器人的 clientId

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "createdAt": "Fri May 10 2024 17:11:52 GMT+0800 (China Standard Time)",
    "isDeleted": false,
    "name": "testdel3",
    "logo": "",
    "secret": "xxx",
    "descriptions": ""
  }
}
```

删除聊天机器人

删除聊天机器人需要：

- 聊天机器人的 ClientId，见上文方式或从聊天机器人控制台的设置页面获得；
- 聊天机器人的密钥，见上文方式或从聊天机器人控制台的设置页面获得。

有了上述两个信息，通过接口：[Chatbot](#) 类中的[删除聊天机器人](#)实现。

更多机器人管理和使用 APIs

更多机器人管理和使用 APIs，参考 [Chatbot 类](#)。

构造函数

构造函数

```
Chatbot(clientId, secret [, botProvider])
```

参数说明

NAME	TYPE	REQUIRED	DESCRIPTION
clientId	string	□	在 机器人控制台/机器人/设置 中获取
secret	string	□	获取办法同上
botProvider	string	□	Chatopera 机器人平台地址， 当使用 Chatopera 云服务时，该值为 https://bot.chatopera.com ，也是默认值

更多实例化例子

不同语言下，`Chatbot`类的包名或引用方式不同，Node.js SDK 的实例化上文已经表述，以下再介绍其它语言。

Java

```
import com.chatopera.bot.sdk.Chatbot;
...
Chatbot chatbot = new Chatbot(clientId, secret);
```

Python

```
from chatopera import Chatbot
bot = Chatbot(clientId, secret)
```

PHP

假设使用[composer](#)作为包管理工具，其它安装方式参考[chatopera-php-sdk](#)。

```
<?php

include_once **DIR** . "/vendor/autoload.php";
$chatbot = new Chatopera\SDK\Chatbot($appId, \secret);
```

Go

```
import (
    "github.com/chatopera/chatopera-go-sdk"
)
...
var chatbot = chatopera.Chatbot(clientId, secret)
```

发送请求

Chatbot实例的核心接口是 `command`，以下也使用 `Chatbot#command` 来指这个接口，该接口是对 RestAPIs 的高级封装，内部完成签名认证，`RequestHeaders`和`RequestBody`等处理。

接口规范

```
result = chatbot.command(method, path [, body])
```

提示： `result` 返回在 Node.js 中使用 `await` 或 `Promise`，参考[快速开始](#)；其它语言直接用 `=` 便可获取。

参数说明

NAME	TYPE	REQUIRED	DESCRIPTION
method	string	☐	对于资源的具体操作类型，由 HTTP 动词表示。有效值包括 <code>GET</code> ， <code>POST</code> ， <code>PUT</code> ， <code>DELETE</code> 和 <code>HEAD</code> 等
path	string	☐	资源的执行路径，通常包含资源实体名称或唯一标识，也可能在 <code>path</code> 中使用 <code>queryString</code> 传递参数
body	JSON 数据结构	☐	<code>body</code> 是请求中的数据，对应 RestAPI 中的 Http Body

`method` 不同动词代表的含义一般如下：

- GET - 从服务器取出一项或多项资源；
- POST - 在服务器创建一个资源；
- PUT - 在服务器更新一个资源；
- DELETE - 在服务器删除一个资源。

还有更多类型的 `method`，没有上述几种常用，在此不进行赘述。

`queryString` 是 URL 的一部分。典型的 URL 看起来像这样: `http://server/resource?foo=A&bar=B`。其中，`foo=A&bar=B`就是 `queryString`，通常用来传递参数，这个例子中包含两个参数：`foo` 值为 `A`；`bar` 值为 `B`。在下文中，`path` 参数中可能包含 `queryString`，形式如`foo={{var1}}&bar={{var2}}`，需要把 `{{var1}}` 和 `{{var2}}` 替换为实际值。

`body` 数据是 JSON 格式的，不同语言对于 JSON 格式支持方式不同。[JSON](#)是一种轻量级的数据交换格式，描述了使用键值对、数组、字符串、数字、日期和布尔类型等值存储对象。[JSON](#)在不同语言下，等价数据结构如下。

语言	JSON OBJECT	JSON ARRAY
JavaScript	<code>{...}</code>	<code>[...]</code>
Java	<code>org.json.JSONObject</code>	<code>org.json.JSONArray</code>
PHP	基本类型 <code>array</code>	基本类型 <code>array</code>
Python	基本类型 <code>dict</code>	基本类型 <code>list</code>
Go	<code>map[string]interface{}</code>	<code>[]map[string]interface{}</code>

提示： 相对而言，JSON 等价的数据结构，在获取 JSON Object 的键值或 JSON Array 的长度和成员时，语法不同，但都易于掌握。在使用时，参考不同 SDK 的[示例程序](#)。

body 是否必填以及是 JSON Object 还是 JSON Array，取决于 method 和 path 的值，不同 method 和 path 的组合对应了不同的接口功能，满足不同需求，下文将介绍满足各种需求的 method 和 path，并各个说明 body 参数。

返回值

返回值即请求结果，针对接口定义， Chatbot#command 的返回值 result 是 JSON Object，并有以下属性。

KEY	TYPE	DESCRIPTION
rc	int	response code，返回码，大于等于 0 的正整型。0 代表服务器端按照请求描述，正常返回结果；rc 不等于 0 是代表异常返回。
data	JSON	数据资源。正常返回时，服务器端执行逻辑成功，比如查询时，data 就是查询结果。
msg	string	消息，当服务器端执行请求成功，并且不需要返回数据资源时，通过 msg 代表文本信息，比如提示信息。
error	string	异常消息，当服务器端返回异常时，具体出错信息包含在 error 中。
status	JSON	全局任务的状态信息。
total	int	分页，所有数据记录条数。
current_page	int	分页，当前页码，（分页从 1 开始）。
total_page	int	分页，所有页数。

每次请求结果中，rc 是必含有的属性，其它属性为可能含有。不同 rc 的正整数形代表不同的异常，data、status 以及分页信息，则因 method 和 path 而异，以下进行详细介绍。

提示： 不同语言对返回值可能进行了封装，但是不离其宗，都是基于以上定义，比如 Java SDK 中，定义 com.chatopera.bot.sdk.Response 作为 Chatbot#command 接口返回值，Response 类提供 getRc、getData 和 toJSON 等方法，提升代码可读性。在使用时，参考不同 SDK 的[示例程序](#)。

下文中使用的 method，path，body 和 result 等均代表以上介绍的概念。

常用 APIs 介绍

在Chatopera 云服务文档中心，仅介绍了关键、常用的 APIs 作为示范，这些 APIs 可以在左侧的子级菜单中获得详细介绍。

全部 APIs 文档

对于高级用户，Chatopera 提供了一个浏览全部 RestAPIs 的站点，这些 APIs 一样可以通过 Chatbot#command 接口调用，因为 Chatbot#command 就是封装了的 RestAPIs。

<https://api-docs.chatopera.com/>

获取机器人画像

```
Chatbot#command("GET", "/")
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "name": "小巴巴",
    "fallback": "请联系客服。",
    "description": "Performs Tasks or retrieves FAQ.",
    "welcome": "你好，我是机器人小巴巴",
    "primaryLanguage": "zh_CN",
    "status": {
      "reindex": 0,
      "retrain": 0
    }
  }
}
```

KEY	TYPE	DESCRIPTION
name	string	机器人名字。
fallback	string	兜底回复，当请求机器人对话时，没有得到来自多轮对话、知识库或意图识别回复时，回复此内容。
welcome	string	机器人问候语。
description	string	机器人描述。
primaryLanguage	string	机器人语言。
status	JSON Object	全局任务的执行状态，reindex代表知识库同步自定义词典的状态；retrain代表意图识别同步自定义词典的状态。

更新机器人画像

```
Chatbot#command("PUT", "/", body)
```

body / JSON Object

```
{
  "fallback": "请联系客服。",
  "description": "我的超级能力是对话",
  "welcome": "你好，我是机器人小巴巴"
}
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "name": "小巴巴",
    "fallback": "请联系客服。",
    "description": "Performs Tasks or retrieves FAQ.",
    "welcome": "你好，我是机器人小巴巴"
  }
}
```

获取全局任务状态

```
Chatbot#command("GET", "/status")
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "status": {
      "reindex": 0,
      "retrain": 0
    }
  }
}
```

删除聊天机器人

删除聊天机器人，该接口销毁机器人资源，并且不可逆。使用时需格外谨慎！

```
Chatbot#command("DELETE", "/")
```

result / JSON Object

```
{
  "rc": 0,
  "msg": "done"
}
```

对话检索

对话检索接口，就是将对话用户的请求发送给机器人，获得机器人的回复。

- 检索多轮对话：从多轮对话获得回复
- 检索知识库：从知识库获得回复
- 检索意图识别：从意图识别模块获得回复

检索多轮对话，也会同时从 **FAQ 知识库**、**RAG 知识库**、意图识别、对话脚本中获得答案并按照算法回复最佳答案，也是 **Chatopera** 官方最推荐的集成形式，使用检索多轮对话接口，可以定制出更为智能的对话机器人。了解详情，请阅读 [《多轮对话的工作机制》](#)。

检索多轮对话

多轮对话是通过脚本规则、函数编程实现问答服务，在 *检索多轮对话* 接口中，同时融合了知识库参与回复决策，返回结果，尤其是通过知识库答案路由到指定话题的指定触发器，非常实用。为了方便使用，宜先理解 [多轮对话的工作机制和工作原理](#)，熟悉多轮对话机制可以真正将 Chatopera 机器人平台的能量发挥到最大。

```
Chatbot#command("POST", "/conversation/query", body)
```

body / JSON Object

```
{
  "fromUserId": "{userId}",
  "textMessage": "想要说些什么",
  "faqBestReplyThreshold": 0.6,
  "faqSuggReplyThreshold": 0.35,
  "extras": {},
  "isDebug": false
}
```

KEY	TYPE	REQUIRED	DESCRIPTION
fromUserId	string	□	用户唯一 ID，用户 ID 由业务系统传递或生成，保证每个用户用唯一字符串
textMessage	string	□	用户输入的对话文字
faqBestReplyThreshold	number	□	知识库最佳回复阈值，知识库中置信度超过该值通过返回值 <code>string</code> 和 <code>params</code> 返回；可以在机器人平台管理控制台的设置页面设置默认值，使用 API 传递参数覆盖默认值
faqSuggReplyThreshold	number	□	知识库建议回复阈值，知识库中置信度超过该值的问答对通过返回值 <code>faq</code> 属性返回；可以在机器人平台管理控制台的设置页面设置默认值，使用 API 传递参数覆盖默认值
isDebug	boolean	□	是否返回调试信息，调试信息包括匹配信息等
extras	JSONObject 或 JSONArray	□	在消息中，添加自定义的信息，然后在多轮对话脚本的函数 <code>this.message.extras</code> 和 <code>this.user.history</code> 中使用

其中，`extras` 用以支持更灵活，自定义的场景，使用 [参考](#)。

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "state": "default",
    "string": "方法",
    "logic_is_unexpected": false,
    "logic_is_fallback": false,
    "service": {
      "provider": "faq",
      "docId": "{{doctId}}",
      "score": 0.3781,
      "faqBestReplyThreshold": 0.37,
      "faqSuggReplyThreshold": 0.1,
      "categories": ["x", "y"]
    },
    "botName": "小巴巴",
    "faq": [
      {
        "id": "{{doctId}}",
        "score": 0.3781,
        "post": "查看相似问题不可能的",
        "categories": ["x", "y"]
      }
    ]
  }
}
```

state: 业务字段，可以在多轮对话脚本中设置

string: 机器人回复的文本内容

topicName: 机器人会话主题

logic_is_fallback: 是否是兜底回复

botName: 机器人的名字

faq: 知识库中匹配 textMessage 的相似度超过 faqSuggReplyThreshold的记录，数组类型

service 代表返回的数据来源，provider:conversation指多轮对话，provider:faq指知识库，provider:intent指意图识别；不同数据来源也会提供相应信息。

PROVIDER	KEY	解释
faq	FAQ 知识库	
	docId	文档 ID
	post	标准问
	score	分数
rag	RAG 知识库	
	model	模型信息，e.g. qwen2.5:14b
	think	推理信息
intent	意图识别	更多描述参考意图匹配器

PROVIDER	KEY	解释
	intent.name	意图名称
	intent.state	意图会话状态
	intent.entities	意图中的命名实体
llm	大语言模型（LLM）	
conversation	多轮对话	
fallback	兜底回复	
mute	该用户被该机器人屏蔽	

检索 RAG 知识库

RAG 知识库通过用户上传的文档生成答案，结合大语言模型和向量数据库等技术。

```
Chatbot#command("POST", "/rag/query", body)
```

body / JSON Object

```
{
  "query": "海南有几个机场"
}
```

result/ JSON Object

```
{
  "rc": 0, // rc = 0 当对话请求正确处理时；rc 等于其它值，对话请求异常，服务未能顺利执行
  "data": {
    "think": null, // 机器人推理内容
    "content": "海南有三个民用机场：海口美兰国际机场、三亚凤凰国际机场和琼海博鳌机场。", // 机器人回复内容
    "is_logic_fallback": false, // 是否解答题：1) false - 已经解答；2) true - 未能解答
    "model": "qwen2.5:14b", // 本次问答使用的模型
    "prompt": "1. 使用下面的上下文信息回答末尾的问题\n2. 如果你不知道，就说\"I_DONT_KNOW\"，不要胡说.\n3. 让答案尽量简洁，只用3、4个句子回答问题.\n\n上下文信息：\n海口有海口美兰国际机场，位于海口市美兰区，航线飞往国内大中城市，也有飞往国际的专机。从海口去美兰国际机场，除了地铁快速到达外，有绕城高速直达，还有琼文高速和223国道，交通非常便利。海南有三个民用机场：海口美兰国际机场、三亚凤凰国际机场和琼海博鳌机场.\n\n问题：海南有几个机场\n答案：" // 系统提示词
  }
}
```

检索 FAQ 知识库

```
Chatbot#command("POST", "/faq/query", body)
```

body / JSON Object

```
{
  "query": "查找相似的问题",
  "fromUserId": "{{userId}}",
  "faqBestReplyThreshold": 0.5,
  "faqSuggReplyThreshold": 0.1
}
```

查询匹配是根据阈值设置的, 也就是 `faqBestReplyThreshold` 和 `faqSuggReplyThreshold`, 前者是最佳回复阈值, 后者是建议回复阈值, 前者高于后者, 都在 [0-1] 区间, 是问题相似度, 值越大, 越从知识库查询相似度高的记录, 1 代表问题和查询完全一样。

result/ JSON Object

```
{
  "rc": 0,
  "data": [
    {
      "id": "{{docId}}",
      "score": 0.48534,
      "post": "查看相似问题不可能的",
      "replies": [
        {
          "rtype": "plain",
          "enabled": true,
          "content": "方法"
        }
      ],
      "categories": [
        "节日",
        "农历"
      ]
    },
    {
      "id": "{{docId}}",
      "score": 0.32699,
      "post": "聊天",
      "replies": [
        {
          "rtype": "plain",
          "content": "foo",
          "enabled": true
        },
        {
          "rtype": "plain",
          "content": "bar",
          "enabled": true
        }
      ],
      "categories": []
    }
  ]
}
```

`categories` 是分类信息。分类是树状结构, 返回值是按照树状结构顺序的数组。

检索意图识别

意图识别是基于请求者的文本内容分析意图, 然后基于意图追问意图槽位信息的对话, 这部分的详细介绍参考<https://docs.chatopera.com/products/chatbot-platform/intent.html>, 下面重点介绍在系统集成中, 通过意图识别服务提供智能问答。

什么是“会话”

“会话”(session)在代表一个用户对话的周期, 认为用户在这个周期内是为了完成某个任务的。从确定任务, 到得到和这个任务相关的信息, 这个 session 就正常结束了, 但是如果用户变化了任务, 这个 session 就不能正常结束。开发者选择什么时候创建新的 session, 但是服务器端决定什么时候完成这个 session, session 的管理涉及: 意图的确定, 意图参数的确定, 会话最大空闲时间, 会话是否解决(resolved)。

- 训练完成后请求对话, 需要先创建会话, 会话会绑定 0-1 个任务: 刚开始不知道用户意图, 当确定用户意图后, 该 session

就只和这个意图相关;

- 会话有最大空闲日期, 如果在半个小时内没有更新, 会被服务器删除;
- 会话可以任意创建, 只要没有超过最大空闲日期都是有效的;
- 不同的用户使用不同的会话, 同一个用户可以同时有多个会话, 但是为了实际效果, 用户最好同时只使用一个会话;
- 当用户的意图和槽位信息被全部确认, 会话包含的 resolved 字段会被设置为 true, 这时开发者可以再次创建一个新的会话。

创建会话

```
Chatbot#command("POST", "/clause/prover/session", body)
```

body / JSON Object

```
{
  "uid": "{{userId}}",
  "channel": "{{channelId}}"
}
```

KEY	TYPE	REQUIRED	DESCRIPTION
userID	string	□	用户标识, 由字母和数字组成的字符串。开发者自定义, 保证每个用户唯一
channelId	string	□	用户来源的渠道标识, 由字母和数字组成的字符串。由开发者自定义, 保证每个渠道唯一

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "intent_name": null,
    "uid": "{{userId}}",
    "channel": "{{channelId}}",
    "resolved": null,
    "id": "{{sessionId}}",
    "entities": null,
    "createdate": "2019-08-28 18:08:51",
    "updatedate": "2019-08-28 18:08:51"
    "ttl": 3600
  },
  "error": null
}
```

intent_name: 意图名字

id: 会话 ID

resolved: 该会话是否完成收集参数

entities: 参数列表, 完成填槽或待填槽

ttl: 该会话信息在多少秒后过期, 每个会话默认是 1 小时的空闲周期, 在该时间内没有跟进的对话, 则会话过期

检索意图识别

```
Chatbot#command("POST", "/clause/prover/chat", body)
```

body / JSON Object

```
{
  "fromUserId": "{{userId}}",
  "session": {
    "id": "{{sessionId}}"
  },
  "message": {
    "textMessage": "我想购买明天火车票"
  }
}
```

KEY	TYPE	REQUIRED	DESCRIPTION
userId	string	□	用户唯一 ID，用户 ID 由业务系统传递或生成，保证每个用户用唯一字符串
sessionId	string	□	使用创建会话接口创建
textMessage	string	□	用户输入的对话文字

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "session": {
      "intent_name": "{{intentName}}",
      "uid": "{{userId}}",
      "channel": "{{channelId}}",
      "resolved": false,
      "id": "{{sessionId}}",
      "entities": [
        {
          "name": "cityName",
          "val": "中国首都"
        }
      ],
      "createdate": "2019-08-28 18:15:24",
      "updatedate": "2019-08-28 18:15:24",
      "ttl": 3595
    },
    "message": {
      "textMessage": "你想做什么工具",
      "is_fallback": null,
      "is_proactive": true
    }
  },
  "error": null
}
```

查看会话详情

```
Chatbot#command("GET", "/clause/prover/session/{{sessionId}}")
```

result/ JSON Object


```
{
  "rc": 0,
  "data": {
    "intent_name": "{{intentName}}",
    "uid": "{{userId}}",
    "channel": "{{channelId}}",
    "resolved": false,
    "id": "{{sessionId}}",
    "entities": null,
    "createdate": "2019-08-28 18:41:56",
    "updatedate": "2019-08-28 18:41:56",
    "ttl": 3600
  },
  "error": null
}
```

创建自定义词典

```
Chatbot#command("POST", "/clause/customdicts", body)
```

body / JSON Object

```
{
  "name": "{{customDictName}}",
  "type": "vocab"
}
```

KEY	TYPE	REQUIRED	DESCRIPTION
name	string	□	自定词典名称，使用小写字母和数据组成的字符串

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "name": "{{customDictName}}",
    "description": "",
    "samples": null,
    "createdate": "2019-08-07 19:59:14",
    "updatedate": "2019-08-07 19:59:14"
  }
}
```

获取自定义词典列表

```
Chatbot#command("GET", "/clause/customdicts?limit={{limit}}&page={{page}}")
```

result/ JSON Object

```
{
  "rc": 0,
  "total": 3,
  "current_page": 1,
  "total_page": 3,
  "data": [
    {
      "name": "{{customDictName}}",
      "description": "",
      "samples": null,
      "createdate": "2019-08-07 19:58:08",
      "updatedate": "2019-08-07 19:58:08"
    }
  ]
}
```

更新自定义词典

```
Chatbot#command("PUT", "/clause/customdicts/{{customDictName}}", body)
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
customDictName	string	无默认值, 必填	自定义词典标识

body / JSON Object

```
{
  "description": "高级轿车品牌"
}
```

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "name": "pizza",
    "description": "",
    "samples": null,
    "createdate": "2020-07-20 20:52:00",
    "updatedate": "2020-07-20 20:51:59",
    "type": "vocab",
  }
}
```

删除自定义词典

```
Chatbot#command("DELETE", "/clause/customdicts/{{customDictName}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
customDictName	string	无默认值, 必填	自定义词典标识

result/ JSON Object

```
{
  "rc": 0,
  "msg": "success",
  "error": null,
  "data": {
    "status": {
      "needReindex": 2,
      "needRetrain": 2
    }
  }
}
```

创建知识库分类

```
Chatbot#command("POST", "/faq/categories", body)
```

body / JSON Object

```
{
  "label": "{{categoryText}}"
}
```

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "value": "{{categoryId}}",
    "categories": [
      {
        "value": "{{categoryId}}",
        "label": "{{categoryText}}",
        "children": [
          {
            "value": "I7vfx47i5I",
            "label": "二级分类名"
          }
        ]
      },
      {
        "value": "{{categoryId}}",
        "label": "x2"
      }
    ]
  }
}
```

获取知识库分类信息

```
Chatbot#command("GET", "/faq/categories")
```

result / JSON Object

```
{
  "rc": 0,
  "data": [
    {
      "value": "{{categoryId}}",
      "label": "{{categoryText}}",
      "children": [
        {
          "value": "{{categoryId}}",
          "label": "{{categoryText}}"
        }
      ]
    }
  ]
}
```

更新知识库分类

```
Chatbot#command("", "/faq/categories", body)
```

```
{
  "value": "{{categoryId}}",
  "label": "新的名字"
}
```

result/ JSON Object

```
{
  "rc": 0,
  "data": [
    {
      "value": "wwQyjs310",
      "label": "一级分类名",
      "children": [
        {
          "value": "{{categoryId}}",
          "label": "新的名字"
        }
      ]
    }
  ]
}
```

删除知识库分类

```
Chatbot#command("DELETE", "/faq/categories/{{categoryId}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
categoryId	string	无默认值，必填	分类唯一标识

result/ JSON Object

```
{
  "rc": 0,
  "data": [
    {
      "value": "TSDD-W6T9",
      "label": "x2"
    }
  ]
}
```

创建问答对

```
Chatbot#command("post", "/faq/database", body)
```

body / JSON Object

```
{
  "post": "如何查看快递单号",
  "replies": [
    {
      "rtype": "plain",
      "content": "foo",
      "enabled": true
    },
    {
      "rtype": "plain",
      "content": "bar",
      "enabled": true
    }
  ],
  "enabled": true,
  "categoryTexts": [
    "一级分类名",
    "二级分类名"
  ]
}
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "id": "{docId}",
    "replyLastUpdate": "{replyLastUpdate}"
  }
}
```

更新知识库问答对

```
Chatbot#command("PUT", "/faq/database/{docId}", body)
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
docId	string	无默认值, 必填	问答对标识

body / JSON Object

```
{
  "post": "怎么开通微信支付?",
  "replyLastUpdate": "{replyLastUpdate}",
  "replies": [
    {
      "rtype": "plain",
      "content": "foo2",
      "enabled": true
    },
    {
      "rtype": "plain",
      "content": "bar2",
      "enabled": true
    }
  ],
  "enabled": true
}
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "id": "{{docId}}",
    "replyLastUpdate": "{{replyLastUpdate}}"
  }
}
```

获取问答对列表

```
Chatbot#command("GET", "/faq/database?limit={{limit}}&page={{page}}&q={{q}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
limit	int	20	返回最多多少条数据
page	int	1	返回第多少页
q	string	空	问答对匹配时，问题应包含的关键字

result / JSON Object

```
{
  "total": 3,
  "current_page": 1,
  "total_page": 1,
  "data": [
    {
      "post": "如何查看快递单号",
      "categories": [
        "wwQyjS310",
        "I7vfx47i5I"
      ],
      "enabled": true,
      "id": "{{docId}}"
    }
  ],
  "rc": 0,
  "status": {
    "reindex": 0,
    "retrain": 0
  }
}
```

创建问答对相似问

```
Chatbot#command("POST", "/faq/database/{{docId}}/extend", body)
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
docId	string	无默认值, 必填	问答对标识

body / JSON Object

```
{
  "post": "怎样支持微信支付?"
}
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "id": "{{extendId}}"
  }
}
```

获取问答对相似问列表

```
Chatbot#command("GET", "/faq/database/{{docId}}/extend")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
docId	string	无默认值, 必填	问答对标识

result / JSON Object

```
{
  "total": 1,
  "current_page": 1,
  "total_page": 1,
  "data": [
    {
      "post": "怎样支持微信支付?",
      "postId": "{{docId}}",
      "enabled": true,
      "id": "{{extendId}}"
    }
  ],
  "rc": 0
}
```

更新问答对相似问

```
Chatbot#command("PUT", "/faq/database/{{docId}}/extend/{{extendId}}", body)
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
docId	string	无默认值, 必填	问答对标识
extendId	string	无默认值, 必填	扩展问标识

body / JSON Object

```
{
  "post": "怎样支持微信支付?"
}
```

result / JSON Object


```
{
  "rc": 0,
  "data": {
    "id": "{{extendId}}"
  }
}
```

删除问答对相似问

```
Chatbot#command("DELETE", "/faq/database/{{docId}}/extend/{{extendId}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
docId	string	无默认值, 必填	问答对标识
extendId	string	无默认值, 必填	扩展问标识

result / JSON Object

```
{
  "rc": 0,
  "msg": "done"
}
```

删除问答对

```
Chatbot#command("DELETE", "/faq/database/{{docId}}")
```

result / JSON Object

```
{
  "rc": 0,
  "msg": "done"
}
```

获取知识库热门问题

```
Chatbot#command("GET", "/faq/database/inquiryrank?topN={{topN}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
topN	int	10	获得热门问题的条数，即导出数据的条数

result / JSON Object

```
{
  "rc": 0,
  "data": [
    {
      "docId": "{{docId}}",
      "inquiryscore": "13",
      "post": "{{post}}",
      "enabled": true,
      "categories": [
        "s1",
        "s2"
      ]
    },
    {
      "docId": "{{docId}}",
      "inquiryscore": "0",
      "post": "{{post}}",
      "enabled": true,
      "categories": []
    },
    ...
  ]
}
```

其中，data 内元素按照 `inquiryscore` 的值，降序排列，即该问题越热门，越靠上。

用户和对话历史

获取用户列表

```
Chatbot#command("GET", "/users")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
limit	int	1	返回最多多少条数据
page	int	20	返回第多少页

result / JSON Object

```
{
  "rc": 0,
  "total": 5,
  "current_page": 1,
  "total_page": 1,
  "data": [
    {
      "userId": "{{userId}}",
      "lasttime": "2020-07-19T14:12:13.690Z",
      "created": "2020-07-19T13:48:02.225Z"
    }
  ]
}
```

userId: 和机器人对话的用户标识

lasttime: 最后沟通时间

created: 第一次沟通时间

屏蔽用户

```
Chatbot#command("POST", "/users/{{userId}}/mute")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
userId	string	无默认值, 必填	用户唯一标识

result / JSON Object

```
{
  "rc": 0,
  "data": {}
}
```

取消屏蔽

```
Chatbot#command("POST", "/users/{{userId}}/unmute")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
userId	string	无默认值, 必填	用户唯一标识

result / JSON Object

```
{
  "rc": 0,
  "data": {}
}
```

是否被屏蔽

```
Chatbot#command("POST", "/users/{{userId}}/ismute")
```

result / JSON Object

```
{
  "rc": 0,
  "data": {
    "mute": false
  }
}
```

data.mute 返回 boolean 类型值。

获取用户画像信息

```
Chatbot#command("GET", "/users/{{userId}}/profile")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
userId	string	无默认值, 必填	用户唯一标识

result/ JSON Object

```
{
  "rc": 0,
  "data": {
    "userId": "postman9",
    "name": null,
    "lasttime": "2020-07-19T14:12:13.690Z",
    "created": "2020-07-19T13:48:02.225Z",
    "profile": {},
    "mute": false
  }
}
```

获取聊天历史

```
Chatbot#command("GET", "/users/{{userId}}/chats?limit={{limit}}&page={{page}}")
```

path

KEY	TYPE	DEFAULT	DESCRIPTION
userId	string	无默认值, 必填	用户唯一标识
limit	int	1	返回最多多少条数据
page	int	20	返回第多少页

result / JSON Object

```
{
  "rc": 0,
  "total": 16,
  "current_page": 1,
  "total_page": 1,
  "data": [
    {
      "userId": "postman9",
      "textMessage": "方法",
      "direction": "outbound",
      "service": "faq",
      "confidence": 0.3781,
      "docId": "AXNCspoufX0JhfysI3-Z",
      "created": "2020-07-19T14:12:13.802Z"
    }
  ]
}
```

total: 该用户和机器人之间对话总数

current_page: 当前页

total_page: 总页数

userId: 用户标识

textMessage: 文本内容

direction: 消息传递方向，【inbound】为消费者发送，【outbound】为机器人发送

service: 提供回复的服务

confidence: 置信度

created: 消息创建时间

常见问题

SDK

返回错误 invalid appId

```
{"rc":1,"error":"invalid appId."}
```

这是因为 `clientId` 和 `secret` 配置不正确。

返回错误 invalid timestamp

```
{"rc":1,"error":"invalid timestamp."}
```

操作系统需要设置的时间同步为互联网时间，参考

<https://www.sysgeek.cn/manage-time-server-windows-10/>

也可以设置为阿里云时间同步器

https://blog.csdn.net/qq_35448976/article/details/78977164

Java SDK / Maven 执行抛出异常

`mvn install` 抛出异常，`SunCertPathBuilderException`

```
sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target
```

[解决方案](#)

计费及保障

Chatopera 云服务是企业级的服务，依托于 Chatopera 的 Platform as a Service 服务上，使用监控、告警、负载均衡、集群等功能保证服务的高可靠性。

计费及发票

Chatopera 云服务计费及发票，参考[文档](#)。

私有部署

Chatopera 云服务私有部署，即 Chatopera 机器人平台，参考[文档](#)。

服务条款

Chatopera 云服务用户 & 私有部署客户使用规范和须知等，参考[文档](#)。

服务水平协议

Chatopera 承诺的可靠性保证等，参考[文档](#)。

计费及发票

Chatopera 云服务的费用中心可以查询计费情况、资源包使用和购买、开具发票等信息。

<https://bot.chatopera.com/billing>

以下是对使用该模块的进一步的介绍。

API 计费预估

Chatopera 云服务的机器人平台消费情况，按照现有用户统计，90% 的用户的年使用费用在 ¥ 600元 ~ ¥ 1000 元之间，这个区间可以帮助您做出适合的决策，因行业和用途的差异较大，不做进一步的分析。

您可以通过以下两个方式更准确的预估费用：

- 购买¥ 60 元的资源包，评测使用的天数；
- 通过下文的内容详细了解计费标准，预估您的业务量，计算费用。

在您获得了价格费用之后，只需要衡量产品是否满足了您的需求。您是不需要与之对比其他聊天机器人厂商的价格的，因为 Chatopera 云服务的独创性、稳定性和开放性。

- 独创性：Chatopera 云服务的机器人平台在产品定位、功能设计、技术实现等方面具有**独创性**，率先定位于为开发者提供定制多轮对话机器人的集成开发环境，与其他厂商之间有产品差异，不具备比较的标准；
- 稳定性：Chatopera 云服务的机器人平台上线于 **2018 年 11 月**，持续迭代，为用户提供了多年持续稳定、可靠的服务；
- 开放性：Chatopera 云服务的机器人平台通过发布多项**开源技术**和不断优化 SDK，来为用户提供开发者友好的产品，加速聊天机器人的应用和普及。

API 计费服务接口

Chatopera 云服务的计费接口包括对话检索 API，按照 **API 调用次数**进行计费：

- 检索知识库对话，[说明文档](#)
- 检索多轮对话，[说明文档](#)
- 检索意图识别，[说明文档](#)

API 计费标准

购买的资源包数额越大，则 **API 调用配额次数**越大、资源包周期越长、换算为单次请求越便宜。

购买须知

- 使用说明

服务调用换算为计数，从资源包配额中扣除，扣除顺序为“赠送资源包>购买资源包”。当您的资源包配额耗尽或过期时，服务将不可用，直至购买资源包后恢复。支付后，可从【费用中心-开具发票】页面，提交开票信息。
- 退订规则

不支持退款及剩余次数冻结。

选择配置

服务范围

所有计费接口

计费接口

检索知识库对话 检索意图识别 检索多轮对话（含大语言模型）

资源包内容

包月 60 元资源包

5,000 次

5000 次对话检索 API 请求，购买日期起 1 个月有效。

0.012 元/次

包年 600 元资源包

100,000 次

10 万次对话检索 API 请求，购买日期起 12 个月有效。

0.006 元/次

包年 6,000 元资源包

2,000,000 次

200 万次对话检索 API 请求，购买日期起 12 个月有效。

0.003 元/次

资源包购买地址 -

<https://bot.chatopera.com/billing/purchase>

最终服务、计费接口和价格以【资源包购买地址】为准。

发票

发票管理地址 -

<https://bot.chatopera.com/billing/open-invoice>

在控制台右上角的导航栏进入【费用中心-开具发票】进行申请，点击【发票管理】查看开具状态，在开票过程中，可能通过短信、邮件和电话方式联系，请保证填写正确信息和联系畅通。

提交申请发票如下图：

245/290

<https://bot.chatopera.com>

开具发票 [回到费用中心](#)

1. 开通文字识别服务后，您将拥有对应额度的免费资源包，详情请参考[免费额度规则](#)
2. 预付费资源包有效期均为1年，1年内若资源包次数未使用完，则过期作废；预付费资源包使用后不支持剩余次数冻结理由退款。
3. 调用量的扣费顺序为“免费资源包->付费资源包->后付费”。当您的免费资源包耗尽时，服务将面临不可用风险，为保障预付费资源包或前往设置页开通后付费模式。
4. 当该服务仅剩一个可用资源包，且余量小于20%和余量为0时，系统都会通过微信/短信/邮件/站内信向您推送预警消息([查看提醒设置](#))

* 发票抬头

比如：北京华夏春松科技有限公司

* 纳税人识别号

纳税人办理各类涉税事项的唯一代码标识

发票类型

增值税普通发票

分类

研发和技术服务

备注

企业技术服务

其它使用须知

1. 资源包

资源包有有效期和配额，用户在使用服务过程中，按照 API 调用次数换算为计数，从配额中扣除。

在购买同一款式的多件资源包时，只从配额数量上叠加，而有效期不改变。比如购买【包月 100 元资源包】3 件，那么配额容量就是 5000 次 x 3，而有效期依然是 1 个月有效。

2. 配额扣除

API 调用的计数定期从配额中扣除，记录查看【配额扣除】。配额扣除的顺序为“赠送资源包 > 购买资源包”。

<https://bot.chatopera.com/billing/quota>

配额扣除的记录如下图：

购买资源包

开具发票

资源包管理

配额扣除

发票管理

开始时间	结束时间	配额计数	资源包类型
2023-02-10 14:27	2023-02-10 15:07	-5	通用 100 元资源包
2023-02-10 14:07	2023-02-10 14:27	-25	通用 100 元资源包
2023-02-10 13:47	2023-02-10 14:07	-15	通用 100 元资源包
2023-02-10 13:27	2023-02-10 13:47	-1	通用 100 元资源包
2023-02-10 13:06	2023-02-10 13:27	-1	通用 100 元资源包
2023-02-10 12:26	2023-02-10 13:06	-4	通用 100 元资源包
2023-02-10 11:46	2023-02-10 12:26	-3	通用 100 元资源包

3. 注册送资源包

注册账户成功后，您将获得赠送资源包一个，请在其有效期内尽量使用：体验产品和上线服务。

4. 无资源包停服

当没有可用配额时，用户服务请求被拒绝，服务不可用，直至购买资源包后恢复。为保证业务不受影响，前往【购买资源包】页面购买。

5. 配额不足告警

资源包从下单支付后生效。若在有效期内配额未用尽，余量作废。仅剩一个可用资源包，且其余量小于 20% 时，平台会通过短信/邮件向您推送预警消息，消息通知可能有数小时延迟，收到通知后请尽快购买资源包。

私有部署

对于企业客户，Chatopera 提供 **Chatopera 机器人平台** 的私有部署支持，**Chatopera 机器人平台** 除计费和支付模块、账号注册和管理模块与**Chatopera 云服务** 不同外，核心的智能问答服务模块和操作均一致，在产品层面是同一套代码，集成 SDK，SDK 接口等均适用，具体的产品功能介绍，参考[文档](#)。

企业用户获得 Chatopera 机器人平台实例，是安装到企业的指定的设备上，私有部署收取软件的商业授权证书费用，不按 API 请求量计费，具体资费见下。

价格

-	专业版	豪华版
画像设置	☐	☐
使用统计	☐	☐
词典管理	☐	☐
RAG 知识库	☐	☐
FAQ 知识库	☐	☐
多轮对话	☐	☐
测试对话	☐	☐
对话历史	☐	☐
SDK 集成	☐	☐
春松客服集成	☐	☐
H5 聊天控件，聊天码	☐	☐
意图识别	☐	☐
聚类分析	☐	☐
大语言模型	☐	配置云服务 API 或私有部署开源 LLMs
第1套实例价格	联系咨询顾问	联系咨询顾问
使用年限	永久	永久

补充说明：

- 各版本均不提供 Chatopera 云服务中的[费用中心](#)模块以及其它计费、支付相关功能；
- **大语言模型**可通过：1）配置云服务的形式；2）私有部署开源大语言模型完成。Chatopera 工程师会协助完成该配置的过程，但是需要采购第三方云服务或增加的硬件购置成本，需要客户额外自行支付；
- 春松客服兼容 Chatopera 机器人平台私有部署实例，使用方法参考[文档](#)；

- 因为没有意图识别模块，专业版中，不支持[意图匹配器](#)；
- 每套实例，代表一个独立的软件服务；软件运行在一个固定的操作系统提供访问地址，为一个单独的实例；
- 第2套实例价格（及以后每套）：该版本第一套价格的 10%；
- 升配或降配：从专业班可以升级到豪华版，升级办法为补差价；不支持降配，比如从豪华版降至专业版；
- 质保和升级服务单独付费；第一年免费，第二年及以后每年价格为私有部署总价的 15%；质保和升级累积原则，即第二年没有购买质保和升级服务，第三年需要时，要一并支付第二年的费用；累积原则上限为 5 年，超过 5 年的，按 5 年计算；质保和升级服务说明见后文；
- 该价格会根据市场和季节进行调整，在下单时，合同商定价格如果与上述表格不一致，本公司具有最终解释权。

部署硬件和软件配置要求

部署方案使用 Docker 和 Docker Compose；Docker 部署可实现跨平台和操作系统的目的。

以下以单机部署（不含大语言模型 LLMs 私有部署）为例，最低配置要求：

-	专业版	豪华版
内存	16GB	64GB
CPU 颗数	4	16
硬盘	125GB	125GB
架构体系	x86_64	x86_64
支持并发	10 qps	20 qps

补充说明：

- Chatopera 机器人平台，在密集计算和机器学习方面，使用 C++，因此具备良好的性能，对操作系统的资源要求不高
- 操作系统：Linux，兼容不同发布版本(Ubuntu（*推荐）, CentOS, Redhat, Debian, etc.)
- 并发：10 qps 代表每秒处理 10 个 API 请求，在 1 秒内的请求未得到处理，会延迟到下一秒，从 Chatopera 机器人平台处理上，每个请求在 100ms 内，具体请求响应时间，依赖于网络情况，以上数据为预估；qps, Query Per Second 的缩写
- 部署私有部署的大语言模型，需要采购 GPU 机器，价格根据客户的需求而不同，请与销售代表洽谈，确定价格

检查 CPU

Intel 处理器

支持目前常见 Intel, AMD 处理器，如：

- Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz 及更高
- AMD Ryzen 7 2700X Eight-Core Processor 及更高

AVX 指令集

检查一台机器的CPU是否支持 AVX 指令集，执行命令：

```
grep flags /proc/cpuinfo |grep avx
```

质保和升级服务说明

- 每次 Chatopera 机器人平台有新功能，推送更新内容，由企业客户自行决定是否升级，升级操作为 Chatopera 团队远程执行
- 运行过程出现网络延迟大等情况， Chatopera 团队远程联机解决
- 备份和恢复等咨询

采购下单

采购下单、商务合作和售后技术支持等，查看[联系方式（https://www.chatopera.com/mail.html）](https://www.chatopera.com/mail.html)。

Chatopera 对公银行账户信息：

Bank 银行	招商银行 支行：北京上地支行
Name 账户名	北京华夏春松科技有限公司
Acct. 账户	110934004410902
Tax ID 税号	91110108MA01CN41XA

交付准备

付款后，由甲方准备好机器，支持连接互联网。Chatopera 团队远程连接，执行部署，部署过程小于 1 天；如果需要现场执行，差旅费用由甲方额外支付；部署环境不支持互联网情况下，每套部署额外支付 RMB 6,000

法律条款

- [服务条款](#)
- [服务水平协议](#)

服务附加条款

- 不允许将 Chatopera 私有部署上线为 SaaS 服务

Chatopera 机器人平台的私有部署版本，不允许将 Chatopera 上线为 SaaS 服务，即将 Chatopera 机器人管理控制台 Dashboard 发布到互联网，供互联网用户注册使用和计费。

在 Chatopera 机器人平台私有部署服务上，机器人平台账户管理使用配置用户名和密码的方式，不支持使用邮箱注册。

- 私有部署不包括源码

企业客户因为安全、合规审计等要查看源码，Chatopera 提供浏览和查看代码的支持，在 Chatopera 的电脑上，仅供查看，不支持拷贝、录像等形式；查看代码需要额外支付费用，具体办法另行协商。

企业客户在私有部署后，对 Chatopera 服务的源码进行拟协议、反编译，甚至修改源码等，是不被允许的，Chatopera 有权利在此种情况下停止服务并不予退款，如果给 Chatopera 造成经济和商业损失，Chatopera 有权利进行法律维权。

免责声明

对于购买 Chatopera 私有部署版本的企业客户，需保证对本公司提供的材料合法、合规。如发生下列情形之一，本公司有权随时中断或终止向用户提供软件或服务而无需通知用户，并拒绝用户于现在和未来使用本公司所提供服务之全部或任何部分：（1）用户提供任何错误、不实、过时、不完整等具有误导性的资料，或者本公司有理由怀疑前述资料为错误、不实、过时、不完整等具误导性的；（2）用户违反本法律声明或相关使用协议中规定的使用规则。

法律适用与司法管辖

本法律声明的制定、执行和解释及争议的解决均应适用中华人民共和国法律。双方因使用本公司所产生的争议，协商不成的，任何一方可以向本公司所在地人民法院提起诉讼。

本协议作用于Chatopera 云服务付费用户, 即在 [Chatopera 云服务](#) 充值并发生扣款记录。

服务水平协议

可靠性保证

- 服务可靠性 99.1%, 即每年 (365 天) 有不超过 3 天的累积服务中断时间
- 由第三方造成的服务中断, 数据丢失并不由 Chatopera 承担责任, 比如第三方机房意外断电、断网
- 由用户操作失误造成的异常, 数据丢失并不由 Chatopera 承担责任
- 由自然不可抗力因素造成的服务中断, 数据丢失并不由 Chatopera 承担责任
- 因 Chatopera 运维事故造成的数据丢失、无法恢复, 服务中断时间超过承诺, 由 Chatopera 及用户协商赔偿用户损失, 赔偿手段包括代金券或退款

工单提交及处理

【工单处理】发送描述至邮箱, info@chatopera.com, 16 小时内 Chatopera 运营人员反馈。

【紧急联系方式】需要快速获得支持, 应对服务宕机等, 紧急联系方式为企业微信

专属顾问

有疑问?
需要帮助?
私有部署?



扫一扫 加我的企业微信/支持微信&企业微信

投诉

联系方式: 见 <https://www.chatopera.com/mail.html>

微信或企业微信扫一扫, 告知工作人员、服务等问题。



服务条款

“Chatopera 云服务”（以下简称“本服务”）是由北京华夏春松科技有限公司（以下简称本公司）在向用户提供的聊天机器人云服务。本服务主要提供定制化聊天机器人的工具、API 接口、PC 软件等。

您对本协议的接受即自愿接受全部条款的约束，包括接受本公司对任一服务条款随时所做的任何修改。本协议可由本公司随时更新，更新后的协议条款一旦公布即代替原来的协议条款，恕不再另行通知，用户可在本网站查阅最新版协议条款。在本公司修改本协议相关条款之后，如果用户不接受修改后的条款，请立即停止使用本公司提供的服务，用户继续使用本公司提供的服务将被视为已接受了修改后的协议。

除非您接受本协议所有条款，否则您无权注册、登录或使用本协议所涉相关服务。您的注册、登录、使用等行为将视为对本协议的接受，并同意接受本协议各项条款的约束。非云服务用户不受以下条款约束，具体内容以双方签订合作协议为准。

1、用户使用规则

- 1.1、用户须是具备完全民事行为能力 and 完全民事行为能力的自然人、法人或其他组织，并保证所提供的个人或单位身份资料信息真实、完整、有效。
- 1.2、用户须同意并接受本公司通过电子邮件，手机号向用户登记的联系方式发送相关商业信息。
- 1.3、用户不得为了任何非法目的而使用本公司上的服务。
- 1.4、用户须遵守所有与本公司服务有关的网络协议、规定和程序。
- 1.5、用户不得恶意占用网络带宽等本公司服务资源。
- 1.6、用户不得使用爬虫等手段收集本服务网站、API 信息。
- 1.7、用户不得在请求 API、脚本中注入 SQL 等破坏条件、代码。
- 1.8、不得逆向工程、反编译或试图以其他方式发现本公司提供的软件的源代码。
- 1.9、不得以任何方式登录或者尝试登录提供服务的主机。
- 1.10、不得以任何方式获取或者尝试获取提供服务的主机的管理权限。
- 1.11、不得利用账户进行任何可能对互联网的正常运转造成不利影响的行为；
- 1.12、不得利用本公司服务系统发布任何骚扰性的、中伤他人的、辱骂性的、恐吓性的、庸俗淫秽的或其他任何非法的信息；
- 1.13、不得利用本公司服务系统进行任何不利于本公司的行为；
- 1.14、就本公司及合作商业伙伴的服务、产品、业务咨询应采取相应机构提供的沟通渠道，不得在公众场合发布有关本公司及相关服务的负面宣传；
- 1.15、如发现任何非法使用用户账号或账号出现安全漏洞的情况，应立即通告本公司。

2、用户权利及义务

- 2.1、默认情况下，在服务协议终止后或者在用户主动删除数据或用户服务期满后，用户使用过程中产生的数据会立即删除。
- 2.2、本公司承诺用户能够控制数据的迁移，保证启用或弃用该服务时，数据能有偿迁入和迁出。
- 2.3、用户私自登录节点进行操作，导致的数据丢失，本公司不提供恢复服务。
- 2.4、本公司不会将用户数据、个人信息等资料泄露给任何第三方，除非政府监管部门监管审计需要。

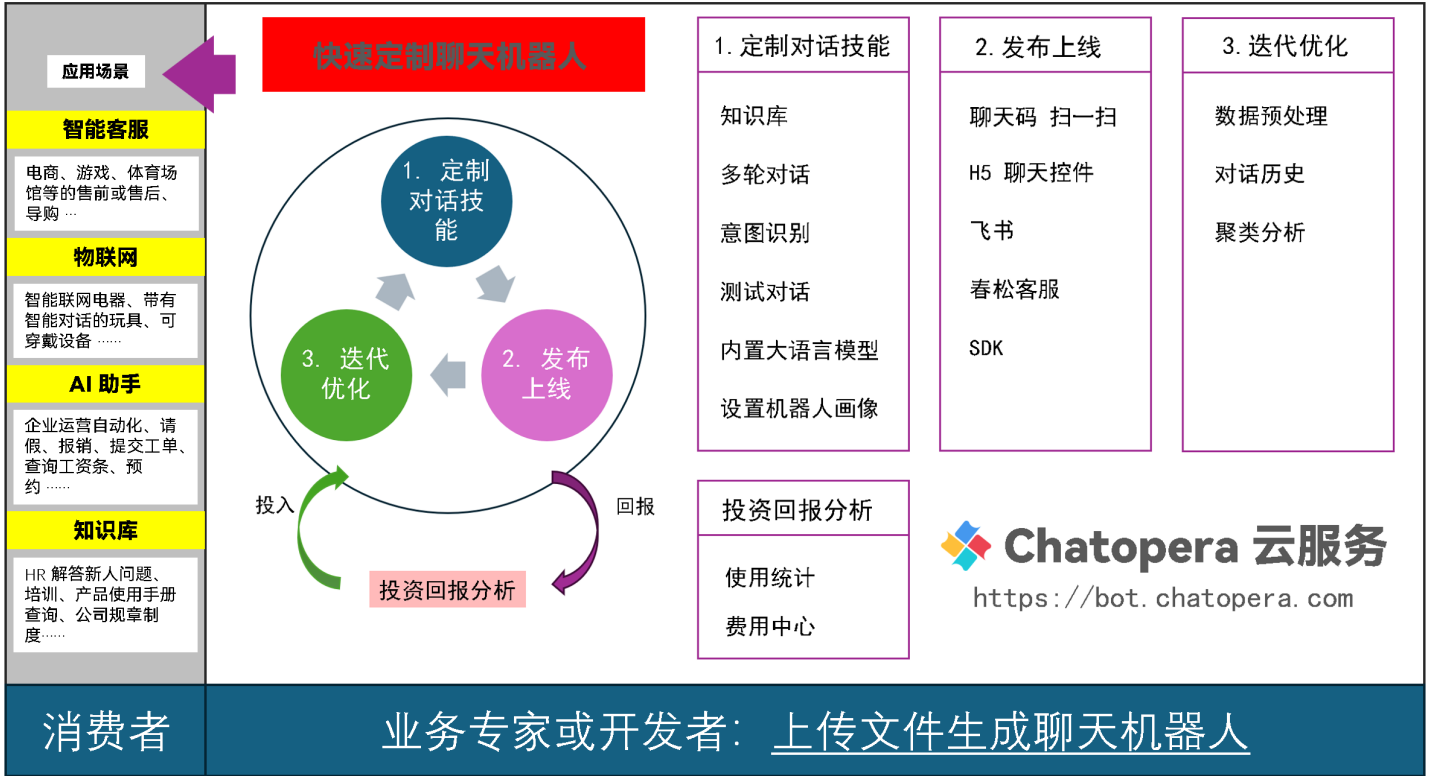
- 2.5、本公司承诺用户在必要的条件下，由于合规或是安全取证调查等原因，可以提供相关的信息。本公司承诺遵守国家相应的法律法规，配合政府监管部门的监管审查。提供的相关信息包括关键组件运行日志、运维人员操作记录等。
- 2.6、为了便于运维，本公司保留查看用户操作记录的权利，登录主机进行运维操作的权利。

3、免责条款

- 3.1、用户实施违反包括但不限于本协议的行为，本公司有权拒绝向用户提供服务，终止本协议，用户已支付的“服务费”预付款不予退还，同时本公司有进一步追究用户法律责任的权利。
- 3.2、本公司通过内容部检测程序发现或经其他机构或用户举报而发现用户有发布违法信息、严重违背社会公德以及其他违反法律禁止性规定或本协议约定的行为时，本公司有权立即终止对用户提供服务并不退还任何款项
- 3.3、不论在何种情况下，本公司均不对由于互联网正常的设备维护，互联网网络联接故障，电脑、通讯或其他系统的故障，电力故障，黑客攻击、计算机病毒侵入或发作、电信部门技术调整导致的影响、因政府管制而造成的暂时性关闭、由于第三方原因(包括但不限于不可抗力，例如火灾、水灾、雷击、地震、洪水、台风、龙卷风、火山爆发、瘟疫和传染病流行、罢工、战争或暴力行为或类似事件等，政府行为，司法行政机关的命令或第三方的不作为等不可抗力而造成的不能服务或延迟服务、用户数据丢失等承担责任，但本公司应及时通知用户，并及时采取措施防止损失扩大。
- 3.4、用户了解并同意，本公司不对因下述任一情况而导致用户的任何损害赔偿承担责任，包括但不限于利润、商誉、使用、数据等方面的损失或其它无形损失的损害赔偿（无论本公司是否已被告知该等损害赔偿的可能性）：使用或未能使用本公司服务；第三方未经批准的使用用户的账户或更改用户的数据；用户对本公司服务的误解；任何非因本公司的原因而引起的与本公司服务有关的其它损失。
- 3.5、特别提示：在受限于本协议其他规定的前提下，为更好的提供服务，本公司将定期或不定期对本公司平台进行功能及（或）页面上的改造及（或）升级。本公司会通过本公司平台或其他有效方式将相关事项及用户注意事项告知用户，经上述告知，本公司不对因用户未能遵守相关注意事项而造成的损失承担任何责任，本公司经改造及（或）升级后，对于用户因操作习惯性而引起的损失，本公司不承担任何责任。
- 3.6、受限于本协议其他规定的前提下，因本公司平台中接入的其他方的支付渠道与用户就支付关系发生任何争议的，与本公司无关，本公司不承担任何责任。
- 3.7、当出现来自或针对客户网站的互联网攻击行为或者接到政府相关部门的监管要求时，本公司将通知客户立即进行处理，对未按照要求及时处理的，本公司有权采取相应措施，以避免网络或内容安全事件的进一步扩大；当出现紧急网络或内容安全事件(例如内容涉及国家监管部门禁止范围，被国家监管部门如网监勒令封闭 IP 等事件)时，为保护广大用户的合法权益，本公司有权在事先不通知对方的情况下采取相应措施。本公司对上述紧急网络或内容安全事件的处理措施免于承担责任。
- 3.8、用户接入第三方服务，比如百度统计等，并不是本公司负责安全保护，如果造成用户财产损失，与本公司无关，本公司不承担任何责任。

2020 年 10 月 [北京华夏春松科技有限公司](#)

Chatopera 机器人平台的定位是低代码上线智能对话机器人的工具和服务。本篇的主要目的是帮助 Chatopera 机器人平台用户从系统的角度理解 Chatopera 的多轮对话的工作机制。在开始之前，您先要对 Chatopera 云服务有一个系统的认识，那就是下面这张图。



作为定制聊天机器人的工具，Chatopera 将定制机器人的生命周期分成了三个：

- 1、定制对话技能；
- 2、发布上线；
- 3、迭代优化。
- 针对全部活动，进行投入产出的分析。

刚刚开始认识 Chatopera 云服务的新用户，待阅读背景知识后，相信你可以更好的使用 Chatopera 机器人平台定制出满足各种需求的聊天机器人。

多轮对话的定义

Chatopera 的智能对话机器人解决方案主要面向企业在客户服务、营销和企业内部协作中使用自然语言交互完成信息查询、任务和自动化流程。目前，市场中比较常见的智能对话机器人解决方案主要是一问一答：设定问答对，使用信息检索形式，提供信息查询功能，在 Chatopera 看来，这不是智能对话机器人的体验。

在 Chatopera，我们认为多轮对话是在一定时间内，一定交互次数内，考虑对话的上下文，每次人发送聊天内容时，机器人具备从上下文情境中分析最合理的回答，准确的为人提供回复。

什么场景下使用多轮对话

Chatopera 的智能对话机器人解决方案是工具，用于设计、实现和发布聊天机器人。从价值提供上，Chatopera 所面向的是封闭域聊天，更关注智能对话机器人在生产、消费和制造等领域的聊天机器人应用；在开放域聊天，尤其是以闲聊为主的应用，不是 Chatopera 智能对话机器人的关注点，虽然我们提供了标准的通用的工具，但是作为开放域聊天，在目前还没有好的技术方案，强人工智能还没有成熟，对于实际应用上，还不能大规模的提供价值。

Chatopera 的多轮对话服务，可以面向不同行业，比如教育、电商、游戏和生产制造等。尤其是客服和企业内部协作中，将以前

通过表单、人工等的服务使用自然语言对话的形式实现。应用场景比如智能客服、智能招聘面试、智能提交工单、智能提交请假、智能分析潜在客户意向等。

实现原理

Chatopera 将问答技术中的不同组成部分构建为基础模块，提供每个模块的管理工具、APIs，Chatopera 机器人平台用户可以单独使用。在基础模块之上，再融合为一个多轮对话方案。

基础模块包括：

- [深度学习](#): 实现语义理解、回答检索、聚类分析等
- [词典](#)：机器人能理解和捕捉的概念
- [知识库](#)：以搜索引擎技术为主
- [意图识别](#)：以机器学习为主，实现任务型对话
- [对话脚本](#)：以脚本规则为主，设定对话规则同时融合 FAQ 和意图识别

以对话脚本为中心融合知识库和意图识别，形成多轮对话，实现定制聊天机器人的标准，是 Chatopera 机器人平台的主要特色：

- [多轮对话的检索](#)：Chatopera 多轮对话如何融合基础模块为一个标准技术
- [模糊匹配器](#)
- [意图匹配器](#)

其它资料

- [视频教程](#)

深度学习

智能问答技术及开源项目

Chatopera 云服务使用深度神经网络。深度学习技术应用于以下任务：

- 近义词匹配，Chatopera 开放相关技术参考 [中文近义词：聊天机器人，智能问答工具包](#)
- 句子相似度，词向量距离预训练模型
- 依存关系分析，Chatopera 开放相关技术参考 [text-dependency-parser](#)
- 词性标注和命名实体识别
- 意图识别，Chatopera 开放相关技术参考 [Clause 中文语义理解引擎](#)
- 聚类分析

出版物

2018 年底，Chatopera 会同行业内其他工作于智能问答与深度学习领域的专家、研究员，与电子工业出版社合作，出版《智能问答与深度学习》一书，该书部分内容是 Chatopera 探索智能问答领域的分享。目前在京东等电商平台图书中有售。



快速查看本书链接: [《智能问答与深度学习》](#)

内容概述

对于需要了解 Chatopera 云服务实现原理的用户，该书将会大有帮助，本书是智能问答的知识介绍，也可以作为入门、工具书。该书介绍的内容相对丰富，目前在京东售出逾 7,000 册，好评 98%。



智能问答深度学习

王海良 李卓桓 林旭鸣 著

试读

自营 5折封顶 (3.1-3.12)

< 关注 举报

企业购更优惠

智能问答与深度学习(博文视点出品)

深度学习人工智能参考书 介绍近年来自然语言处理和机器阅读的成果，配有翔实示例，助力实际应用，请下载，业内大咖力荐 团购电话4006186622

王海良, 李卓桓, 林旭鸣 著

京 东 价 **¥ 34.50** [5折] [定价 ¥69.00] [降价通知](#)

增值业务 [助力环保, 传递知识, 旧书换新](#)

排 行 榜 入选『通俗易懂的人工智能书TOP』 [详情>>](#)

配 送 至 北京朝阳区八里庄街道 [可配货](#)，预计下单后5-10天发货

京东物流 京准达 | 211限时达 | 京尊达 [v](#)

由 **京东** 发货，并提供售后服务。

服务支持 **放心购** 7天价保 | [上门换新](#) | [破损包退换](#) [v](#)

可配送全球49元免基础运费

京东服务 [季度意外换新 ¥1.50](#) [?](#)

白条分期 不分期 ¥11.88 × 3期 ¥6.13 × 6期 ¥3.16 × 12期 ¥1.76 × 24期

1 + -

加入购物车

购买电子书 ¥15.99

开源代码

该书中介绍的示例程序的源码，已经以开源项目的形式发布：

<https://github.com/chatopera/book-of-qna-code>

开放语料

随着 Chatopera 云服务的不断优化，在研发过程中，形成了一些语料，Chatopera 也以开放数据的形式开放，这增加了我们的工作量，而得到的回报是不多的。这个工作的主要目的是回馈给客户、开源社区的同仁们，以及期待更多的研究者可以用这些数据做出技术上的突破、创新，Chatopera 竭诚与业内同仁们一起做好智能问答，早日实现聊天机器人的普及，让科技使我们的生活更美好。

- [保险行业中文语料库](#)
- [心理咨询问答语料库](#)

更多开源项目

- [DeepQA](#): Chatopera 于生成式智能问答的探索、实践
- [docsbot](#): 将使用 Markdown 的文档转化为问答服务

更多开源项目，敬请通过 Chatopera GitHub 主页了解：

<https://github.com/chatopera>

概述

词典就是机器人在对话中所掌握的概念，机器人凭借词典区分不同实体，词典通过不同形式约定了一个词汇的集合。

什么是实体呢？以一个例子说明，比如订一份外卖需要知道用户想定什么外卖，什么时候定，定多少，以及口味要求和地址。那么此时需要的实体就包括：food（食物）、time（时间）、num（数量）、taste（口味）、address（地址）。

Chatopera 机器人平台提供多种形式的词典。

词典类型

词典分为：**系统词典** 和 **自定义词典** 两种类型。

系统词典是 Chatopera 机器人平台预置的词典，词典标识以 "@" 为前缀开头。自定义词典，是用户创建和维护的词典，包括 **词汇表词典** 和 **正则表达式词典** 两种类型。

在智能问答需求中，有一些问题可以提前设置好答案，不涉及上下文环境，是一问一答。对于这类问答，就可以使用知识库模块。

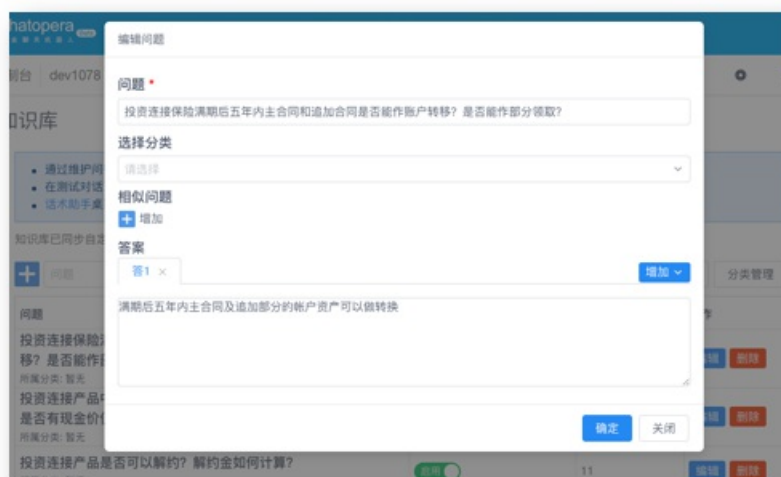
- 录入问题和答案
- 设计标准问、相似问和近义词
- 使用搜索和排序，根据相似度返回结果，同搜索引擎

产品特点

- 内置的检索模型基于大数据训练
- 完善的问答对管理和词典管理

实现原理

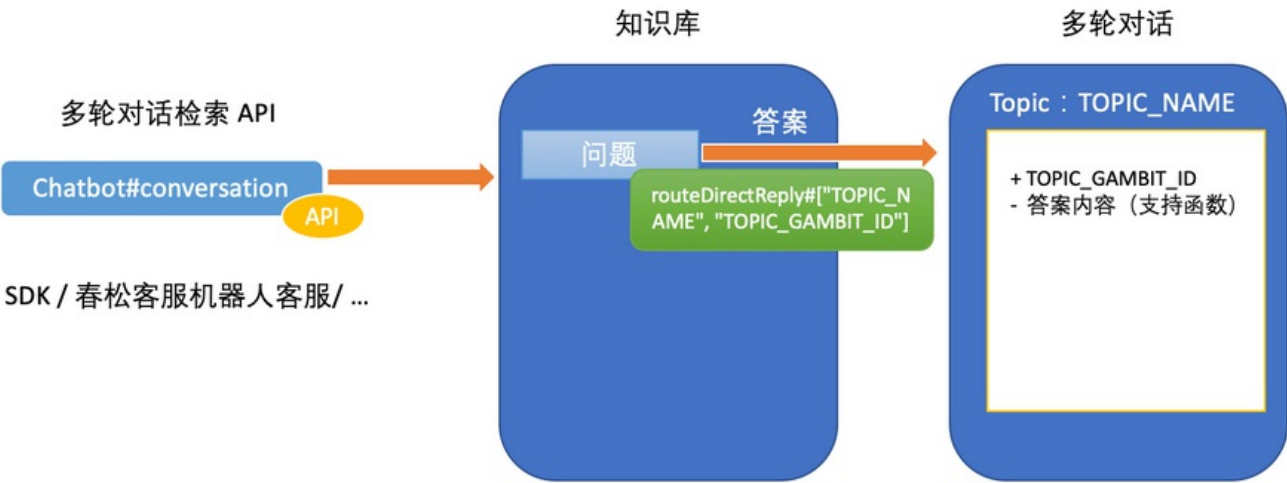
知识库通过标准问、相似问、自定义词典和答案维护对话内容，为 Chatopera 机器人平台用户提供标准的、快速的对话检索服务。对话用户的请求文本和问答对里的标准问或扩展问比较，计算相似度，当相似度高于阈值时，即认定该问答对的答案是对话用户需要的回复。



编辑知识库问答对

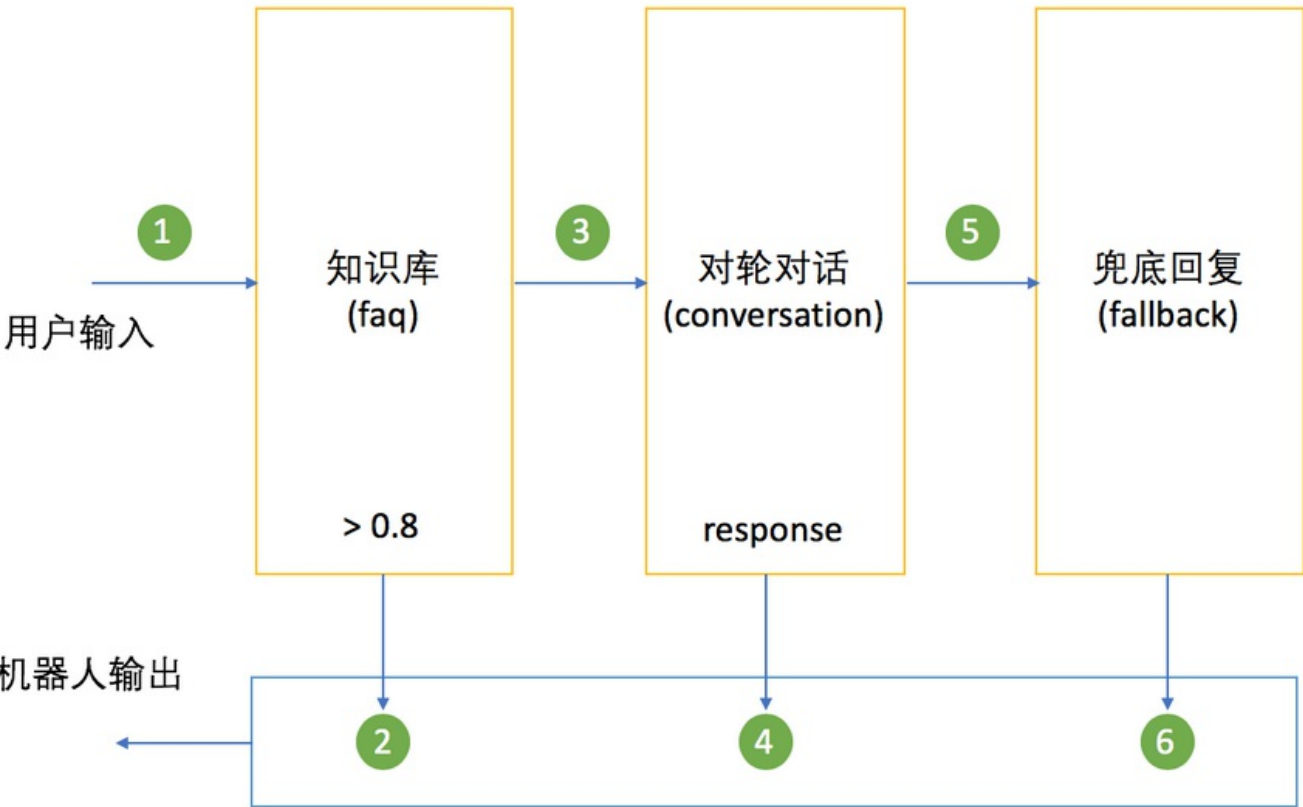
动态答案

将知识库答案路由到多轮对话的话题，然后从多轮对话获得答案。



目前，知识库路由（routeDirectReply答案）只在多轮对话检索上支持，多轮对话检索中，自动优先检索知识库。

Chatopera 多轮对话检索过程，优先级：知识库 > 脚本 > 兜底回复。



应用场景

尤其是在客服场景中，来访者有大量的问题都是重复的一问一答的问题，答案相对来说固定，或者在一段时间内固定，这样最适合通过完善知识库提升客服工作效率。

提示：一问一答是指一个问题对应固定的答案。比如“世界上有几个大洲？”就属于这类为题；但是如果问“今天股市大盘走势如何？”，答案不是固定的，该类对话使用意图识别模块和多轮对话脚本解决。

使用过程

学习使用**知识库**的过程很简单，历史数据也不是必需的。通常先由业务人员整理一些常见问题，并在**知识库管理页面**添加问答对，在**对话测试**页面进行简单的验证就可以集成上线了。

进阶优化知识库包括：

- 1) 设置自定义词典，增强知识库检索时处理近义词的能力；
- 2) 为标准问增加扩展问；
- 3) 在对话**历史记录**页面查看沉寂问题和兜底回复，创建新的问答对；
- 4) 使用**聚类分析**服务，进行大规模的对话历史的自动机器人学习分析。

增强知识库的智能水平，知识库的优化，知识库的优化也是长期的过程，也是企业重要的很有价值的资产。

意图识别

在应用聊天机器人的场景中，有一大类是任务驱动型的问答，比如来访者说“我想购买车票”，那么接下来机器人围绕订票这个需求，询问出行时间、目的地等信息，并将收集到的信息返回给聊天机器人开发者，完成订单确认、支付和下单订票等。这就是一个典型的任务驱动的问答：以识别意图并根据意图收集相关信息为目的。类似的对话需求，在 Chatopera 云服务中，是通过意图识别模块支持。

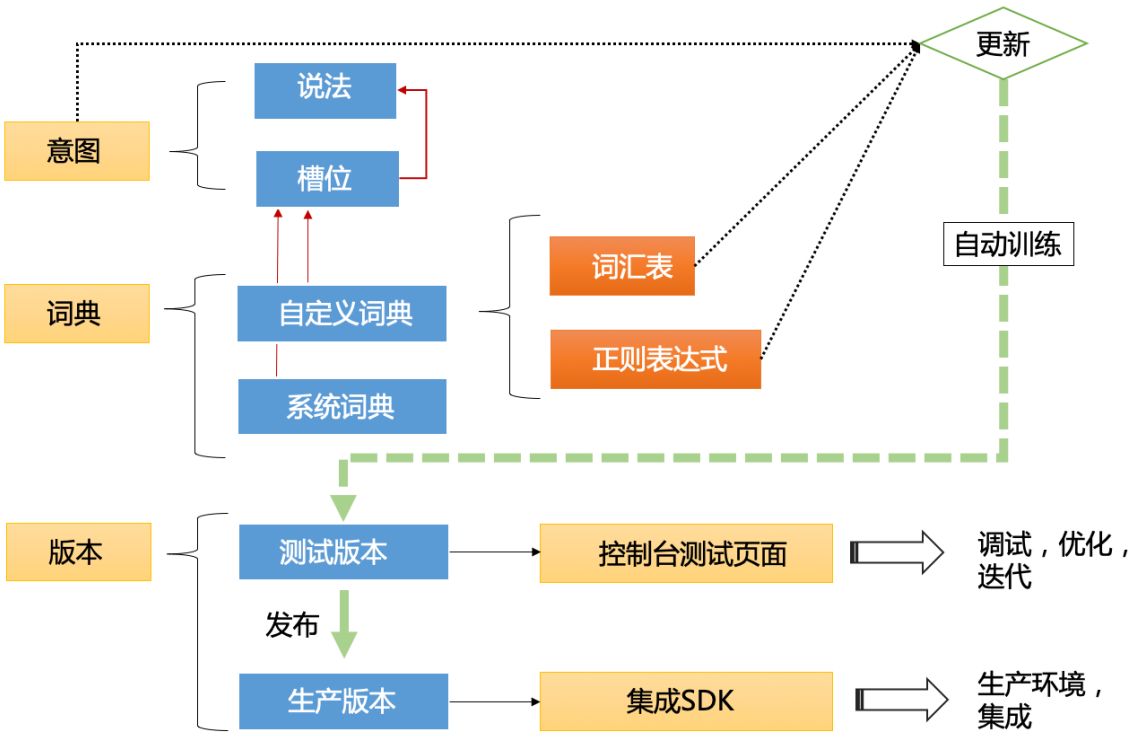
- 使用说法定义意图分类模型
- 利用序列标注识别槽位信息

产品特点

- 高性能算法
- 支持小规模数据量训练
- 在线标注和训练

实现原理

意图识别是语义理解的一个重要话题：与机器人对话时，是需要机器首先理解人的意图的；然后，根据这个意图，机器人继续与人进行问答；得到了这个意图的相关信息，机器才去执行这个意图代表的任务。



使用过程

意图：通过为每个意图添加说法和槽位，训练机器学习模型。对话用户的文本被分析为某一个意图，如果识别了意图，则计算槽位信息，并且对于没有识别到的槽位进行追问。

槽位：和一个意图相关的关键信息，比如时间、地点和专有名词等。槽位可以绑定到某个词典。

Chatopera 机器人平台提供自定义词典和系统词典：

- * 自定义词典包括词汇表词典和正则表达式词典；
- * 词汇表词典主要用于处理业务上的关键字段、同义词和专有名词；
- * 正则表达式词典则用于识别手机号、身份证号、订单号、邮箱等一些具有规则的信息；
- * 系统词典是用机器学习训练的命名实体提取判定的信息，比如人名、地名、组织机构和时间等。

输入用户可能的说法，用[]使用槽位。例如：我想预订[CityName]的机票

添加按Enter键可以快速添加

用户说法	操作
我想打车	删除

槽位

槽位名称TIME词典names必填

追问什么时间出发?

添加

槽位名称	词典	必填	追问	操作
暂无数据				

标注意图识别

意图匹配器

在 Chatopera 机器人平台，意图识别模块也是集成进入了多轮对话模块，参考使用意图匹配器。

对话脚本

- 以语法规则定义对话
- 融合知识库问答和意图识别
- 可执行自定义的 JavaScript 函数

产品特点

- 规范、灵活的脚本语法
- 语法支持编程语言，内置 NLP 函数
- 对话脚本可共享分发，Chatopera 提供了[对话模板](#)供快速定制对话服务

实现原理

对话脚本是通过书写 Chatopera 多轮对话脚本语法建立的规则实现问答。

- 对话脚本将对话分为**匹配器**，即和对话用户的输入进行比较的规则
- **回复**，匹配规则后的回复内容，支持文本和函数
- **上下轮钩子**，将规则与规则进行关联
- 对话脚本中的**函数**，使用 JavaScript 方式低代码编程，内置函数库，即方便了系统集成，同时也提供大量自然语言处理帮助函数，不需要掌握自然语言处理 NLP 的知识，开发者可以专注的实现业务逻辑

使用过程

匹配器是对话的基础，当用户向聊天机器人发送一条消息时，Chatopera 机器人平台会从所有定义的**匹配器**中找到匹配的规则。**匹配器**用半角字符加号`+`开始，对应的**回复**是紧邻的下行，用半角字符减号`-`开始。

例如，我们可以这样定义一个对话：

```
+ 晚饭吃什么  
- 烤鸭
```

注意：这里`+`和`-`和文字之间需要隔一个空格。

匹配器有多种，分别通过不同形式建立匹配规则，以适应不同场景的灵活使用。

- **通配符匹配器：**[使用语法建立规则](#)
- **模糊匹配器：**[容错能力更强和智能的匹配器](#)
- **意图匹配器：**[借助意图识别模块，轻松实现任务型对话](#)

在脚本匹配器语法和函数中，融合知识库和意图识别，所以，对话脚本是多轮对话的中心，围绕脚本实现不同问答技术的融合。所以，很多时候，文档和产品中，也用【多轮对话】表述对话脚本，就是这个原因。对话脚本独立构成了一个模块，在 Chatopera 机器人平台内部，实现了对话脚本引擎。



管理多轮对话

模糊匹配器

通配符匹配器的不足

以前, 在多轮对话中, 使用通配符匹配器存在一个问题: 匹配器规则的书写要尽量和输入的对话文本一致。使用通配符, 用一些特殊符号, 比如 `*n` 去定义一些目标词汇的集合, 然而这依然在构建对话流程时显得不够智能, 对于目标词汇的识别以及按照位置匹配, 是通配符匹配器的优势。

举一个例子:



The screenshot displays a chatbot interface with a conversation history on the left and a script editor on the right. The chat history shows a user saying "你好" and the bot replying "你好!". The script editor shows a rule for "你好" with a red arrow pointing to the rule definition.

```
1 /**
2  * 获得示例程序, 快入开始!
3  * https://github.com/chatopera/chatbot-samples
4  */
5
6
7 + (你好|您好|你好啊)
8 - 你好!
```

在上面的脚本中, 定义了一个通配符匹配器 "+ (你好|您好|你好啊)", 它支持三种问法, 但是用户可能用更为多变的说法, 为了支持这些说法, 可以选择使用更宽松的通配符, 但很多时候并不好把握:

- 1) 使用枚举的思路, 当要支持所有相关意图的说法时, 耗时费力;
- 2) 使用通用通配符, 太过灵活, 有的时候匹配到了期望以外的词语。

为了解决“容错”和表达能力不足的问题, 需要提供更多形式的匹配器来解决。

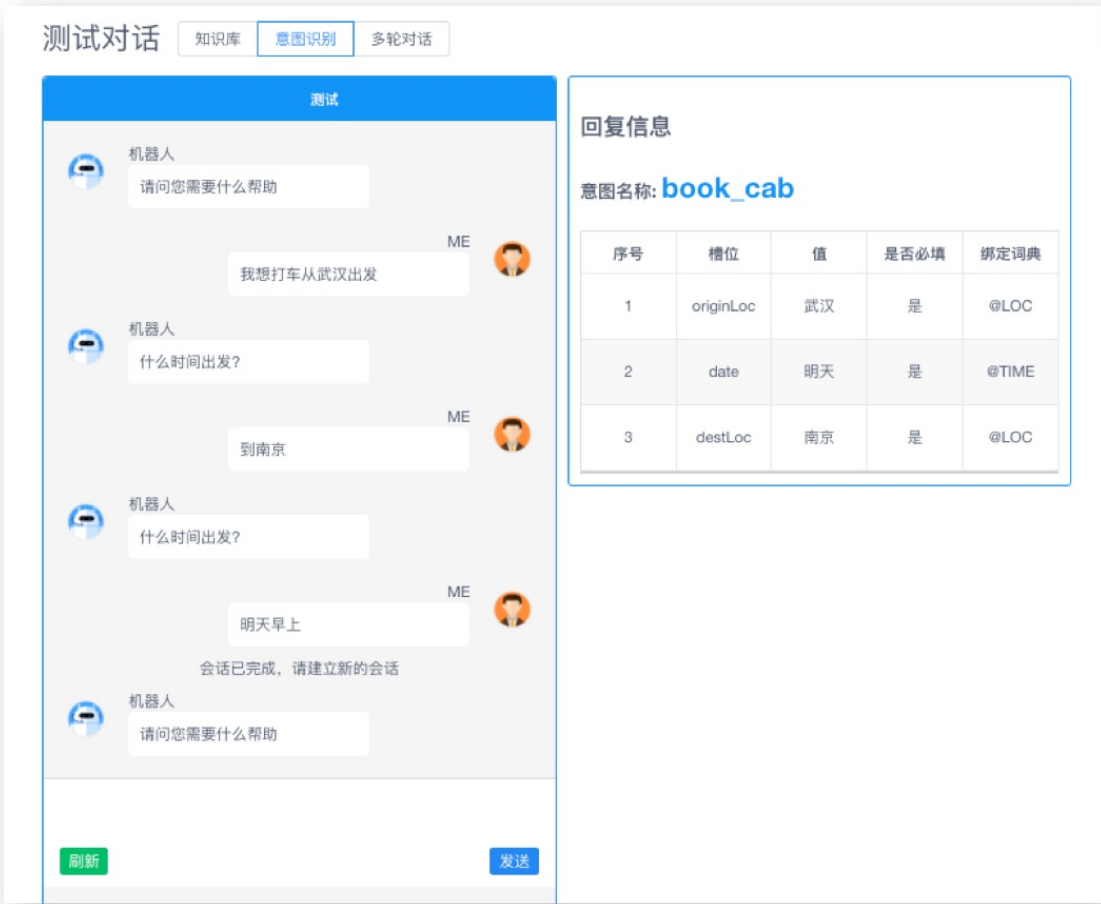
接下来

- 在多轮对话中使用模糊匹配器

设计初衷

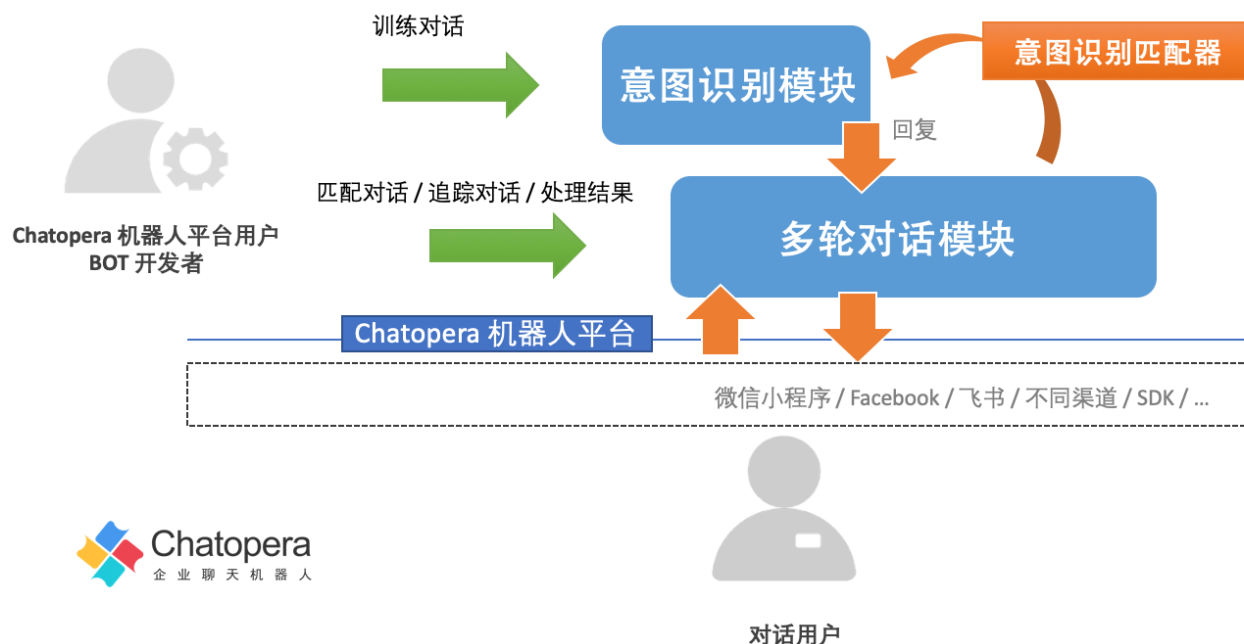
Chatopera 机器人平台 2019 年就发布了意图识别模块。意图识别模块使用机器学习技术，通过少量的数据标注，就可以匹配很多种用户说法，实现智能的任务型对话机器人，意图识别模块基于大规模的通用语料，内置多个预训练模型，以适应不同行业应用。在过去两年，Chatopera 团队一直在探索，将对话脚本和意图识别结合，促成更为强大的多轮对话定制能力。现在，一系列最佳实践通过意图匹配器的形式实现并发布了。

在没有意图匹配器以前，对话脚本，是按照一个个固定的流程进行的，通过上下轮钩子，比如用户说：我想打车，写好了开始的规则，并匹配上以后，按照脚本顺序问打车的数据：时间、出发地、目的地等。如果机器人问“您想从哪里出发？”，而对话用户发送了时间“今天下午 5 点”，那么在脚本中，处理起来费事，还需要增加学习成本。而针对于类似的场景，机器人需要能识别时间，然后继续追问出发地。智能的处理任务中的关键信息，是普遍存在的需求。而 Chatopera 意图识别对话，就是解决这个问题的。



意图识别模块，简单说就是识别并分析意图信息，根据意图需要的关键信息进行追问，直到获取到所有必须的关键信息或超过追问限制次数。

以前，在 Chatopera 机器人平台内，意图识别模块和多轮对话模块，是两个独立模块，Chatopera 提供了[开源项目](#)的形式提供最佳实践：在业务端，BOT 开发者用 SDK 灵活的使用不同模块。这样的形式，学习成本加大，怎么样通过低代码的方式上线有意图识别能力的机器人呢？现在，这个答案就是意图匹配器。在多轮对话脚本中，借助意图匹配器很好的融合了脚本、函数和意图识别。Chatopera 机器人平台用户，可以用更为简单和快捷的方式，上线更为强大的智能对话机器人。



对话脚本和意图识别二者的结合，至少获得了如下几点好处：

- 1) 会话周期在多轮对话中管理，以前意图识别需要 BOT 开发者自行维护会话周期，机器人管理控制台提供参数进行配置；
- 2) 增加对话灵活性，识别意图后，转入意图识别对话，但是对于一个意图中关键信息，没有获取到，则追问；对一个关键信息连续追问一定设置次数，则继续查询知识库和脚本，提升了处理的能力（这个继续查找行为也称为“穿透”）；
- 3) 意图识别得到的意图信息，在多轮对话脚本的函数可以读取，可处理意图识别成功或者意图识别槽位追问次数超过设定最大值，两种情况的信息；
- 4) 利用多轮对话函数中，内置的高级 NLP 函数，继续分析意图信息，比如使用 `extractTime` 将相对时间转化为绝对时间，也就是“今天”识别为具体的日期 2021-08-31；
- 5) 利用多轮对话函数中，`http` 工具类，和业务系统、互联网服务等集成。

以上的这些好处，让 Chatopera 机器人平台可以诞生更多的创新的对话机器人，帮助 BOT 开发者尽情的释放创造力！

欲了解更多 Chatopera 多轮对话工作机制，查看[详情链接](#)，您也可以阅读完本篇后再阅读多轮对话工作机制。

意图匹配器的功能

在对话机器人中，一个常见的应用场景是通过自然语言形式，自动化的完成流程，比如：“我想打车”、“我想看病挂号”和“我想点外卖”等。对话用户说了这些后，就是要明确的希望机器人帮助完成一个任务：查询信息、下单或管理智能家居等。这个任务和这个任务里的关键信息，比如时间、地点和专有名词，就是机器人需要捕捉的，机器人通过追问的形式交互。

在 Chatopera 机器人平台上，也使用如下的术语表达这些概念。

概念	术语
任务	意图
任务的说法	说法
关键信息	槽位
概念； 词语	词典

这些对话内容的管理，就在意图识别模块中：开发者或业务人员，提供说法；在说法中可以写槽位；槽位对应的是不同类型的词典。

那么对话用户的哪些说法对应了意图呢？哪些词对应了槽位呢？在 Chatopera 意图识别模块中，提供维护意图、说法、槽位和词典的页面。

← book_cab

用户说法 ⓘ

输入用户可能的说法，用{}使用槽位，例如：我想预订{CityName}的机票

添加

按Enter键可以快速添加

用户说法

我想打车

我想打车，从{originPlace}出发

我想打车到{destPlace}

词典建设，是意图识别中很重要的方面，在 Chatopera 机器人平台上，词典管理非常完善。词典包括系统词典，自定义词典。系统词典是 Chatopera 机器人平台上开箱即用的词典，包括人名、组织机构名、时间和地点的。自定义词典有词汇表词典和正则表达式词典，是 Chatopera 机器人平台用户自行维护的。

训练意图识别对话机器人的工作，不需要开发技能，无代码工作；不管是开发者，还是业务人员，只要熟悉上述的几个概念就可以训练。

那么，训练好了意图识别对话，那么要上线就还存在两个问题：

1) 该如何处理识别结果，比如“我想打车”，得到了对话用户的出发地、目的地和时间等信息，如何发布订单呢？

2) 如果对话要被集成到一个客服机器人里，这个客服机器人也可以回答常见问题，在 Chatopera 知识库里设定了，如何让不同模块的对话融合起来呢？

上述两个问题的解决方案，就是使用意图匹配器。

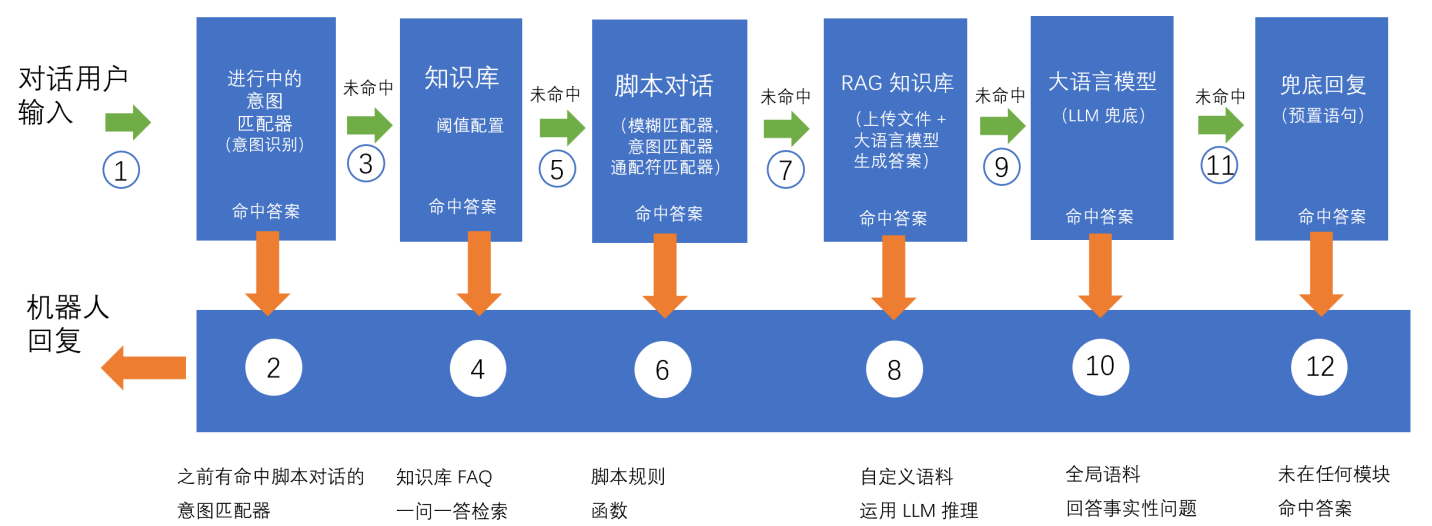
接下来

- [在多线程对话中使用意图匹配器](#)

在 Chatopera 多轮对话中，理解检索顺序是熟悉多轮对话原理的关键知识点。首先是基础模块间的检索顺序，然后是对话脚本中的话题的检索顺序。

模块间检索

对话用户请求时，多轮对话会先检查是否有进行中的意图识别对话，然后是知识库检索，匹配知识库问答对，当有问答对高于知识库阈值时，机器人回复问答对中的答案内容；未匹配知识库，进入对话脚本，从话题中匹配，匹配上则回复内容；否则则回复兜底回复。从概念上有下图关系。



之所以说是从概念上，是因为整个检索过程更为复杂，不方便理解，先用概念图从直观上快速理解 Chatopera 多轮对话的框架。

关于知识库、对话脚本和意图识别的相互之间的调用关系，后文会有更多介绍。

创建对话脚本的话题

聊天机器人的多轮对话主要就是很多对话规则的组合，可以设想这些对话规则组成了一个一个的话题，在对话时，可以聊一个话题，可以跳跃到其它的话题。那么，每次获得了聊天用户的文本，机器人进行答案的检索就是选择最合理的话题并找到匹配的规则，计算回复内容。

在创建好机器人后，下载多轮对话设计器，在多轮对话设计器中，创建话题。



话题里使用脚本实现对话逻辑，用一个简单的例子说明脚本语法非常容易掌握。

+ <noun>是个好地方

- 我很喜欢<cap>

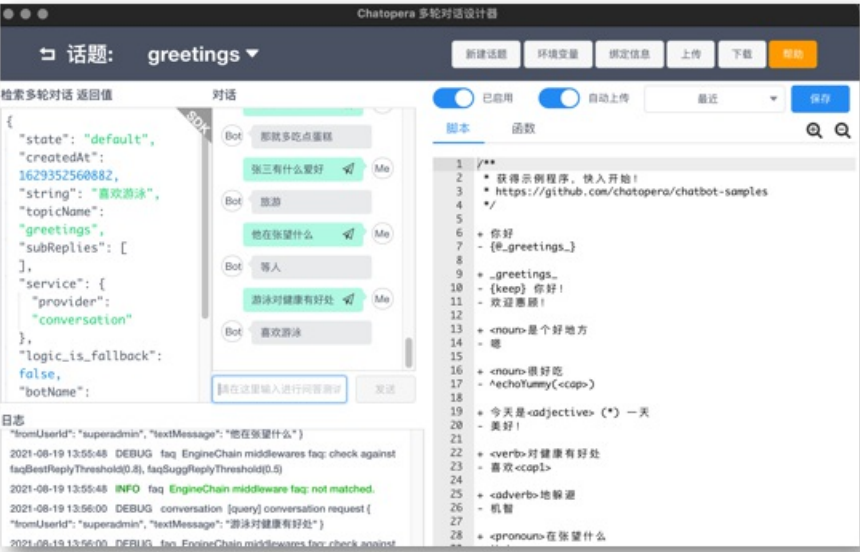
+ \${0.5}{上一次去是什么时候}

% 我很喜欢<cap>

- 去年10月份

+ 开始的是匹配器，目前 Chatopera 多轮对话支持通配符匹配器和模糊匹配器；- 开始的是回复，目前支持文本和函数，函数是 JavaScript 脚本；% 开始的是上下轮钩子，用以关联规则。

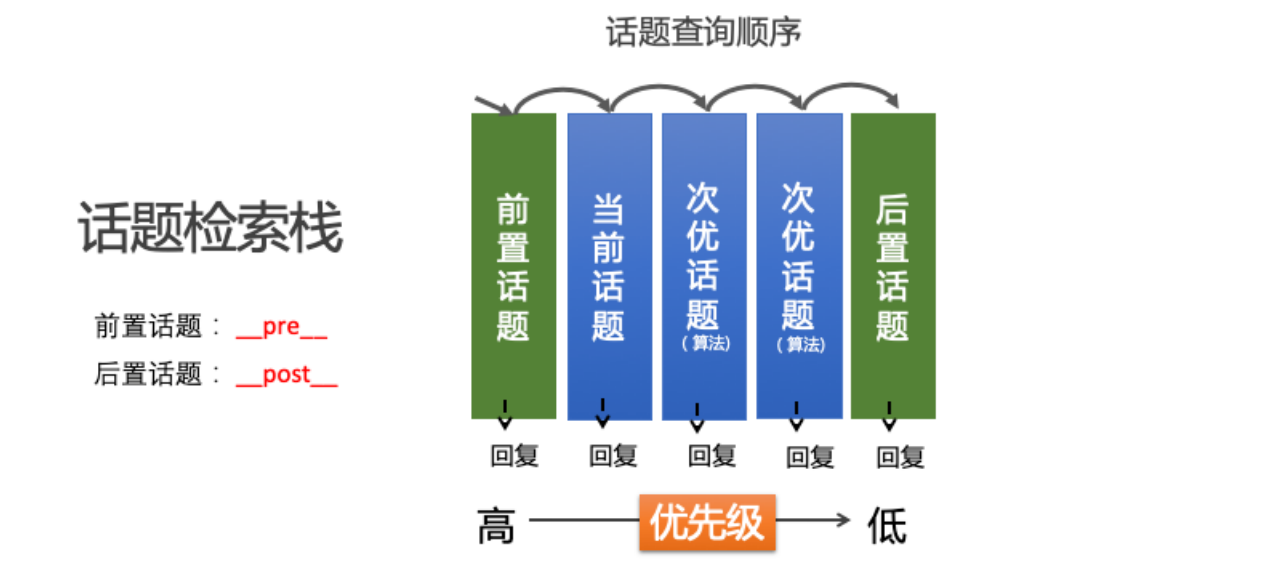
使用多轮对话设计器撰写对话脚本的用户体验，经过了多年的打磨，已经非常成熟和稳定。



在多轮对话设计内调试对话，现在对于多轮对话设计器还不需要详细了解，本节内容旨在介绍原理，而话题作为重要的概念，需要先介绍如何创建以及在哪里编辑。

话题检索顺序

每个话题包含名字和一些规则，每个规则包括了匹配器和回复，业务上有明确的上下轮依赖的规则使用上下轮钩子进行关联。话题检索也是有顺序的，匹配到了某一个话题的规则，后续的话题就被跳过。匹配从高优先级到低优先级进行，最高优先级是【前置话题】，话题名称为系统约定：__pre__；最低优先级是【后置话题】，话题名称为系统约定：__post__。优先级仅低于前置话题的话题是当前话题，就是上一次对话用户匹配到的规则所在的话题；其余的话题根据算法动态的排序。

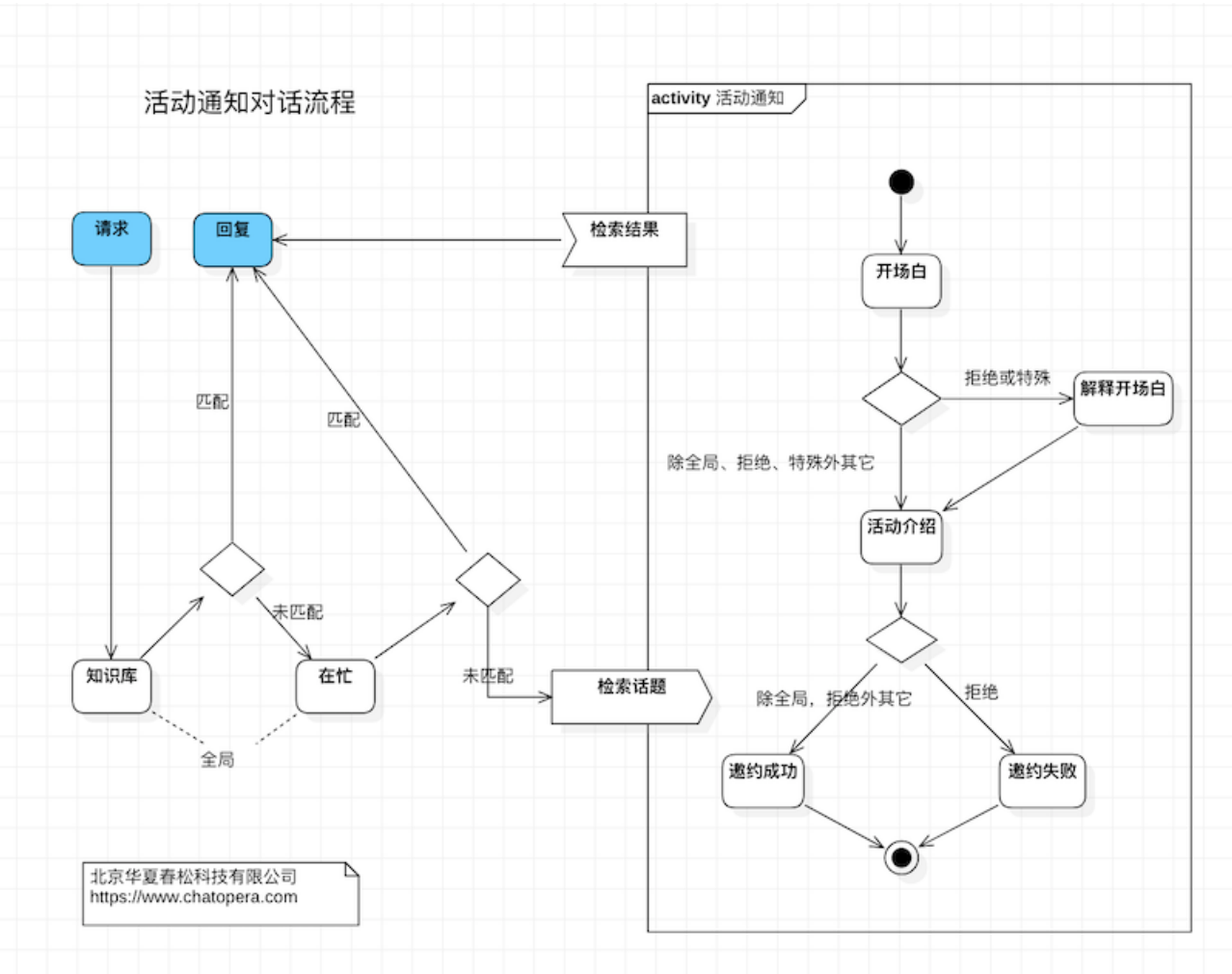


如果使用了上下轮钩子，则最先匹配携带有上下文钩子（就是 % 上次回复内容）的规则。

整体上，对话脚本在检索时，检索栈是动态变化的。

对话状态机

现在，我们从另一个角度，状态机的角度思考多轮对话，因为是在多个连续的交互中，完成一个对话目标，那么就存在一个状态的问题，状态机是状态可以转移的图，两个状态之间的关系通过状态机约束。比如，某个活动通知的对话状态机如下：



这是个图示，仅为了说明原理。“请求”节点代表每次对话用户发送了文本，“回复”节点代表机器人处理结果，回复文本。因为知

识库在检索中最先发生，可以放入一些一问一答的问答对，而一些全局的关键词放入前置话题【__pre__】中，图中右侧方框内，则是由其他话题组成的对话脚本，整个对话构成了状态机。

这个问题在对话脚本中，尤其需要注意：Chatopera 对话脚本引擎会考虑过去一段时间内，一定对话轮次的历史，机器人会回看这些记录来分析最合理的回复。这个时间长度和轮次的约束，在 Chatopera 机器人平台管理控制台内可以设定，是每个机器人的属性：[会话回溯最大时长](#)和[会话回溯最大轮次](#)。



当对话用户的输入匹配到对话脚本的规则时，即是对话到达了一个状态。

擦除状态

开发者在多轮对话脚本的函数中，也可以擦除这个状态，这样到下次对话用户再请求时，话题检索的栈回到初始状态，状态机回到原始，这个擦除的方法是在回复或函数中添加 `{CLEAR}` 前缀，关于这个知识点的使用说明见[文档](#)。

使用函数切换状态

如果想从一个话题，切换到另外一个话题获得回复。那么可以在函数中使用 `"topicRedirect(TOPIC_NAME, TOPIC_GAMBIT_ID)"`。将 `TOPIC_NAME` 替换为话题名字，`TOPIC_GAMBIT_ID` 替换为匹配器。就可以切换到该规则下获得回复。

`topicRedirect` 函数的更多介绍，[参考文档](#)。

知识库路由

对于状态机的状态跳转，Chatopera 多轮对话方案中，还有一个高级方法，通过知识库路由对话到对话状态机任意状态。

```
routeDirectReply#["TOPIC_NAME", "TOPIC_GAMBIT_ID "]
```

设置知识库问答对

设定知识库的问答对中的答案，内容使用上述格式，将 `TOPIC_NAME` 替换为话题名字，`TOPIC_GAMBIT_ID` 替换为匹配器。就可以切换到该规则下获得回复。

关于知识库路由的更多介绍，[参考文档](#)。

延申说明

《[对话实现的更多详细说明（查看大图）](#)》虽然更为复杂，但和概念图含义基本一致，其中的要点是模块间有更多的状态检查和穿透行为（从一个模块进入另外一个模块）。在检索的过程中，涉及到一些参数，这些参数可以在 Chatopera 机器人平台对话机器人设置页面设定或者在 SDK 中传入参数。比如知识库阈值默认为 `0.9`，该阈值可以通过在请求中设定参数来调整，[介绍链接](#)。

视频教程

本系列视频帮助企业在 Chatopera 机器人平台上开发聊天机器人，通过自然语言交互的形式，定制开发聊天机器人，提升业务流程自动化。Chatopera 机器人平台包括知识库、多轮对话、意图识别和大语言模型等组件，标准化聊天机器人开发，支持企业 OA 智能问答、HR 智能问答、智能客服和网络营销等场景。

课程内容分为两大部分：

- **【初级入门】** 《Chatopera 机器人平台使用指南》
- **【高级深入】** 《Chatopera 机器人平台 Deep Dive》

内容大纲(部分)

1.1 自然语言交互的当下与未来

- 自动化、就业和生产力
- 聊天机器人的商业应用
- Chatopera 机器人平台概览
- 从入门到精通

1.2 聊天机器人应用场景及前提准备

- 需求、场景设定、系统组成
- 对话流程设计
- 建模工具介绍
- 设计“聊天机器人实现工具”的思考

1.3 实现对话应用：活动通知

- 知识库用途和管理
- 脚本实现话题
- 多轮对话的检索顺序
- 动手完成活动通知话题
- 更多对话模板及总结

1.4 深入理解聊天机器人实现和更多实用技巧

- 信息检索系统概述
- 话题脚本引擎
- 多轮对话：从知识库路由到脚本引擎
- Chatopera 文档中心的使用

1.5 系统集成

- SDK 语言及安装
- 实例化 Chatbot

- Command 接口介绍
- 对话检索：知识库、多轮对话

1.6 聊天机器人上线后优化

- 分析投资回报率（RoI）
- 优化沉寂问题和错误回复
- 总结
- 后续分享计划

课程地址：<https://www.bilibili.com/read/cv7526530>

Chatopera 机器人平台使用指南

自动化、智能化的自然语言交互



通过左侧导航菜单了解不同的附录内容。

词性标注

词性标注（Part-of-Speech tagging 或 POS tagging），又称词类标注或者简称标注，是指为分词结果中的每个单词标注一个正确的词性的程序，也即确定每个词是名词、动词、形容词或其他词性的过程。

使用位置

在 Chatopera 机器人平台内，较多的使用词性标注，为了方便 BOT 开发者实现更为强大的对话能力的机器人，部分模块开放词性标注的结果，供灵活的支持更多场景。

多轮对话内置函数

在 Chatopera 机器人平台中，多轮对话模块，使用[函数](#)处理请求和回复时，支持在函数中，使用 `this.message.words` 和 `this.message.tags` 访问经过自然语言处理的输入文本信息。

比如输入：商场的开门时间。那么，则有

```
this.message.words = ["商场","的","开门","时间"];
this.message.tags = ["nis","ude1","vi","n"]
```

针对，不同的语言，tags 所使用的定义集合是不一样的。以下正是说明不同语言，Chatopera 机器人平台使用的词性标注集合。

中文

包括简体中文和繁体中文，使用的词性标注集。在汉语中，词性标注比较简单，因为汉语词汇词性多变的情况比较少见，大多词语只有一个词性，或者出现频次最高的词性远远高于第二位的词性。

TAG	词性
a	形容词
ad	副形词
ag	形容词性语素
al	形容词性惯用语
an	名形词
b	区别词
begin	仅用于始##始
bg	区别语素
bl	区别词性惯用语
c	连词
cc	并列连词
d	副词

TAG	词性
dg	辄,俱,复之类的副词
dl	连语
e	叹词
end	仅用于终##终
f	方位词
g	学术词汇
gb	生物相关词汇；或 Chatopera Built-in 词汇，在 Chatopera 意图识别模块使用该标签作为自定义词汇
gbc	生物类别
gc	化学相关词汇
gg	地理地质相关词汇
gi	计算机相关词汇
gm	数学相关词汇
gp	物理相关词汇
h	前缀
i	成语
j	简称略语
k	后缀
l	习用语
m	数词
mg	数语素
Mg	甲乙丙丁之类的数词
mq	数量词
n	名词
nb	生物名
nba	动物名

TAG	词性
nbc	动物纲目
nbp	植物名
nf	食品，比如“薯片”
ng	名词性语素
nh	医药疾病等健康相关名词
nhd	疾病
nhm	药品
ni	机构相关（不是独立机构名）
nic	下属机构
nis	机构后缀
nit	教育相关机构
nl	名词性惯用语
nm	物品名
nmc	化学品名
nn	工作相关名词
nnd	职业
nnt	职务职称
nr	人名
nr1	复姓
nr2	蒙古姓名
nrf	音译人名
nrj	日语人名
ns	地名
nsf	音译地名
nt	机构团体名

TAG	词性
ntc	公司名
ntcb	银行
ntcf	工厂
ntch	酒店宾馆
nth	医院
nto	政府机构
nts	中小学
ntu	大学
nx	字母专名
nz	其他专名
o	拟声词
p	介词
pba	介词“把”
pbei	介词“被”
q	量词
qg	量词语素
qt	时量词
qv	动量词
r	代词
rg	代词性语素
Rg	古汉语代词性语素
rr	人称代词
ry	疑问代词
rys	处所疑问代词
ryt	时间疑问代词

TAG	词性
ryv	谓词性疑问代词
rz	指示代词
rzs	处所指示代词
rzt	时间指示代词
rzv	谓词性指示代词
s	处所词
st	停用词，stopword 缩写
t	时间词
tg	时间词性语素
u	助词
ud	助词
ude1	的 底
ude2	地
ude3	得
udeng	等 等等 云云
udh	的话
ug	过
uguo	过
uj	助词
ul	连词
ule	了 喽
ulian	连 （“连小学生都会”）
uls	来讲 来说 而言 说来
usuo	所
uv	连词

TAG	词性
uyy	一样 一般 似的 般
uz	着
uzhe	着
uzhi	之
v	动词
vd	副动词
vf	趋向动词
vg	动词性语素
vi	不及物动词（内动词）
vl	动词性惯用语
vn	名动词
vshi	动词“是”
vx	形式动词
vyou	动词“有”
w	标点符号
wb	百分号千分号，全角：％‰ 半角：%
wd	逗号，全角：， 半角：，
wf	分号，全角：； 半角：；
wh	单位符号，全角：¥ \$ £ °℃ 半角：\$
wj	句号，全角：。
wky	右括号，全角：）〕】〉 半角：)}{>
wkz	左括号，全角：（〔【〈 半角：([[{<
wm	冒号，全角：： 半角：：
wn	顿号，全角：、
wp	破折号，全角：—— — — 半角：——

TAG	词性
ws	省略号，全角：
wt	叹号，全角： ！
ww	问号，全角： ？
wyy	右引号，全角： ” ’ 』
wyz	左引号，全角： “ ‘ 『
x	字符串
xu	网址 URL
xx	非语素字
y	语气词(delete yg)
yg	语气语素
z	状态词
zg	状态词

English / 英语

TAG	DESCRIPTION	EXAMPLES
CC	Coord Conjunctn	and,but,or
CD	Cardinal number	one,two
DT	Determiner	the,some
EX	Existential there	there
FW	Foreign Word	mon dieu
IN	Preposition	of,in,by
JJ	Adjective	big
JJR	Adj., comparative	bigger
JJS	Adj., superlative	biggest
LS	List item marker	1,One
MD	Modal	can,should

TAG	DESCRIPTION	EXAMPLES
NN	Noun, sing. or mass	dog
NNP	Proper noun, sing.	Edinburgh
NNPS	Proper noun, plural	Smiths
NNS	Noun, plural	dogs
POS	Possessive ending	's
PDT	Predeterminer	all, both
PRP\$	Possessive pronoun	my,one's
PRP	Personal pronoun	I,you,she
RB	Adverb	quickly
RBR	Adverb, comparative	faster
RBS	Adverb, superlative	fastest
RP	Particle	up,off
SYM	Symbol	+,%,&
TO	'to'	to
UH	Interjection	oh, oops
VB	verb, base form	eat
VBD	verb, past tense	ate
VBG	verb, gerund	eating
VCN	verb, past part	eaten
VBP	Verb, present	eat
VBZ	Verb, present	eats
WDT	Wh-determiner	which,that
WP	Wh pronoun	who,what
WP\$	Possessive-Wh	whose
WRB	Wh-adverb	how,where

TAG	DESCRIPTION	EXAMPLES
,	Comma	,
.	Sent-final punct	. ! ?
:	Mid-sent punct.	: ; Ñ
\$	Dollar sign	\$
#	Pound sign	#
"	quote	"
(	Left paren	(
)	Right paren	)

日语 / Japanese

TAG	説明
N	名詞 # Noun
PRO	代名詞 # Pronoun
DT	連体詞 # Adjectival determiner
V	動詞 # Verb
ADJ	形容詞 # Adjective
ADJV	形状詞 # Adjectival verb
ADV	副詞 # Adverb
PRT	助詞 # Particle
AUXV	助動詞 # Auxiliary verb
PUNCT	補助記号 # Punctuation
SYM	記号 # Symbol
SUF	接尾辞 # Suffix
PRE	接頭辞 # Prefix
TAIL	語尾 # Word tail (conjugation)
CC	接続詞 # Conjunction

TAG	説明
PRP	代名詞 # Pronoun
URL	URL # URL
ENG	英単語 # English word
FIL	言いよどみ # Filler
MSP	web 誤脱 # Misspelling
INT	感動詞 # Interjection
KYTEA_JA_DEFAULT_TAG	新規未知語 # Unclassified unknown word